[toc]

前言和致谢（Foreword and acknowledgments）

> This is a draft text intended for a class on operating systems. It explains the main concepts of operating systems by studying an example kernel, named xv6. Xv6 is modeled on Dennis Ritchie’s and Ken Thompson’s Unix Version 6 (v6) [17]. Xv6 loosely follows the structure and style of v6, but is implemented in ANSI C [7] for a multi-core RISC-V [15].

这是一篇用于操作系统课程的教材草案。它通过研究一个名为 xv6 的示例内核来解释操作系统的主要概念。xv6 以 Dennis Ritchie 和 Ken Thompson 的 Unix 版本 6 (v6) [17] 为蓝本。xv6 大致沿用了 v6 的结构和风格，但采用 ANSI C [7] 语言实现，适用于 “多核（multi-core）” 的 RISC-V [15]。

> This text should be read along with the source code for xv6, an approach inspired by John Lions’ Commentary on UNIX 6th Edition [11]; the text has hyperlinks to the source code at https://github.com/mit-pdos/xv6-riscv. See https://pdos.csail.mit.edu/6.1810 for additional pointers to on-line resources for v6 and xv6, including several lab assignments using xv6.

本文应结合 xv6 的源代码一起阅读，其灵感源自 John Li ons 的《UNIX 第六版评注》[11]；本文提供了源代码的超链接，网址为 https://github.com/mit-pdos/xv6-riscv。https://pdos.csail.mit.edu/6.1810 上提供了更多 v6 和 xv6 的在线资源，包括一些 xv6 的实验作业。

> We have used this text in 6.828 and 6.1810, the operating system classes at MIT. We thank the faculty, teaching assistants, and students of those classes who have all directly or indirectly contributed to xv6. In particular, we would like to thank Adam Belay, Austin Clements, and Nickolai Zeldovich. Finally, we would like to thank people who emailed us bugs in the text or suggestions for improvements: Abutalib Aghayev, Sebastian Boehm, brandb97, Anton Burtsev, Raphael Carvalho, Tej Chajed,Brendan Davidson, Rasit Eskicioglu, Color Fuzzy, Wojciech Gac, Giuseppe, Tao Guo, Haibo Hao, Naoki Hayama, Chris Henderson, Robert Hilderman, Eden Hochbaum, Wolfgang Keller, Paweł Kraszewski, Henry Laih, Jin Li, Austin Liew, lyazj@github.com, Pavan Maddamsetti, Jacek Masiulaniec, Michael McConville, m3hm00d, Mes0903, miguelgvieira, Mark Morrissey, Muhammed Mourad, Harry Pan, Harry Porter, Siyuan Qian, Zhefeng Qiao, Askar Safin, Salman Shah, Huang Sha, Vikram Shenoy, Adeodato Simó, Ruslan Savchenko, Pawel Szczurko, Warren Toomey, tyfkda, tzerbib, Vanush Vaswani, Chen Wang, Xi Wang, and Zou Chang Wei, Sam Whitlock, Qiongsi Wu, LucyShawYang, ykf1114@gmail.com, and Meng Zhou.

我们在麻省理工学院 (MIT) 的操作系统课程 6.828 和 6.1810 中使用了本书。我们感谢这些课程的教师、助教和学生，他们为 xv6 做出了直接或间接的贡献。特别感谢 Adam Belay、Austin Clements 和 Nickolai Zeldovich。最后，我们要感谢通过电子邮件向我们报告文本中的错误以及提出改进建议的人们：Abutalib Aghayev、Sebastian Boehm、brandb97、Anton Burtsev、Raphael Carvalho、Tej Chajed、Brendan Davidson、Rasit Eskicioglu、Color Fuzzy、Wojciech Gac、Giuseppe、Tao Guo、Haibo Hao、Naoki Hayama、Chris Henderson、Robert Hilderman、Eden Hochbaum、Wolfgang Keller、Paweł Kraszewski、Henry Laih、Jin Li、Austin Liew、lyazj@github.com、Pavan Maddamsetti、Jacek Masiulaniec、Michael McConville、

m3hm00d、Mes0903、miguelgvieira、Mark Morrissey、Muhammed Mourad、Harry Pan、Harry Porter、Siyuan Qian、Zhefeng Qiao、Askar Safin、Salman Shah、Huang Sha、Vikram Shenoy、 Adeodato Simó、Ruslan Savchenko、Pawel Szczurko、Warren Toomey、tyfkda、tzerbib、Vanush Vaswani、Chen Wang、Xi Wang 和 Zou Chang Wei、Sam Whitlock、Qiongsi Wu、LucyShawYang、ykf1114@gmail.com 和 Meng Zhou。

> If you spot errors or have suggestions for improvement, please send email to Frans Kaashoek and Robert Morris (kaashoek,rtm@csail.mit.edu).

如果您发现错误或有改进建议，请给 Frans Kaashoek 和 Robert Morris 发送电子邮件 (kaashoek,rtm@csail.mit.edu)。

第 1 章 操作系统接口（Chapter 1 Operating system interfaces）

> The job of an operating system is to share a computer among multiple programs and to provide a more useful set of services than the hardware alone supports. An operating system manages and abstracts the low-level hardware, so that, for example, a word processor need not concern itself with which type of disk hardware is being used. An operating system shares the hardware among multiple programs so that they run (or appear to run) at the same time. Finally, operating systems provide controlled ways for programs to interact, so that they can share data or work together.

操作系统的任务是让多个程序共享一台计算机，并（通过封装等工作）提供更多有用的服务，而不是让程序直接访问和操作硬件。操作系统管理底层硬件并面向应用抽象这些硬件，这样一些上层应用（譬如文字处理器）就无需关心使用哪种具体类型的磁盘设备。此外，多个程序通过一个操作系统共享硬件，使得它们可以(或者至少看起来可以)同时运行。最后，操作系统为程序提供了可控的交互方式，这样它们就可以共享数据或者协同工作。

> An operating system provides services to user programs through an interface. Designing a good interface turns out to be difficult. On the one hand, we would like the interface to be simple and narrow because that makes it easier to get the implementation right. On the other hand, we may be tempted to offer many sophisticated features to applications. The trick in resolving this tension is to design interfaces that rely on a few mechanisms that can be combined to provide much generality.

操作系统通过一套 “接口（interfaces）” （译者注：一个接口可以理解成一个函数调用，譬如典型的文件操作 `open`， `read` 和 `write`）向应用程序提供服务。设计一个良好的接口是很困难的事情。一方面，我们希望一个接口是简单的并且功能单一，因为这样更利于正确地实现。另一方面，我们可能倾向于为应用程序提供许多复杂的功能特性。解决这个矛盾的诀窍在于设计接口时，基于一些良好的机制，保证单个接口简单可靠，同时将它们组合起来又可以实现复杂的功能，从而提供良好的通用性。

> This book uses a single operating system as a concrete example to illustrate operating system concepts. That operating system, xv6, provides the basic interfaces introduced by Ken Thompson and Dennis Ritchie’ s Unix operating system [17], as well as mimicking Unix’ s internal design. Unix provides a narrow interface whose mechanisms combine well, offering a surprising degree of generality. This interface has been so successful that modern operating systems—BSD, Linux, macOS, Solaris, and even, to a lesser extent, Microsoft Windows—have Unix-like interfaces. Understanding xv6 is a good start toward understanding any of these systems and many others.

本书以一个操作系统作为具体的例子来阐述操作系统的概念。这个操作系统就是 xv6，它提供了 Ken Thompson 和 Dennis Ritchie 在开发 Unix 时引入的一套基本的接口 [17]，同时模仿了 Unix 的内部设计。Unix 提供了一套很精简的接口，通过组合使用这套接口，可以实现令人惊讶的通用性。这个接口非常成功，现代的操作系统，譬如 BSD、Linux、Mac OSX、Solaris，甚至在一定程度上，连

Microsoft 的 Windows 都提供了这些 “和 Unix 相似的接口（Unix-like interfaces）”。学好 xv6 能为理解这些系统和其他更多的系统打下良好的基础。

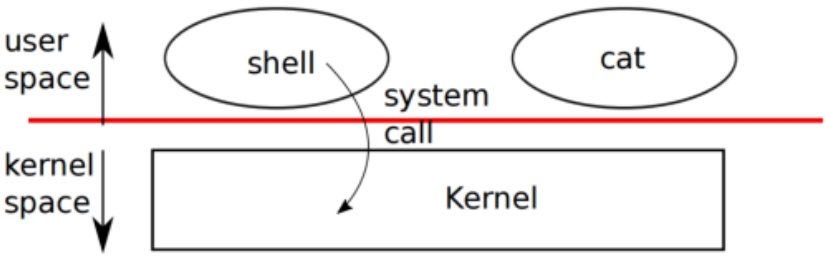

Figure 1.1: A kernel and two user processes.

> As Figure 1.1 shows, xv6 takes the traditional form of a kernel, a special program that provides services to running programs. Each running program, called a process, has memory containing instructions, data, and a stack. The instructions implement the program’ s computation. The data are the variables on which the computation acts. The stack organizes the program’ s procedure calls. A given computer typically has many processes but only a single kernel.

如图 1.1 所示，xv6 实现了一个传统意义上的 “内核（kernel）”，内核可以看成是一个特殊的 “程序（program）”，它为其他正在运行的程序提供服务。每个正在运行的程序，称为 “进程（process）”，都拥有一块自己的内存，进程的 “指令（instructions）”、“数据（data）” 和 “栈（stack）” 都存放在这块内存中。程序的计算功能通过指令实现，数据是计算操作的变量，程序的 “过程调用（procedure calls）” （译者注，即我们常说的函数调用）通过栈进行组织。一台给定的计算机上通常运行有多个进程，但同时只会运行一个内核。

> When a process needs to invoke a kernel service, it invokes a system call, one of the calls in the operating system’ s interface. The system call enters the kernel; the kernel performs the service and returns. Thus a process alternates between executing in user space and kernel space.

当一个进程需要请求一项内核服务时，它会调用一个 “系统调用（system call）”，每个系统调用就是操作系统所提供的一套接口中的一个实例。内核执行系统调用，完成服务并返回。因此，我们可以认为一个进程在 “用户空间（user space）” 和 “内核空间（kernel space）” 之间交替运行。

> As described in detail in subsequent chapters, the kernel uses the hardware protection mechanisms provided by a CPU (This text generally refers to the hardware element that executes a computation with the term CPU, an acronym) to ensure that each process executing in user space can access only its own memory. The kernel executes with the hardware privileges required to implement these protections; user programs execute without those privileges. When a user program invokes a system call, the hardware raises the privilege level and starts executing a pre-arranged function in the kernel.

正如在接下里的章节中所介绍的那样，内核使用 CPU (本文通常用 “CPU” 来指代执行计算的硬件元素，这是一个术语) 提供的硬件保护机制来确保每个在用户空间执行的进程只能访问它自己的内存。内核运行时拥有访问硬件的权限，这实现了对硬件的保护；而用户程序执行时不具备这些权限。当用户程序调用系统调用时，硬件会提升权限级别，并开始执行内核中预先安排好的函数。

> The collection of system calls that a kernel provides is the interface that user programs see. The xv6 kernel provides a subset of the services and system calls that Unix kernels traditionally offer.

内核提供的系统调用集合正是用户程序看到的接口。xv6 作为一个内核实现了传统 Unix 内核提供的服务和系统调用的子集。

> The rest of this chapter outlines xv6’ s services—processes, memory, file descriptors, pipes, and a file system—and illustrates them with code snippets and discussions of how the shell, Unix’ s command-line user interface, uses them. The shell’ s use of system calls illustrates how carefully they have been designed.

本章的其余部分概述了 xv6 中有关进程、内存、文件描述符、管道和文件系统的服务，并给出了相关的代码片段，同时还讨论了 shell（Unix 的命令行用户界面）以及如何使用它们。通过学习 shell 是如何使用这些系统调用，可以帮助我们理解这些接口是如何被精心设计的。

> The shell is an ordinary program that reads commands from the user and executes them. The fact that the shell is a user program, and not part of the kernel, illustrates the power of the system call interface: there is nothing special about the shell. It also means that the shell is easy to replace; as a result, modern Unix systems have a variety of shells to choose from, each with its own user interface and scripting features. The xv6 shell is a simple implementation of the essence of the Unix Bourne shell.

shell 是一个普通的程序，它从用户那里读取命令并执行它们。shell 是一个用户态程序，而不是内核的一部分，这一事实说明了系统调用接口的强大之处：shell 没有什么特别之处。这也意味着 shell 很容易被替换；因此，现代 Unix 系统有多种 shell 可供选择，每种 shell 的操作界面和对脚本的支持都各有各的特色。xv6 的 shell 是 Unix Bourne shell 的简单实现。

> The implementation of the xv6 shell can be found at (7850). The link is a hyperlink to the relevant xv6 source code at https://github.com/mit-pdos/xv6-riscv/ and the specific number refers to the sheet and line number in xv6-src-booklet.pdf, as in the Lions’ Commentary on UNIX 6th Edition [11]. A good practice is to try to read the source code first on your own (in your favorite development environment, on github, or in a PDF viewer) and then come back to this book. By the end of this book you should be able to understand every line of xv6 source code without having to consult this book.

xv6 的 shell 的实现可以在 (7850) 找到。该链接是指向相关 xv6 源代码的超链接，地址是 https://github.com/mit-pdos/xv6-riscv/，7850 这个编号指的是 `xv6-src-booklet.pdf` 中的工作表和行号，类似于 Lions 的《UNIX 第六版评注》[11]。一个好的做法是先自己尝试阅读源代码（在你最喜欢的开发环境中、在 GitHub 上或使用 PDF 阅读器），然后再回过头来阅读本书。读完本书后，你应该能够理解 xv6 源代码的每一行，而无需查阅本书。（译者注：原文 PDF 的行号，譬如 “7850” 是一个超链接，但译文这里暂不支持，仅列出行号，读者可以在 `xv6-src-booklet.pdf` 中搜索该行号值找到对应代码行。）

1.1 进程和内存（Processes and memory）

> An xv6 process consists of user-space memory (instructions, data, and stack) and per-process state private to the kernel. Xv6 time-shares processes: it transparently switches the available CPUs among the set of processes waiting to execute. When a process is not executing, xv6 saves the process's CPU registers, restoring them when it next runs the process. The kernel associates a process identifier, or `PID`, with each process.

一个 xv6 进程由用户空间内存（用于存放指令、数据和实现栈）以及内核私有的每个进程的状态组成。xv6 对进程实现 分时共享（time-shares），也就是说，它透明地在等待执行的多个进程之间切换 CPU。当某个进程不再使用 CPU 时，xv6 会保存该进程的 CPU 寄存器值，并在下次运行该进程时恢复它们。内核为每个进程指定一个 “进程标识符（process identifier）”，即 `PID`。

> A process may create a new process using the `fork` system call. `fork` gives the new process an exact copy of the calling process's memory: `fork` copies the instructions, data, and stack of the calling process into the new process's memory. `fork` returns in both the original and new processes. In the original process, `fork` returns the new process's PID. In the new process, `fork` returns zero. The original and new processes are often called the parent and child.

一个进程可以调用系统调用 `fork` 创建一个新的进程。`fork` 会将原调用进程的内存中的内容完整地复制一份到新建进程的内存中：复制的内容包括调用进程的指令、数据和栈。`fork` 在原进程和新进程中都会返回。在原进程中，`fork` 返回新进程的 PID。在新进程中，`fork` 返回零。原进程和新进程通常被称为 父进程（parent） 和 子进程（child）。

System call	Description
int fork()	Create a process, return child's PID.
int exit(int status)	Terminate the current process; status reported to wait(). No return.
int wait(int *status)	Wait for a child to exit; exit status in *status; returns child PID.
int kill(int pid)	Terminate process PID. Returns 0, or -1 for error.
int getpid()	Return the current process's PID.
int pause(int n)	Pause for n clock ticks.
int exec(char *file, char *argv[])	Load a file and execute it with arguments; only returns if error.
char *sbrk(int n)	Grow process's memory by n zero bytes. Returns start of new memory.
int open(char *file, int flags)	Open a file; flags indicate read/write; returns an fd (file descriptor).
int write(int fd, char *buf, int n)	Write n bytes from buf to file descriptor fd; returns n.
int read(int fd, char *buf, int n)	Read n bytes into buf; returns number read; or 0 if end of file.
int close(int fd)	Release open file fd.
int dup(int fd)	Return a new file descriptor referring to the same file as fd.
int pipe(int p[])	Create a pipe, put read/write file descriptors in p[0] and p[1].
int chdir(char *dir)	Change the current directory.
int mkdir(char *dir)	Create a new directory.
int mknod(char *file, int, int)	Create a device file.
int fstat(int fd, struct stat *st)	Place info about an open file into *st.
int link(char *file1, char *file2)	Create another name (file2) for the file file1.
int unlink(char *file)	Remove a file.

Figure 1.2: Xv6 system calls. If not otherwise stated, these calls return 0 for no error, and -1 if there's an error.

> For example, consider the following program fragment written in the C programming language [7]:

例如，参考下面用 C 语言编写的程序片段 [7]：

```c
int pid = fork();
if(pid > 0){
  printf("parent: child=%d\n", pid);
  pid = wait((int *) 0);
  printf("child %d is done\n", pid);
} else if(pid == 0){
  printf("child: exiting\n");
  exit(0);
} else {
  printf("fork error\n");
}
```

> The `exit` system call causes the calling process to stop executing and to release resources such as memory and open files. Exit takes an integer status argument, conventionally 0 to indicate success and 1 to indicate failure. The `wait` system call returns the PID of an exited (or killed) child of the current process and copies the exit status of the child to the address passed to wait; if none of the caller's children has exited, `wait` waits for one to do so. If the caller has no children, `wait` immediately returns -1. If the parent doesn't care about the exit status of a child, it can pass a 0 address to `wait`.

`exit` 系统调用会导致调用进程停止执行并释放资源，例如内存和打开的文件。`exit` 接受一个整数类型的参数，通常 0 表示成功，1 表示失败。`wait` 系统调用返回当前进程的已退出（或被杀死）的子进程的 PID，并将子进程的退出状态值复制到传递给 `wait` 的地址；如果调用者的子进程均未退出，`wait` 会等待，直到其中任一个子进程退出。如果调用者没有子进程，`wait` 会立即返回 `-1`。如果父进程不关心子进程的退出状态，则可以将空地址（值为 0）传递给 `wait`。

> In the example, the output lines

上面这个例子中，假设输出如下内容：

```
parent: child=1234
child: exiting
```

> might come out in either order (or even intermixed), depending on whether the parent or child gets to its `printf` call first. After the child exits, the parent's `wait` returns, causing the parent to print

以上打印可能会以任意顺序（甚至混杂方式）输出，具体取决于父进程还是子进程先执行 `printf` 调用。子进程退出后，父进程的 `wait` 返回，导致父进程打印:

```
parent: child 1234 is done
```

> Although the child starts with a copy of the parent's memory, the parent and child execute with separate memory and separate registers: changing a variable in one does not affect the other. For example, when the return value of `wait` is stored into `pid` in the parent process, it doesn't change the variable `pid` in the child. The value of `pid` in the child will still be zero.

尽管子进程开始运行时拥有一个父进程内存的副本，但父进程和子进程在不同的内存和寄存器中执行：更改其中一个进程的变量不会影响另一个进程。例如，当 `wait` 的返回值存储到父进程的 `pid` 中时，它不会更改子进程中的变量 `pid`。子进程中的 `pid` 值仍然为零。

> The `exec` system call replaces the calling process's memory with a new memory image loaded from a file stored in the file system. The file must have a particular format, which specifies which part of the file holds instructions, which part is data, at which instruction to start, etc. Xv6 uses the ELF format, which Chapter 3 discusses in more detail. Usually the file is the result of compiling a program's source code. When `exec` succeeds, it does not return to the calling program; instead, the instructions loaded from the file start executing at the entry point declared in the ELF header. `exec` takes two arguments: the name of the file containing the executable and an array of string arguments. For example:

`exec` 系统调用会替换调用进程的内存内容，具体来说，它将存储在文件系统中的文件中的内容加载到内存中，替换了原有的内容。该文件必须具有特定的格式，该格式指定了文件的哪部分包含指令、哪部分包含数据、从哪条指令开始执行等等。xv6 使用 ELF 格式，第 3 章将对此进行更详细的讨论。通常，该文件是编译程序源代码的输出结果。当 `exec` 成功执行时，调用它的程序不会返回；而是加载文件中的指令，并从文件的 ELF 头中声明的入口点开始执行加载后的指令。`exec` 接受两个参数：一个是可执行文件的文件名，另一个是字符串数组。例如：

```
char *argv[3];

argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
exec("/bin/echo", argv);
printf("exec error\n");
```

> This fragment replaces the calling program with an instance of the program `/bin/echo` running with the argument list `echo hello`. Most programs ignore the first element of the argument array, which is conventionally the name of the program.

此代码片段将调用程序替换为程序 `/bin/echo` 的实例，该实例运行时使用参数列表 `echo hello`。大多数程序会忽略参数数组的第一个元素，通常情况下，该元素是程序的名称。

> The xv6 shell uses the above calls to run programs on behalf of users. The main structure of the shell is simple; see `main` (8001). The main loop reads a line of input from the user with `getcmd`. Then it calls `fork`, which creates a copy of the shell process. The parent calls `wait`, while the child runs the command. For example, if the user had typed "`echo hello`" to the shell, `runcmd` would have been called with "`echo hello`" as the argument. `runcmd` (7903) runs the actual command. For "`echo hello`", it would call `exec` (7927). If `exec` succeeds then the child will execute instructions from `echo` instead of `runcmd`. At some point `echo` will call `exit`, which will cause the parent to return from `wait` in `main` (8001).

xv6 的 shell 使用上述调用来代表用户运行程序。shell 的主要结构很简单；参见 `main` (8001)。主循环使用 `getcmd` 从用户那里读取一行输入。然后它调用 `fork`，创建 shell 进程的副本。父进程调用 `wait`，而子进程运行命令。例如，如果用户在 shell 中输入了“echo hello”，则会以 “`echo`

`hello` ” 作为参数调用 `runcmd`。`runcmd` (7903) 运行实际命令。对于 “`echo hello`”，它将调用 `exec` (7927)。如果 `exec` 成功，则子进程将执行来自 `echo` 而不是 `runcmd` 的指令。在某些时候，`echo` 将调用 `exit`，这将导致父进程从 `main` （8001）中的 `wait` 返回。

> You might wonder why `fork` and `exec` are not combined in a single call; we will see later that the shell exploits the separation in its implementation of I/O redirection. To avoid the wastefulness of creating a duplicate process and then immediately replacing it (with `exec`), operating kernels optimize the implementation of `fork` for this use case by using virtual memory techniques such as copy-on-write (see Section 5).

你可能想知道为什么 `fork` 和 `exec` 不合并实现为一个系统调用；稍后我们将看到，shell 在实现 “输入/输出重定向（I/O redirection）” 中利用了这种分离的设计。创建并复制进程内存后立即调用 `exec` 会导致进程内存被替换，这在操作上是一种浪费，为了避免这种浪费，操作系统内核针对这种情况会通过使用虚拟内存技术优化 `fork` 的实现，例如 “写时复制（copy-on-write）”（具体参见第 5 章）。

> Xv6 allocates most user-space memory implicitly: `fork` allocates the memory required for the child’s copy of the parent’s memory, and `exec` allocates enough memory to hold the executable file. A process that needs more memory at run-time (perhaps for `malloc`) can call `sbrk(n)` to grow its data memory by `n` zero bytes; `sbrk` returns the location of the new memory.

xv6 会隐式分配大部分用户空间内存：`fork` 会在子进程复制父进程内存时分配所需的内存，`exec` 则会分配足够的内存来保存可执行文件。如果进程在运行时需要更多内存（例如用于 `malloc`），可以调用 `sbrk(n)` 来增加 `n` 个字节（数据初始化为零）；`sbrk` 会返回新分配的内存的位置。

1.2 输入/输出和文件描述符（I/O and File descriptors）

> A file descriptor is a small integer representing a kernel-managed object that a process may read from or write to. A process may obtain a file descriptor by opening a file, directory, or device, or by creating a pipe, or by duplicating an existing descriptor. For simplicity we’ll often refer to the object a file descriptor refers to as a “file”; the file descriptor interface abstracts away the differences between files, pipes, and devices, making them all look like streams of bytes. We’ll refer to input and output as I/O.

文件描述符（file descriptor） 是一个小整数，用来代表一个由内核管理的对象，进程可以对其执行读取或写入操作。当进程打开文件、目录或设备，或者创建管道时内核会返回一个对应的文件描述符，进程也可以通过复制现有描述符来获取一个文件描述符。为了简单起见，我们通常将文件描述符所引用的对象称为 “文件（file）”；文件描述符通过抽象隐藏了文件、管道和设备之间的差异，使它们看起来都像 “字节流（streams of bytes）”。我们将 “输入（Input）” 和 “输出（Output）” 简称为 I/O。

> Internally, the xv6 kernel uses the file descriptor as an index into a per-process table, so that every process has a private space of file descriptors starting at zero. By convention, a process reads from file descriptor 0 (standard input), writes output to file descriptor 1 (standard output), and writes error messages to file descriptor 2 (standard error). As we will see, the shell exploits the convention to implement I/O redirection and pipelines. The shell

> ensures that it always has three file descriptors open (8007), which are by default file descriptors for the console.

在内部实现中，xv6 内核使用文件描述符作为一个表的索引（这个表每个进程都有一个），因此每个进程都有一个私有的从零开始的文件描述符空间。按照惯例，进程从文件描述符 0（“标准输入（standard input）”）读取输入的数据，将输出写入文件描述符 1（“标准输出（standard output）”），并将错误信息写入文件描述符 2（“标准出错（standard error）”）。正如我们将看到的，shell 利用这个惯例来实现 I/O 重定向和管道。shell 确保始终打开了这三个文件描述符（8007），默认情况下，这些文件描述符对应 “控制台（console）”。

> The `read` and `write` system calls read bytes from and write bytes to open files named by file descriptors. The call `read(fd, buf, n)` reads at most `n` bytes from the file descriptor `fd`, copies them into `buf`, and returns the number of bytes read. Each file descriptor that refers to a file has an offset associated with it. `read` reads data from the current file offset and then advances that offset by the number of bytes read: a subsequent `read` will return the bytes following the ones returned by the first read. When there are no more bytes to read, `read` returns zero to indicate the end of the file.

`read` 和 `write` 这两个系统调用分别从通过文件描述符参数指定的已打开的文件中读取和写入字节。`read(fd, buf, n)` 调用从文件描述符 `fd` 读取最多 `n` 个字节，将它们复制到 `buf` 中，并返回读取的字节数。每个引用文件的文件描述符都有一个与其关联的偏移量。`read` 从当前文件偏移量读取数据，然后将该偏移量向文件末尾移动读取的字节数：下一次 `read` 将返回上一次读取返回的字节数之后的字节。当没有更多字节可读取时，`read` 返回零以指示遇到了文件结尾。

> The call `write(fd, buf, n)` writes `n` bytes from `buf` to the file descriptor `fd` and returns the number of bytes written. Fewer than `n` bytes are written only when an error occurs. Like `read`, `write` writes data at the current file offset and then advances that offset by the number of bytes written: each `write` picks up where the previous one left off.

调用 `write(fd, buf, n)` 会将 `buf` 中的 `n` 个字节写入文件描述符 `fd`，并返回写入的字节数。如果写入少于 `n` 个字节说明发生了错误。与 `read` 类似，`write` 会从当前文件偏移量位置往后写入数据，然后将该偏移量增加写入的字节数：每次 `write` 都会从上一次停止的地方继续。

> The following program fragment (which forms the essence of the program `cat`) copies data from its standard input to its standard output. If an error occurs, it writes a message to the standard error.

以下程序片段（构成了程序 `cat` 的核心逻辑）将数据从其标准输入复制到其标准输出。如果发生错误，它会将一条消息写入标准出错设备。

```
char buf[512];
int n;

for(;;){
  n = read(0, buf, sizeof buf);
  if(n == 0)
    break;
  if(n < 0){
    fprintf(2, "read error\n");
    exit(1);
```

```
  }
  if(write(1, buf, n) != n){
    fprintf(2, "write error\n");
    exit(1);
  }
}
```

> The important thing to note in the code fragment is that `cat` doesn't know whether it is reading from a file, console, or a pipe. Similarly `cat` doesn't know whether it is printing to a console, a file, or whatever. The use of file descriptors and the convention that file descriptor 0 is input and file descriptor 1 is output allows a simple implementation of `cat`.

需要注意的是，这段代码片段中，`cat` 并不知道它是从文件、控制台还是管道读取数据。同样，`cat` 也不知道它是打印到控制台、文件还是其他什么地方。使用文件描述符以及将文件描述符 0 定义为输入、文件描述符 1 定义为输出的约定，可以简化 `cat` 的实现。

> The `close` system call releases a file descriptor, making it free for reuse by a future `open`, `pipe`, or `dup` system call (see below). A newly allocated file descriptor is always the lowest-numbered unused descriptor of the current process.

`close` 系统调用会释放文件描述符，使其可供将来的 `open`、`pipe` 或 `dup` 系统调用（见下文）重用。新分配的文件描述符始终是当前进程中编号最小的未使用的描述符。

> File descriptors and `fork` interact to make I/O redirection easy to implement. `fork` copies the parent's file descriptor table along with its memory, so that the child starts with exactly the same open files as the parent. The system call `exec` replaces the calling process's memory but preserves its file table. This behavior allows the shell to implement I/O redirection by forking, closing and re-opening chosen file descriptors in the child, and then calling `exec` to run the new program. Here is a simplified version of the code a shell runs for the command `cat < input.txt`:

利用文件描述符和 `fork` 可以简化 I/O 重定向的实现。`fork` 会复制父进程的文件描述符表及其内存，这样子进程从一开始就打开了与父进程完全相同的文件。系统调用 `exec` 会替换调用它的进程的内存，但保留了其原先的文件表。此行为允许 shell 在执行完 `fork` 后在子进程中先执行关闭再重新打开指定的文件描述符，然后调用 `exec` 运行新程序来实现 I/O 重定向 (redirection)。以下是对应在 Shell 中运行命令 `cat < input.txt` 的代码的简化版本：

```
char *argv[2];

argv[0] = "cat";
argv[1] = 0;
if(fork() == 0) {
  close(0);
  open("input.txt", O_RDONLY);
  exec("cat", argv);
}
```

> After the child closes file descriptor 0, `open` is guaranteed to use that file descriptor for the newly opened `input.txt`: 0 will be the smallest available file descriptor. `cat` then executes with file descriptor 0 (standard input) referring to `input.txt`. The parent

> process’ s file descriptors are not changed by this sequence, since it modifies only the child’ s descriptors.

子进程关闭文件描述符 0 后，因为 0 将是最小的可用文件描述符，所以内核会保证 `open` 重新打开 `input.txt` 时得到的文件描述符还是 0。随后调用 `exec` 执行 `cat` 时所使用的文件描述符 0（标准输入）将对应 `input.txt`。这些代码不会影响父进程打开的文件描述符，因为它只修改了子进程的描述符。

> The code for I/O redirection in the xv6 shell works in exactly this way (7931). Recall that at this point in the code the shell has already forked the child shell and that `runcmd` will call `exec` to load the new program.

xv6 中 shell 的 I/O 重定向代码正是如此工作的 (7931)。回想一下，此时 shell 已经 fork 了子 shell，并且 `runcmd` 将调用 `exec` 来加载新程序。

> The second argument to `open` consists of a set of flags, expressed as bits, that control what `open` does. The possible values are defined in the file control (fcntl) header (4000-4004): `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREATE`, and `O_TRUNC`, which instruct `open` to open the file for reading, or for writing, or for both reading and writing, to create the file if it doesn’ t exist, and to truncate the file to zero length.

`open` 的第二个参数由一组以比特位表示的标志组成，用于控制 `open` 的行为。可用的值在 “文件控制 (file control，简称 fcntl)” 头文件中定义（4000-4004），这些选项对应指示 `open` 打开文件进行读取（`O_RDONLY`）、写入（`O_WRONLY`）或可读可写（`O_RDWR`）、如果文件不存在则创建文件（`O_CREATE`），以及将文件长度截断为零（`O_TRUNC`）。

> Now it should be clear why it is helpful that `fork` and `exec` are separate calls: between the two, the shell has a chance to redirect the child’ s I/O without disturbing the I/O setup of the main shell. One could instead imagine a hypothetical combined `forkexec` system call, but the options for doing I/O redirection with such a call seem awkward. The shell could modify its own I/O setup before calling `forkexec` (and then un-do those modifications); or `forkexec` could take instructions for I/O redirection as arguments; or (least attractively) every program like `cat` could be taught to do its own I/O redirection.

介绍到这里读者应该清楚为什么我们要将 `fork` 和 `exec` 分开，因为在这两个调用之间，shell 有机会在不干扰父进程 的 I/O 设置的情况下重定向子 shell 的 I/O。也可以设想假设存在一个将 `fork` 和 `exec` 组合在一起的 `forkexec` 系统调用，但使用这样的调用实现 I/O 重定向似乎有点麻烦。（为了实现 I/O 重定向的效果）shell 可以在调用 `forkexec` 之前修改自身的 I/O 设置（在调用 `forkexec` 后再撤消这些修改）；或者 `forkexec` 可以将是否需要 I/O 重定向指令作为参数传入；或者（最不推荐的实现方案）让每个像 `cat` 这样的程序根据命令行选项来决定是否由自己来实现 I/O 重定向。

> Although `fork` copies the file descriptor table, each underlying file offset is shared between parent and child. Consider this example:

虽然 `fork` 会复制文件描述符表，但这些文件描述符对应的文件实际只有一份，这个文件的偏移量在父进程和子进程之间共享。请考虑以下示例：

```
if(fork() == 0) {
  write(1, "hello ", 6);
  exit(0);
} else {
  wait(0);
  write(1, "world\n", 6);
}
```

> At the end of this fragment, the file attached to file descriptor 1 will contain the data `hello world`. The `write` in the parent (which, thanks to `wait`, runs only after the child is done) picks up where the child's `write` left off. This behavior helps produce sequential output from sequences of shell commands, like `(echo hello; echo world) >output.txt`.

在此代码片段的最后，文件描述符 1 对应的文件将包含数据 `hello world`。父进程中的 `write` 操作（由于 `wait` 的作用，仅会在子进程完成后才能运行）将从子进程 `write` 操作停止的地方继续执行。此行为有助于确保一系列 shell 命令（例如 `(echo hello; echo world) >output.txt`）按顺序打印输出。

> The `dup` system call duplicates an existing file descriptor, returning a new one that refers to the same underlying I/O object. Both file descriptors share an offset, just as the file descriptors duplicated by `fork` do. This is another way to write `hello world` into a file:

`dup` 系统调用复制一个现有的文件描述符，并返回一个指向同一底层 I/O 对象的新文件描述符。两个文件描述符共享同一个偏移量，就像 `fork` 复制的文件描述符一样。以下是将 `hello world` 写入同一个文件的另一种方法：

```
fd = dup(1);
write(1, "hello ", 6);
write(fd, "world\n", 6);
```

> Two file descriptors share an offset if they were derived from the same original file descriptor by a sequence of `fork` and `dup` calls. Otherwise file descriptors do not share offsets, even if they resulted from `open` calls for the same file. `dup` allows shells to implement commands like this: `ls existing-file non-existing-file > tmp1 2>&1`. The `2>&1` tells the shell to give the command a file descriptor 2 that is a duplicate of descriptor 1. Both the name of the existing file and the error message for the non-existing file will show up in the file `tmp1`. The xv6 shell doesn't support I/O redirection for the error file descriptor, but now you know how to implement it.

如果两个文件描述符是通过一系列 `fork` 和 `dup` 调用从同一个原始文件描述符派生而来的，则它们共享同一个偏移量。否则，文件描述符不共享偏移量，即使它们是由对同一个文件的 `open` 调用生成的。`dup` 允许 shell 实现如下命令：`ls existing-file non-existing-file > tmp1 2>&1`。`2>&1` 告诉 shell 为该命令提供一个复制自描述符 1 的文件描述符 2。`ls` 对 `existing-file` 的输出结果和对 `non-existing-file` 的错误提示都将显示在文件 `tmp1` 中。xv6 的 shell 不支持对标准出错文件描述符的 I/O 重定向，但现在你应该知道如何实现它。

> File descriptors are a powerful abstraction, because they hide the details of what they are connected to: a process writing to file descriptor 1 may be writing to a file, to a device like the console, or to a pipe.

文件描述符是一种很高级的抽象，因为该接口隐藏了它们实际连接对象的细节：对文件描述符 1 执行写入操作的进程可能对应着写入文件、或者写入控制台等设备，也可能对应着写入管道。

1.3 管道（Pipes）

> A pipe is a small kernel buffer exposed to processes as a pair of file descriptors, one for reading and one for writing. Writing data to one end of the pipe makes that data available for reading from the other end of the pipe. Pipes provide a way for processes to communicate.

管道(pipe) 是一个小型内核缓冲区，进程通过打开一对文件描述符来访问它，一个文件描述符用于读取，一个文件描述符用于写入。将数据从管道的一端写入后，可以从管道的另一端将数据读取出来。管道提供了一种进程间通讯的方式。

> The following example code runs the program `wc` with standard input connected to the read end of a pipe.

下面的示例代码运行程序 `wc`，并将标准输入连接到管道的读取端。

```
int p[2];
char *argv[2];

argv[0] = "wc";
argv[1] = 0;

pipe(p);
if(fork() == 0) {
  close(0);
  dup(p[0]);
  close(p[0]);
  close(p[1]);
  exec("/bin/wc", argv);
} else {
  close(p[0]);
  write(p[1], "hello world\n", 12);
  close(p[1]);
}
```

> The program calls `pipe`, which creates a new pipe and records the read and write file descriptors in the array `p`. After `fork`, both parent and child have file descriptors referring to the pipe. The child calls `close` and `dup` to make file descriptor zero refer to the read end of the pipe, closes the file descriptors in `p`, and calls `exec` to run `wc`. When `wc` reads from its standard input, it reads from the pipe. The parent closes the read side of the pipe, writes to the pipe, and then closes the write side.

程序首先调用 `pipe` 函数，创建一个新的管道，传入的参数 `p` 是一个整型数组，用于记录管道两端的读写文件描述符（译者注，具体来说，当 `pipe` 成功返回时，`p[0]` 指向管道的读取端，`p[1]`

指向管道的写入端）。`fork` 之后，父进程和子进程都拥有指向该管道的文件描述符。子进程调用 `close` 和 `dup` 函数，使文件描述符 0 指向管道的读取端，然后关闭文件描述符 `p[1]`，并调用 `exec` 函数运行 `wc` 程序。当 `wc` 从其标准输入读取时，它实际上也就是从管道读取数据。父进程关闭管道的读取端，向管道写入数据，然后关闭写入端。

> If no data is available, a `read` on a pipe waits for either data to be written or for all file descriptors referring to the write end to be closed; in the latter case, `read` will return 0, just as if the end of a data file had been reached. The fact that `read` blocks until it is impossible for new data to arrive is one reason that it's important for the child to close the write end of the pipe before executing `wc` above: if one of `wc`'s file descriptors referred to the write end of the pipe, `wc` would never see end-of-file.

如果管道中没有数据，读端的 `read` 会等待数据写入，或者等待所有指向写入端的文件描述符关闭；在后一种情况下，`read` 将返回 0，就像读到了数据文件的末尾。`read` 会阻塞直到不可能有新数据到达，这也是为什么子进程在执行上述 `wc` 之前需要关闭管道写入端的非常重要的原因之一：如果 `wc` 的某个文件描述符指向管道的写入端，`wc` 将一直等待并且无法结束。

> The xv6 shell implements pipelines such as `grep fork sh.c | wc -l` in a manner similar to the above code (7950). The child process creates a pipe to connect the left end of the pipeline with the right end. Then it calls `fork` and `runcmd` for the left end of the pipeline and `fork` and `runcmd` for the right end, and waits for both to finish. The right end of the pipeline may be a command that itself includes a pipe (e.g., `a | b | c`), which itself forks two new child processes (one for `b` and one for `c`). Thus, the shell may create a tree of processes. The leaves of this tree are commands and the interior nodes are processes that wait until the left and right children complete.

xv6 中的 shell 对管道（例如 `grep fork sh.c | wc -l`）的实现方式（7950）与上述代码类似。子进程创建一个管道，连接管道的两边（译者注：管道的两边是指上面命令行例子中 “|” 的两边的子命令）。然后，它为管道的两边均调用 `fork` 和 `runcmd`，并等待两者完成。管道的右边可以继续嵌套一个包含管道的命令（例如 `a | b | c`），如果是这样，则该命令自身又会 `fork` 两个新的子进程（一个用于 `b`，一个用于 `c`）。因此，shell 可以构建一个进程树。这棵树的叶子对应的是命令（译者注：每个命令对应创建一个进程），内部节点是等待左右子进程完成的进程。

> Pipes may seem no more powerful than temporary files: the pipeline

管道似乎并不比临时文件更强大，下面这个管道写法：

```
echo hello world | wc
```

> could be implemented without pipes as

可以不用管道来实现，如下所示。

```
echo hello world >/tmp/xyz; wc </tmp/xyz
```

> Pipes have at least three advantages over temporary files in this situation. First, pipes automatically clean themselves up; with the file redirection, a shell would have to be careful

> to remove `/tmp/xyz` when done. Second, pipes can pass arbitrarily long streams of data, while file redirection requires enough free space on disk to store all the data. Third, pipes allow for parallel execution of pipeline stages, while the file approach requires the first program to finish before the second starts.

在这种情况下，管道至少比临时文件有三个优势。首先，管道会自动清理；而使用文件重定向时，shell 必须小心地在清理完成后删除 `/tmp/xyz`。其次，管道可以传递任意长度的数据流，而文件重定向需要磁盘上有足够的可用空间来存储所有数据。第三，管道允许并行执行各个用 “|” 分隔的部分，而文件方法要求第一个程序在第二个程序启动之前完成。

1.4 文件系统（File system）

> The xv6 file system provides data files, which contain uninterpreted byte arrays, and directories, which contain named references to data files and other directories. The directories form a tree, starting at a special directory called the root. A path like `/a/b/c` refers to the file or directory named `c` inside the directory named `b` inside the directory named `a` in the root directory `/`. Paths that don’t begin with `/` are evaluated relative to the calling process’s current directory, which can be changed with the `chdir` system call. Both these code fragments open the same file (assuming all the directories involved exist):

xv6 文件系统提供 “数据文件（data files）”（数据内容是原始的，未经解释的二进制字节数组）和 “目录（directory）”（包含对数据文件和其他目录的名字引用）。这些目录构成一棵树状结构，这棵树的根是一个我们称之为 根目录（root） 的特殊目录。形式为 `/a/b/c` 的字符串表达的是一个路径名，在这个路径中，`/` 为起始目录，其子目录名字叫 `a`，`a` 的子目录名字叫 `b`，而 `b` 目录中还有一个名字叫做 `c` 的子目录或者文件。不以 `/` 开头的路径的起始目录位置则是相对于调用进程的 当前目录（current directory），我们可以使用 `chdir` 系统调用更改当前目录。以下两个代码片段都可用于打开同一个文件（假设所有涉及的目录都存在）：

```
chdir("/a");
chdir("b");
open("c", O_RDONLY);

open("/a/b/c", O_RDONLY);
```

> The first fragment changes the process’s current directory to `/a/b`; the second neither refers to nor changes the process’s current directory.

第一个代码片段将进程的当前目录更改为 `/a/b`；第二个片段没有用到进程的当前目录（译者注：`/a/b/c` 的形式采用的是绝对路径的概念），也没有改变进程的当前目录。

> There are system calls to create new files and directories: `mkdir` creates a new directory, `open` with the `O_CREATE` flag creates a new data file, and `mknod` creates a new device file. This example illustrates all three:

有一些系统调用可以创建新文件和目录：`mkdir` 创建一个新目录，`open` 带上 `O_CREATE` 标志用于新建一个数据文件，`mknod` 用于创建一个新的设备文件。下面的示例演示了这三种调用：

```
mkdir("/dir");
fd = open("/dir/file", O_CREATE|O_WRONLY);
close(fd);
mknod("/console", 1, 1);
```

> `mknod` creates a special file that refers to a device. Associated with a device file are the major and minor device numbers (the two arguments to `mknod`), which uniquely identify a kernel device. When a process later opens a device file, the kernel diverts `read` and `write` system calls to the kernel device implementation instead of passing them to the file system.

`mknod` 会创建一个代表设备的特殊文件。与设备文件关联的是主设备号和次设备号（分别对应传入 `mknod` 的后面两个参数），它们唯一地标识了一个内核设备。当进程稍后打开设备文件时，内核会将 `read` 和 `write` 系统调用分发给内核中针对设备的实现（译者注：即设备驱动部分），而不是将它们传递给文件系统。

> A file’s name is distinct from the file itself; the same underlying file, called an `inode`, can have multiple names, called `links`. Each link consists of an entry in a directory; the entry contains a file name and a reference to an inode. An inode holds `metadata` about a file, including its type (file or directory or device), its length, the location of the file’s content on disk, and the number of links to a file.

一个文件的名字与文件本身不是同一个概念；同一个底层的文件（我们称之为 “索引节点（inode）”）可以有多个文件名（我们称之为 “链接（link）”）。每个链接对应目录中的一个 “目录项（entry）”；该目录项由文件名和对 inode 的引用组成。inode 保存了文件的 “元数据（metadata）”，元数据包括文件的类型（文件、目录或设备）、长度、磁盘上保存文件内容的位置以及指向该文件的链接数量。

> The `fstat` system call retrieves information from the inode that a file descriptor refers to. It fills in a `struct stat`, defined in `stat.h` (4050) as:

`fstat` 系统调用根据指定的文件描述符参数找到其对应的 inode，并返回 inode 中的信息。它将 inode 中的信息填充到类型为 `struct stat` 的变量中，该结构体在 `stat.h` (4050) 中定义如下：

```
#define T_DIR 1 // Directory
#define T_FILE 2 // File
#define T_DEVICE 3 // Device
struct stat {
  int dev; // File system’s disk device
  uint ino; // Inode number
  short type; // Type of file
  short nlink; // Number of links to file
  uint64 size; // Size of file in bytes
};
```

> The `link` system call creates another file system name referring to the same inode as an existing file. This fragment creates a new file named both `a` and `b`.

`link` 系统调用在文件系统中为一个现存的文件 inode 创建一个新的名称。下面的代码片段创建了一个文件，这个文件有两个名字，一个叫 `a`，另一个叫 `b`。

```
open("a", O_CREATE|O_WRONLY);
link("a", "b");
```

> Reading from or writing to `a` is the same as reading from or writing to `b`. Each inode is identified by a unique inode number. After the code sequence above, it is possible to determine that `a` and `b` refer to the same underlying contents by inspecting the result of `fstat`: both will return the same inode number (`ino`), and the `nlink` count will be set to 2.

对 `a` 的读取或写入等价于对 `b` 的读取或写入。每个 inode 都通过一个唯一的 索引节点编号（inode number） 来标识。执行上述代码片段后，可以通过检查 `fstat` 的结果来确定 `a` 和 `b` 指向相同的底层内容：两者都将返回相同的索引节点编号 (参考 `struct stat` 的成员 `ino`)，并且（结构体 `struct stat` 的成员） `nlink` 的值将被设置为 2。

> The `unlink` system call removes a name from the file system. The file's inode and the disk space holding its content are only freed when the file's link count is zero and no file descriptors refer to it. Thus adding

`unlink` 系统调用会从文件系统中删除一个名称。只有当一个文件的链接计数值为零且没有文件描述符引用该文件时，文件的 inode 和保存其内容的磁盘空间才会被释放。因此，添加下面这行代码：

```
unlink("a");
```

> to the last code sequence leaves the inode and file content accessible as `b`. Furthermore,

到上面代码片段的末尾将使我们只能用 `b` 来访问这个文件 inode。此外参考以下两行代码：

```
fd = open("/tmp/xyz", O_CREATE|O_RDWR);
unlink("/tmp/xyz");
```

> is an idiomatic way to create a temporary inode with no name that will be cleaned up when the process closes `fd` or exits.

这两行代码演示了我们创建没有名称的临时 inode 的习惯性方法，当进程关闭 `fd` 或退出时，该 inode 将被自动清理。

> Unix provides file utilities callable from the shell as user-level programs, for example `mkdir`, `ln`, and `rm`. This design allows anyone to extend the command-line interface by adding new user-level programs. In hindsight this plan seems obvious, but other systems designed at the time of Unix often built such commands into the shell (and built the shell into the kernel).

Unix 提供了一些可以在 shell 中执行的操作文件的实用程序，例如 `mkdir`、`ln` 和 `rm`，它们都是用户级程序。这种设计允许任何人通过添加新的用户级程序来扩展命令行接口。事后看来，这种设计似乎再自然不过了，但与 Unix 同时代设计的其他系统通常会将此类命令内置到 shell 中（甚至将 shell 内置到内核中）。

> One exception is `cd`, which is built into the shell (8021). `cd` must change the current working directory of the shell itself. If `cd` were run as a regular command, then the shell would fork a child process, the child process would run `cd`, and `cd` would change the child ’ s working directory. The parent’ s (i.e., the shell’ s) working directory would not change.

但有一个例外是 `cd` 命令，它是 shell 内置的（8021）。`cd` 必须更改 shell 本身的当前工作目录。如果 `cd` 作为常规命令运行，shell 会 fork 一个子进程，子进程会运行 `cd`，而 `cd` 只会更改子进程的当前工作目录。父进程（即 shell）的工作目录并不会被改变。

1.5 现实世界（Real world）

> Unix’ s combination of “standard” file descriptors, pipes, and convenient shell syntax for operations on them was a major advance in writing general-purpose reusable programs. The idea sparked a culture of “software tools” that was responsible for much of Unix’ s power and popularity, and the shell was the first so-called “scripting language.” The Unix system call interface persists today in systems like BSD, Linux, and macOS.

Unix 将 “标准（standard）” 文件描述符、管道以及便捷的 shell 语法结合在一起，为编写通用的可复用的程序带来了重大进步。这一理念催生了一种 “软件工具” 文化，这在很大程度上成就了 Unix 的强大功能和受欢迎程度，而 shell 则是第一个所谓的 “脚本语言”。Unix 系统调用接口至今仍被 BSD、Linux 和 macOS 等系统沿用。

> The Unix system call interface has been standardized through the Portable Operating System Interface (POSIX) standard. Xv6 is not POSIX compliant: it is missing many system calls (including basic ones such as `lseek`), and many of the system calls it does provide differ from the standard. Our main goals for xv6 are simplicity and clarity while providing a simple UNIX-like system-call interface. Several people have extended xv6 with a few more system calls and a simple C library in order to run basic Unix programs. Modern kernels, however, provide many more system calls, and many more kinds of kernel services, than xv6. For example, they support networking, windowing systems, user-level threads, drivers for many devices, and so on. Modern kernels evolve continuously and rapidly, and offer many features beyond POSIX.

Unix 系统调用接口已经通过 “可移植操作系统接口 (Portable Operating System Interface，简称 POSIX)” 标准实现了标准化。但 xv6 并 不 符合 POSIX 标准：它缺少许多系统调用（包括 `lseek` 等基本调用），并且它提供的许多系统调用与标准也不一致。xv6 的主要目标是简洁明了，同时提供一个简单的 “类似 UNIX（UNIX-like）” 的系统调用接口。有些人扩展了 xv6，为它添加了更多系统调用和一个简单的 C 库，以便运行基本的 Unix 程序。然而，现代内核提供的系统调用和内核服务种类远多于 xv6。例如，它们支持网络、窗口系统、用户级线程、多种设备的驱动程序等等。现代内核持续快速地发展，并提供了许多超越 POSIX 的功能。

> Unix unified access to multiple types of resources (files, directories, and devices) with a single set of file-name and file-descriptor interfaces. This idea can be extended to more kinds of resources; a good example is Plan 9 [16], which applied the “resources are files” concept to networks, graphics, and more. However, most Unix-derived operating systems have not followed this route.

Unix 使用了单一的一组文件名和文件描述符接口统一了对多种资源（包括文件、目录和设备）的访问。这一理念可以扩展到更多类型的资源；Plan 9 [16] 就是一个很好的例子，它将 “资源即文件（resources are files）” 的概念应用于网络、图形等领域。然而，大多数从 Unix 衍生的操作系统并没有遵循这条路线。

> The file system and file descriptors have been powerful abstractions. Even so, there are other models for operating system interfaces. Multics, a predecessor of Unix, abstracted file storage in a way that made it look like memory, producing a very different flavor of interface. The complexity of the Multics design had a direct influence on the designers of Unix, who aimed to build something simpler.

虽然文件系统和文件描述符的抽象能力已经足够强大，但操作系统的接口设计绝非只有这一种方式。Unix 的前身 Multics 以一种类似于内存的方式对文件存储进行了抽象，从而产生了一种截然不同的接口风格。Multics 在设计上的复杂性直接影响了 Unix 的设计者们，而他们的目标则是希望系统设计变得更简单。

> Xv6 does not provide a notion of users or of protecting one user from another; in Unix terms, all xv6 processes run as root.

xv6 没有提供用户的概念，也不支持对用户的保护（即一个用户的操作不会影响另一个用户）；用 Unix 术语来说，所有 xv6 进程都以 root 身份运行。

> This book examines how xv6 implements its Unix-like interface, but the ideas and concepts apply to more than just Unix. Any operating system must multiplex processes onto the underlying hardware, isolate processes from each other, and provide mechanisms for controlled inter-process communication. After studying xv6, you should be able to look at other, more complex operating systems and see the concepts underlying xv6 in those systems as well.

本书探讨了 xv6 如何实现其 Unix-like 接口，但其理念和概念不局限于 Unix。任何操作系统都必须在底层硬件上实现进程复用，进程隔离，并提供受控的进程间通信的机制。学习 xv6 之后，你应该能够理解其他更复杂的操作系统，并了解这些系统中和 xv6 类似的底层概念。

1.6 练习（Exercises）

> 1. Write a program that uses UNIX system calls to “ping-pong” a byte between two processes over a pair of pipes, one for each direction. Measure the program’s performance, in exchanges per second.

1. 编写一个程序，使用 UNIX 系统调用通过一对管道（每个方向一个）在两个进程之间 “来回（ping-pong）” 传输一个字节。测量该程序的性能，以每秒交换次数为单位。

第 2 章 操作系统组织（Chapter 2 Operating system organization）

> A key requirement for an operating system is to support several activities at once. For example, one might use the `fork` and `exec` system calls from Chapter 1 to start both a compiler and a text editor as processes. The operating system must time-share resources such as CPUs and memory among these processes. The operating system must also arrange for isolation between the processes. If one process has a bug and malfunctions, it shouldn’ t affect unrelated processes. Complete isolation, however, is too strong, since it should be possible for processes to intentionally interact; pipelines are an example. Thus an operating system must fulfill three requirements: multiplexing, isolation, and interaction.

对于一个操作系统来说，一个关键的需求是要同时支持多个 “活动（activity）” 。例如，可以使用第 1 章中描述的系统调用 `fork` 和 `exec` 来启动编译器和文本编辑器两个进程。操作系统必须在这些进程之间采用 分时（time-share） 的方式共享 CPU 和内存等资源。操作系统还必须确保进程之间的 隔离性（isolation）。如果一个进程的实现有问题导致了一些非法操作，它也不应该影响那些和它无关的其他进程。当然，也没有必要实现完全的隔离，进程之间有时候也有交互的需要；管道就是一个例子。因此，操作系统必须满足三个需求：“复用（multiplexing）”、“隔离（isolation）” 和 “交互（interaction）”。

> This chapter provides an overview of how operating systems are organized to achieve these three requirements. It turns out there are many ways to do so, but this text focuses on mainstream designs centered around a monolithic kernel, which is used by many Unix operating systems. This chapter also provides an overview of an xv6 process, the unit of isolation in xv6.

本章概述了如何设计操作系统来实现这三个要求。事实证明，有很多方法可以做到这一点，但是本文侧重于介绍主流的 “宏内核（monolithic kernel）” 设计理念，许多 Unix 操作系统都采用了这种设计方式。本章还概述了 xv6 中的 “进程（process）”（它是 xv6 中实现隔离的基本单元）。

> Xv6 runs on a multi-core RISC-V microprocessor (By "multi-core" this text means multiple CPUs that share memory but execute in parallel, each with its own set of registers. This text sometimes uses the term multiprocessor as a synonym for multi-core, though multiprocessor can also refer more specifically to a computer with several distinct processor chips.), and much of its low-level functionality (for example, its process implementation) is specific to RISC-V. RISC-V is a 64-bit CPU, and xv6 is written in “LP64” C, which means long (L) and pointers (P) in the C programming language are 64 bits, but an `int` is 32 bits. This book assumes the reader has done a bit of machine-level programming on some architecture, and will introduce RISC-V-specific ideas as they come up. The user-level ISA [2] and privileged architecture [3] documents are the complete specifications. You may also refer to “The RISC-V Reader: An Open Architecture Atlas” [15].

xv6 支持 多核(multi-core) RISC-V 微处理器 (本文中的 “多核（multi-core）” 是指多个 CPU 共享内存且同时并发运行，每个 CPU 都有各自的寄存器组。本文有时使用 “多处理器

（multiprocessor）” 作为 “多核” 的同义词，但 “多处理器” 有时更特指一台计算机具有多个不同的处理器芯片。），它的许多底层功能（例如，它的进程实现）是特定于 RISC-V 的。运行 xv6 的 RISC-V 处理器是 64 位的，xv6 用 C 语言编写，且编译时采用 “LP64” 方式，这意味着 C 语言中的 `long`（L）类型的变量和指针（P）变量都是 64 位长度，但 `int` 类型变量是 32 位的。这本书假设读者已经在一些架构上具备一些机器指令级别（译者注：即采用汇编语言）编程的经验，本文在涉及相关内容时会适当介绍一些 RISC-V 相关的知识。完整的 ISA 规范请参考用户级别（非特权） ISA 手册 [2] 和特权 ISA 手册 [3]。另一个有用的参考文献是 “The RISC-V Reader: An Open Architecture Atlas” [15]。

> The CPU in a complete computer is surrounded by support hardware, much of it in the form of I/O interfaces. Xv6 is written for the support hardware simulated by qemu’ s “-machine virt” option. This includes RAM, a ROM containing boot code, a serial connection to the user’ s keyboard/screen, and a disk for storage.

作为一个完整的计算机系统，除了 CPU 外还包括其他外围设备硬件，其中大部分外设可以通过 I/O 接口方式访问。xv6 是针对 qemu 的 virt 机器开发的 (所谓 virt 机器是指运行 qemu 模拟器时用 “-machine virt” 选项启动的一种仿真平台) 。这个仿真平台模拟了内存（RAM）、包含引导代码的 ROM、一个连接了用户键盘和屏幕的串口，以及一个支持持久存储的磁盘。

2.1 抽象物理资源（Abstracting physical resources）

> The first question one might ask when encountering an operating system is why have it at all? That is, one could implement the system calls in Figure 1.2 as a library, with which applications link. In this plan, each application could even have its own library tailored to its needs. Applications could directly interact with hardware resources and use those resources in the best way for the application (e.g., to achieve high or predictable performance). Some operating systems for embedded devices or real-time systems are organized in this way.

当人们初次接触操作系统概念时，可能会问的第一个问题是我们为什么需要它？我们完全可以将图 1.2 中的系统调用实现为一个库，应用程序链接这个库就好了。如果按照这个思路更进一步，每个应用程序甚至可以根据自己的需求定制自己的库。应用程序可以直接与硬件资源交互，并从应用角度出发以各自最佳的方式使用这些资源（例如，实现高性能或可预期的性能）。（现实中）一些针对嵌入式设备或实时系统设计的操作系统就是这样组织的。

> The downside of this library approach is that, if there is more than one application running, the applications must be well-behaved. For example, each application must periodically give up the CPU so that other applications can run. Such a cooperative time-sharing scheme may be OK if all applications trust each other and have no bugs. It’ s more typical for applications to not trust each other, and to have bugs, so one often wants stronger isolation than a cooperative scheme provides.

采用这种库函数方式实现操作系统的缺点是，如果有多个应用程序在运行，这些应用程序必须要仔细设计。例如，每个应用程序必须定期放弃处理器，以便其他应用程序能够有机会运行。如果所有应用程序都相互信任并且没有错误，这种基于 协同（cooperative） 的 “分时（time-sharing）” 方案可能是可行的。然而更常见的情况是，应用程序之间互不了解且具体实现中不可避免会出现问题（译者注：所谓问题最简单的例子就是出现诸如死循环导致不能定期放弃处理器），所以人们通常希望采用比 “协同” 更好的方法实现隔离。

> To achieve strong isolation it's helpful to forbid applications from directly accessing sensitive hardware resources, and instead to abstract the resources into services. For example, Unix applications interact with storage only through the file system's `open`, `read`, `write`, and `close` system calls, instead of reading and writing the disk directly. This provides the application with the convenience of pathnames, and it allows the operating system (which provides the interface) to manage the disk. Even if isolation is not a concern, programs that interact intentionally (or just wish to keep out of each other's way) are likely to find a file system a more convenient abstraction than direct use of the disk.

为了实现更好的隔离，最好禁止应用程序直接访问敏感的硬件资源，代之以将对资源的访问抽象为服务。例如，Unix 上的应用程序只能通过文件系统提供的系统调用，诸如 `open`、`read`、`write` 和 `close` 访问存储，而不是直接读写磁盘。这些系统调用为应用程序提供了方便实用的路径名（来访问磁盘上的数据），而将具体对磁盘的管理工作交给了操作系统（作为接口的提供者）。即使不考虑隔离问题，需要交互的程序（如果只是希望它们之间不会互相干扰）可能会发现通过文件系统访问磁盘比直接使用磁盘会更方便。

> Similarly, Unix transparently switches hardware CPUs among processes, saving and restoring register state as necessary, so that applications don't have to be aware of time-sharing. This transparency allows the operating system to share CPUs even if some applications are in infinite loops.

同样，Unix 在进程之间透明地切换硬件处理器，根据需要保存和恢复寄存器状态，这样应用程序就不必意识到 “分时共享（time-sharing）” 的存在。这种透明性允许操作系统（为进程）公平地分享处理器，即使有些应用程序会处于无限循环中。

> As another example, Unix processes use `exec` to build up their memory image, instead of directly interacting with physical memory. This allows the operating system to decide where to place a process in memory; if memory is tight, the operating system might even store some of a process's data on disk. `exec` also provides users with the convenience of a file system to store executable program images.

另一个例子是，Unix 进程使用 `exec` 来创建它们的（虚拟）内存空间，而不是让程序直接访问物理内存。最终将由操作系统来决定一个进程所使用的物理内存；如果内存很紧张，操作系统甚至可以将一个进程的部分数据暂存在磁盘上。`exec` 还给用户提供了一种便利，即我们可以通过文件系统将程序保存在磁盘上，只有在需要运行它的时候再将程序从磁盘上加载到内存中。

> Many forms of interaction among Unix processes occur via file descriptors. Not only do file descriptors abstract away many details (e.g., where data in a pipe or file is stored), they are also defined in a way that simplifies interaction. For example, if one application in a pipeline exits or fails, the kernel automatically generates an end-of-file signal for the next process in the pipeline.

Unix 进程之间的许多交互都是通过 “文件描述符（file descriptors）” 实现的。文件描述符不仅抽象了许多细节（例如，管道或文件中的数据实际存储在哪里），还简化了进程之间交互的方式。譬如，当程序关闭管道或者管道操作失败时，内核会自动为管道另一端的进程产生 “文件结束（end-of-file）” 信号。

> The system-call interface in Figure 1.2 is carefully designed to provide both programmer convenience and the possibility of strong isolation. The Unix interface is not the only way to abstract resources, but it has proved to be a good one.

图 1.2 中的系统调用接口是精心设计的产物，既为程序员提供了便利，又为实现强有力的隔离提供了可能性。Unix 接口不是抽象资源的唯一方案，但它已经被证明是一个非常好的方案。

2.2 用户模式，管理员模式和系统调用（User mode, supervisor mode, and system calls）

> Strong isolation requires a hard boundary between applications and the operating system. Applications shouldn’ t be allowed to disturb the operation of the operating system or other programs, even if the application has a bug or is malicious. To achieve strong isolation, the operating system must arrange that applications cannot modify (or even read) the operating system’ s data structures and instructions and that applications cannot access other processes’ memory.

“强隔离（strong isolation）” 需要在应用程序和操作系统之间建立一个明显的边界。我们不希望一个应用程序的错误操作（无论是有意还是无意）会影响操作系统或其他应用程序。要达到强隔离的目标，操作系统必须保证应用程序不能修改（甚至读取）操作系统的数据结构和指令，以及确保这个应用程序不能访问其他进程的内存。

> CPUs provide hardware support for strong isolation. For example, RISC-V has three privilege levels which constrain what code can do: machine mode, supervisor mode, and user mode. Instructions executing in machine mode have full privilege; a CPU starts in machine mode. Machine mode is mostly intended for setting up the computer during boot. Xv6 executes briefly in machine mode and then changes to supervisor mode.

CPU 为实现强隔离从硬件层面提供了支持。例如，RISC-V 的 CPU 有三种 “特权级别（privilege level）”：机器模式（machine mode）、管理员模式（supervisor mode） 和 用户模式（user mode）。在机器模式下执行指令时权限最大；CPU 上电后首先运行在机器模式下。机器模式主要用于在计算机启动过程中执行配置操作。xv6 在机器模式下执行很少的几行代码，随后就将处理器切换为管理员模式。

> In supervisor mode the CPU is allowed to execute privileged instructions: for example, enabling and disabling interrupts, reading and writing the register that holds the address of the page table, etc. If an application in user mode attempts to execute a privileged instruction, then the CPU doesn’ t execute the instruction, but "traps" to special code in supervisor mode that can terminate the application. Figure 1.1 in Chapter 1 illustrates this organization. An application can execute only user-mode instructions (e.g., adding numbers, etc.) and is said to be running in user space, while the software in supervisor mode can also execute privileged instructions and is said to be running in kernel space. The software running in kernel space (or in supervisor mode) is called the kernel.

在管理员模式下，CPU 被允许执行 特权指令（privileged instructions）：例如，启用和禁用中断、对保存页表地址的寄存器进行读取和写入等。如果在用户模式下应用程序试图执行特权指令，那么 CPU 不仅会拒绝执行，还会将处理器通过 “陷入（trap）” 方式切换到管理员模式并执行一段特殊

的代码来终止应用程序。第 1 章中的图 1.1 描述了这种设计。一个应用程序只能执行用户模式下允许执行的指令（例如，对数字求和等），这被称为在 用户空间（user space） 中运行，而处于管理员模式下程序可以执行特权指令，这被称为在 内核空间（kernel space） 中运行。在内核空间（对应 RISC-V 的管理员模式概念）中运行的软件被称为 内核（kernel）。

> Applications interact with the kernel via system calls calls such as `read`. Applications are not allowed to directly call kernel functions or access the kernel’ s memory. RISC-V provides the `ecall` instruction for system calls; it switches the CPU from user to supervisor mode and jumps to a kernel-specified entry point. Once the CPU has switched to supervisor mode, the kernel can then validate the arguments of the system call (e.g., check if the address passed to the system call is part of the application’ s memory), decide whether the application is allowed to perform the requested operation (e.g., check if the application is allowed to write the specified file), and then deny it or execute it. It is important that the kernel control the entry point for transitions to supervisor mode; if the application could decide the kernel entry point, a malicious application could, for example, enter the kernel at a point where the validation of arguments is skipped.

应用程序通过系统调用（例如 `read`）与内核交互。应用程序不允许直接调用内核函数或访问内核内存。RISC-V 为系统调用提供了 `ecall` 指令；该指令将 CPU 从用户模式切换到管理员模式，并跳转到内核指定的 “入口点（entry point）” 。(译者注：所谓 entry point 可以认为是一条指令的地址，当处理器从用户模式切换到管理员模式时，处理器的 PC 自动跳转到这个地址开始执行内核定义的函数)。一旦 CPU 切换到管理员模式，内核就可以验证系统调用的参数（例如，检查传递给系统调用的地址是否是应用程序内存的一部分），决定是否允许应用程序执行请求的操作（例如，检查是否允许应用程序写入指定的文件），结果无非是拒绝该请求或执行该请求。这个转换到管理员模式后执行的入口点必须由内核来定义，这点非常重要；如果应用程序可以决定内核入口点，那么心存恶意的应用程序就可以跳过参数验证，直接进入内核（执行内核的函数）。

2.3 内核的组织（Kernel organization）

> A key design question is what part of the operating system should run in supervisor mode. One possibility is that the entire operating system resides in the kernel, so that the implementations of all system calls run in supervisor mode. This organization is called a monolithic kernel.

有一个关键的设计问题是：操作系统的哪些部分应该在管理员模式下运行。一种可能性是整个操作系统驻留在内核中，这样所有系统调用的实现都以管理员模式运行。这种组织方式被称为 宏内核（monolithic kernel）。

> In a monolithic organization the entire operating system consists of a single program running in supervisor mode. One reason this organization is convenient is that the OS designer doesn’ t have to divide code into parts that do and do not require supervisor privileges. Furthermore, it is easy for different parts of the operating system to cooperate, since they are parts of a single program. For example, a monolithic kernel might share a disk block cache with the file system and the virtual memory system.

在宏内核方式下，整个操作系统由单个程序组成，运行在管理员模式下。这种运行方式的方便之处在于，操作系统设计人员无需区分操作系统的哪些部分需要运行在管理员模式下，哪些又不需要。

此外，由于操作系统的不同部分都属于同一个程序，因此它们之间很容易协作。例如，宏内核可以创建一个磁盘块缓存供文件系统和虚拟内存系统共享使用。

> A downside is that monolithic kernels tend to grow large and complex, so that no one developer understands all of the interactions between different parts of the code; this is a recipe for bugs. A bug in the kernel is particularly troublesome because it may cause the entire computer to crash, or cause many applications to malfunction, or make the entire computer vulnerable to security attacks.

宏内核的一个缺点是，它往往会变得庞大而复杂，以至于没有哪个开发人员能够理解代码不同部分之间的所有交互；这很容易滋生 “问题（bug）”。内核中的 bug 尤其麻烦，因为它可能导致整个计算机崩溃，或导致许多应用程序无法正常工作，甚至使整个计算机容易受到安全攻击。

> A microkernel aims to reduce the incidence of bugs in the kernel. The idea is to put an absolute minimum of functionality in the kernel itself, so that little code executes in supervisor mode, and so that the kernel is easy to understand and analyze for correctness. The bulk of the operating system runs as user-level server processes. For example, the file system code would execute as a server process, in user mode rather than supervisor mode.

微内核（microkernel） 旨在降低内核中错误的发生率。其理念是将极少的功能放入内核中，以便只有极少量的代码在管理员模式下执行，从而易于理解和分析内核的正确性。操作系统的大部分代码以用户态服务进程的形式运行。例如，文件系统代码将作为服务器进程在用户模式下执行，而不是在管理员模式下。

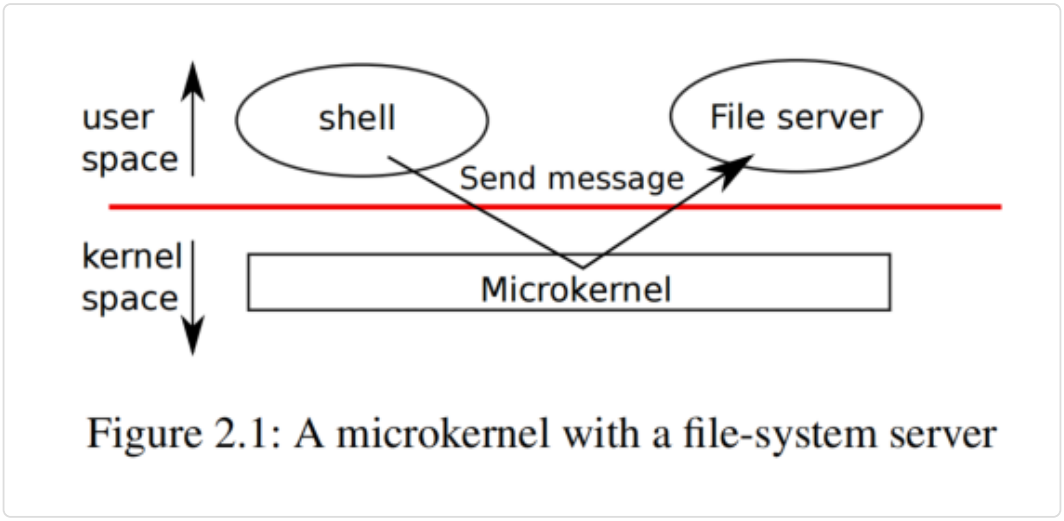

Figure 2.1: A microkernel with a file-system server

> Figure 2.1 illustrates this microkernel design. In the figure, the file system runs as a user-level server process. To allow applications to interact with the file server, the kernel provides an inter-process communication mechanism to send messages from one user-mode process to another. For example, if an application like the shell wants to read or write a file, it sends a message to the file server and waits for a response.

图 2.1 展示了这种微内核设计。图中，文件系统作为用户态服务进程运行。为了允许应用程序与文件服务器交互，内核提供了一种进程间通信机制，用于在用户模式进程之间发送消息。例如，像 shell 这样的应用程序如果想要读取或写入文件，它需要向文件服务器发送消息并等待响应。

> In a microkernel, the kernel interface consists of a few low-level functions for starting applications, sending messages, accessing device hardware, etc. This organization allows the kernel to be relatively simple, as most of the operating system resides in user-level servers.

在微内核中，内核接口由一些底层函数组成，包括启动应用程序、发送消息、访问设备硬件等。这种组织方式使内核相对简单，因为操作系统的大部分代码都运行在用户态层面的服务器中。

> In the real world, both monolithic kernels and microkernels are popular. Many Unix kernels are monolithic. For example, Linux has a monolithic kernel, although some OS functions run as user-level servers (e.g., the window system). Linux delivers high performance to OS-intensive applications, partially because the subsystems of the kernel can be tightly integrated.

在现实世界中，宏内核和微内核都很流行。许多 Unix 内核都是宏内核的。例如，Linux 就是采用的宏内核方式，除了一部分操作系统功能作为用户态服务器运行（例如窗口系统）。Linux 能够支持实现高性能的 “操作系统密集型（OS-intensive）” 应用程序，部分原因就是因为其内核中的子系统之间可以紧密集成。

> Operating systems such as Minix, L4, and QNX are organized as a microkernel with servers, and have seen wide deployment in embedded settings. A variant of L4, seL4, is small enough that it has been verified for memory safety and other security properties [8].

Minix、L4 和 QNX 等操作系统则采用微内核架构，大部分功能以服务器形式提供，它们在嵌入式环境中得到了广泛的部署。其中，L4 的一个变体 seL4 的体积足够小，并且已通过内存安全和其他安全特性的验证 [8]。

> There is much debate among developers of operating systems about which organization is better, but there is no conclusive evidence one way or the other. Furthermore, it depends much on what “better” means: faster performance, smaller code size, reliability of the kernel, reliability of the complete operating system (including user-level services), etc.

至于究竟那种组织方式更好，在操作系统开发者之间存在很多争论，目前尚无定论。此外，这很大程度上取决于如何理解什么是 “更好”：这涉及系统更倾向于实现更快的性能、更小的代码大小、更可靠的内核，还是更多考虑整个操作系统（包括用户级别服务）的可靠性等等方面的问题。

> There are also practical considerations that may be more important than the question of which organization. Some operating systems have a microkernel but run some of the user-level services in kernel space for performance reasons. Some operating systems have monolithic kernels because that is how they started and there is little incentive to move to a pure microkernel organization, because new features may be more important than rewriting the existing operating system to fit a microkernel design.

还有一些需要实际考虑的因素可能比单纯选择哪种组织方式更重要。有些操作系统采用微内核，但出于性能考虑，仍然会将部分用户级别服务放在内核空间运行。有些操作系统采用宏内核架构，仅仅是因为它们最初就是这样设计的，而且几乎没有什么动力再转向纯粹的微内核架构，与其重写现有操作系统以适应微内核设计还不如在开发新功能上多花点精力。

> From this book’ s perspective, microkernel and monolithic operating systems share many key ideas. They implement system calls, they use page tables, they handle interrupts, they support processes, they use locks for concurrency control, they implement a file system, etc. This book focuses on these core ideas.

从本书的角度来看，微内核和宏内核的操作系统有许多共同的核心思想。它们都实现了系统调用，使用了页表，处理了中断，支持进程，使用锁进行并发控制，并且都实现了文件系统等等。本书将重点介绍这些核心思想。

> Xv6 is implemented as a monolithic kernel, like most Unix operating systems. Thus, the xv6 kernel interface corresponds to the operating system interface, and the kernel implements the complete operating system. Since xv6 doesn’ t provide many services, its kernel is smaller than some microkernels, but conceptually xv6 is monolithic.

与大多数 Unix 操作系统一样，xv6 采用了宏内核的组织方式。因此，xv6 内核的接口就是操作系统的接口，内核实现了完整的操作系统。由于 xv6 提供的服务较少，其内核比一些微内核更小，但从概念上讲，xv6 仍然属于宏内核。

2.4 代码讲解：xv6 的组织（Code: xv6 organization）

	File	Description
Boot	entry.S	Very first boot instructions.
	main.c	Control initialization of other modules.
	start.c	Early machine-mode boot code.
Processes	exec.c	exec() system call.
	proc.c	Processes and scheduling.
	swtch.S	Thread switching.
	sysproc.c	Process-related system calls.
Traps	kernelvec.S	Handle traps from kernel code.
	trampoline.S	Handle traps from user code.
	trap.c	C code to handle and return from traps and interrupts.
	syscall.c	Dispatch system calls to handling function.
Memory	vm.c	Manage page tables and address spaces.
	kalloc.c	Physical page allocator.
Devices	console.c	Connect to the user keyboard and screen.
	plic.c	RISC-V interrupt controller.
	printf.c	Formatted output to the console.
	uart.c	Serial-port console device driver.
	virtio_disk.c	Disk device driver.
FS	bio.c	Disk block cache for the file system.
	file.c	File descriptor support.
	fs.c	File system.
	log.c	File system logging and crash recovery.
	sysfile.c	File-related system calls.
	pipe.c	Pipes.
Misc	sleeplock.c	Locks that yield the CPU.
	spinlock.c	Locks that don't yield the CPU.
	string.c	C string and byte-array library.

Figure 2.2: Xv6 kernel source files.

> The xv6 kernel source is in the `kernel/` sub-directory. Figure 2.2 lists the files, divided into the major areas of kernel responsibility: starting the system (booting), creating and controlling processes, handling traps (interrupts and system calls), allocating memory and configuring virtual addresses, controlling devices, and managing the file-system.

xv6 内核源代码位于 `kernel/` 子目录中。图 2.2 列出了这些文件，并按内核的主要职责进行划分，这包括：启动系统（引导）、创建和控制进程、处理 “陷阱（traps）”（中断和系统调用）、分配内存和配置虚拟地址、控制设备以及管理文件系统。

2.5 进程概述（Process overview）

> The unit of isolation in xv6 (as in other Unix operating systems) is a process. The process abstraction prevents one process from wrecking or spying on another process' s memory, CPU, file descriptors, etc. It also prevents a process from wrecking the kernel itself, so that a process can' t subvert the kernel' s isolation mechanisms. The kernel must implement the

> process abstraction with care because a buggy or malicious application may trick the kernel or hardware into doing something bad (e.g., circumventing isolation). The mechanisms used by the kernel to implement processes include the user/supervisor mode flag, address spaces, and time-slicing of threads.

（和其他 Unix 操作系统一样）xv6 中实现隔离的基本单元是一个 进程（process）。进程是一个抽象的概念，一个进程无法访问（更谈不上破坏）另一个进程所拥有的内存、CPU、文件描述符等。一个进程也无法破坏内核本身，自然这样一个进程就不能突破内核的隔离机制。内核必须小心地实现进程这个抽象设计，因为一个有缺陷或恶意的应用程序可能会欺骗内核或硬件做坏事（例如，绕过隔离机制）。内核用来实现进程的机制包括区分用户模式和管理员模式，引入地址空间的概念以及对线程的运行划分时间片。

> To help enforce isolation, the process abstraction provides the illusion to a program that it has its own private machine. A process provides a program with what appears to be a private memory system, or address space, which other processes cannot read or write. A process also provides the program with what appears to be its own CPU to execute the program’ s instructions.

为了加强隔离，进程这种抽象设计给程序提供了一种错觉，即好像它拥有自己专有的机器。一个程序似乎拥有一块私有的内存区域，也叫作 地址空间（address space），其他进程对它不能读取也不能写入。这个程序还拥有自己的 CPU 来执行它的指令。

> Xv6 uses page tables (which are implemented by hardware) to give each process its own address space. The RISC-V page table translates (or “maps”) a virtual address (the address that an RISC-V instruction manipulates) to a physical address (an address that the CPU sends to main memory).

xv6 使用 “页表（page tables）”（在硬件的帮助下）为每个进程提供自己的地址空间。RISC-V 的页表将 虚拟地址（virtual address）（RISC-V 的指令操作的地址）“翻译（translate）”（或 “映射（map）”）为 物理地址（physical address）（CPU 芯片发送到主存储器的地址）。（译者注：页表是内存中的一个数据结构，存放了虚拟地址和物理地址之间的对应关系，也就是这里说的映射的概念。当 CPU 执行指令时，由专门的 MMU 硬件单元根据页表中定义的映射关系，将指令中的虚拟地址转换成物理地址并进而访问实际的物理内存。）

> Xv6 maintains a separate page table for each process that defines that process’ s address space. As illustrated in Figure 2.3, an address space includes the process’ s user memory starting at virtual address zero. Instructions come first, followed by global variables, then the stack, and finally a “heap” area (for malloc) that the process can expand as needed. There are a number of factors that limit the maximum size of a process’ s address space: pointers on the RISC-V are 64 bits wide; the hardware uses only the low 39 bits when looking up virtual addresses in page tables; and xv6 uses only 38 of those 39 bits. Thus, the maximum address is $2^{38} - 1 = 0x3fffffffff$, which is `MAXVA` (0899). At the top of the address space xv6 places a trampoline page (4096 bytes) and a trapframe page. Xv6 uses these two pages to transition into the kernel and back; the trampoline page contains the code to transition in and out of the kernel, and the trapframe is where the kernel saves the process’ s user registers, as Chapter 4 explains.

xv6 为每个进程维护一个单独的页表，定义了该进程的地址空间。如图 2.3 所示是一个进程的地址空间，它对应了进程的 用户内存（user memory），这块内存以虚拟内存地址 0 作为起始地址。向上（虚拟地址增加的方向），首先存放的是指令，然后是全局变量，接着是栈区，最后是一个 “堆” 区（用于 malloc）供进程根据需要进行扩展。有许多因素限制了进程地址空间的最大范围：RISC-V 的指针宽度是 64 位；硬件在页表中查找虚拟地址时只使用低 39 位；xv6 只使用了这 39 位中的 38 位。因此，最大地址是 $2^{38} - 1 = 0x3fffffffff$，即代码中定义的常量 `MAXVA`（0899）。在地址空间的顶部，xv6 为 trampoline 和 trapframe 各映射了一个 “页（page）”（大小为 4096 个字节），xv6 利用这两个页实现用户态和内核态之间的切换，trampline 页中包含了用于内核态切换时会执行的指令，trapframe 页会被内核用于保存进程的用户寄存器，有关这部分的详细内容将在第 4 章中进一步解释。

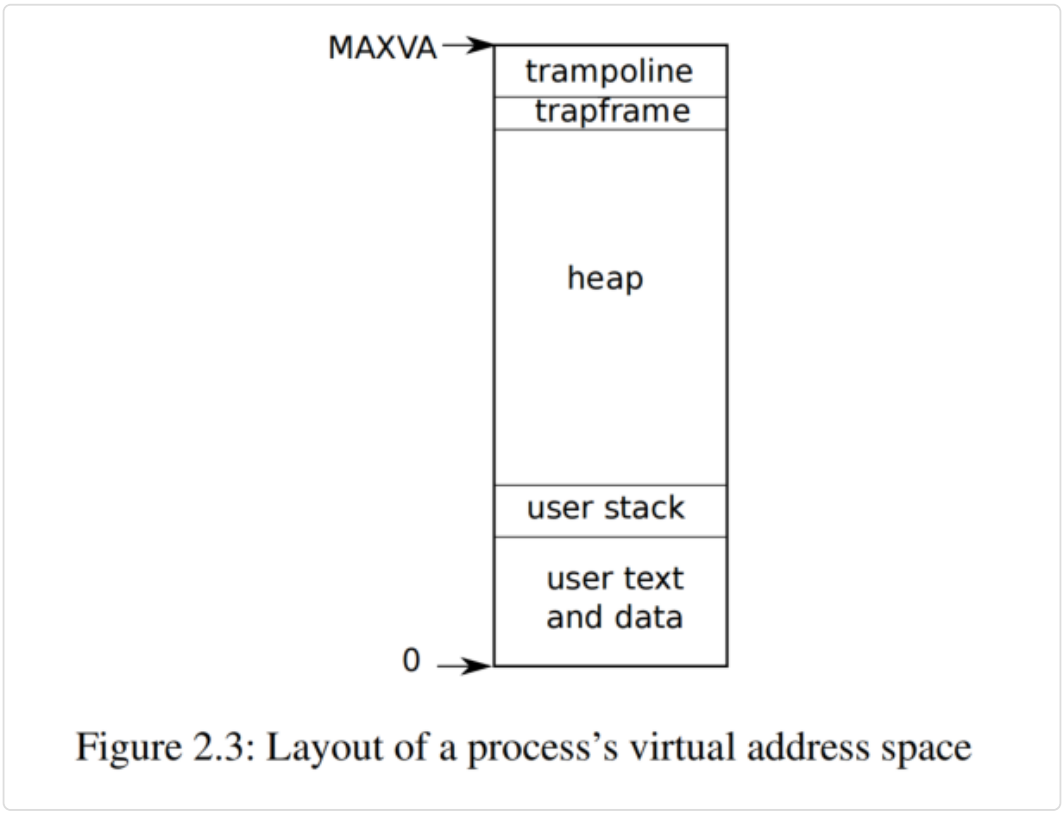

Figure 2.3: Layout of a process’s virtual address space

> The xv6 kernel maintains many pieces of state for each process, which it gathers into a `struct proc` (2034). A process’ s most important pieces of kernel state are its page table, its kernel stack, and its run state. We’ ll use the notation `p->xxx` to refer to elements of the `proc` structure; for example, `p->pagetable` is a pointer to the process’ s page table.

xv6 内核为每个进程维护许多信息，所有的这些内容都定义在一个结构体 `struct proc` (2034) 中。一个进程中最重要的内核信息包括它的页表、内核栈区和运行状态。我们将使用 `p->xxx` 来引用 `proc` 结构体中的属性；例如，`p->pagetable` 是一个指向该进程页表的指针。

> At this point, please read `kernel/proc.h`, which defines `struct proc`. The xv6 code is more important for you to understand than this book; you should prioritize the code, and consult this book as needed to clarify the code. The purpose of some of the code may not be apparent at first, but further reading and searching the code will help. Feel free to explore and modify the code.

此时，请阅读 `kernel/proc.h`，其中定义了 `struct proc`。xv6 代码比本书更重要；您应该优先阅读代码，并根据需要参考本书来理解代码。某些代码的用途可能一开始并不明显，但进一步阅读和搜索代码会有所帮助。请大胆探索和修改代码。

> Each process has a thread of control (or thread for short) that holds the state needed to execute the process. At any given time, a thread might be executing on a CPU, or suspended (not executing, but capable of resuming executing in the future). To switch a CPU between processes, the kernel suspends the thread currently running on that CPU and saves its state, and restores the state of another process's previously-suspended thread. Much of the state of a thread (local variables, function call return addresses) is stored on the thread's stacks. Each process has two stacks: a user stack and a kernel stack (`p->kstack`). When the process is executing user instructions, only its user stack is in use, and its kernel stack is empty. When the process enters the kernel (for a system call or interrupt), the kernel code executes on the process's kernel stack; while a process is in the kernel, its user stack still contains saved data, but isn't actively used. A process's thread alternates between actively using its user stack and its kernel stack. The kernel stack is separate (and protected from user code) so that the kernel can execute even if a process has wrecked its user stack.

每个进程都对应一个 “控制流（thread of control）”（或简称 线程(thread)），并维护着进程执行过程中所需的上下文信息。在任意一个时间点上，一个线程要么是在处理器上执行，要么挂起（不执行，但稍后能够恢复继续执行）。为了在处理器上交替执行进程，内核会挂起当前在处理器上运行的线程，保存它的上下文，然后恢复另一个当前被挂起的进程所对应的线程的上下文。线程的大部分状态（包括本地变量、函数调用返回地址）存储在线程的栈上。每个进程有两个栈：一个用户栈和一个内核栈（ `p->kstack` ）。当进程执行用户指令时，只使用它的用户栈，它的内核栈是空的。当进程进入内核（由于系统调用或中断）时，内核代码使用进程的内核栈；当一个进程进入内核态后，它的用户栈仍然包含保存的数据，只是不被内核指令使用。进程的线程交替使用它的用户栈和内核栈。内核栈是独立的（用户代码不能访问），因此即使一个进程搞坏了它的用户栈，内核依然可以正常运行。

> A process's user code can make a system call by executing the RISC-V `ecall` instruction. This instruction switches to supervisor mode and changes the program counter to a kernel-defined entry point. The code at the entry point switches to the process's kernel stack and executes the kernel instructions that implement the system call. When the system call completes, the kernel returns to user space by executin the `sret` instruction, which switches to user mode and resumes executing user instructions just after the system call instruction. A process's thread can "block" in the kernel to wait for I/O, and resume where it left off when the I/O has finished.

一个进程的用户态代码可以通过执行 RISC-V 的 `ecall` 指令发起系统调用，该指令将处理器切换到管理员模式，并将程序计数器（program counter）更改为内核定义的入口点。入口点的代码执行切换操作使线程转为使用内核栈并开始执行实现系统调用的内核指令，当系统调用完成时，内核通过调用 `sret` 指令切换回用户模式从而返回用户空间，并从发起系统调用的指令（译者注：即 `ecall` ）后的那条用户指令开始继续恢复执行。一个进程的线程可以在内核中以 “阻塞（block）” 方式等待 I/O，并在 I/O 完成后恢复执行。

> `p->state` indicates whether the process is allocated, ready to run, currently running on a CPU, waiting for I/O, or exiting.

`p->state` 用于记录当前进程进程的状态是 “已分配（allocated）”、“就绪（ready to run）”、“运行（running）”、“等待 I/O 中（waiting）” 还是 “正在退出（exiting）”。

> `p->pagetable` holds the process’s page table, in the format that the RISC-V hardware expects. Xv6 causes the paging hardware to use a process’s `p->pagetable` when executing that process in user space. A process’s page table also serves as the record of the addresses of the physical pages allocated to store the process’s memory.

`p->pagetable` 指向进程的页表，页表中的格式必须符合 RISC-V 硬件所期望的格式。当进程在用户空间执行，xv6 会设置分页硬件使用进程的 `p->pagetable`。一个进程的页表记录着哪些物理页已分配给该进程。

> In summary, a process bundles two design ideas: an address space to give a process the illusion of its own memory, and a thread to give the process the illusion of its own CPU. In xv6, a process consists of one address space and one thread. In real operating systems a process may have more than one thread to take advantage of multiple CPUs.

总之，进程的概念包含了两个设计思想：一个是地址空间的概念，它让进程感觉拥有自己专属的内存；另一个是线程的概念，它让进程感觉拥有自己专属的 CPU。在 xv6 中，一个进程由一个地址空间和一个线程组成。在实际操作系统中，一个进程可能含有多个线程，以充分利用处理器中的多个核。

2.6 代码讲解：启动 xv6，第一个进程以及系统调用（Code: starting xv6, the first process and system call）

> To make xv6 more concrete, we’ll outline how the kernel starts and runs the first process. The subsequent chapters will describe the mechanisms that show up in this overview in more detail. Please read `kernel/entry.S`, `kernel/start.c`, `kernel/main.c`, and `user/init.c`.

为了使读者更具体地了解 xv6，我们将概述内核如何启动和运行第一个进程。后面的章节将更详细地描述这里概述中介绍的机制。请阅读代码：`kernel/entry.S`，`kernel/start.c`，`kernel/main.c`，以及 `user/init.c`。

> When the RISC-V computer powers on, it initializes itself and runs a boot loader which is stored in read-only memory. The boot loader copies the xv6 kernel into memory at physical address `0x80000000`. The reason it places the kernel at `0x80000000` rather than `0x0` is because the address range `0x0:0x80000000` contains I/O devices.

当 RISC-V 计算机上电时，它会初始化自己并运行一个存储在只读内存中的 “引导加载程序（boot loader）”。引导加载程序将 xv6 内核复制到内存中物理地址为 `0x80000000` 处。它将内核放在 `0x80000000` 而不是 `0x0` 的原因是 `0x0:0x80000000` 这段地址范围被预留用于访问 I/O 设备。

> Then the boot loader jumps to xv6 starting at `_entry` (1006). The RISC-V starts with paging hardware disabled: virtual addresses map directly to physical addresses. The instructions at `_entry` set up a stack so that xv6 can run C code. Xv6 declares space for this stack, `stack0`, in the file `start.c` (1060). The code at `_entry` loads the stack pointer register `sp` with the

> address `stack0+4096`, the top of the stack, because the stack on RISC-V grows down. Now that the kernel has a stack, `_entry` calls into C code at `start` (1064).

然后引导加载程序跳转到 `_entry` (1006) 开始运行 xv6。启动阶段中 “分页硬件（paging hardware，即 MMU）” 处于禁用模式，也就是说此时虚拟地址直接映射为等值的物理地址。汇编函数 `_entry` 中首先建立了一个栈，这为在 xv6 中使用 C 语言写代码了做好了准备。xv6在 `start.c` (1060) 中定义了最初的栈 `stack0`。由于在 RISC-V 上的栈是向低地址方向扩展，所以在 `_entry` 中将栈顶地址 `stack0+4096` 加载到栈指针寄存器 `sp` 中。现在内核有了栈区，`_entry` 便可以调用 C 函数 `start` (1064) 了。

> The function `start` performs some setup that the CPU only allows in machine mode, most crucially programming the clock chip to generate timer interrupts. Then `start` uses the RISC-V `mret` instruction to switch to supervisor mode and jump to `main` (1160). `mret` requires a bit of setup: `start` sets the previous privilege mode to supervisor in the register `mstatus`, sets the destination address to `main` by writing `main`’s address into the register `mepc`, disables virtual address translation in supervisor mode by writing `0` into the page-table register `satp`, and delegates all interrupts and exceptions to supervisor mode.

函数 `start` 执行一些仅在机器模式下允许的配置操作，最主要的是对时钟芯片进行编程来产生定时器中断。然后 `start` 函数通过调用 `mret` 指令将 CPU 切换到管理员模式并跳转到 `main` 函数（1160）。在执行 `mret` 之前需要提前做好如下设置：它在寄存器 `mstatus` 中将上一个特权模式（译者注：即 `mstatus.MPP`）设置为管理员模式，此外它通过将 `main` 函数的地址写入寄存器 `mepc` 使得处理器返回管理员模式后从 `main` 恢复执行，它通过向页表寄存器 `satp` 写入 `0` 确保管理员模式下虚拟地址转换功能被禁用，并将所有的中断和异常委托（delegate）给管理员模式。

> After `main` (1160) initializes several devices and subsystems, it creates the first process by calling `userinit` (2327). All newly created processes start executing in the kernel in `forkret` (2653). As a special case for the first process, `forkret` calls `kexec` to load the user program `/init`.

在 `main`（1160）初始化多个设备和子系统后，它通过调用 `userinit`（2327）创建第一个进程。所有新创建的进程都通过 `forkret`（2653）在内核中开始执行。对于第一个进程，作为特例，`forkret` 通过调用 `kexec` 加载了用户程序 `/init`。

> After calling `kexec`, `forkret` returns to user space in the `/init` process. `init` (7764) creates a new console device file if needed and then opens it as file descriptors 0, 1, and 2. Then it starts a shell on the console. The system is up.

调用 `kexec` 后，`forkret` 在 `/init` 进程中返回到用户空间。`init` (7764) 会尝试打开一个 “控制台（console）” 设备文件，如果该文件不存在就创建一个新的，最终会打开该文件从而得到文件描述符 0、1 和 2。最后它在控制台上启动一个 shell。系统就这样启动了。

2.7 安全模型（Security Model）

> You may wonder how the operating system deals with buggy or malicious code. Because coping with malice is strictly harder than dealing with accidental bugs, it’s reasonable to focus mostly on providing security against malice. Here’s a high-level view of typical security assumptions and goals in operating system design.

您可能想知道操作系统如何处理错误或恶意代码。由于应对恶意代码比处理意外错误更困难，因此在考虑安全问题时将主要精力放在针对恶意代码上是合理的。以下概述是操作系统设计中典型的从安全角度出发的假设和开发目标。

> The operating system must assume that a process’s user-level code will do its best to wreck the kernel or other processes. User code may try to dereference pointers outside its allowed address space; it may attempt to execute instructions not intended for user code; it may try to read and write RISC-V control registers; it may try to access device hardware; and it may pass clever values to system calls in an attempt to trick the kernel into crashing or doing something stupid.

操作系统必须假设一个进程的用户态代码随时可能会破坏内核或其他进程。用户代码可能会尝试将指针指向其被允许访问的地址空间之外的任何地方；它可能会尝试执行任何 RISC-V 指令，即使这些指令并非专为用户代码设计；它可能会尝试读写任何 RISC-V 的控制寄存器；它可能会尝试直接访问硬件设备；它还可能会在调用系统调用时通过参数传入一些特殊的值，并尝试利用这些值使内核崩溃或让它执行某些愚蠢的操作。

> The kernel’s goal is to restrict each user processe so that it can only access its own user memory, use the 32 general-purpose RISC-V registers, and affect the kernel and other processes in the ways that system calls are intended to allow. The kernel must prevent any other actions. These are typically absolute requirements in kernel design.

内核的目标是限制每个用户进程，使其只能访问自己用户内存上的内容和指令，此外还要限制用户进程只能使用 32 个通用的 RISC-V 寄存器，并只允许它以系统调用允许的方式影响内核和其他进程。内核必须阻止任何其他操作。这通常是内核设计中必须要满足的需求。

> Expectations for the kernel’s own code are different. Kernel code is assumed to be written by well-meaning and careful programmers, to be bug-free, and to contain nothing malicious. This assumption affects how we analyze kernel code. For example, there are many internal kernel functions (e.g., the spin locks) that would cause serious problems if kernel code used them incorrectly. We assume, however, that the kernel uses its own functions correctly. At the hardware level, the RISC-V CPU, RAM, disk, etc. are assumed to operate as advertised in the documentation, with no hardware bugs.

对内核自身代码的期望则截然不同。内核代码被假定由善意且细心的程序员编写。内核代码（被假设）应该没有缺陷，当然也不应该含有任何恶意的代码。这一假设会影响我们分析内核代码的方式。例如，许多内核内部函数（例如自旋锁）如果被内核代码错误地使用，当然会导致严重的问题。但我们总是假设内核正确地使用自己的函数。在硬件层面，RISC-V 的 CPU、RAM、磁盘等都按文档中宣传的那样运行，没有任何硬件上的缺陷。

> Real life is not so straightforward. It’s difficult to prevent abusive user programs from calling system calls in a way that makes the system unusable by consuming kernel-protected resources: disk space, CPU time, process table slots, etc. It’s usually impossible to write 100% bug-free kernel code or design bug-free hardware; if the writers of malicious user code are aware of kernel or hardware bugs, they will exploit them. Even in mature, widely-used kernels, such as Linux, people often discover previously-unknown vulnerabilities [1]. Finally, the distinction between user and kernel code is sometimes blurred: some privileged user-level processes may provide essential services and effectively

> be part of the operating system, and in some operating systems privileged user code can insert new code into the kernel (as with Linux's loadable kernel modules and eBPF).

现实情况并非如此简单。很难阻止滥用系统调用的用户程序通过消耗受内核保护的资源（例如磁盘空间、CPU 时间、进程表槽等）并导致系统不可用。编写 100% 无错误的内核代码或设计 100% 无错误的硬件通常是不可能的；如果恶意用户代码的编写者意识到内核或硬件的错误，他们就会利用这些错误。即使在 Linux 等成熟且广泛使用的内核中，人们也经常会发现以前未知的漏洞 [1]。最后，用户代码和内核代码之间的区别有时很模糊：一些特权用户级进程可能提供必要的服务，并有效地成为操作系统的一部分；而在某些操作系统中，特权用户代码可以将新代码插入内核（例如通过 Linux 的可加载内核模块和 eBPF）。

> As a partial defense against kernel bugs, xv6 code includes checks for inconsistencies and unrecoverable errors, and will "panic" in response, by calling `panic()`. This function prints an error message and halts the system. Panicking is not desirable, but is preferable to continuing execution. Typically a panic results from a kernel bug that causes kernel data to be incorrect or causes the kernel to perform an illegal action such as referencing non-existent memory; in such a situation it is safer to halt execution with `panic()` than to try to continue in an inconsistent state. A kernel developer would react to a panic by working to identify and fix the underlying code bug.

作为对内核错误的应对措施的一部分，xv6 代码包含对不一致和不可恢复错误的检查，并通过调用 `panic()` 来处理这些异常（译者注，这里我们称该动作为 "崩溃（Panicking）"）。此函数会打印一条错误消息并暂停系统。虽然这会导致系统崩溃，但总比继续执行要好。通常，崩溃是由内核错误引起的，该错误会导致内核数据不正确或导致内核执行非法操作（例如引用不存在的内存）；在这种情况下，使用 `panic()` 暂停执行比尝试在不一致的状态下继续执行更安全。内核开发人员要做的是根据崩溃时的输出修复底层代码错误。

2.8 现实世界（Real world）

> Most operating systems have adopted the process concept, and most processes look similar to xv6's. Modern operating systems, however, support several threads within a process, to allow a single process to exploit multiple CPUs. Supporting multiple threads in a process involves quite a bit of machinery that xv6 doesn't have, often including interface changes (e.g., Linux's `clone`, a variant of `fork`), to control which aspects of a process threads share.

大多数操作系统都采纳了进程的概念，并且大多数操作系统的进程看起来与 xv6 的很像。然而，现代操作系统支持在一个进程中创建多个线程，使得一个进程能够利用多个处理器。在一个进程中支持多个线程涉及许多新的机制以控制一个进程中多个线程之间可以共享哪些内容。这些机制 xv6 并没有实现，包括潜在的接口更改（例如，Linux 下的 `clone`，一个 `fork` 的变体）。

2.9 练习（Exercises）

> 1. Add a system call to xv6 that returns the amount of free memory available.

1. 为 xv6 增加一个系统调用，返回当前可用的内存的大小。

第 3 章 页表（Chapter 3 Page tables）

> Page tables are the most popular mechanism through which the operating system provides each process with its own private address space and memory. Page tables determine what memory addresses mean, and what parts of physical memory can be accessed. They allow xv6 to isolate different processes' address spaces and to multiplex them onto a single physical memory. Page tables provide a level of indirection that allows operating systems to perform many useful tricks. Xv6 performs a few: mapping the same memory (a trampoline page) in several address spaces, guarding kernel and user stacks with an unmapped page, and allocating user heap memory lazily. The rest of this chapter explains the page tables that the RISC-V hardware provides and how xv6 uses them.

“页表（Page tables）” 是操作系统为每个进程实现私有地址空间和内存的最常用的机制。理解页表的概念有助于我们理解内存地址的含义，页表也定义了物理内存中哪些部分可以被访问。xv6 利用页表隔离不同进程的地址空间，从而实现多个进程对单个物理内存空间的复用。页表是一种常用的设计，它提供了一层隔离（a level of indirection），方便操作系统基于该设计实现一些有趣的应用。xv6 利用页表实现了各种操作，譬如：它将同一个内存页（即下文中将介绍的 trampoline 页）映射到不同的地址空间中；用一个未映射的页保护用户栈和内核栈；延迟分配用户的堆内存，等等。本章的其余部分将介绍 RISC-V 硬件提供的页表机制以及 xv6 将如何使用它。

3.1 分页硬件（Paging hardware）

> As a reminder, RISC-V instructions (both user and kernel) manipulate virtual addresses. The machine's RAM, or physical memory, is indexed with physical addresses. The RISC-V page table hardware connects these two kinds of addresses, by mapping each virtual address to a physical address.

值得注意的是，RISC-V 指令（包括用户态指令和内核态指令）使用的都是虚拟地址，而机器的 RAM（物理内存）是通过物理地址访问的。RISC-V 的页表硬件通过将每个虚拟地址映射到物理地址来为这两种地址建立关联。

> Xv6 uses RISC-V's Sv39 mode, which means that only the bottom 39 bits of a 64-bit virtual address are used; the top 25 bits are not used. In this Sv39 configuration, a RISC-V page table is logically an array of 2^{27} (134,217,728) page table entries (PTEs). Each PTE contains a 44-bit physical page number (PPN) and some flags. The paging hardware translates a virtual address by using the top 27 bits of the 39 bits to index into the page table to find a PTE, and making a 56-bit physical address whose top 44 bits come from the PPN in the PTE and whose bottom 12 bits are copied from the original virtual address. Figure 3.1 shows this process with a logical view of the page table as a simple array of PTEs (the RISC-V page table is actually a tree; see Figure 3.2 for a fuller story). A page table gives the operating system control over virtual-to-physical address translations at the granularity of aligned chunks of 4096 (2^{12}) bytes. Such a chunk is called a page.

xv6 使用了 RISC-V 的 Sv39 模式，这意味着它只使用一个 64 位虚拟地址的低 39 位；高 25 位不使用。在这种 Sv39 模式下，RISC-V 页表在逻辑上可以看成是一个由 2^{27} 个 页表项（Page Table Entry，下文简称 PTE） 组成的数组，每个 PTE 中有 44 个比特位用于存放 “物理页编号（Physical Page Number，简称 PPN）”， 另外还包括一些标志位（Flags）。分页硬件使用虚拟地址中低 39 位中的高 27 位作为下标来索引页表，以找到该虚拟地址对应的一个 PTE，然后生成一个 56 位的物理地址，其高 44 位来自 PTE 中的 PPN，低 12 位来自对应虚拟地址的低 12 位。图 3.1 描述了这个过程，页表可以被看作一个简单的由 PTE 组成的数组（最终的 RISC-V 的页表实现是一棵树，更完整的介绍可以参见图 3.2）。页表使得操作系统能够以大小为 4096 (2^{12}) 字节的内存块为单元控制虚拟地址到物理地址的转换（同时注意这些内存单元必须按 4096 字节为边界进行对齐）。这样的内存块在术语上称为 页（page）。

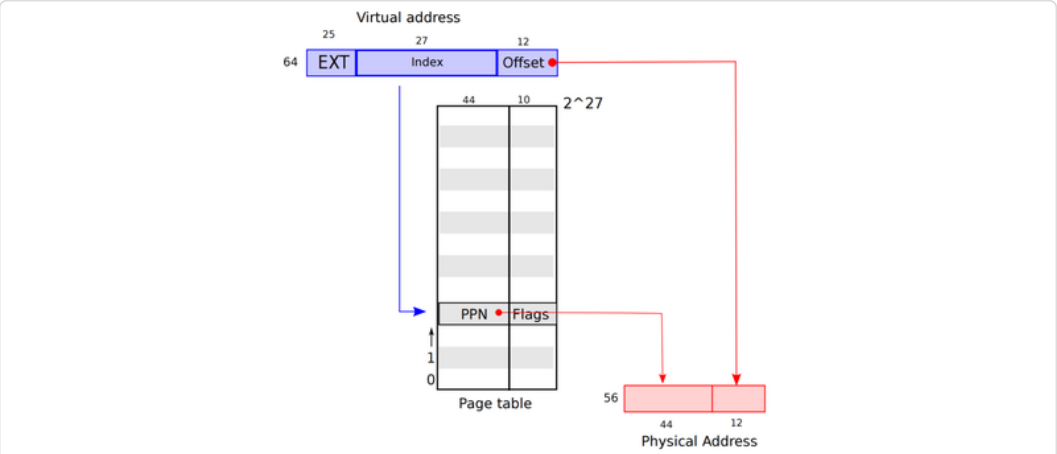

Figure 3.1: An abstract view of a flat page table mapping virtual to physical addresses.

> RISC-V’ s design leaves room for expansion of both virtual and physical addresses. If more virtual address space is needed, RISC-V supports an Sv48 mode, with 48-bit virtual addresses [3]. Physical addresses also have room for growth: there is room in the PTE format for the physical page number to grow by another 10 bits. The designers of RISC-V chose address sizes based on technology predictions. 2^{48} bytes is 262,144 GB, a much larger user virtual address space than any application is likely to use today. 2^{56} bytes of physical address space is 65,536 terabytes, much more RAM than any computer can currently be equipped with.

RISC-V 的设计为虚拟地址和物理地址的扩展预留了空间。如果需要更多虚拟地址空间，RISC-V 还支持 Sv48 模式，该模式具有 48 位虚拟地址 [3]。物理地址范围也有进一步扩展的空间，这体现在 PTE 中还有 10 个比特位没有使用。RISC-V 的设计者基于对未来技术上的考量确定了这些地址的长度。2^{48} 字节对应 262,144 GB（译者注：即 256 TB），这比当今任何应用程序可能使用的用户态虚拟地址空间都要大得多。2^{56} 字节的物理地址空间对应 65,536 TB（译者注：即 64 PB），这比目前任何计算机所能配备的 RAM 都要大得多（译者注，Sv39 和 Sv48 支持的虚拟地址空间大小不同，Sv39 是 512 GB (2^{39})，Sv48 是 256 TB (2^{48})，但 Sv39 和 Sv48 所支持的物理地址空间大小是相同的，都是 64 PB (2^{56})）。

> As Figure 3.2 shows, a RISC-V CPU page table is stored in physical memory as a three-level tree. The root of the tree is a 4096-byte page-table page that contains 512 PTEs, which

> contain the physical addresses for page-table pages in the next level of the tree. Each of those pages contains 512 PTEs for the final level in the tree. The paging hardware uses the top 9 bits of the 27 bits to select a PTE in the root page-table page, the middle 9 bits to select a PTE in a page-table page in the next level of the tree, and the bottom 9 bits to select the final PTE. (In Sv48 RISC-V a page table has four levels, and bits 39 through 47 of a virtual address index into the top-level.)

如图 3.2 所示，一个 RISC-V 的处理器页表以三级的树状结构存储在物理内存中。该树的根是一个大小为 4096 字节的 “页表页（page-table page）” （译者注：在图 3.2 中中标记为 “Page Directory”），包含了 512 个 PTE，每个 PTE 中存放了该树下一级页表页的物理地址信息。第二级的页表页同样每个页包含了 512 个 PTE，每一个 PTE 存放了该树最后一级（也就是第三级）页表页的物理地址信息。分页硬件使用虚拟地址中的 27 位（Index）中的最高的 9 个比特位（译者注：图 3.2 中 Virtual Address 中的 L2）在根（root）页表页中找到一个 PTE，中间的 9 个比特位（译者注：图 3.2 中 Virtual Address 中的 L1）在树的下一级页表页面中定位第二级 PTE，最低的 9 个比特位（译者注：图 3.2 中 Virtual Address 中的 L0）选择最后一级 PTE（对于 Sv48， 一个页表有四级，使用虚拟地址中的第 39 位到 47 位来索引根页表页。）。

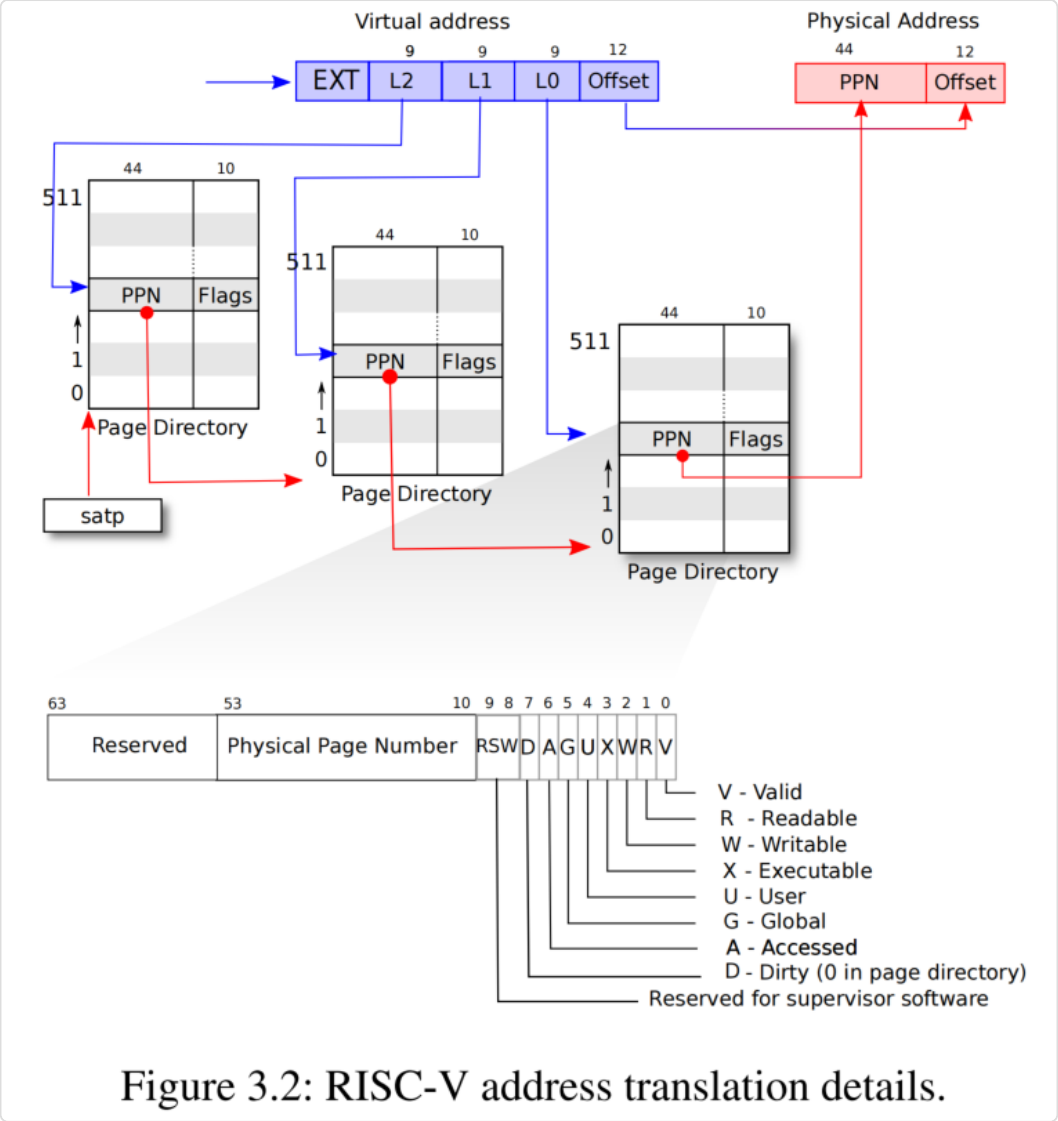

Figure 3.2: RISC-V address translation details.

> If any of the three PTEs required to translate an address is not present, the paging hardware raises a page-fault exception, leaving it up to the kernel to handle the page fault (see Chapter 4 and 5).

如果转换地址过程中所需的三个 PTE 中的任何一个不存在，分页硬件就会触发 缺页异常（page-fault exception），是否处理该异常取决于内核的行为（具体参见第 4 章和第 5 章的介绍）。

> The three-level structure of Figure 3.2 allows a memory-efficient way of recording PTEs, compared to the single-level design of Figure 3.1. In the common case in which large ranges of virtual addresses have no mappings, the three-level structure can omit entire page directories. For example, if an application uses only a few pages starting at address zero, then the entries 1 through 511 of the top-level page directory are invalid, and the kernel doesn’ t have to allocate pages those for 511 intermediate page directories. Furthermore, the kernel also doesn’ t have to allocate pages for the bottom-level page directories for those 511 intermediate page directories. So, in this example, the three-level design saves 511 pages for intermediate page directories and `511 × 512` pages for bottom-level page directories.

与图 3.1 的单级页表设计相比，图 3.2 的三级结构在存放 PTE 时更节省内存。考虑到通常情况下绝大部分的虚拟地址并不会实际映射物理内存，三级结构下我们可以省略很多存放 PTE 的 “页目录（Page Directory）”。举个例子，如果一个应用程序只使用了从地址 0 开始的有限的几个物理页，那么第一级页目录中只有第一项（编号为 0）的 PTE 有效，编号为 1 的 PTE 到编号为 511 的 PTE 都用不上，这也意味着内核不必为这 511 个 PTE 所对应的第二级页目录分配物理页。自然也更不必为这 511 个第二级页目录分配第三级的页目录。所以，以这个例子为例计算，采用三级页表设计后，我们节省了第二级页目录的 511 个物理页，以及 `511 x 512` 个第三级的页目录的内存。

> Although a CPU walks the three-level structure in hardware as part of executing a load or store instruction, a potential downside of three levels is that the CPU must load three PTEs from memory to perform the translation of the virtual address in the load/store instruction to a physical address. To avoid the cost of loading PTEs from physical memory, a RISC-V CPU caches page table entries in a Translation Look-aside Buffer (TLB).

每次执行 “加载（load）” 或者 “存储（store）” 指令时处理器都需要遍历三级页表结构将指令中的虚拟地址转换为物理地址，虽然这个操作完全由硬件负责执行，但缺点是每次都需要去内存中访问 PTE。为了减少从物理内存加载 PTE 的开销，RISC-V 处理器会将 PTE 缓存在 TLB（Translation Look-aside Buffer） 中。

> Each PTE contains flag bits that tell the paging hardware how the associated virtual address is allowed to be used. `PTE_V` indicates whether the PTE is present: if it is not set, a reference to the page causes a page fault (i.e., is not allowed). `PTE_R` controls whether instructions are allowed to read to the page. `PTE_W` controls whether instructions are allowed to write to the page. `PTE_X` controls whether the CPU may interpret the content of the page as instructions and execute them. `PTE_U` controls whether instructions in user mode are allowed to access the page; if `PTE_U` is not set, the PTE can be used only in supervisor mode. Figure 3.2 shows where the flag bits sit in a PTE. The flags and all other page hardware-related structures are defined in (0500)

每个 PTE 包含一些标志位，这些标志位用于告诉分页硬件如何访问和使用对应的虚拟地址。标志位 `PTE_V` 说明该条 PTE 是否有效：如果它没有被设置，对该页的引用会导致 page fault 异常（即不允许访问该页）。标志位 `PTE_R` 指示是否允许指令读取该页。标志位 `PTE_W` 控制是否允许指令写入该页。标志位 `PTE_X` 控制处理器是否可以从该页读取指令并解释执行。标志位 `PTE_U` 控制用户模式下的指令是否允许访问该页；如果没有设置 `PTE_U`，该 PTE 只能在管理员模式下被访问。图 3.2 显示了 PTE 中 flag 比特位的具体定义。标志位常量和所有其他与页硬件相关的结构体都定义在（0500）。

> To tell a CPU to use a page table, the kernel must write the physical address of the root page-table page into the satp register. A CPU will translate all addresses generated by subsequent instructions using the page table pointed to by its own `satp`. Each CPU has its own `satp` so that different CPUs can run different processes, each with a private address space described by its own page table.

为了告诉处理器使用页表，内核必须将 “根页表页（root page-table page）” 的物理地址写入到 `satp` 寄存器中。一旦 `satp` 寄存器被更新，该 CPU 对于此后执行的指令都将基于自己的 `satp` 所指向的页表对指令中的虚拟地址进行翻译（生成物理地址）。每个 CPU 都有自己的 `satp`，因此多个 CPU 可以运行不同的进程，每个进程都可以使用自己的页表从而拥有它们私有的地址空间。

> From the kernel’s point of view, a page table is data stored in memory, and the kernel creates and modifies page tables using code much like you might see for any tree-shaped data structure.

从内核的角度来看，一个页表和其他存储在内存中的数据并没有什么区别，内核像操作其他树状数据结构一样，用类似的代码来创建和访问页表。

> A few notes about terms used in this book. Physical memory refers to storage cells in RAM. A byte of physical memory has an address, called a physical address. Instructions that dereference addresses (such as loads, stores, jumps, and function calls) use only virtual addresses, which the paging hardware translates to physical addresses, and then sends to the RAM hardware to read or write storage. An address space is the set of virtual addresses that are valid in a given page table; each xv6 process has a separate user address space, and the xv6 kernel has its own address space as well. User memory refers to a process’s user address space plus the physical memory that the page table allows the process to access. Virtual memory refers to the ideas and techniques associated with managing page tables and using them to achieve goals such as isolation.

这里再强调一下本书中用到的一些术语。物理内存（Physical memory） 是指 RAM 中的存储单元。物理内存中的每个字节有一个地址，称为 物理地址（physical address）。指令中涉及地址时（譬如加载，存储，跳转以及函数调用）用的都是虚拟地址，分页硬件将其转换为物理地址，然后将其发送到 RAM 硬件去执行实际的读写。所谓的一个 地址空间（address space） 是指在一个给定页表中有效映射的一段虚拟地址范围；xv6 中每个进程都有一个独立的用户地址空间，而 xv6 的内核也同样有它自己的地址空间。用户内存（User memory） 指的是一个进程的用户地址空间以及进程的页表允许进程访问的物理内存范围。虚拟内存（Virtual memory） 则指的是通过管理和使用页表实现隔离的概念和技术。

3.2 内核地址空间（Kernel address space）

> When it starts, xv6 creates a single page table describing the kernel’ s address space. The kernel configures the layout of its address space to give itself access to physical memory and various hardware resources at predictable virtual addresses. Figure 3.3 shows how this layout maps kernel virtual addresses to physical addresses. The file (0200) declares the constants for xv6’ s kernel memory layout.

启动时，xv6 会创建一个单独的描述内核地址空间的页表。内核采用了一种 “可预期（predictable）” 的映射方式设定其地址空间的布局，方便自己通过虚拟地址访问物理内存和各种硬件资源。图 3.3 显示了在这种布局下内核如何将虚拟地址映射到物理地址。文件 (0200) 中定义了 xv6 内核内存布局相关的常量。

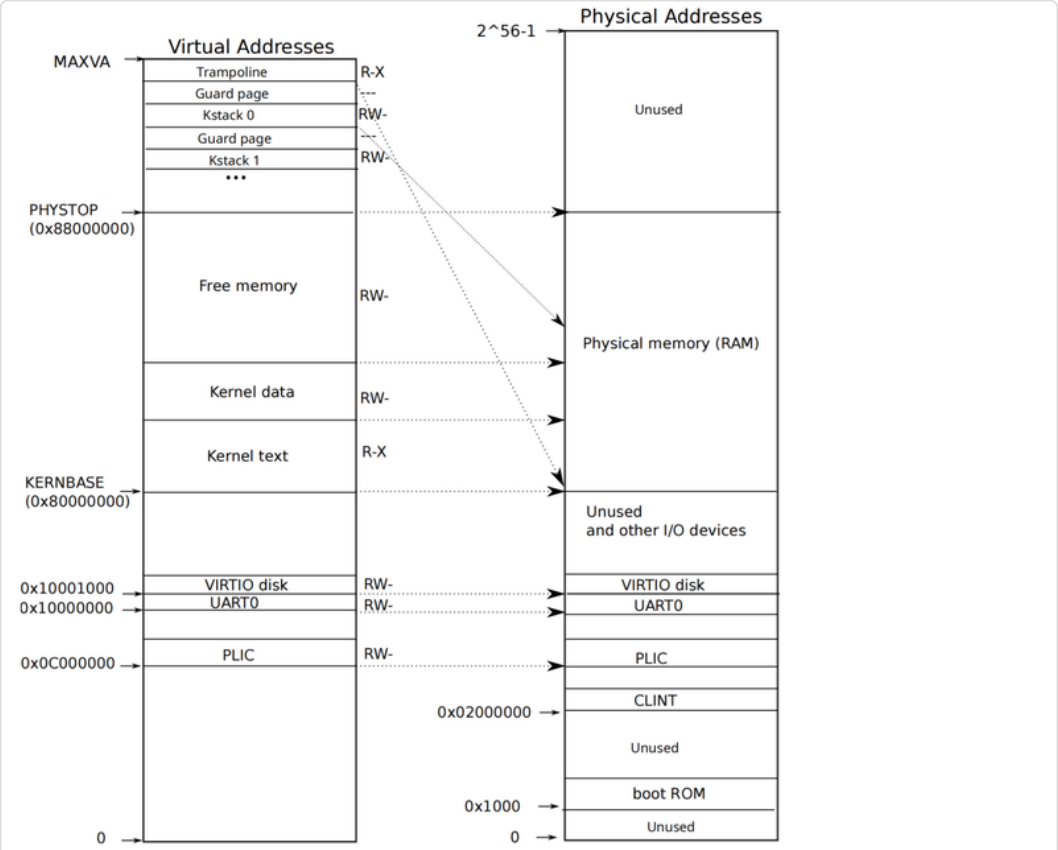

Figure 3.3: On the left, xv6’s kernel virtual address space. RWX refer to PTE read, write, and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.

> QEMU simulates a computer that includes RAM (physical memory) starting at physical address `0x80000000` and continuing through at least `0x88000000`, which xv6 calls `PHYSTOP`. The QEMU simulation also includes I/O devices such as a disk interface. QEMU exposes the device interfaces to software as memory-mapped control registers that sit below `0x80000000` in the physical address space. The kernel can interact with the devices by reading/writing these special physical addresses; such reads and writes communicate with the device hardware rather than with RAM. Chapter 4 explains how xv6 interacts with devices.

QEMU 模拟了一台计算机，这台计算机中的 RAM（物理内存）拥有一段连续的物理地址，其地址范围是从 `0x80000000` 开始，到 `0x88000000` 结束。对于这个结束地址，在 xv6 的代码中给它起了个名字叫 `PHYSTOP`。QEMU 模拟的对象还包括一些 I/O 设备，如磁盘接口。QEMU 将设备接口，即一些控制寄存器通过 内存映射（memory-mapped） 的方式提供给软件进行访问，这些寄存器的地址值位于物理地址 `0x80000000` 以下。内核可以通过读取和写入这些特殊的物理地址与设备交互；需要注意的是这里读取和写入访问的是设备硬件，而不 RAM。第 4 章解释了 xv6 如何与设备进行交互。

> The kernel maps all physical RAM and device registers at virtual addresses equal to the physical addresses. This is called “direct mapping,” and allows the kernel to read or write physical address `x` simply by loading or storing to virtual address `x`. The kernel code itself is located at `KERNBASE=0x80000000` in both the virtual address space and in physical memory. When `kfork` (2373) allocates user memory for the child process, the allocator returns the physical address of that memory; `kfork` uses that address directly as a virtual address when it is copying the parent’s user memory to the child.

内核将所有物理 RAM 和设备寄存器映射到与物理地址相等的虚拟地址。这称为 “直接映射（direct mapping）”，也就是说当内核对虚拟地址 `x` 进行读写时就等价于对物理地址 `x` 的读和写。内核的指令本身在虚拟地址空间和物理内存中都位于 `KERNBASE=0x80000000` 的位置。当 `kfork`（2373）为子进程分配用户内存时，分配器返回该内存的物理地址；当 `kfork` 将父进程的用户内存复制给子进程时，它会将该地址值当作虚拟地址直接使用。

> There are a couple of kernel virtual addresses that aren’t direct-mapped:
> - The trampoline page. It is mapped at the top of the virtual address space; user page tables have this same mapping. Chapter 4 discusses the role of the trampoline page, but we see here an interesting use case of page tables; a physical page (holding the trampoline code) is mapped twice in the virtual address space of the kernel: once at the top of the virtual address space and once with a direct mapping.
> - The kernel stack pages. Each process has its own kernel stack, which is mapped at a high kernel virtual address so that below it xv6 can leave an unmapped guard page. The guard page’s PTE is invalid (i.e., `PTE_V` is not set), so that if the kernel overflows a kernel stack, it will likely cause a page fault and the kernel will panic. Without a guard page an overflowing stack would overwrite other kernel memory, resulting in incorrect operation. A panic crash is preferable.

有几处内核虚拟地址没有采用直接映射的方式：
- “蹦床页(trampoline page)”。它被映射在虚拟地址空间的顶部；用户页表对其采用了相同的映射方式。第 4 章讨论了蹦床页的作用，但我们在这里看到了一个有趣的页表使用案例；一个（含有蹦床代码的）物理页在内核的虚拟地址空间中被映射了两次：一次在虚拟地址空间的顶部，一次是直接映射。
- 内核的栈页。每个进程都有一个自己的内核栈（译者注，这里指的是物理内存），它们被映射为位置偏高一些的内核虚拟地址，xv6 在每个内核栈对应的虚拟地址区间下面都预留了一个没有映射物理内存的虚拟页，我们称之为 保护页（guard page）。保护页的 PTE 是无效的（也就是说其比特位 `PTE_V` 没有设置），这样一旦内核栈溢出就会触发一个 page fault 异常，对该异常的内核处理结果就是内核 “崩溃（panic）”。如果没有保护页，栈溢出将会覆盖其他内核内存，导致不可预期的错误操作。相比来说，使内核直接崩溃更加可取。

> While the kernel uses its stacks via the high-memory mappings, each is also accessible to the kernel through a direct-mapped address. An alternate design might have just the direct mapping, and use the stacks at the direct-mapped address. In that arrangement, however, providing guard pages would involve unmapping virtual addresses that would otherwise refer to physical memory, which would then be hard to use.

虽然 xv6 内核在使用内核栈时采用了将其映射为较高虚拟地址的方式，但通过直接映射的方式也是可以的。譬如我们可以将所有的内核映射统一为直接映射映射方式，这样我们就可以通过直接映射的虚拟地址访问栈。然而，如果这么做的话，为了提供保护页将涉及取消映射虚拟地址的操作，这将增加实现的复杂度（译者注：这里原文的意思是说，直接映射方式下保护页所占用的虚拟地址空间也默认直接映射了物理页，为了不浪费这些物理内存，我们可能需要先取消这些保护页直接映射的物理内存再将它们重新映射到其他虚拟地址去访问）。

> The kernel maps the pages for the trampoline and the kernel text with the permissions `PTE_R` and `PTE_X`, but not `PTE_W`. The kernel maps other pages with the permissions `PTE_R` and `PTE_W`, but not `PTE_X`. The mappings for the guard pages are invalid. The purpose of these restricted permissions is to help catch kernel bugs that access pages in unexpected ways, for example if kernel code accidentally tried to write over kernel instructions.

内核在映射蹦床页和含有内核 “文本（text，即指令部分）” 的物理页时设置了 `PTE_R` 和 `PTE_X` 标志位，但没有设置 `PTE_W`。内核在映射其他物理页时设置了 `PTE_R` 和 `PTE_W` 标志位，但没有设置 `PTE_X`。对于保护页则采用了无效的映射（译者注：指设置 `PTE_V` 标志位为 0）。这些对访问权限进行限制的目的是用于捕获那些内核中非法访问物理页的错误，例如，内核中的代码在执行过程中可能会意外覆盖了内核指令所占的内存区。

> The kernel creates a single kernel page table, used by all CPUs when they execute in the kernel. xv6 does not modify the kernel page table after initially creating it.

内核会单独创建一个内核页表，所有 CPU 在内核中执行时都会使用该页表。xv6 在最初创建好内核页表后就不会再修改它。

3.3 代码讲解：创建一个地址空间（Code: creating an address space）

> Please read `kernel/vm.c` through the end of `mappages()` before proceeding.

在继续之前，请阅读 `kernel/vm.c` 中直到 `mappages()` 结尾的代码。

> Most of the xv6 code for manipulating address spaces and page tables resides in `vm.c` (1400). The central data structure is `pagetable_t`, which is really a pointer to a RISC-V root page-table page; a `pagetable_t` may be either the kernel page table, or one of the per-process page tables. The central functions are `walk`, which finds the PTE for a virtual address, and `mappages`, which installs PTEs for new mappings. Functions starting with `kvm` manipulate the kernel page table; functions starting with `uvm` manipulate a user page table; other functions are used for both. `copyout` and `copyin` copy data to and from user

> virtual addresses provided as system call arguments; they are in `vm.c` because they need to explicitly translate those addresses in order to find the corresponding physical memory.

xv6 中大多数用于操作地址空间和页表的代码都在 `vm.c` (1400) 中。其核心数据结构是 `pagetable_t`，它的类型实际上是指向 RISC-V 的 “根页表页（root page-table page）” 的指针；一个 `pagetable_t` 可以是内核页表（译者注：即 `kernel_pagetable`），也可以是某个进程的页表（译者注：文中也叫 “用户页表（user page table）”，对应 `struct proc` 中的 `pagetable`）。最核心的函数是 `walk` 和 `mappages`，前者根据虚拟地址找到对应的 PTE，后者为新的映射关系创建 PTE。以 `kvm` 为前缀的函数用于操作内核页表；以 `uvm` 为前缀的函数用于操作用户页表；其他函数则可能两者兼而有之。 `copyout` 和 `copyin` 这一对函数用于将数据从内核复制到给定的用户虚拟地址或反过来从给定的用户虚拟地址复制数据到内核，这里给定的虚拟地址一般来自系统调用传入的参数（译者注：对应 `copyout` 函数的参数 `dstva` 以及 `copyin` 函数的参数 `dst`）；由于这两个函数需要显式地翻译这些地址，以便找到对应的物理内存，因此我们把这两个函数定义在 `vm.c` 中。

> Early in the boot sequence, `main` calls `kvminit` (1465) to create the kernel’s page table using `kvmmake` (1421). This call occurs before xv6 has enabled paging on the RISC-V, so addresses refer directly to physical memory. `kvmmake` first allocates a page of physical memory to hold the root page-table page. Then it calls `kvmmap` to install the translations that the kernel needs. The translations include the kernel’s instructions and data, physical memory up to `PHYSTOP`, and memory ranges which are actually devices. `proc_mapstacks` (2132) allocates a kernel stack for each process. It calls `kvmmap` to map each stack at the virtual address generated by `KSTACK`, which leaves room for the invalid stack-guard pages.

在启动的早期阶段， `main` 调用 `kvminit` (1465)，而 `kvminit` 又继续调用 `kvmmake` (1421) 创建内核的页表。由于此时 xv6 还未在 RISC-V 上开启分页，因此这些函数中的指令直接引用物理内存地址。 `kvmmake` 首先分配一个物理页来保存根页表页。然后它调用 `kvmmap` 来设置内核的映射。映射的内容包括内核的指令区和数据区（上限直到物理内存的 `PHYSTOP` 处），以及对应设备 I/O 的内存地址范围。（ `kvmmake` 中所调用的） `proc_mapstacks` (2132) 为每个进程分配一个内核栈。它调用 `kvmmap` 将每个栈映射到由宏 `KSTACK` 计算得到的虚拟地址处， `KSTACK` 的计算结果为无效的栈保护页留出空间。

> `kvmmap` (1457) calls `mappages` (1556), which installs mappings into a page table for a range of virtual addresses to a corresponding range of physical addresses. It does this separately for each virtual address in the range, at page intervals. For each virtual address to be mapped, `mappages` calls `walk` to find the address of the PTE for that address. It then initializes the PTE to hold the relevant physical page number, the desired permissions (`PTE_W`, `PTE_X`, and/or `PTE_R`), and `PTE_V` to mark the PTE as valid (1577).

`kvmmap` (1457) 调用 `mappages` (1556)， `mappages` 为一段范围内的虚拟地址和物理地址之间建立映射关系并将这些映射关系保存到页表中。它以页大小为单位（译者注：即代码中的 `PGSIZE`），为范围内的所有虚拟地址逐个建立映射。对于要映射的每个虚拟地址， `mappages` 调用 `walk` 来找到该地址的 PTE 地址。然后，它初始化 PTE 以保存相关的物理页号，所需的权限（ `PTE_W`、 `PTE_X` 和/或 `PTE_R`）以及用于标记 PTE 是否有效的 `PTE_V` (1577)。

> `walk` (1497) mimics the RISC-V paging hardware as it looks up the PTE for a virtual address (see Figure 3.2). `walk` descends the page table tree one level at a time, using each level’s 9 bits of virtual address to index into the relevant page directory page. At each level it finds

> either the PTE of the next level's page directory page, or the PTE of final page (1503). If a PTE in a first or second level page directory page isn't valid, then the required directory page hasn't yet been allocated; if the `alloc` argument is set, `walk` allocates a new page-table page and puts its physical address in the PTE. It returns the address of the PTE in the lowest layer in the tree (1513).

在根据虚拟地址查找对应的 PTE 时（参见图 3.2），`walk` (1497) 的实现模仿了 RISC-V 的分页硬件的行为。`walk` 从根页表开始逐层往下搜索，一次从 3 级页表中获取 9 个比特位用于在对应层级的页目录页中定位表项。在每一层中它将获得下一级页目录页对应的 PTE 或叶子物理页的 PTE (1503)。如果第一级或者第二级页目录页中的 PTE 无效，则说明所需的目录页还没有分配；如果 `alloc` 参数值为 1，`walk` 就会分配一个新的页表页，并将其物理地址记录在 PTE 中。它返回树中最低一级的 PTE 的地址 (1513)。

> The above code depends on physical memory being direct-mapped into the kernel virtual address space. For example, as `walk` descends levels of the page table, it pulls the (physical) address of the next-level-down page table from a PTE (1505), and then uses that address as a virtual address to fetch the PTE at the next level down (1503).

上面的代码可以工作的前提是内核中虚拟地址和物理地址采用的是直接映射的方式。例如，当 `walk` 逐级遍历页表时，它从 PTE (1505) 中提取下一级页表的（物理）地址，然后直接使用该地址值作为虚拟地址来获取更下一级的 PTE (1503)。

> On each CPU, `main` calls `kvminithart` (1473) to install the kernel page table, placing the physical address of the root page-table page into the CPU's `satp` register. After this the CPU translates addresses using the kernel page table. The kernel continues to execute correctly because the kernel page table is direct-mapped, so that addresses refer to the same locations in RAM before and after this change.

在每个 CPU 上，`main` 调用 `kvminithart` (1473) 来安装内核页表，该函数将根页表页面的物理地址放入 CPU 的 `satp` 寄存器中。至此开始 CPU 将基于内核页表进行地址转换。在此操作之后内核之所以可以继续正确执行，正是因为内核页表采用的是直接映射方式，虚拟地址指向的 RAM 位置并没有发生改变。

> Each RISC-V CPU caches page table entries in a Translation Look-aside Buffer (TLB), and when xv6 changes a page table, it must tell the CPU to invalidate corresponding cached TLB entries. If it didn't, then at some point later the TLB might use an old cached mapping, pointing to a physical page that in the meantime has been allocated to another process, and as a result, a process might be able to scribble on some other process's memory. The RISC-V has an instruction `sfence.vma` that flushes the current CPU's TLB. Xv6 executes `sfence.vma` in `kvminithart` after reloading the `satp` register, and in the trampoline code for `uservec` and `userret`.

每个 RISC-V CPU 都会将 PTE 缓存在 Translation Look-aside Buffer (TLB) 中。当 xv6 切换页表时，它必须告知 CPU 将 TLB 中缓存的相关 PTE 失效。否则，TLB 稍后可能会使用旧的缓存的映射，而这些无效的映射可能会指向此时已分配给其他进程的物理页，其结果可能会造成，一个进程在其他进程的内存上执行写入操作。RISC-V 有一个指令 `sfence.vma`，用于刷新当前 CPU 的 TLB。在 `kvminithart` 中 xv6 在重新加载 `satp` 寄存器之后会执行 `sfence.vma`，此外在 trampoline 相关代码中的 `uservec` 和 `userret` 中也有类似操作。

> It is also necessary to issue `sfence.vma` before changing `satp`, in order to wait for completion of all outstanding loads and stores. This wait ensures that preceding updates to the page table have completed, and ensures that preceding loads and stores use the old page table, not the new one.

同样，也有必要在更改 satp 之前执行 `sfence.vma` 指令，以等待所有未完成的加载和存储操作完成。此等待可确保先前对页表的更新已完成，并确保先前的加载和存储操作使用的是旧页表，而不是新页表。

3.4 物理内存分配（Physical memory allocation）

> The kernel must allocate and free physical memory at run-time for page tables, user memory, kernel stacks, and pipe buffers.

内核必须在运行时为页表、用户内存、内核栈和管道缓冲区分配和释放物理内存。

> Xv6 uses the physical memory between the end of the kernel and `PHYSTOP` for run-time allocation. It allocates and frees whole 4096-byte pages at a time. It keeps track of which pages are free by threading a linked list through the pages themselves. Allocation consists of removing a page from the linked list; freeing consists of adding the freed page to the list.

xv6 将从内核自身占用内存空间的末尾到 `PHYSTOP` 之间的物理内存用于运行期间的内存分配。它一次分配或者释放一整个物理页（4096 字节）。为了方便跟踪哪些物理页可用，xv6 使用链表将所有的物理页串联起来。分配时从链表中移除一个物理页；释放时将被释放的物理页添加回到链表中。

> Please read `kernel/kalloc.c`.

请阅读 `kernel/kalloc.c` 中的代码。

3.5 代码讲解：物理内存分配器（Code: Physical memory allocator）

> The allocator resides in `kalloc.c` (2950). The allocator's data structure is a free list of physical memory pages that are available for allocation. Each free page's "next" pointer resides in a `struct run` (2966). The allocator stores each free page's `run` structure in the free page itself, since there's nothing else stored there while the page is free. The free list is protected by a spin lock (2970-2973). The list and the lock are wrapped in a struct to make clear that the lock protects the fields in the struct. For now, ignore the lock and the calls to `acquire` and `release`; Chapter 7 will examine locking in detail.

"分配器 (allocator)" 位于 `kalloc.c` (2950) 中。分配器的数据结构设计是将可供分配的物理内存页组织为一个 空闲链表（free list）。这个链表的每个元素的 next 指针存放在一个结构体 `struct run` (2966) 中。分配器将每个空闲页的 `run` 结构就存放在这个空闲页自身中，因为当这个页没有被分配使用时该页并没有存储其他信息。空闲链表受到 "自旋锁（spin lock）" 的保护 (2970-2973)。链表和锁被封装在一个结构体中，以明确这个锁就是用于保护其所在结构体中的链表成员。让我们先忽略锁以及代码中对 `acquire` 和 `release` 的调用；第 7 章将详细介绍锁。

> The function `main` calls `kinit` to initialize the allocator (2976). `kinit` initializes the free list to hold every page of physical RAM between the end of the kernel and `PHYSTOP`. Xv6 ought to determine how much physical memory is available by parsing configuration information provided by the hardware. Instead xv6 assumes that the machine has 128 megabytes of RAM. `kinit` calls `freerange` to add memory to the free list via per-page calls to `kfree`. A PTE can only refer to a physical address that is aligned on a 4096-byte boundary (is a multiple of 4096), so `freerange` uses `PGROUNDUP` to ensure that it frees only aligned physical addresses. The allocator starts with no memory; these calls to `kfree` give it some to manage.

`main` 函数调用 `kinit` 来初始化分配器 (2976)。`kinit` 负责初始化空闲链表，可用的物理内存范围从内核结束的位置（译者注：即 `end[]`）一直到 `PHYSTOP` 之间，以页（译者注：`PGSIZE`）为单位进行划分。更严格的做法，xv6 应该通过解析硬件提供的配置信息来确定究竟有多少物理内存可用。而 xv6 的做法是直接假定机器有 128M 字节的内存。`kinit` 通过调用 `freerange` 将内存添加到空闲链表中，`freerange` 的实现是针对每个物理页调用 `kfree`。PTE 的格式定义要求对应的物理地址必须是按 4096 字节边界对齐（即物理地址必须是 4096 的倍数），所以 `freerange` 通过使用 `PGROUNDUP` 来确保它释放的物理页地址满足对齐要求。通过对这些物理页逐个调用 `kfree`，将它们加入分配器中。

> The allocator sometimes treats addresses as integers in order to perform arithmetic on them (e.g., traversing all pages in `freerange`), and sometimes uses addresses as pointers to read and write memory (e.g., manipulating the `run` structure stored in each page); this dual use of addresses is the main reason that the allocator code is full of C type-casts.

分配器有时将地址视为整数，以便对其执行算术运算（例如，在 `freerange` 中遍历所有物理页），有时将地址用作读写内存的指针（例如，访问存储在每个物理页中的 `run` 结构体）；这种对地址的双重使用导致分配器代码中多处使用 C 语言的类型转换。

> The function `kfree` (3005) begins by setting every byte in the memory being freed to the value 1. This will cause code that uses memory after freeing it (uses "dangling references") to read garbage instead of the old valid contents; hopefully that will cause such code to break faster. Then `kfree` prepends the page to the free list: it casts `pa` to a pointer to `struct run`, records the old start of the free list in `r->next`, and sets the free list equal to `r`. `kalloc` removes and returns the first element in the free list.

函数 `kfree` (3005) 首先将需要释放的物理页中的每一个字节的值设置为 1。这么做的目的是将原先内存中的信息擦除，确保一旦出现 "悬空引用（dangling references）"，即一块内存被释放后未经分配而被访问的情况下不会暴露原先存放的有效信息，同时也希望籍此让错误的代码更快崩溃。然后 `kfree` 将物理页插入到空闲链表的头部，具体做法是：先将 `pa` 强制转换为一个指向 `struct run` 的指针 `r`，然后将空闲链表原来的头部节点地址记录在 `r->next` 中，最后将空闲链表的头节点指向 `r`。`kalloc` 负责删除并返回空闲链表中的第一个元素。

3.6 进程地址空间（Process address space）

> Each process has its own page table, and when xv6 switches between processes, it also changes page tables. Figure 3.4 shows a process's address space in more detail than Figure 2.3. A process's user address space starts at zero and in principle ends at `MAXVA`

> `(0x4000000000)` (0896), though in practice only a small fraction of this is mapped to physical memory.

每个进程都有它自己的页表，当 xv6 在进程之间切换时，也会更改页表。相比于图 2.3，图 3.4 在进程的地址空间上给出了更详细的描述。进程的用户地址空间从零开始，原则上结束于 `MAXVA (0x4000000000)` (0896)，但实际上只有一小部分被映射为物理内存。

> A process’ s address space consists of pages that contain the text of the program (which xv6 maps with the permissions `PTE_R` , `PTE_X` , and `PTE_U`), pages that contain the pre-initialized data of the program, a page for the stack, and pages for the heap. Xv6 maps the data, stack, and heap with the permissions `PTE_R` , `PTE_W` , and `PTE_U` .

一个进程的地址空间由包含程序指令的物理页（xv6 映射时会赋予 `PTE_R`、`PTE_X` 和 `PTE_U` 权限），包含已初始化数据的物理页，一个用于栈的物理页以及用于堆的多个物理页组成。xv6 映射数据、栈和堆时指定 `PTE_R`、`PTE_W` 和 `PTE_U` 权限。

> Using permissions within a user address space is a common technique to harden a user process. If the text were mapped with `PTE_W` , then a process could accidentally modify its own program; for example, a programming error may cause the program to write to a null pointer, modifying instructions at address 0, and then continue running, perhaps creating more havoc. To detect such errors immediately, xv6 maps the text without `PTE_W` ; if a program accidentally attempts to store to address 0, the hardware will refuse to execute the store and raises a page fault (see Chapter 4). The kernel then kills the process and prints out an informative message so that the developer can track down the problem.

在用户地址空间中映射时指定权限是保证用户进程安全的常用技术。如果文本段所在内存映射时指定了 `PTE_W`，则进程可能会意外修改其自身的程序；例如，代码编程错误可能导致程序向空指针所指向的位置写数据，从而修改地址 0 处的指令，允许这样的程序继续运行，甚至可能造成更大的破坏。为了立即检测到此类错误，xv6 映射文本段所在内存区域时不使用 `PTE_W`；如果程序意外尝试往地址 0 处写入，硬件将拒绝执行该写操作并引发 “页错误（page fault）”（参见第 4 章）。内核会终止该进程并打印一条信息，以便开发人员追踪该问题。

> Similarly, by mapping data without `PTE_X` , a user program cannot accidentally jump to an address in the program’ s data and start executing at that address.

类似地，映射数据段时不指定 `PTE_X` 权限，这样用户程序就不会意外跳转到程序数据段所在的地址并从该地址开始执行。

> In the real world, hardening a process by setting permissions carefully also aids in defending against security attacks. An adversary may feed carefully-constructed input to a program (e.g., a Web server) that triggers a bug in the program in the hope of turning that bug into an exploit [14]. Setting permissions carefully and other techniques, such as randomizing of the layout of the user address space, make such attacks harder.

在现实世界中，通过小心地设置权限来加固进程也有助于防御安全攻击。攻击者可能会向程序（例如，Web 服务器）输入精心构造的输入，尝试触发程序中的错误并将该错误转化为可利用的漏洞 [14]。通过谨慎设置权限以及其他技术（例如随机化用户地址空间的布局）可以提高此类攻击的难度系数。

> The stack is a single page, and is shown with the initial contents as created by the `exec` system call. Strings containing the command-line arguments, as well as an array of pointers to them, are at the very top of the stack. Just under that are values that allow a program to start at `main` as if the function `main(argc, argv)` had just been called.

xv6 为用户栈分配了一个物理页，（图 3.4）上显示了执行系统调用 `exec` 时栈上的初始内容。这包含存放命令行参数的字符串以及指向这些参数的指针数组，它们都位于栈的最顶部。在其下方是允许程序从 `main` 开始执行的值，栈的布局就好像刚刚调用了函数 `main(argc, argv)` 一样。

> To detect a user stack overflowing the allocated stack memory, xv6 places an inaccessible guard page right below the stack by clearing the `PTE_U` flag. If the user stack overflows and the process tries to use an address below the stack, the hardware will generate a page-fault exception because the guard page is inaccessible to a program running in user mode. A real-world operating system might instead automatically allocate more memory for the user stack when it overflows.

为了检测用户栈是否溢出，xv6 在栈的正下方放置一个不可访问的保护页，同时清除了该保护页的 `PTE_U` 标志。如果用户栈溢出，即进程尝试访问栈区下方的虚拟地址时，将触发硬件产生 page-fault 异常，因为该保护页被禁止在用户模式下被程序访问。真实世界中的操作系统也可能会在用户栈溢出时自动为其分配更多内存。

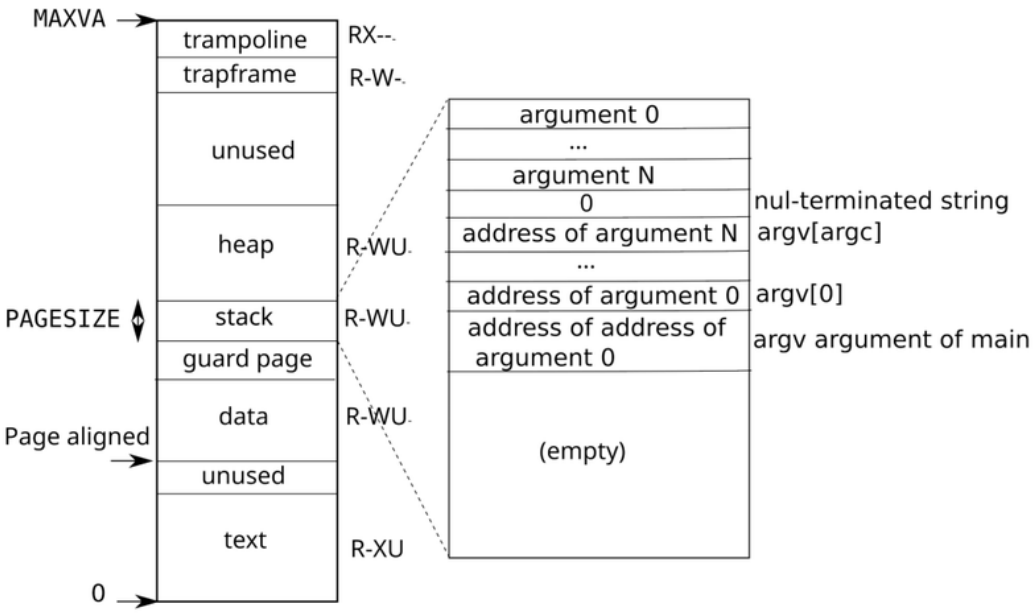

Figure 3.4: A process's user address space, with its initial stack.

> We see here a few nice examples of use of page tables. First, different processes' page tables translate user addresses to different pages of physical memory, so that each process has private user memory. Second, each process sees its memory as having contiguous virtual addresses starting at zero, while the process's physical memory can be non-contiguous. Third, the kernel maps a page with trampoline code at the top of the user

> address space (without `PTE_U`), thus a single page of physical memory shows up in all address spaces, but can be used only by the kernel.

我们在这里看到了一些非常好的使用页表的例子。首先，不同进程的页表将用户地址转换为不同的物理内存页，这样每个进程就都拥有了私有的用户内存。其次，每个进程都将其内存视为具有从零开始的连续虚拟地址，而进程的物理内存可以是不连续的。第三，内核将一个包含 “蹦床（trampoline）” 代码的物理页映射到用户地址空间的顶部（但未设置 `PTE_U`），这样，所有进程地址空间中都映射了这个物理内存页，可是这个页却只能被内核访问。

3.7 代码讲解：exec（Code: exec）

> Please read `kernel/exec.c` and `kernel/vm.c` starting at `uvmcreate()`.

请阅读 `kernel/exec.c` 中的代码以及 `kernel/vm.c` 中从 `uvmcreate()` 开始的代码部分。

> `exec` is a system call that replaces a process’ s user address space with data read from a file, called a binary or executable file. A binary is typically the output of the compiler and linker, and holds machine instructions and program data. `kexec` (6426), the kernel’ s internal implementation of `exec`, opens the named binary `path` using `namei` (6440), which is explained in Chapter 10. Then, it reads the ELF header. Xv6 binaries are formatted in the widely-used ELF format, defined in (0950). An ELF binary consists of an ELF header, `struct elfhdr` (0955), followed by a sequence of program section headers, `struct proghdr` (0974). Each `proghdr` describes a section of the application that must be loaded into memory; xv6 programs have two program section headers: one for instructions and one for data.

`exec` 是一个系统调用，它将进程用户地址空间的内容替换为从一个二进制的可执行文件中读取的数据。这个二进制文件通常是利用编译器和链接器产生，包含了机器指令和程序数据。`kexec`（6426），作为 `exec` 系统调用的内核内部实现，通过调用 `namei` (6440) 打开参数 `path` 指定的二进制文件，`namei` 具体将在第 10 章中解释。然后，`kexec` 读取文件的 ELF 头。xv6 的二进制文件采用广泛使用的 ELF 格式，这个格式定义在 (0950) 文件中。ELF 二进制文件由一个 ELF 头（采用 `struct elfhdr` (0955) 表示）和一系列 “程序节头（program section header）”（采用 `struct proghdr` (0974) 表示）组成。每个 `proghdr` 描述应用程序中必须加载到内存中的部分；xv6 程序有两个程序节头：一个包含指令，一个包含数据。

> The first step is a quick check that the file probably contains an ELF binary. An ELF binary starts with the four-byte “magic number” `0x7F`, `'E'`, `'L'`, `'F'`, or `ELF_MAGIC` (0952). If the ELF header has the right magic number, `kexec` assumes that the binary is well-formed.

（`kexec` 函数中读入文件后）第一步是快速检查该文件是否符合 ELF 二进制格式。ELF 二进制文件的最前部是一个四字节的 “魔数（magic number）” `0x7F`、`'E'`、`'L'`、`'F'`，即代码 (0952) 中定义的宏 `ELF_MAGIC`。如果 ELF 头包含正确的魔数，`kexec` 则假定该二进制文件格式满足 ELF 标准要求。

> `kexec` allocates a new page table with no user mappings with `proc_pagetable` (6454), allocates memory for each ELF segment with `uvmalloc` (6470), and loads each segment into memory with `loadseg` (6409). `loadseg` uses `walkaddr` to find the physical address of

> the allocated memory at which to write each page of the ELF segment, and `readi` to read from the file.

接下来 `kexec` 调用函数 `proc_pagetable` (6454) 创建一个新页表（还没有映射任何的用户内存），然后调用 `uvmalloc` (6470) 为每个 ELF “段（segment）” 分配内存，并调用 `loadseg` (6409) 将每个段加载到内存中。`loadseg` 函数中使用 `walkaddr` 来查找分配内存的物理地址，在该地址写入 ELF 段的每一页，并使用 `readi` 从文件中读取数据。

> The program section header for `/init`, the first user program created with `exec`, looks like this:

下面是 “/init” 的程序节头信息，它是 xv6 通过调用 `exec` 函数加载的第一个用户程序（译者注，这是似乎是写错了，应该是通过 `kexec` 函数加载的第一个用户程序）：

```
objdump -p user/_init
user/_init: file format elf64-little
Program Header:
0x70000003 off 0x0000000000006bb0 vaddr 0x0000000000000000
                                  paddr 0x0000000000000000 align 2**0
         filesz 0x000000000000004a memsz 0x0000000000000000 flags r--
    LOAD off 0x0000000000001000 vaddr 0x0000000000000000
                                  paddr 0x0000000000000000 align 2**12
         filesz 0x0000000000001000 memsz 0x0000000000001000 flags r-x
    LOAD off 0x0000000000002000 vaddr 0x0000000000001000
                                  paddr 0x0000000000001000 align 2**12
        filesz 0x0000000000000010 memsz 0x0000000000000030 flags rw
   STACK off 0x0000000000000000 vaddr 0x0000000000000000
                                  paddr 0x0000000000000000 align 2**4
        filesz 0x0000000000000000 memsz 0x0000000000000000 flags rw-
```

> We see that the text segment should be loaded at virtual address 0x0 in memory (without write permissions) from content at offset 0x1000 in the file. We also see that the data should be loaded at address 0x1000, which is at a page boundary, and without executable permissions.

我们看到，文本段（位于文件中偏移 0x1000 处）会被加载到内存中的虚拟地址 0x0 处（无写入权限）。我们还看到，数据段会被加载到地址 0x1000 处，该地址和页边界对齐，并且无可执行权限。

> A program section header’s `filesz` may be less than the `memsz`, indicating that the gap between them should be filled with zeroes (for C global variables) rather than read from the file. For `/init`, the data `filesz` is 0x10 bytes and `memsz` is 0x30 bytes, and thus `uvmalloc` allocates enough physical memory to hold 0x30 bytes, but reads only 0x10 bytes from the file `/init`.

一个程序节头的 `filesz` 的值可能小于 `memsz` 的值，这表明（在内存中）这些节之间的空隙会被零填充（对于 C 全局变量），而不是从文件中读取。对于 `/init`，数据段的 `filesz` 的值为 0x10（单位是字节），`memsz` 的值为 0x30，因此 `uvmalloc` 会分配足够的物理内存来容纳 0x30 个字节，但只会从文件 `/init` 中读取 0x10 个字节。

> Now `kexec` allocates and initializes the user stack. It allocates just one stack page. `kexec` copies the argument strings to the top of the stack one at a time, recording the pointers to

> them in `ustack`. It places a null pointer at the end of what will be the `argv` list passed to `main`. The values for `argc` and `argv` are passed to `main` through the system-call return path: `argc` is passed via the system call return value, which goes in `a0`, and `argv` is passed through the `a1` entry of the process's trapframe.

接下来 `kexec` 分配并初始化用户栈。它只会为栈分配一个物理页的内存。`kexec` 依次将所有参数的字符串复制到栈的顶部，并将指向它们的指针记录在 `ustack`（数组）中。它将一个空指针放置在将要传递给 `main` 的 `argv` 列表的末尾。`argc` 和 `argv` 的值通过系统调用返回路径传递给 `main`：`argc` 通过系统调用返回值传递，该返回值位于寄存器 `a0` 中，而 `argv` 则通过进程 "陷阱帧（trapframe）" 中的 `a1` 成员传递。

> `kexec` places an inaccessible page just below the stack page, so that programs that try to use more than one page will fault. This inaccessible page also allows `kexec` to deal with arguments that are too large; in that situation, the `copyout` (1754) function that `kexec` uses to copy arguments to the stack will notice that the destination page is not accessible, and will return -1.

`kexec` 在紧挨着栈页下方放置了一个不可访问的页，这样如果程序在使用栈的过程中越界就会报错。这个不可访问的页使得 `kexec` 能够处理参数过长的问题；在这种情况下，`kexec` 在调用 `copyout` (1754) 将参数从内核复制到用户栈的过程中中能检测出来目标页不可访问，从而返回 -1。

> During the preparation of the new memory image, if `kexec` detects an error like an invalid program segment, it jumps to the label `bad`, frees the new image, and returns -1. `kexec` must wait to free the old image until it is sure that the system call will succeed: if the old image is gone, the system call cannot return -1 to it. The only error cases in `kexec` happen during the creation of the image. Once the image is complete, `kexec` can commit to the new page table (6531) and free the old one (6535).

在准备新的进程页表期间，如果 `kexec` 检测到错误（例如无效的程序段），它会跳转到标签 `bad` 处，释放新的进程页表并返回 `-1`。`kexec` 必须在系统调用执行成功，也就是函数的最后才能释放旧的页表，如果过早释放，系统调用连返回 `-1` 都会有问题。`kexec` 中唯一可能失败的地方发生在页表创建过程中。新的页表创建成功后，`kexec` 可以为进程安装新的页表 (6531) 并释放旧的页表 (6535)。

> The `exec` system call loads bytes from the ELF file into memory at addresses specified by the ELF file. Users or processes can place whatever addresses they want into an ELF file. Thus `exec` is risky, because the addresses in the ELF file may refer to the kernel, accidentally or on purpose. The consequences for an unwary kernel could range from a crash to a malicious subversion of the kernel's isolation mechanisms (i.e., a security exploit). Xv6 performs a number of checks to avoid these risks. For example `if(ph.vaddr + ph.memsz < ph.vaddr)` checks for whether the sum overflows a 64-bit integer. The danger is that a user could construct an ELF binary with a `ph.vaddr` that points to a user-chosen address, and `ph.memsz` large enough that the sum overflows to 0x1000, which will look like a valid value. In an older version of xv6 in which the user address space also contained the kernel (but not readable/writable in user mode), the user could choose an address that corresponded to kernel memory and would thus copy data from the ELF binary into the kernel. In the RISC-V version of xv6 this cannot happen, because the kernel has its own

> separate page table; `loadseg` loads into the process’ s page table, not in the kernel’ s page table.

系统调用 `exec` 将 ELF 文件中的内容加载到 ELF 文件中指定的内存地址。这些地址可以在 ELF 文件中任意指定。因此，执行 `exec` 存在风险，因为 ELF 文件中的地址可能指向内核（无论是有意的还是无意的）。如果内核不仔细检查，轻则崩溃，重则可能导致内核的隔离机制被恶意破坏（即发生安全漏洞）。xv6 会执行一系列检查来避免这些风险。例如，`if(ph.vaddr + ph.memsz < ph.vaddr)` 这行代码会检查求和计算结果针对一个 64 位整数是否会溢出。风险在于，用户可能会构造一个 ELF 二进制文件，其中 `ph.vaddr` 指向用户选择的地址，而 `ph.memsz` 足够大，导致两者相加后的和溢出后返回到 0x1000，而这看起来像是一个有效值（译者注：如果不检查溢出后返回到 0x1000 的情况，会导致原本不该被加载到 text 段的内容被加载到 text 段并覆盖原有合法指令）。（另外一种风险情况是）在旧版本的 xv6 中，用户地址空间也包含内核（但在用户模式下不可读写），用户可以选择一个与内核内存对应的地址，从而将数据从 ELF 二进制文件复制到内核。在 RISC-V 版本的 xv6 中，这种情况不会发生，因为内核有自己独立的页表；`loadseg` 负责将数据加载到进程的页表中，而不是加载到内核的页表中。

> It is easy for a kernel developer to omit a crucial check, and real-world kernels have a long history of missing checks whose absence can be exploited by user programs to obtain kernel privileges. It is likely that xv6 doesn’ t do a complete job of validating user-level data supplied to the kernel, which a malicious user program might be able to exploit to circumvent xv6’ s isolation.

内核开发人员很容易忽略对关键点的检查，现实世界中的内核长期以来都容易忽视这些检查，用户程序可以利用这些问题来尝试获取内核权限。xv6 很可能无法完全验证提供给内核的用户级数据，恶意用户程序可能会利用这些数据来绕过 xv6 的隔离机制。

3.8 现实世界（Real world）

> Like most operating systems, xv6 uses the paging hardware for memory protection and mapping. Most operating systems make far more sophisticated use of paging than xv6 by combining paging and page-fault exceptions, which we will discuss in Chapter 4.

与大多数操作系统一样，xv6 使用分页硬件实现对内存保护和映射。大多数操作系统对分页的使用比 xv6 更加复杂，往往将分页和缺页异常处理结合起来，我们将在第 4 章讨论这一点。

> Xv6 is simplified by the kernel’ s use of a direct map between virtual and physical addresses, and by its assumption that there is physical RAM at address 0x80000000, where the kernel expects to be loaded. This works with QEMU, but on real hardware it turns out to be a bad idea; real hardware places RAM and devices at unpredictable physical addresses, so that (for example) there might be no RAM at 0x80000000, where xv6 expect to be able to store the kernel. More serious kernel designs exploit the page table to turn arbitrary hardware physical memory layouts into predictable kernel virtual address layouts.

xv6 在设计上的简化之处在于在内核中对虚拟地址和物理地址之间采用直接映射的方式，并假设内核的加载地址恰好就是物理内存的起始地址 0x80000000。这对于 QEMU 是没有问题的，但在实际硬件上却并非如此简单；在真实的硬件上 RAM 和设备的物理地址并不固定，（譬如）xv6 认为存放内核的物理地址 0x80000000 处可能根本就不是 RAM。更严谨的内核设计不应该对物理内存布局有任

何假定，而是通过定义确定的内核虚拟地址布局，然后利用页表将任意的硬件物理内存布局转换为这个虚拟的地址布局。

> RISC-V supports protection at the level of physical addresses, but xv6 doesn’ t use that feature.

RISC-V 支持物理地址级别的保护，但 xv6 没有使用该功能。

> On machines with lots of memory it might make sense to use RISC-V’ s support for “super pages.” Small pages make sense when physical memory is small, to allow allocation and page-out to disk with fine granularity. For example, if a program uses only 8 kilobytes of memory, giving it a whole 4-megabyte super-page of physical memory is wasteful. Larger pages make sense on machines with lots of RAM, and may reduce overhead for page-table manipulation.

在拥有大量内存的机器上，使用 RISC-V 对 “超级页（super pages）” 的支持或许是明智之举。当物理内存较小时，使用较小的物理页比较合理，这样可以以更精细的颗粒度分配内存并将其 “换出（page-out）” 到磁盘。例如，如果一个程序只使用 8 KB 的内存，那么给它分配一个完整的 4 MB 的超级页就是一种浪费。在拥有大量 RAM 的机器上，使用更大的物理页比较合理，因为这可以减少页表操作的开销。

> To avoid having to flush the complete TLB when changing page tables, RISC-V CPUs may support address space identifiers (ASIDs) [3]. The kernel can then flush just the TLB entries for a particular address space. Xv6 does not use this feature.

了避免在切换页表时刷新整个 TLB，RISC-V CPU 需要支持 address space identifiers (ASID) [3]。这样，内核就可以只刷新特定地址空间的 TLB 条目。但 xv6 没有利用此功能。

> The xv6 kernel’ s lack of a malloc-like allocator that can provide memory for small objects prevents the kernel from using sophisticated data structures that would require dynamic allocation. A more elaborate kernel would likely allocate many different sizes of small blocks, rather than (as in xv6) just 4096-byte blocks; a real kernel allocator would need to handle small allocations as well as large ones.

xv6 内核缺少类似 `malloc` 的分配器来为小的结构体对象分配内存，这使得内核在动态分配复杂的数据结构时比较浪费内存。更复杂的内核实现可能会需要分配许多不同大小的小内存块，而不是（像 xv6 中那样）每次分配的内存块最小也要 4096 字节；真正的内核分配器需要既能够分配小的内存块也能够分配大的内存块。

> Memory allocation is a perennial hot topic, the basic problems being efficient use of limited memory and preparing for unknown future requests [9]. Today people care more about speed than space efficiency.

内存分配是一个长久以来一直存在的热门话题，其基本问题是如何高效利用有限的内存并为将来未知的请求做好准备 [9]。如今，人们更倾向于关心分配的速度而非空间效率。

3.9 练习（Exercises）

> 1. Parse RISC-V’s device tree to find the amount of physical memory the computer has.

1. 解析 RISC-V 的设备树以确定计算机拥有的物理内存大小。

> 1. The functions `copyin` and `copyinstr` walk the user page table in software. Set up the kernel page table so that the kernel has the user program mapped, and `copyin` and `copyinstr` can use `memcpy` to copy system call arguments into kernel space, relying on the hardware to do the page table walk.

1. 目前函数 `copyin` 和 `copyinstr` 用软件方式遍历用户页表。要求修改代码，在设置内核页表时使得内核映射用户程序，这样 `copyin` 和 `copyinstr` 可以使用 `memcpy` 将系统调用参数复制到内核空间，而且是依赖硬件对页表进行遍历。

> 1. Modify xv6 to use super pages for the kernel.

1. 修改 xv6 内核使用 “超级页（super page）”。

> 1. Unix implementations of `exec` traditionally include special handling for shell scripts. If the file to execute begins with the text `#!`, then the first line is taken to be a program to run to interpret the file. For example, if `exec` is called to run `myprog arg1` and `myprog`’s first line is `#!/interp`, then `exec` runs `/interp` with command line `/interp myprog arg1`. Implement support for this convention in xv6.

1. Unix 上对 `exec` 的实现传统上包含对 Shell 脚本的特殊处理。如果要执行的文件以文本 `#!` 开头，则第一行将被视为指明了一个用于解释该文件的程序。例如，如果调用 `exec` 来运行 `myprog arg1`，而 `myprog` 的第一行是 `#!/interp`，则 `exec` 会使用命令行 `/interp myprog arg1` 来运行 `/interp`。为 xv6 实现以上功能。

> 1. Implement address space layout randomization for the kernel.

1. 为内核实现地址空间布局随机化。

第 4 章 陷阱和系统调用（Chapter 4 Traps and system call）

> There are three kinds of event which cause the CPU to set aside ordinary execution of instructions and force a transfer of control to special kernel code that handles the event. One situation is a system call, when a user program executes the `ecall` instruction to ask the kernel to do something for it. Another situation is an exception: an instruction (user or kernel) does something illegal, such as load from an invalid virtual address. The third situation is a device interrupt, when a device signals that it needs attention, for example when the disk hardware finishes a read or write request.

有三种事件会导致 CPU 暂停正常的指令执行，并强制将执行流转移到处理该事件的特殊的内核指令处。第一种情况是 “系统调用（system call）”，即用户程序执行 `ecall` 指令，发起对内核的请求，请求内核代替它执行某些操作。另一种情况是 “异常（exception）”：发生的场景是当一条（用户态或内核态的）指令执行了非法操作，例如访问（读取）了无效的虚拟地址。第三种情况是设备 中断（interrupt），即设备主动发出信号，提醒处理器注意某个事件发生了，例如当磁盘硬件完成读写请求时。

> This book uses trap as a generic term for these situations. Typically whatever code was executing at the time of the trap will later need to resume, and shouldn’t need to be aware that anything special happened. That is, we often want traps to be transparent; this is particularly important for device interrupts, which the interrupted code typically doesn’t expect. A trap forces a transfer of control into the kernel; the kernel saves registers and other state so that execution can be resumed; the kernel executes appropriate handler code (e.g., a system call implementation or device driver); the kernel restores the saved state and returns from the trap; and the original code resumes where it left off.

在本书中使用 陷阱（trap） 作为这些情况的统称。通常，当 trap 发生时（被打断的）正在执行的指令序列稍后都需要恢复，并且从执行指令序列的角度来看并不需要知道发生了什么特殊情况。也就是说，我们通常希望 trap 是透明的；这对于设备中断尤其重要，因为被中断的代码通常对这种情况并没有预期。Trap 强制将（对 CPU 的）控制权转移给内核；内核保存寄存器和其他状态，以便将来恢复执行；内核执行适当的处理程序代码（例如，系统调用函数或设备驱动程序）；内核恢复保存过的状态并从 trap 返回；原先的代码从被中断处继续恢复执行。

> Xv6 handles all traps in the kernel; traps are not delivered to user code. Handling traps in the kernel is natural for system calls. It makes sense for interrupts since isolation demands that only the kernel be allowed to use devices, and because the kernel is able to share devices among multiple processes. It also makes sense for exceptions since the kernel may be able to handle the exception from user space (for an example see Chapter 5) or respond by killing the offending program.

对于 xv6 来说，所有的 trap 处理都在内核中；也就是说，我们不会在用户态下执行 trap 的处理代码。这对于系统调用来说是很自然的事情。对于中断来说，也很有意义，因为这实现了隔离，确保只有在内核态才可以直接访问设备，所以我们可以把内核看成是一种供多个进程共享的访问设备的

机制。异常也是内核需要重点处理的 trap 类型，内核需要正确处理来自用户空间的异常（具体例子请参考第 5 章的介绍），除此之外，内核只是简单地终止（kill）触发异常的进程。

> Xv6 trap handling proceeds in four stages: hardware actions taken by the RISC-V CPU, some assembly instructions that prepare the way for kernel C code, a C function that decides what to do with the trap, and the system call or device-driver service routine. While commonality among the three trap types suggests that a kernel could handle all traps with a single code path, it turns out to be convenient to have separate code for two distinct cases: traps from user space, and traps from kernel space. Kernel code (assembler or C) that processes a trap is often called a handler; the first handler instructions are usually written in assembler (rather than C) and are sometimes called a vector.

xv6 的 trap 处理分为四个阶段：最开始是 RISC-V CPU 内部的硬件操作，然后执行一段为内核 C 代码做准备的汇编指令，接下来是决定如何处理 trap 的 C 函数，最终决定是调用系统调用函数还是设备驱动程序服务例程。虽然这三种 trap 类型的共性表明内核可以用同一条代码路径处理所有 trap，但实践证明，将 trap 处理按两种不同的情况分开，分别编写代码更为方便：具体是区分该 trap 来自用户空间还是来自内核空间。处理 trap 的内核代码（汇编语言或 C 语言）通常被称为 “处理程序（handler）”；处理程序中最开始的一段指令通常用汇编语言（而不是 C 语言）编写，所以有时我们也把处理程序叫做 向量（vector）。

> Before proceeding, please read `kernel/trampoline.S`, and `usertrap()` and `prepare_return()` in `kernel/trap.c`.

在继续之前，请阅读 `kernel/trampoline.S`，以及 `kernel/trap.c` 中的 `usertrap()` 和 `prepare_return()`。

4.1 RISC-V 的陷阱机制（RISC-V trap machinery）

> Each RISC-V CPU has a set of hardware control registers that the kernel writes to tell the CPU how to handle traps, and that the kernel can read to find out about a trap that has occurred. The RISC-V documents contain the full story [3]. `riscv.h` (0500) contains definitions that xv6 uses. Here’s an outline of the most important registers:

每个 RISC-V CPU 都有一组硬件控制寄存器，内核通过设置这些寄存器来告诉 CPU 如何处理 trap，内核也可以读取这些寄存器来了解一个已发生的 trap。RISC-V 文档包含完整的内容 [3]。`riscv.h` (0500) 中定义了 xv6 所使用的和 RISC-V 相关的宏和 inline 函数。以下是一些最重要的寄存器的概述：

> - `stvec`: The kernel writes the address of its trap handler code here; the RISC-V jumps to the address in `stvec` to handle a trap.
> - `sepc`: When a trap occurs, RISC-V saves the program counter here (since the `pc` is then overwritten with the value in `stvec`). The `sret` (return from trap) instruction copies `sepc` to the `pc`. The kernel can write `sepc` to control where `sret` goes.
> - `scause`: RISC-V puts a number here that describes the reason for the trap.
> - `sscratch`: The kernel trap handler code uses `sscratch` to help it avoid overwriting user registers before saving them.

> - `sstatus`: The SIE bit in `sstatus` controls whether device interrupts are enabled. If the kernel clears SIE, the RISC-V will defer device interrupts until the kernel sets SIE. The SPP bit indicates whether a trap came from user mode or supervisor mode, and controls to what mode `sret` returns.

- `stvec`：内核在此处写入其 trap 处理程序的地址；RISC-V 会跳转到 `stvec` 中记录的地址来处理 trap。
- `sepc`：当一个 trap 发生时，RISC-V 会在此处保存 “程序计数器（program counter）” 的值（因为当 trap 发生时，`pc` 会被 `stvec` 中的值覆盖）。指令 `sret`（ret 是 return 的缩写，表示从 trap 返回）将 `sepc` 的值复制回 `pc` 中。内核可以设置 `sepc` 的值来控制执行 `sret` 后从哪里开始恢复取指执行。
- `scause`：RISC-V 在此处放置一个数字来描述 trap 发生的原因。
- `sscratch`：内核 trap 处理程序的代码使用 `sscratch` 来帮助其避免在保存用户寄存器之前覆盖它们。
- `sstatus`：`sstatus` 中的 SIE 比特位控制是否启用设备中断。如果内核清除了 SIE 比特位，RISC-V 将屏蔽设备中断，直到内核重新设置 SIE 比特位。SPP 比特位指示发生 trap 时机器处于用户模式还是管理员模式，从而控制执行 `sret` 时返回到哪个模式。

> The above registers can only be accessed in supervisor mode (i.e., by the kernel); the CPU prevents user code from reading or writing them.

上述寄存器只能在管理员模式下访问（也就是说只能被内核访问），在用户模式下它们无法被读取或写入。

> Each CPU on a multi-core chip has its own set of these registers, and more than one CPU may be handling a trap at any given time.

对于此类寄存器，在多核芯片上的每个 CPU 核都有一组，也就是说在任意给定时刻，每个 CPU 可以独立地处理自己的 trap。

> When it forces a trap, the RISC-V hardware does the following for all trap types:
>
> 1. If the trap is a device interrupt, and the `sstatus` SIE bit is clear, don’t do any of the following.
> 2. Disable interrupts by clearing the SIE bit in `sstatus`.
> 3. Copy the `pc` to `sepc`.
> 4. Save the current mode (user or supervisor) in the SPP bit in `sstatus`.
> 5. Set `scause` to a number indicating the trap’s cause.
> 6. Set the mode to supervisor.
> 7. Copy `stvec` to the `pc`.
> 8. Start executing at the new `pc`.

硬件针对各种 trap 类型的处理都是相同的，当一个 trap 发生时，RISC-V 处理器会按顺序执行以下动作：

1. 如果 trap 是设备中断，并且 `sstatus` 的 SIE 位已清零，则不会执行以下任何操作（译者注：即关中断情况下设备中断即使发生也不会触发 trap）。
2. 清除 `sstatus` 中的 SIE 比特位来禁用中断。
3. 将 `pc` 的值复制到寄存器 `sepc` 中。

4. 将当前模式（用户模式或管理员模式）保存在 `sstatus` 的 SPP 比特位中。
5. 给寄存器 `scause` 设置一个特定的值以反映 trap 发生的原因。
6. 将模式设置为管理员模式。
7. 用寄存器 `stvec` 中的值覆盖 `pc`。
8. 根据 `pc` 中新的指令地址取指执行。

> The CPU doesn’t switch to the kernel page table, doesn’t switch to a stack in the kernel, and doesn’t save any registers other than the `pc`. Kernel software must perform these tasks. One reason that the CPU does minimal work during a trap is to provide flexibility to software; for example, some operating systems omit a page table switch in some situations to increase trap performance.

（当 trap 发生时）CPU 不会自动切换为使用内核页表，也不会自动切换为使用内核栈，同时也不会保存除 `pc` 之外的任何寄存器。内核的代码必须自己执行这些操作。CPU 在 trap 发生期间只执行少量工作的原因之一是为软件实现提供灵活性；例如，某些操作系统在某些情况下可能会省略页表切换以提高执行 trap 的性能。

> It’s worth thinking about whether any of the steps listed above could be omitted, perhaps in search of faster traps. Though there are situations in which a simpler sequence can work, many of the steps would be dangerous to omit in general. For example, suppose that the CPU didn’t switch program counters. Then a trap from user space could switch to supervisor mode while still running user instructions. Those user instructions could break user/kernel isolation, for example by modifying the `satp` register to point to a page table that allowed accessing all of physical memory. It is thus important that the CPU switch to a kernel-specified instruction address, namely `stvec`.

这里我们可以深入思考一下，如果为了寻求更快的 trap 执行速度，（CPU 硬件）是否可以省略上面列出来的步骤中的任何一步。虽然在某些情况下可以使用更简单的执行步骤，但通常情况下，省略某些步骤是危险的。例如，假设 CPU 不切换 program counter。那么当在用户模式下发生 trap 时虽然模式已经切换到管理员模式，可是 CPU 仍然在执行用户指令。这些用户指令可能会破坏我们要保持用户和内核之间隔离的要求，譬如用户指令可以通过修改 `satp` 寄存器使其指向一个允许访问所有物理内存的页表。因此，由 CPU 来确保自动切换到内核指定的指令地址（即寄存器 `stvec` 中设定的值）是非常重要的。

4.2 用户空间触发的陷阱（Traps from user space）

> Xv6 handles traps differently depending on whether the trap occurs while executing in the kernel or in user code. Here is the story for traps from user code; Section 4.5 describes traps from kernel code.

在处理 trap 的方式上，xv6 区分 trap 发生在内核态还是在用户态。本小节介绍了当 trap 发生在用户态时的处理；第 4.5 节描述了 trap 发生在内核态时的处理。

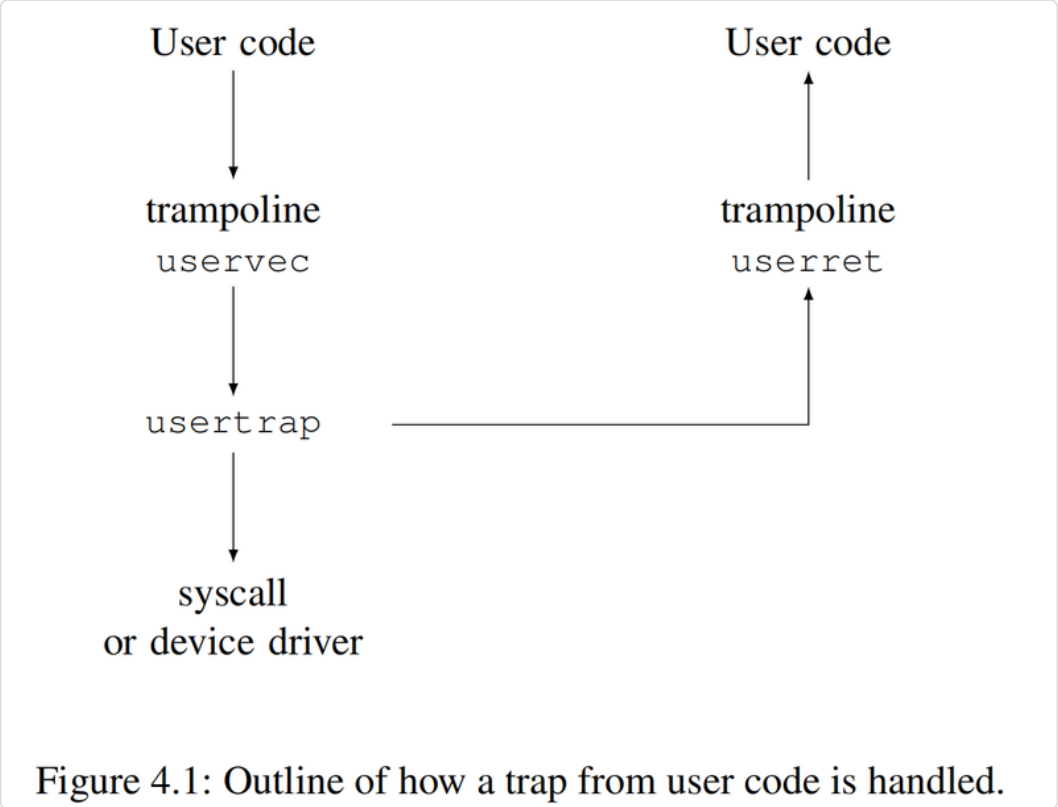

Figure 4.1: Outline of how a trap from user code is handled.

> A trap may occur while executing in user space if the user program makes a system call (`ecall` instruction), or does something illegal, or if a device interrupts. As shown in Figure 4.1, the high-level path of a trap from user space is `uservec` (3071), then `usertrap` (3337); and when the kernel is ready to return, `usertrap` returns to `userret` (3151) which executes `sret` to user space.

如果用户程序在用户空间执行时发生系统调用（执行 `ecall` 指令），或执行了非法操作，或者发生设备中断，都会触发 trap。如图 4.1 所示，用户态 trap 的大致执行路径为：`uservec`（3071），然后是 `usertrap`（3337）；当内核准备返回时，`usertrap` 返回到 `userret`（3151），`userret` 会执行 `sret` 返回用户空间。

> A major constraint on the design of xv6's trap handling is the fact that the RISC-V hardware does not switch page tables when it forces a trap. This means that the trap handler address in `stvec` must have a valid mapping in the user page table, since that's the page table in force when the trap handling code starts executing. Furthermore, xv6's trap handling code needs to switch to the kernel page table; in order to be able to continue executing after that switch, the kernel page table must also have a mapping for the handler pointed to by `stvec`.

xv6 在 trap 处理设计上存在的一个主要限制在于，RISC-V 硬件在触发 trap 时不会自动切换页表。也就是说当 trap 处理程序的代码开始执行时当前激活的仍然是用户态的页表，所以这意味着 `stvec` 中所指向的 trap 处理程序的地址必须在用户页表中具备有效的映射。此外，xv6 的 trap 处理代码需要负责将页表切换为内核页表；为了能够在切换后能够继续执行，内核页表中对 `stvec` 所指向的 trap 处理程序也要具备有效的映射。

> Xv6 satisfies these requirements using a trampoline page. This page contains `uservec`, the xv6 trap handling code that `stvec` points to. The trampoline page is mapped in every process's page table at virtual address `0x3ffffff000` (called `TRAMPOLINE`), which is the last page in the virtual address space so that it will be above memory that programs use for themselves. The trampoline page is mapped at the same virtual address in the kernel page table. See Figure 2.3 and Figure 3.3. Because the trampoline page is mapped in the user page table, traps can start executing there in supervisor mode. Because the trampoline page is mapped at the same address in the kernel address space, the trap handler can continue to execute after it switches to the kernel page table.

xv6 使用一个 “蹦床页（trampoline page）” 来满足以上要求。这个 trampoline page 中存放了 `uservec` 函数的指令，即 `stvec` 指向的 xv6 的 trap 处理程序。每个进程的页表（即用户页表）都会将这个 trampoline page 映射到进程地址空间中的虚拟地址 `0x3ffffff000` 处（这个地址在代码中定义为 `TRAMPOLINE`），这个 trampoline page 是虚拟地址空间的最后一个 page，因此它位于程序会使用的内存地址空间的最上方。trampoline page 在内核页表中被映射为相同的虚拟地址。参见图 2.3 和图 3.3。由于用户页表中有效映射了 trampoline page，因此当处理器进入管理员模式开始处理 trap 时可以从 `TRAMPOLINE` 那里开始执行。同时由于 trampoline page 也映射到内核地址空间中的相同地址，因此 trap 处理程序在切换到内核页表后依然可以继续执行。

> The code for the `uservec` trap handler is in `trampoline.S` (3071). When `uservec` starts, all 32 registers contain values owned by the interrupted user code. These 32 values need to be saved somewhere in memory, so that later on the kernel can restore them before returning to user space. Storing to memory requires use of a register to hold the store's destination address, but at this point there are no general-purpose registers available! Luckily RISC-V provides a helping hand in the form of the `sscratch` register. The `csrw` instruction at the start of `uservec` saves `a0` in `sscratch`. Now `uservec` has one register (`a0`) to play with.

`uservec` 函数（译者注：即用户态 trap 处理程序）的代码位于 `trampoline.S` 中 (3071)。当 `uservec` 刚开始执行时，所有 32 个寄存器的值对应着被中断的用户程序的上下文。这 32 个值需要保存在内存中的某个位置，以便内核稍后在返回用户空间之前恢复它们。执行保存的操作需要使用寄存器来记录保存的目的地址，但目前没有可用的通用寄存器！（译者注：所有的通用寄存器的内容此时都需要保存，此时使用它们中的任何一个，都会修改它们的内容，这显然是不可以的）幸运的是，RISC-V 有一个 `sscratch` 寄存器可以提供帮助。`uservec` 函数中一开始利用 `csrw` 指令将 `a0` 保存在 `sscratch` 中。这样此时 `uservec` 至少有一个寄存器 (`a0`) 可以使用。

> uservec's next task is to save the 32 user registers. The kernel allocates, for each process, a page of memory for a `trapframe` structure that (among other things) has space to save the 32 user registers (1992). Because `satp` still refers to the user page table, `uservec` needs the trapframe to be mapped in the user address space. Xv6 maps each process's trapframe at virtual address `TRAPFRAME` (`0x3ffffe000`) in that process's user page table; one page below `TRAMPOLINE`. Each process's `p->trapframe` contains a kernel virtual address for the process's trapframe.

`uservec` 的下一个任务是保存 32 个用户寄存器（译者注：严格说是 31 个，x0/zero 不需要保存和恢复）。内核为每个进程分配一个内存物理页（译者注：下文直接用 trapframe 指代这个物理页），用于存放 `trapframe` 结构体（以及其他一些内容），该结构体的部分空间用于保存 32 个用户寄存器 (1992)。由于 `satp` 此时仍然指向用户页表，`uservec` 需要将 trapframe 映射到用户地

址空间。xv6 将每个进程的 trapframe 映射到该进程用户页表中的虚拟地址 `TRAPFRAME`（`0x3ffffe000`）；`TRAPFRAME` 就位于 `TRAMPOLINE` 的下方。每个进程的 `p->trapframe` 保存了自己的 trapframe 的内核虚拟地址。（译者注：内核代码中可以通过 `p->trapframe` 直接访问 trapframe，因为内核采用的是 direct mapping，具体参考 3.2 节）。

> `uservec` sets register `a0` to address `TRAPFRAME` and saves all the user registers there. Then it retrieves the user `a0` from `sscratch` and saves it in the trapframe.

因此，`uservec` 将地址 `TRAPFRAME` 设置到 `a0` 中（译者注：接上文，此时 `a0` 中原先的值已经保存在 `sscratch` 中），然后（根据这个地址）将所有用户寄存器保存在 trapframe 中。然后从 `sscratch` 取出备份的用户态 `a0` 并保存到 trapframe 中。

> The kernel previously initialized the trapframe to contain some values useful to `uservec`: the address of the current process's kernel stack, the current CPU's hartid, the address of the `usertrap` function, and the address of the kernel page table. `uservec` retrieves these values, switches `satp` to the kernel page table, and jumps to `usertrap`, a C function.

内核原先在 trapframe 中初始化了一些对 `uservec` 有用的数据，这包括：当前进程的内核栈地址、当前 CPU 的 hartid、`usertrap` 函数的地址以及内核页表地址，`uservec` 将读取和使用这些值，然后将 `satp` 切换到内核页表，并跳转到 `usertrap` (这是一个 C 函数)。

> The job of `usertrap` is to determine the cause of the trap, process it, and return (3337). It first changes `stvec` so that a trap while in the kernel will be handled by `kernelvec` rather than `uservec`. It saves the `sepc` register (the saved user program counter) for future use when returning back to user space. If the trap is a system call, `usertrap` calls `syscall` to handle it; if a device interrupt, `devintr`; if a page fault, `vmfault`; otherwise it's an exception (e.g., use of an invalid address), and the kernel kills the faulting process. The system call path adds four to the saved user program counter because RISC-V, in the case of a system call, leaves the program pointer pointing to the `ecall` instruction but user code needs to resume executing at the subsequent instruction. `usertrap` checks if the process has been killed or should yield the CPU (if this trap is a timer interrupt).

`usertrap` 的工作包括确定 trap 发生的原因，进行相应处理然后返回 (3337)。它首先修改 `stvec` 的值，以便在内核态中一旦发生 trap 处理器会跳转到 `kernelvec` 而不是依然交给 `uservec` 处理。然后，它将 `sepc` 寄存器的值保存到 trapframe 中（`sepc` 的值是 trap 发生时的用户态下的 program counter），这是为了将来返回到用户空间所做的准备。接下来 `usertrap` 会判断，如果 trap 是系统调用，则调用 `syscall` 来处理；如果是设备中断，则调用 `devintr`；如果是 page fault，则调用 `vmfault`；其他情况是异常（譬如，访问了一个非法地址），内核会终止出错的进程。针对 RISC-V 的系统调用，我们并不会在这里直接修改处理器的 `sepc` 寄存器（即仍然让它指向调用 `ecall` 指令的地址），而是将已保存的用户态 program counter（译者注：即 `p->trapframe->epc`）的值加上 4，这么做的目的是确保系统调用执行完毕后，在返回用户态时，我们能从 `ecall` 指令的下一条指令处开始恢复执行（译者注：考虑到本节前面也说到 `usertrap` 中可能会发生进程切换返回用户态并修改 `sepc`，所以 `p->trapframe->epc` 的值只有在本进程真正确定需要返回用户态之前才会被恢复到 `sepc` 中，具体见下文针对 `usertrapret` 函数的解释）。`usertrap` 会检查进程是否已被终止或是否应该放弃 CPU（如果触发此 trap 的是定时器中断）。

> The first step in returning to user space is the call to `prepare_return` (3404). This function sets up the RISC-V control registers to prepare for a future trap from user space: setting

> `stvec` to `uservec` and preparing the trapframe fields that `uservec` relies on. `prepare_return` sets `sepc` to the previously saved user program counter. Finally, `usertrap` returns back to `userret` in the trampoline page (3151), passing back a pointer to the user page table in `a0`.

当我们开始准备返回用户空间的时候，第一步是调用 `prepare_return` (3404)。此函数设置 RISC-V 控制寄存器，为下一次用户空间的 trap 做好准备，这些准备工作包括：将 `stvec` 设置为 `uservec`，并为执行 `uservec` 准备好所需要的 trapframe。`prepare_return` 将 `sepc` 设置为先前保存的用户 program counter。最后，`usertrap` 返回 trampoline page 上的 `userret` (3151)，并通过 `a0` 传递了一个指向进程用户页表的指针。

> `userret` switches `satp` to the process' s user page table. Recall that the user page table maps both the trampoline page and `TRAPFRAME`, but nothing else from the kernel. The trampoline page mapping at the same virtual address in user and kernel page tables allows `userret` to keep executing after changing `satp`. From this point on, the only data `userret` can use is the register contents and the content of the trapframe. `userret` loads the `TRAPFRAME` address into `a0`, restores saved user registers from the trapframe via `a0`, restores the saved user `a0`, and executes `sret` to return to user space.

`userret` 将 `satp` 切换到进程的用户页表。回想一下，用户页表中只映射了 trampoline（映射到 `TRAMPOLINE`）和 trapframe（映射到 `TRAPFRAME`），除此之外，并没有映射内核的其他内容。由于 trampoline 在用户和内核页表中的虚拟地址相同，因此 `userret` 在更改 `satp` 后仍能继续执行。从此时起，`userret` 唯一可以使用的数据是寄存器内容和 trapframe 的内容。`userret` 先将 `TRAPFRAME` 地址加载到 `a0`，然后通过 `a0` 从 trapframe 恢复保存的（除 `a0` 之外其他的）用户寄存器，最后恢复保存的 `a0`，`userret` 最后通过执行 `sret` 返回用户空间。

> `uservec` and `userret` are written in assembly language because it is difficult to write C code to save or restore all the registers or survive switching page tables.

`uservec` 和 `userret` 是用汇编语言编写的，因为对于保存或恢复所有寄存器或切换页表这些操作很难用 C 代码来编写。

4.3 代码讲解：执行系统调用（Code: Calling system calls）

> User programs call library functions in order to make system calls. For example, the shell displays a prompt with this function call (in `user/sh.c`):

用户程序通过调用库函数来发起系统调用。例如，shell 应用程序通过以下函数调用打印一个提示符（在 `user/sh.c` 中）：

```
write(2, "$ ", 2);
```

> Here' s the library function, in `user/usys.S`:

该库函数定义如下，具体定义在 `user/usys.S` 中：

```
write:
  li a7, SYS_write
  ecall
  ret
```

> The code that the C compiler generates for the function call loads the three arguments into registers `a0`, `a1`, and `a2`. Then the `write()` function loads the system call number, `SYS_write` (16), into `a7`. The kernel will look at those registers to find out what system call is intended, and what the arguments are. The `ecall` instruction traps from user space into the kernel and causes `uservec`, `usertrap`, and then `syscall` to execute.

C 编译器为这个函数调用生成相关的指令将三个参数分别存放在寄存器 `a0`、`a1` 和 `a2` 中。然后 `write()` 函数负责将系统调用号，即 `SYS_write`（其具体值为 16）放在 `a7` 中。内核会检查这些寄存器的值从而知道要执行哪个系统调用，以及传入的参数是多少。`ecall` 指令触发 trap，将处理器从用户态切换到内核态，并顺序执行 `uservec`、`usertrap` 和 `syscall`。

> At this point, please read `kernel/syscall.c`, `sys_write()` in `kernel/sysfile.c`, and `copyout()`, `copyin()`, and `copyinstr()` in `kernel/vm.c`.

此时，请阅读 `kernel/syscall.c`、`kernel/sysfile.c` 中的 `sys_write()` 以及 `kernel/vm.c` 中的 `copyout()`、`copyin()` 和 `copyinstr()`。

> `syscall` (3731) retrieves the system call number from the saved `a7` in the trapframe and uses it to index into `syscalls` (3706). For our example, `a7` contains `SYS_write` (3566), resulting in a call to the system call implementation function `sys_write`.

`syscall` (3731) 从 trapframe 中保存的 `a7` 中取到系统调用号，并用它作为索引在 `syscalls` 中找到对应的项。对于这里的例子代码，`a7` 的值是 `SYS_write` (3566)，所以最终会调用其对应的系统调用实现函数 `sys_write`。

> When `sys_write` returns, `syscall` records its return value in `p->trapframe->a0`. This will cause the original user-space call to `write()` to return that value, since the C calling convention on RISC-V places return values in `a0`. System calls conventionally return negative numbers to indicate errors, and zero or positive numbers for success.

当 `sys_write` 返回时，`syscall` 会将其返回值记录在 `p->trapframe->a0` 中。因为根据 RISC-V 上的 C 函数调用约定，规定将返回值放在 `a0` 中，所以这将导致原先用户空间对 `write()` 的调用返回该值。系统调用通常返回负数表示错误，返回零或正数表示成功。

4.4 代码讲解：系统调用参数（Code: System call arguments）

> System call arguments start out in the user registers, and are then moved to the trap frame by the kernel trap code. The kernel functions `argint`, `argaddr`, and `argfd` retrieve the n'th system call argument from the trap frame as an integer, pointer, or a file descriptor.

系统调用参数首先在用户态下被写入寄存器，然后由内核的 trap 处理代码移动到 trapframe 中。内核函数 `argint`、`argaddr` 和 `argfd` 从 trapframe 中找到第 n 个系统调用参数，再分别转化成整数、指针或文件描述符的形式。

> Some system calls pass pointers as arguments, and the kernel must use those pointers to read or write user memory. The `write` system call, for example, passes the kernel a user-space pointer to the data to be written. Such pointers pose two challenges. First, the user program may be buggy or malicious, and may pass the kernel an invalid pointer or a pointer intended to trick the kernel into accessing kernel memory instead of user memory. Second, the xv6 kernel page table mappings are not the same as the user page table mappings, so the kernel cannot use ordinary instructions to load or store from user-supplied addresses.

一些系统调用通过参数传递指针，内核需要使用这些指针来读取或写入用户内存。例如，`write` 系统调用向内核传递一个指针，该指针指向用户空间中存放有待写出数据的缓存。传递这样的指针带来了两个挑战。首先，用户程序可能存在缺陷或者纯粹怀有恶意，它们可能会向内核传递无效指针，或传递一个指针试图诱骗内核进而访问内核内存而不是用户内存。其次，由于 xv6 内核页表与用户页表的映射方式不同，因此内核无法直接使用普通指令根据用户提供的地址读取或存储数据。

> The kernel implements functions that safely transfer data to and from user-supplied addresses. `fetchstr` is an example (3624). File system calls such as `exec` use `fetchstr` to retrieve string file-name arguments from user space. `fetchstr` calls `copyinstr` to do the hard work.

内核实现了安全地根据用户提供的地址传输数据的函数。例如 `fetchstr` (3624)。诸如 `exec` 之类的系统调用通过使用 `fetchstr` 从用户空间获取字符串形式的文件名参数。`fetchstr` 中通过调用 `copyinstr` 来完成这项复杂的工作。

> `copyinstr` (1833) copies up to `max` bytes to `dst` from virtual address `srcva` in the user page table `pagetable`. Since `pagetable` is not the current page table, `copyinstr` uses `walkaddr` (which calls `walk`) to look up `srcva` in `pagetable`, yielding physical address `pa0`. The kernel's page table maps all of physical RAM at virtual addresses that are equal to the RAM's physical address. This allows `copyinstr` to directly copy string bytes from `pa0` to `dst`. `walkaddr` (1520) checks that the user-supplied virtual address is part of the process's user address space, so programs cannot trick the kernel into reading other memory. A similar function, `copyout`, copies data from the kernel to a user-supplied address.

基于参数中给定的用户页表 `pagetable`，函数 `copyinstr` (1833) 将最多 `max` 个字节从用户态虚拟地址 `srcva` 复制到 `dst`。由于 `pagetable` 不是 当前页表，`copyinstr` 使用 `walkaddr`（内部调用 `walk`）在 `pagetable` 中查找 `srcva`，从而得到物理地址 `pa0`。因为内核的页表将所有物理内存的地址映射到与其值相等的虚拟地址处。所以 `copyinstr` 可以直接将字符串字节从 `pa0` 复制到 `dst`。`walkaddr` (1520) 会检查用户提供的虚拟地址是否属于进程的用户地址空间，因此程序无法欺骗内核读取其他内存。类似的函数 `copyout` 将数据从内核空间复制到用户提供的地址。

4.5 内核空间触发的陷阱（Traps from kernel space）

> Please read `kernel/kernelvec.S`, and `kerneltrap()` in `kernel/trap.c`.

请阅读 `kernel/kernelvec.S` 和 `kernel/trap.c` 中的 `kerneltrap()`。

> Xv6 handles traps from kernel code in a different way than traps from user code. When entering the kernel, `usertrap` points `stvec` to the assembly code at `kernelvec` (3211).

> Since `kernelvec` only executes if xv6 was already in the kernel, `kernelvec` can rely on `satp` being set to the kernel page table, and on the stack pointer referring to a valid kernel stack. `kernelvec` pushes all 32 registers onto the current stack, from which it will later restore them so that the interrupted kernel code can resume without disturbance.

xv6 对于处理内核态 trap 的方式与处理用户态 trap 不同。进入内核时，`usertrap` 会将 `stvec` 指向汇编函数 `kernelvec`（3211）。由于 `kernelvec` 仅在 xv6 已进入内核后才会执行，因此当 `kernelvec` 被执行时我们知道 `satp` 已经设置为指向内核页表，以及栈指针也已经指向有效的内核栈（译者注：`uservec` 中跳转 `usertrap` 之前已经完成内核页表以及内核栈指针的设置）。`kernelvec` 会将所有 32 个寄存器压入当前栈，之后再从栈中恢复它们，以便被中断的内核代码能够不受干扰地恢复执行（译者注：原文这里所说的会将所有 32 个寄存器压栈的描述并不准确，实际只会保存 caller-saved 寄存器，具体见代码）。

> `kernelvec` saves the registers on the stack of the interrupted kernel thread, which makes sense because the register values belong to that thread. This is particularly important if the trap causes a switch to a different thread – in that case the trap will actually return from the stack of the new thread, leaving the interrupted thread's saved registers safely on its stack.

`kernelvec` 将寄存器保存在被中断的内核线程的栈中，这么做是合理的，因为每个线程需要独立地保存一份自己被中断时的寄存器上下文信息。尤其重要的是我们需要意识到 trap 发生时可能导致线程切换，一旦发生线程切换，trap 返回时我们就可以从新线程的栈中恢复寄存器上下文，而同时被中断的线程的寄存器信息则被保存在自己的栈中，这也是安全的。

> `kernelvec` jumps to `kerneltrap` (3453) after saving registers. `kerneltrap` is prepared for just one type of trap: device interrupts. It calls `devintr` (3506) to handle them. If the trap isn't a device interrupt, it must be an exception, such as kernel code trying to use an invalid pointer. This could only be caused by a bug in the kernel code. The kernel does not have a way to recover in this situation, so it calls `panic()`, which prints an error message and then halts.

保存好寄存器后，`kernelvec` 跳转到 `kerneltrap` (3453)。`kerneltrap` 只准备处理一种类型的 trap，即设备中断。它调用 `devintr` (3506) 来处理中断。如果 trap 不是设备中断，则一定是异常，譬如内核代码用到了一个非法指针。这只能说明内核代码中有错误发生。在这种情况下，内核无法恢复，所以对于内核的异常，内核会调用 `panic()` 打印出错信息然后停止内核的执行。

> If `kerneltrap` was called due to a timer interrupt, and a process's kernel thread is running (as opposed to a scheduler thread), `kerneltrap` calls `yield` to give other threads a chance to run. At some point one of those threads will yield, and let our thread and its `kerneltrap` resume again. Chapter 8 explains what happens in `yield`.

对于定时器中断触发执行 `kerneltrap` 的情况，如果被打断的是普通的进程的内核线程（而不是调度器线程），`kerneltrap` 会调用 `yield` 让其他线程有机会运行。此后的某个时刻，当其他运行的线程中的某一个让出处理器时，我们这里先前让出处理器的线程会再次恢复运行并完成 `kerneltrap`。第 8 章将解释 `yield` 中发生的细节。

> When `kerneltrap`'s work is done, it needs to return to whatever code was interrupted by the trap. Because a `yield` may have disturbed `sepc` and the previous mode in `sstatus`, `kerneltrap` saves them when it starts. It now restores those control registers and returns

> to `kernelvec` (3237). `kernelvec` pops the saved registers from the stack and executes `sret`, which copies `sepc` to `pc` and resumes the interrupted kernel code.

当 `kerneltrap` 的工作完成后，它需要返回到被 trap 中断的代码。考虑到这期间可能发生过 `yield` 导致 `sepc` 和 `sstatus` 中原先存放的内容被修改，我们在刚进入 `kerneltrap` 时会保存它们的值。在 `kerneltrap` 的最后会恢复这些控制寄存器并返回到 `kernelvec` (3237)。`kernelvec` 会从栈中弹出保存的寄存器上下文并执行 `sret`，这条指令会将 `sepc` 复制到 `pc`，并恢复执行被中断的内核代码。

> Xv6 sets a CPU's `stvec` to `kernelvec` when that CPU enters the kernel from user space; you can see this in `usertrap` (3346). But there's a window of time when the kernel has started executing but `stvec` is still set to `uservec`, and it's crucial that no device interrupt occur during that window. Luckily the RISC-V always disables interrupts when it starts to take a trap, and `usertrap` doesn't enable them again until after it sets `stvec`.

当 CPU 从用户态进入内核时，xv6 会将 CPU 的 `stvec` 设置为 `kernelvec`；你可以在 `usertrap` (3346) 中看到这一点。但在内核开始执行后存在一个时间窗口期，在此期间 `stvec` 仍然指向 `uservec`，需要特别关注的一点是在此期间不能发生任何设备中断。所幸的是，RISC-V 规定当 trap 发生时硬件会自动禁用中断，而 `usertrap` 直到设置 `stvec` 后才会再次启用中断。

4.6 现实世界（Real world）

> The need for trampoline pages could be eliminated if kernel memory were mapped into every process' s user page table (with `PTE_U` clear). That would also eliminate the need for a page table switch when trapping from user space into the kernel. That in turn would allow system call implementations in the kernel to take advantage of the current process' s user memory being mapped, allowing kernel code to directly dereference user pointers. Many operating systems have used these ideas to increase efficiency. Xv6 avoids them in order to reduce the chances of security bugs in the kernel due to inadvertent use of user pointers, and to reduce some complexity that would be required to ensure that user and kernel virtual addresses don't overlap.

如果将内核的内存在每个进程的用户页表中都进行映射（同时清除 `PTE_U` 标志位），则可以消除对特殊的 trapmpoline 页的需求。这也将消除从用户空间进入内核时进行页表切换的需要。同时这将允许内核在实现系统调用时利用当前进程映射的用户内存，从而允许内核代码直接利用传入的用户态指针访问内存。许多操作系统已经采用了这些想法来提高效率。xv6 避免了这些想法，以减少由于无意间错误使用用户指针而导致内核出现安全漏洞的可能性，并降低确为了保用户和内核虚拟地址不重叠而引入的复杂性。

4.7 练习（Exercises）

> 1. Could some or all of the code in `trampoline.S` and `kernelvec.S` be written in C rather than assembler?

1. `trampoline.S` 和 `kernelvec.S` 中的部分或全部代码可以改成用 C 语言编写而不是汇编语言编写吗？

> 1. Is there a way to eliminate the special `TRAPFRAME` page mapping in every user address space? For example, could `uservec` be modified to simply push the 32 user registers onto the kernel stack, or store them in the `proc` structure?

1. 有没有办法消除每个用户地址空间中对特殊的 TRAPFRAME 页的映射？例如，是否可以修改 `uservec`，将 32 个用户寄存器简单地保存到内核栈中，或者将它们存储在 `proc` 结构体中？

> 1. Could xv6 be modified to eliminate the special `TRAMPOLINE` page mapping?

1. 是否可以修改 xv6，消除针对 TRAMPOLINE 页的映射？

第 5 章 缺页异常（Chapter 5 Page faults）

> The RISC-V CPU raises a page-fault exception when a virtual address is used that has no mapping in the page table, or has a mapping whose `PTE_V` flag is clear, or a mapping whose permission bits (`PTE_R` , `PTE_W` , `PTE_X` , `PTE_U`) forbid the operation being attempted. RISC-V distinguishes three kinds of page fault: load page faults (caused by load instructions), store page faults (caused by store instructions), and instruction page faults (caused by fetches of instructions to be executed). The `scause` register indicates the type of the page fault and the `stval` register contains the address that couldn’ t be translated.

在以下情况下会触发 RISC-V CPU 产生 “缺页（page-fault）” 异常：当使用的虚拟地址在页表中没有被映射，或者映射的 `PTE_V` 标志为 0（清除状态），或者指令尝试执行针对该虚拟地址禁止的操作（体现在映射的权限位 `PTE_R` 、 `PTE_W` 、 `PTE_X` 、 `PTE_U` 上）。RISC-V 区分三种类型的缺页异常：“加载页异常（load page fault）”（由加载指令触发）、“存储页异常（store page fault）”（由存储指令触发）和 “指令页异常（instruction page fault）”（由取指操作触发）。 `scause` 寄存器指示缺页异常的类型， `stval` 寄存器会指出异常发生时当前访问的内存地址。

> The combination of page tables and page faults is a powerful tool. Page tables give the kernel a level of indirection between virtual and physical addresses, so that the kernel can control the structure and content of address spaces. Page faults allow the kernel to intercept loads and stores and, by modifying the page table, specify on the fly what data those references refer to. The kernel can use these capabilities to increase efficiency: for example, copy-on-write fork allows the kernel to transparently share memory between parent and child, avoiding the cost of copying pages that neither write. Application programmers can also benefit. One possibility is memory-mapped files, where the kernel uses paging to cause a file’ s content to appear in an application’ s address space, transparently reading file blocks in response to page faults. Another is lazy memory allocation, which allows a program to ask for a huge virtual address space, but only to pay the cost of allocating physical memory for the pages the program actually reads and writes. xv6 uses page faults for only one purpose: lazy allocation.

组合利用页表和缺页异常可以实现强大的功能。页表为内核在虚拟地址和物理地址之间实现了一层封装（译者注，即内核也不会直接访问物理地址，而是在页表和 MMU 的帮助下通过虚拟地址访问物理内存），这使得内核可以控制地址空间的布局结构和内容。缺页异常使得内核可以拦截加载和存储操作，并通过修改页表，动态指定这些引用（即虚拟地址）指向的数据（物理内存）。内核可以利用这些功能来提高运行效率：例如，基于 “写时复制（copy-on-write）” 实现 fork 允许内核在父进程和子进程之间 “透明地（transparently）” 共享内存，从而避免在父子进程都没有发生写内存操作之前复制物理页的开销。编写应用程序得程序员也可以从中受益。譬如利用 内存映射文件（memory-mapped files） 技术，该技术在内核中利用分页将文件内容映射到应用程序的地址空间中（译者注，内存映射后虽然文件的内容可能还在磁盘上不在内存中，但给应用程序的感觉是可以使用访问内存的加载和存储指令对文件直接进行读取和写入），并在实际发生缺页异常时才将文件内容从磁盘上读入内存。另一种可能性是 “延迟内存分配（lazy memory allocation,）”，它允许程序请求一段巨大的虚拟地址空间，但只需为程序实际读写的页分配物理内存。在 xv6 中利用缺页异常只实现了一个特性，即 “延迟分配（lazy allocation）”。

> Before proceeding, please read the functions `sys_sbrk()` in `kernel/sysproc.c`, and `vmfault` in `kernel/vm.c`. Search for calls to `vmfault` in `kernel/trap.c` and `kernel/vm.c`.

在继续之前，请阅读 `kernel/sysproc.c` 中的 `sys_sbrk()` 函数，以及 `kernel/vm.c` 中的 `vmfault` 函数。在 `kernel/trap.c` 和 `kernel/vm.c` 中搜索哪里调用了函数 `vmfault`。

5.1 延迟分配（Lazy allocation）

> xv6's lazy allocation has two parts. First, when an application asks for memory by calling `sbrk` with the flag `SBRK_LAZY`, the kernel notes the increase in size, but does not allocate physical memory and does not create PTEs for the new range of virtual addresses. Second, on a page fault on one of those new addresses, the kernel allocates a page of physical memory and maps it into the page table. The kernel implements lazy allocation transparently to applications: no modifications to applications are necessary for them to benefit.

xv6 的 延迟分配 实现由两部分组成。首先，应用程序通过调用 `sbrk` 并在请求分配内存时传入 `SBRK_LAZY` 标志，内核只会记录内存大小的增加，但不会分配物理内存，也不会为新的虚拟地址范围创建页表项 (PTE)。其次，当涉及到的任何一个新地址上发生缺页异常时，内核会分配一页物理内存并将其映射到页表中。内核的延迟分配实现对应用程序来说是无感的：应用程序无需任何修改即可从中受益。

> Lazy allocation is convenient for applications because they don't have to accurately predict how much memory they will need. For example, an application may process input, but not know in advance how large the input will be. With lazy allocations an application can ask for memory for the worst case, but not have to pay for this worst case: the kernel doesn't have to do any work at all for pages that the application never uses.

延迟分配对于应用程序来说非常方便，因为对于应用程序来说无需准确预测所需的（实际）内存量。例如，应用程序可能会处理输入，但事先并不知道输入的大小。基于延迟分配，应用程序可以按照最极端的情况申请尽可能大的内存，而无需为此付出代价，因为内核并不需要为应用程序从未使用过的页分配实际的内存。

> Furthermore, if the application is asking to grow the address space by a lot, then `sbrk` without lazy allocation is expensive: if an application asks for a gigabyte of memory, the kernel has to allocate and zero 262,144 4096-byte physical pages. Lazy allocation allows this cost to be spread over time. On the other hand, lazy allocation incurs the extra overhead of page faults, which involve a user/kernel transition. Operating systems can reduce this cost by allocating a batch of consecutive pages per page fault instead of one page and by specializing the kernel entry/exit code for such page-faults (though xv6 does neither).

此外，如果应用程序要求大幅增加地址空间，那么如果 `sbrk` 不支持延迟分配将会非常耗时：因为假设应用程序请求 1GB 内存，内核必须（一次性）分配并清零 262144 个大小为 4096 字节的物理页。延迟分配机制可以将这部分开销分摊到多个时间点上去。但另一方面，延迟分配机制会引入缺页异常处理所带来的额外开销，这包括用户态和内核态之间切换等处理。操作系统可以通过在缺页

异常发生时分配一批连续的物理页（而不只是一页），并专门为此类缺页异常编写内核的入口和出口代码来抵消这部分开销（然而 xv6 并没有做这些优化）。

> On the other hand, when taking a page fault for a lazily-allocated page, the kernel may find that it has not free memory to allocate. In this case, the kernel has no easy way of returning an out-of-memory error to the application and instead kills the application. For applications that prefer an error on a failed allocation, xv6 allows an application to allocate memory eagerly by calling `sbrk` with the flag `SBRK_EAGER`.

另一方面，当延迟分配物理页时，内核可能会发现没有可用的内存可供分配了。在这种情况下，由于内核无法方便地给应用程序返回内存不足的错误（译者注，因为此时错误发生在异步的异常处理过程中，而不是源自应用程序发起的系统调用），所以内核选择直接终止该应用程序。对于那些希望在分配时就明确知道是否成功的应用程序，xv6 允许应用程序在调用 `sbrk` 时传入 `SBRK_EAGER` 标志来指明这一点。

5.2 代码讲解（Code）

> The system call `sbrk(n)` grows (or shrinks if `n` is negative) a process's memory size by n bytes, and returns the start of the newly allocated region (i.e., the old size). The kernel implementation is `sys_sbrk` (3801).

系统调用 `sbrk(n)` 会将进程的内存大小增加 `n` 个字节（如果 `n` 为负，则减少），并返回新分配区域的起始位置（即原先的大小）。内核中的实现对应为 `sys_sbrk` (3801)。

> If the application specifies `SBRK_EAGER`, the system call is implemented by the function `growproc` (2353). `growproc` calls `uvmalloc`. `uvmalloc` (1628) allocates physical memory with `kalloc`, zeros the allocated memory, and adds PTEs to the user page table with `mappages`.

如果应用程序指定了 `SBRK_EAGER`，则系统调用由函数 `growproc`（2353）实现。`growproc` 调用 `uvmalloc`。`uvmalloc`（1628）使用 `kalloc` 分配物理内存，将分配的内存清零，并使用 `mappages` 将 PTE 添加到用户页表中。

> If the applications allocates memory lazily, `sys_sbrk` just increments the process's size (`myproc()->sz`) by `n` and returns the old size; it does not allocate physical memory or add PTEs to the process's page table.

如果应用程序选择延迟分配内存，`sys_sbrk` 只会将进程的大小（`myproc()->sz`）增加 `n` 并返回原先的大小；它不会分配物理内存或为进程的页表添加 PTE。

> When a process loads or stores to a virtual address that lacks a valid page-table mapping, the CPU will raise page-fault exception. `usertrap` checks for this case (3372) and calls `vmfault` (1879) to handle the page fault. `vmfault` checks that the faulting address is within the region previously granted by `sbrk`, allocates a page of physical memory with `kalloc`, zeros the allocated page, and adds a PTE to the user page table with `mappages`. Xv6 sets the `PTE_W`, `PTE_R`, `PTE_U`, and `PTE_V` flags in the PTE for the new page. Then, `usertrap` resumes the process at the instruction that caused the fault. Because the PTE is now valid, the re-executed load or store instruction will execute without a fault.

当进程执行加载或存储操作时访问到没有有效映射的虚拟地址时，CPU 将触发 缺页（page-fault）异常。`usertrap` 中针对这种情况（3372）会调用 `vmfault` （1879）。`vmfault` 首先检查触发异常涉及的虚拟地址是否落在 `sbrk` 先前分配的区域内，然后使用 `kalloc` 分配一个物理内存页并将分配的页清零，最后调用 `mappages` 向用户页表添加 PTE。xv6 为新页的 PTE 设置 `PTE_W` 、`PTE_R` 、 `PTE_U` 和 `PTE_V` 标志。`usertrap` 返回后 CPU 从导致异常指令处恢复进程执行（译者注，即重新执行导致缺页异常的指令）。由于 PTE 现在有效，重新执行的加载或存储指令将不会出错。

> If an application frees memory using `sbrk` , `sys_sbrk` calls `shrinkproc` , which calls `uvmdealloc` . The real work is done by `uvmunmap` (1604), which uses `walk` to find PTEs. Since some pages may never have been used by the process and thus never have been allocated by `vmfault` , `uvmunmap` skips PTEs without the `PTE_V` flag. If a PTE mapping is valid, `uvmunmap` calls `kfree` to free the physical memory it refers to.

如果应用程序使用 `sbrk` 释放内存， `sys_sbrk` 会调用 `shrinkproc` ，而 `shrinkproc` 又会调用 `uvmdealloc` 。真正的工作由 `uvmunmap` (1604) 完成，它使用 `walk` 来查找 PTE。由于某些页（译者注：虚拟的）可能从未被进程使用过，因此 `vmfault` 也从未为其分配过物理内存， `uvmunmap` 会跳过没有 `PTE_V` 标志的 PTE。如果 PTE 映射有效， `uvmunmap` 会调用 `kfree` 来释放它所引用的物理内存。

> Note that Xv6 uses a process’ s page table not just to tell the hardware how to map user virtual addresses, but also as the only record of which physical memory pages are allocated to that process. That is the reason why freeing user memory (in `uvmunmap`) requires examination of the user page table.

注意 xv6 使用进程的页表不仅是为了告诉硬件如何映射用户虚拟地址，而且也正是在这份页表中唯一记录了为该进程分配了哪些物理内存页。这就是为什么释放用户内存（在 `uvmunmap` 中）需要检查用户页表的原因。

5.3 现实世界：写时拷贝 fork（Real world: Copy-On-Write (COW) fork）

> Many kernels (though not xv6) use page faults to implement copy-on-write (COW) fork. The `fork` system call promises that the child sees memory whose initial content is the same as the parent’ s memory at the time of the fork. One way to implement this is to copy the entire memory of the parent to newly allocated physical memory for the child; this is what xv6 does. Copying can be slow, and it would be more efficient if the child could share the parent’ s physical memory. A straightforward implementation of this would not work, however, since it would cause the parent and child to disrupt each other’ s execution with their writes to the shared stack and heap.

许多内核（xv6 除外）使用缺页异常来实现 写时复制 (copy-on-write，简称 COW) fork。`fork` 系统调用确保子进程看到的内存初始内容与父进程在 fork 时看到的内存内容相同。实现此操作的一种方法是将父进程的整个内存复制到为子进程新分配的物理内存中；xv6 就是这么做的。复制过程可能很慢，如果子进程可以共享父进程的物理内存，效率会更高。然而，直接实现此操作并不可行，因为这么做会使得父进程和子进程也共用了同一份栈和堆，任何一方改写了栈和堆的内容都会干扰另一方的运行。

> Copy-on-write fork causes parent and child to safely share physical memory by appropriate use of page-table permissions and page faults.The basic plan is for the parent and child to initially share all physical pages, but for each to map them read-only (with the `PTE_W` flag clear). Parent and child can then read from the shared physical memory. If either writes a shared page, the RISC-V CPU raises a page-fault exception. A kernel supporting COW would respond by allocating a new page of physical memory and copying the shared page into that new page. Then kernel would change the relevant PTE in the faulting process's page table to point to the copy and to allow writes as well as reads, and then resume the faulting process at the instruction that caused the fault. Because the PTE now allows writes, the re-executed store instruction will execute without a fault, and will modify a private copy of the page rather than the shared page.

COW fork 通过适当使用页表权限和缺页异常，使父进程和子进程可以安全地共享物理内存。其基本方案是父进程和子进程最初共享所有物理页面，但各自只以只读方式映射这些页面（即清除 `PTE_W` 标志）。父进程和子进程可以从共享的物理内存中读取数据。但如果其中任何一方对某个共享的页执行了写操作，RISC-V CPU 就会触发缺页异常。支持 COW 的内核的 trap 处理程序会做出响应，分配一个新的物理内存页，并将共享的物理页内容复制到该页中。内核会将出错进程页表中的相关 PTE 更改为指向新的副本，并允许对其写入和读取，然后在导致错误的指令处恢复出错进程的执行。由于 PTE 现在允许写入，因此重新执行的存储（store）指令将正确执行，此时被修改的页将是当前进程私有的副本而不再是共享的页。

> Copy-on-write requires book-keeping to help decide when physical pages can be freed, since each page can be referenced by a varying number of page tables depending on the history of forks, page faults, execs, and exits. This book-keeping allows an important optimization: if a process incurs a store page fault and the physical page is only referred to from that process's page table, no copy is needed.

因为 fork、缺页异常处理、exec 和 exit 等操作的发生都在不停地改变一个物理页被不同页表所引用的状态，所以引入 COW 处理后需要记录引用信息来帮助决定何时可以释放这些物理页。利用这些记录的引用信息可以实现重要的优化：譬如某个进程在写入物理内存时触发了缺页异常，但由于该物理页仅被当前此进程的页表所引用，则不需要触发 COW。

> Copy-on-write makes `fork` faster, since `fork` need not copy memory. Some of the memory will have to be copied later, when written, but it's often the case that most of the memory never has to be copied. A common example is `fork` followed by `exec`: a few pages may be written after the `fork`, but then the child's `exec` releases the bulk of the memory inherited from the parent. Copy-on-write `fork` eliminates the need to ever copy this memory. Furthermore, COW fork is transparent: no modifications to applications are necessary for them to benefit.

COW 使 `fork` 运行速度更快，因为 `fork` 无需复制内存。部分内存会稍后在发生写入操作时才会被复制，但通常情况下，大部分内存无需复制。一个常见的例子是 `fork` 后紧接着执行 `exec`：在 `fork` 之后可能会对少量页执行写操作，但随后子进程因为执行 `exec` 会释放从父进程继承的大部分内存。支持 COW 的 `fork` 则无需复制这些内存。此外，COW fork 对应用程序是 “透明的（transparent）”，即无需对应用程序进行任何修改即可从中受益。

5.4 现实世界：按需分页（Real world: Demand paging）

> Yet another widely-used feature that exploits page faults is demand paging. In the `exec` system call, xv6 loads all of an application’ s text and data into memory before starting the application. Since applications can be large and reading from disk takes time, this startup cost can be noticeable to users. To decrease startup time, a modern kernel doesn’ t initially load the executable file into memory, but just creates the user page table with all PTEs marked invalid. The kernel starts the program running; each time the program uses a page for the first time, a page fault occurs, and in response the kernel reads the content of the page from disk and maps it into the user address space. Like COW fork and lazy allocation, the kernel can implement this feature transparently to applications.

另一个广泛使用的，利用缺页异常的特性是 按需分页（demand paging）。在系统调用 `exec` 中，xv6 会在启动应用程序之前将应用程序的所有文本和数据加载到内存中。由于应用程序可能很大，并且从磁盘读取数据需要时间，因此用户可能会注意到这种启动成本。为了缩短启动时间，现代内核不会一开始就将可执行文件全部加载到内存中，而是仅创建用户页表，并将所有 PTE 标记为无效。内核启动程序后；每当一个物理页被程序首次访问时，都会发生缺页异常，作为响应，内核会从磁盘读取物理页内容并将其映射到用户地址空间。与 COW fork 和 lazy allocation 类似，内核可以对应用程序透明地实现此功能。

> The programs running on a computer may need more memory than the computer has RAM. To cope gracefully, the operating system may implement paging to disk. The idea is to store only a fraction of user pages in RAM, and to store the rest on disk in a paging area. The kernel marks PTEs that correspond to memory stored in the paging area (and thus not in RAM) as invalid. If an application tries to use one of the pages that has been paged out to disk, the application will incura page fault, and the page must be paged in: the kernel trap handler will allocate a page of physical RAM, read the page from disk into the RAM, and modify the relevant PTE to point to the RAM.

计算机上运行的程序所需要的内存可能比计算机实际安装的内存更多。为了妥善应对这种情况，操作系统可能需要实现 磁盘扩展分页（paging to disk） 的功能。其原理是只将一小部分用户程序的内容存放在内存中，其余部分则存储在磁盘中专门的 分页区域（paging area） 中。内核会将存储在分页区域（而非内存）中的页对应的 PTE 标记为无效。如果应用程序尝试使用已 换出（paged out） 到磁盘的页，则会触发缺页异常，这将导致内核将这些页 换入（paged in）：具体操作就是内核的 trap handler 将分配一个物理页，将已经换出到磁盘上的页从磁盘再次读入内存，并修改相关的 PTE 以指向该内存。

> What happens if a page needs to be paged in, but there is no free physical RAM? In that case, the kernel must first free a physical page by paging it out or evicting it to the paging area on disk, and marking the PTEs referring to that physical page as invalid. Eviction is expensive, so paging performs best if it’ s infrequent: if applications use only a subset of their memory pages and the union of the subsets fits in RAM. This property is often referred to as having good locality of reference. As with many virtual memory techniques, kernels usually implement paging to disk in a way that’ s transparent to applications.

如果某个页需要换入，但没有可用的物理内存，会发生什么情况呢？在这种情况下，内核必须首先释放一些物理页，方法是将某些内存中的页上的内容 “换出（page out）” 或 “驱逐（evicting）” 到磁盘上的分页区域中，并将引用该物理页的 PTE 标记为无效。驱逐操作的开销很

大，因此，如果这个操作不是很频繁的话，那尚可接受，也就是说如果应用程序仅使用其内存页的一部分，并且这部分内容都在内存中。此特性通常被称为具有良好的 “引用局部性（locality of reference）” 。与许多虚拟内存技术一样，内核通常以对应用程序透明的方式实现 paging to disk。

> Computers often operate with little or no free physical memory, regardless of how much RAM the hardware provides. For example, cloud providers multiplex many customers on a single machine to use their hardware cost-effectively. As another example, users run many applications on smart phones in a small amount of physical memory. In such settings allocating a page may require first evicting an existing page. Thus, when free physical memory is scarce, allocation is expensive.

无论我们给硬件配备多少内存，系统都不会觉得多，这导致计算机通常都只能在极少剩余甚至耗尽物理内存的情况下运行。例如，云服务提供商会在一台机器上为众多客户提供多路复用服务，以经济高效地利用硬件资源。再比如，用户在智能手机上只有少量的物理内存，但需要运行的应用程序却很多。在这种情况下，分配一个物理页往往需要先清除一个现有物理页。因此，当可用物理内存稀缺时，分配成本会非常高。

> Lazy allocation and demand paging are particularly advantageous when free memory is scarce and programs actively use only a fraction of their allocated memory. These techniques can also avoid the work wasted when a page is allocated or loaded but either never used or evicted before it can be used.

当可用内存稀缺但程序仅使用分配给它内存的一小部分时，lazy allocation 和 demand paging 的方案就有很大的优势。这些技术还可以避免因分配或加载物理页后但一直没有使用或在使用前被驱逐所造成的浪费。

5.5 现实世界：内存映射文件（Real world: Memory-mapped files）

> Other features that combine paging and page-fault exceptions include automatically extending stacks and memory-mapped files, which are files that a program maps into its address space using the `mmap` system call so that the program can read and write them using load and store instructions.

基于分页和缺页异常机制还可以实现其他功能，包括自动扩展栈和 内存映射文件（memory-mapped files），所谓 memory-mapped files 是指使用 `mmap` 系统调用将文件映射到进程的地址空间，以便程序可以使用加载和存储指令对文件直接进行读取和写入。

5.6 练习（Exercises）

> 1. Write a user program that grows its address space by one byte by calling `sbrk(1)`. Run the program and investigate the page table for the program before the call to

> `sbrk` and after the call to `sbrk`. How much space has the kernel allocated? What does the PTE for the new memory contain?

1. 编写一个用户程序，通过调用 `sbrk(1)` 将其地址空间增加一个字节。运行该程序，并查看调用 `sbrk` 之前和之后程序的页表。内核分配了多少空间？新内存的 PTE 包含什么内容？

> 1. Implement COW fork.

1. 实现 COW fork。

> 1. Implement `mmap`.

1. 实现 `mmap`。

第 6 章 中断和设备驱动（Chapter 6 Interrupts and device drivers）

> A driver is the code in an operating system that manages a particular device: it configures the device hardware, tells the device to perform operations, handles the resulting interrupts, and interacts with processes using the device. Driver code can be tricky because a driver executes concurrently with the device, and often concurrently with processes using the device. In addition, the driver must understand the device’s hardware interface, which can be complex and poorly documented.

驱动（driver） 是操作系统中管理特定设备的代码：它配置硬件设备、通知设备执行操作、处理产生的中断以及与正在使用设备的进程进行交互。编写驱动程序代码可能比较棘手，因为执行驱动程序的硬件（处理器）和驱动程序管理的硬件并非同一个设备，并且通常与使用该设备的进程并行运行。（译者注：这也是下文要提到的中断等异步处理的来源）。此外，编写驱动程序必须了解设备的硬件接口，而这可能很复杂且常常缺少良好的文档说明。

> Devices that need attention from the operating system can usually be configured to generate interrupts, which are one type of trap. The kernel trap handling code recognizes when a device has raised an interrupt and calls the driver’s interrupt handler; in xv6, this dispatch happens in `devintr` (3506).

操作系统管理的设备通常需要配置为产生 “中断（interrupt）”，这是属于 trap 的一种。当设备触发中断时内核的 trap 处理代码会被执行并调用驱动程序的中断处理程序；在 xv6 中，这个处理发生在 `devintr` (3506) 中。

> Many device drivers execute code in two contexts: a top half that runs in a process’s kernel thread, and a bottom half that executes at interrupt time. The top half is called via system calls such as `read` and `write` that want the device to perform I/O. This code may ask the hardware to start an operation (e.g., ask the disk to read a block); then the code waits for the operation to complete. Eventually the device completes the operation and raises an interrupt. The driver’s interrupt handler, acting as the bottom half, figures out what operation has completed, wakes up a waiting process if appropriate, and tells the hardware to start work on the next operation, if any.

许多设备驱动程序的代码在两种上下文环境中执行：一种称之为 上半部分（top half）, 即在进程的内核线程中执行，另一种称之为 下半部分（bottom half），即运行在中断中。上半部分通过 “系统调用（system call）” 触发，譬如调用 `read` 和 `write` 通知设备执行输入输出操作。此代码可能会要求硬件启动某些操作（例如，要求磁盘读取一个 “块（block）”）；然后代码等待操作完成。最终，设备完成操作并触发中断。此时驱动程序的中断处理程序开始运行，执行下半部分，判断已完成的操作是什么，如果需要的话还要唤醒等待的进程，以及告诉硬件启动下一个操作。

6.1 代码讲解：控制台输入（Code: Console input）

> The console driver (6950) is a simple illustration of driver structure. The console driver accepts characters typed by a human, via the UART serial-port hardware attached to the RISC-V. The console driver accumulates a line of input at a time, processing special input characters such as backspace and control-u. User processes, such as the shell, use the `read` system call to fetch lines of input from the console. When you type input to xv6 in QEMU, your keystrokes are delivered to xv6 by way of QEMU’ s simulated UART hardware.

“控制台（console）” 驱动程序 (6950) 是一个很好的描述驱动程序结构的例子。控制台驱动程序通过连接到 RISC-V 处理器的 UART 串行端口设备接收人工输入的字符（译者注：UART 的全称是 Universal Asynchronous Receiver/Transmitter）。控制台驱动程序每次会累积一行的字符输入，这期间会处理其中的一些特殊的输入字符，例如 “退格键（backspace）” 和 control-u（译者注：指同时按下 Ctrl 键和字符 u 键）。用户进程（例如 shell）使用 `read` 系统调用从控制台获取输入行。当你在 QEMU 中向 xv6 输入时，你的按键产生的字符数据将通过 QEMU 模拟的 UART 硬件传递给 xv6。

> The UART hardware that the driver talks to is a 16550 chip [13] emulated by QEMU. On a real computer, a 16550 would manage an RS232 serial link connecting to a terminal or other computer. When running QEMU, it’ s connected to your keyboard and display.

驱动程序负责控制的 UART 硬件是 QEMU 模拟的 16550 芯片 [13]。在真实计算机上，16550 芯片会管理一个连接到终端或其他计算机的 RS232 串行链路。在 QEMU 上，模拟的 16550 芯片直接连接到键盘和显示器。

> The UART hardware appears to software as a set of memory-mapped control registers. That is, there are some physical addresses that are connected to the UART device, so that loads and stores interact with the device hardware rather than RAM. The memory-mapped addresses for the UART start at 0x10000000, or `UART0` (0220). There are a handful of UART control registers, each the width of a byte. Their offsets from `UART0` are defined in (7221). For example, the `LSR` register contains bits that indicate whether input characters are waiting to be read by the driver. These characters (if any) are available for reading from the `RHR` register. Each time one is read, the UART hardware deletes it from an internal FIFO of waiting characters, and clears the “ready” bit in `LSR` when the FIFO is empty. To transmit, the driver writes a byte to the `THR` register, which causes the UART to append the byte to a FIFO of bytes that the UART will send on the RS232 serial link. The UART transmit and receive hardware are largely independent of each other.

对于 UART 硬件，从软件的角度来看，就是一组 内存映射（memory-mapped） 的控制寄存器。也就是说，一些物理地址连接着 UART 设备（译者注：即我们可以通过这些物理地址访问 UART 设备），同样是调用 “加载（load）” 和 “存储（store）” 指令，但此时访问的硬件是外设而不是内存。对于 UART 设备来说其内存映射的起始物理地址是 0x10000000，代码中对应的是宏常量 `UART0` (0220)。UART 有少量控制寄存器，每个寄存器的宽度为一个字节。它们相对于 `UART0` 的偏移量值定义在 (7221) 中。例如， `LSR` 寄存器中的比特位用于指示输入字符是否正在等待驱动读取。我们可以从 `RHR` 寄存器中读取字符（如果有的话）。每次读取一个字符时，UART 硬件都会将其从一个内部缓存字符的 “队列（FIFO，First In First Out 的缩写）” 中删除，并在队列为空时清除 `LSR` 中的 “就绪（ready）” 位。要发送数据时，驱动程序会将一个字节写入 `THR` 寄存器，这会

导致 UART 将该字节添加到另一个队列中，而这个队列包含了 UART 将在 RS232 串行链路上发送的字节。UART 中负责发送和接收的硬件单元在很大程度上是相互独立的。

> Xv6's `main` calls `consoleinit` (7154) to initialize the UART hardware. This code configures the UART to generate a receive interrupt when the UART receives each byte of input, and a transmit complete interrupt each time the UART finishes sending a byte of output (7251).

xv6 的 `main` 函数调用 `consoleinit` (7154) 来初始化 UART 硬件。这段代码配置 UART，使其在接收到每个输入字节时产生一个接收中断，并在每次发送完一个输出字节时产生一个 传输完成（transmit complete） 中断 (7251)。

> The xv6 shell reads from the console by way of a file descriptor opened by `init.c` (7768). Calls to the `read` system call make their way through the kernel to `consoleread` (7040). `consoleread` waits for input to arrive (via interrupts) and be buffered in `cons.buf`, copies the input to user space, and (after a whole line has arrived) returns to the user process. If the user hasn't typed a full line yet, any reading processes will wait in the `sleep` call (7056) (Chapter 9 explains the details of `sleep`).

在 `init.c` (7768) 中, xv6 的 shell 程序通过打开一个文件描述符从控制台读取数据。调用系统调用 `read` 在内核中会执行 `consoleread` (7040)。`consoleread` 等待输入到达（通过中断），并得到缓存在 `cons.buf` 中的输入字符，`consoleread` 在将累积的一整行输入复制到用户空间后结束读取并返回用户空间。如果用户尚未输入整行，则所有读取进程都会阻塞在 `sleep` 调用 (7056) 中（第 9 章详细介绍了 `sleep`）。

> When the user types a character, the UART hardware asks the RISC-V to raise an interrupt, which activates xv6's trap handler. The trap handler calls `devintr` (3506), which looks at the RISC-V `scause` register to discover that the interrupt is from an external device. Then it asks a hardware unit called the PLIC [3] to tell it which device interrupted (3514). If it was the UART, `devintr` calls `uartintr`.

当用户输入字符时，UART 硬件会向 RISC-V 处理器发出中断，从而激活 xv6 的 trap “处理函数（handler）”。trap handler 会调用 `devintr` (3506)，它会查看 RISC-V 的 `scause` 寄存器，以确定中断是否来自外部设备。如果是来自外部设备，它会访问一个名为 PLIC [3] 的硬件模块从而得知是哪个设备发出了中断 (3514)。如果是 UART，`devintr` 则调用 `uartintr`。

> `uartintr` (7354) reads any waiting input characters from the UART hardware and hands them to `consoleintr` (7107); it doesn't wait for characters, since future input will raise a new interrupt. The job of `consoleintr` is to accumulate input characters in `cons.buf` until a whole line arrives. `consoleintr` treats backspace and a few other characters specially. When a newline arrives, `consoleintr` wakes up a waiting `consoleread` (if there is one).

`uartintr` (7354) 从 UART 硬件读取所有等待的输入字符，并将它们传递给 `consoleintr` (7107)；它不会等待字符，因为后续的输入会引发新的中断。`consoleintr` 的作用是将输入字符累积到 `cons.buf` 中，直到满足一行的条件。`consoleintr` 会对 “退格键（backspace）” 和其他一些字符进行特殊处理。当 “换行符（newline）” 到达时，`consoleintr` 会唤醒正在等待的 `consoleread`（如果有的话）。

> Once woken, `consoleread` will observe a full line in `cons.buf`, copy it to user space, and return (via the system call machinery) to user space.

一旦被唤醒，`consoleread` 将会从 `cons.buf` 中获取一整行字符并将其复制到用户空间，然后（通过系统调用机制）返回到用户空间。

6.2 代码讲解：控制台输出（Code: Console output）

> A `write` system call on a file descriptor connected to the console eventually arrives at `uartputc` (7309). The device driver maintains an output buffer (`uart_tx_buf`) so that writing processes do not have to wait for the UART to finish sending; instead, `uartputc` appends each character to the buffer, calls `uartstart` to start the device transmitting (if it isn’t already), and returns. The only situation in which `uartputc` waits is if the buffer is already full.

对连接到控制台的文件描述符执行 `write` 系统调用时，程序执行路径最终会到达 `uartputc` (7309)。设备驱动程序维护一个输出缓冲区 (`uart_tx_buf`)，以便写入进程无需等待 UART 发送完成；`uartputc` 会将每个字符追加到缓冲区，调用 `uartstart` 启动设备发送（如果尚未启动），然后返回。唯一会让 `uartputc` 等待的情况是缓冲区已满。

> Each time the UART finishes sending a byte, it generates an interrupt. `uartintr` calls `uartstart`, which checks that the device really has finished sending, and hands the device the next buffered output character. Thus if a process writes multiple bytes to the console, typically the first byte will be sent by `uartputc`’s call to `uartstart`, and the remaining buffered bytes will be sent by `uartstart` calls from `uartintr` as transmit complete interrupts arrive.

UART 每次发送完一个字节后，都会产生一个中断。`uartintr`（中断处理程序）会调用 `uartstart`，后者会检查设备是否确实已完成发送，并将下一个缓冲的输出字符交给设备。因此，如果一个进程向控制台写入多个字节，通常第一个字节会由 `uartputc` 调用 `uartstart` 发送，其余缓冲的字节则会在 “发送完成” 中断到达时由 `uartintr` 调用 `uartstart` 发送。

译者注：上面两段的描述过时了，和代码匹配不上。下面按照自己的理解重新简单写一下。

对连接到控制台的文件描述符执行 `write` 系统调用时，程序执行路径最终会到达 `uartwrite` (7782)。该函数会和 `uartintr` 配合，在 "transmit complete" 中断的驱动下将参数 `buf` 数组中的字符逐个发送出去。具体做法是在 `uartwrite` 中有一个循环，检测 `tx_busy` 这个标志是否为真（1）。如果为真则调用 `sleep` 在 `tx_chan` 上等待。`uartintr` 这个中断处理函数中如果发现收到 "transmit complete" 中断，则将 `tx_busy` 改为假（0）并唤醒睡眠的进程，进程醒来后会从 `buf` 中取出一个字符发送出去，并将 `tx_busy` 重置为 1 后进入下一个等待循环。如此往复，直到将 `buf` 中的字符全部发送完毕。

> A general pattern to note is the decoupling of device activity from process activity via buffering and interrupts. The console driver can process input even when no process is waiting to read it; a subsequent read will see the input. Similarly, processes can send output without having to wait for the device. This decoupling can increase performance by allowing processes to execute concurrently with device I/O, and is particularly important

> when the device is slow (as with the UART) or needs immediate attention (as with echoing typed characters). This idea is sometimes called I/O concurrency.

注意我们这里用到了一个通用设计模式，即通过缓冲和中断将设备活动与进程活动解耦。即使没有进程在等待读取输入，控制台驱动程序也会处理输入（译者注：驱动会将其收到的数据先缓存起来）；后续（进程的）读取操作将直接读取（驱动程序缓存的）输入。同样，进程可以发送输出而无需等待设备完成。这种解耦允许进程与设备 I/O 并发执行，从而提高性能，尤其是在设备速度较慢（例如 UART）或需要立即处理（例如回显输入字符）时尤为重要。这种理念有时被称为 输入/输出并发(I/O concurrency)。

6.3 驱动中的并发（Concurrency in drivers）

> You may have noticed calls to `acquire` in `consoleread` and in `consoleintr`. These calls acquire a lock, which protects the console driver's data structures from concurrent access. There are three concurrency dangers here: two processes on different CPUs might call `consoleread` at the same time; the hardware might ask a CPU to deliver a console (really UART) interrupt while that CPU is already executing inside `consoleread`; and the hardware might deliver a console interrupt on a different CPU while `consoleread` is executing. Chapter 7 explains how to use locks to ensure that these dangers don't lead to incorrect results.

您可能已经注意到在 `consoleread` 和 `consoleintr` 中对 `acquire` 的调用。这些调用会获取一个锁，用于保护控制台驱动程序的数据结构免受并发访问的影响。这里存在三种并发风险：首先，不同 CPU 上的两个进程可能同时调用 `consoleread`；其次，当 `consoleread` 在一个 CPU 上执行时，硬件可能会触发同一个 CPU 上的控制台中断（实际上是由于收到 UART 中断）；再次，`consoleread` 在一个 CPU 上执行的同时在另一个 CPU 上触发控制台中断。第 7 章将解释如何使用锁来确保这些风险不会导致错误的结果。

> Another way in which concurrency requires care in drivers is that one process may be waiting for input from a device, but the interrupt signaling arrival of the input may arrive when a different process (or no process at all) is running. Thus interrupt handlers are not allowed to think about the process or code that they have interrupted. For example, an interrupt handler cannot safely call `copyout` with the current process's page table. Interrupt handlers typically do relatively little work (e.g., just copy the input data to a buffer), and wake up top-half code to do the rest.

驱动程序中针对并发处理还需要注意的另一个方面是，一个进程可能正在等待来自设备的输入，但输入的中断信号到达时，可能正在运行的是另一个进程（或当前根本没有进程在运行）。因此，中断处理程序不能对被其中断的进程或代码有任何假设。例如，中断处理程序直接基于当前进程的页表来调用 `copyout` 是危险的。中断处理程序通常只执行相对较少的工作（例如，仅将输入数据复制到缓冲区），然后唤醒 top half 代码来完成剩余的工作。

6.4 定时器中断（Timer interrupts）

> Xv6 uses timer interrupts to maintain its idea of the current time and to switch among computebound processes. Timer interrupts come from clock hardware attached to each RISC-V CPU. Xv6 programs each CPU’s clock hardware to interrupt the CPU periodically.

xv6 使用定时器中断来保持对当前时间的感知，并在 “计算密集型（computebound）” 进程之间切换（译者注：所谓 “计算密集型” 进程是指那些不轻易主动放弃处理器的进程，譬如执行 `while (1) i++` 这样的指令序列）。定时器中断来自连接到每个 RISC-V CPU 的时钟硬件。xv6 会对每个 CPU 的时钟硬件进行配置，使其定期产生中断。

> Code in `start.c` (1102) sets some control bits that allow supervisor-mode access to the timer control registers, and then asks for the first timer interrupt. The `time` control register contains a count that the hardware increments at a steady rate; this serves as a notion of the current time. The `stimecmp` register contains a time at which the the CPU will raise a timer interrupt; setting `stimecmp` to the current value of `time` plus x will schedule an interrupt x time units in the future. For `qemu`’s RISC-V emulation, 1000000 time units is roughly a tenth of second.

`start.c` (1102) 中的代码（译者注：即 `timerinit` 函数）设置了一些控制位，允许我们在管理员模式下访问定时器控制寄存器，然后启动第一个定时器中断。控制寄存器 `time` 包含一个计数值，硬件会以稳定的频率递增该计数值；我们可以利用它来表示当前时间。`stimecmp` 寄存器可以用于设置 CPU 触发定时器中断的间隔；将 `stimecmp` 设置为 `time` 的当前值加上 x 将会在未来 x 个时间单位内产生一次中断。对于 `qemu` 仿真的 RISC-V 硬件平台上，1000000 个这样的时间单位大约等于十分之一秒。

> Timer interrupts arrive via `usertrap` or `kerneltrap` and `devintr`, like other device interrupts. Timer interrupts arrive with `scause`’s low bits set to five; `devintr` in `trap.c` detects this situation and calls `clockintr` (3482). The latter function increments `ticks`, allowing the kernel to track the passage of time. The increment occurs on only one CPU, to avoid time passing faster if there are multiple CPUs. `clockintr` wakes up any processes waiting in the `pause` system call, and schedules the next timer interrupt by writing `stimecmp`.

与其他设备中断一样，定时器中断到达时会触发调用 `usertrap` 或 `kerneltrap`，并最终调用 `devintr`。定时器中断到达时， `scause` 的低位值被设置为 5； `trap.c` 中的 `devintr` 会根据这个标记调用 `clockintr` (3482)。后者会增加 `ticks`，使内核能够跟踪时间的流逝。对 `ticks` 的递增只会发生在一个 CPU 上（译者注：即 `cpuid() == 0` 的 CPU 上），以避免在多处理器情况下时间流逝过快（译者注： `ticks` 是一个全局变量，如果多个处理器都会修改它自然会加快计时的频率，而且整体频率也不固定）。此外 `clockintr` 还会唤醒在 `pause` 系统调用中等待的进程，并通过写入 `stimecmp` 来调度下一个定时器中断。

> `devintr` returns 2 for a timer interrupt in order to indicate to `kerneltrap` or `usertrap` that they should call `yield` so that CPUs can be multiplexed among runnable processes.

`devintr` 针对定时器中断的情况其返回值是 2，如果是这种情况， `kerneltrap` 或 `usertrap` 会调用 `yield`，这样 CPU 就可以被其他 “就绪（runnable）” 的进程复用（译者注： `yield` 会迫使 CPU 上的进程主动让出处理器给其他就绪的进程）。

> The fact that kernel code can be interrupted by a timer interrupt that forces a context switch via `yield` is part of the reason why early code in `usertrap` is careful to save state such as `sepc` before enabling interrupts. These context switches also mean that kernel code must be written in the knowledge that it may move from one CPU to another without warning.

内核代码的执行可以被定时器中断打断，并通过 `yield` 强制进行上下文切换，这也是 `usertrap` 在函数开始部分在启用中断之前小心地保存 `sepc` 等状态的原因之一。这些上下文切换也意味着内核代码必须在编写时考虑到进程可能会在没有任何感觉的情况下从一个 CPU 被转移到另一个 CPU。

6.5 现实世界（Real world）

> Xv6, like many operating systems, allows interrupts and even context switches (via `yield`) while executing in the kernel. The reason for this is to retain quick response times during complex system calls that run for a long time. However, as noted above, allowing interrupts in the kernel is the source of some complexity; as a result, a few operating systems allow interrupts only while executing user code.

与许多操作系统一样，xv6 在内核态执行时允许中断，甚至允许上下文切换（通过 `yield`）。这样做的目的是为了在执行一些耗时较长的复杂系统调用时保持快速响应。然而，如上所述，在内核中允许中断会带来一些复杂性；因此，一些操作系统仅在用户态执行代码时才允许中断。

> Supporting all the devices on a typical computer in its full glory is much work, because there are many devices, the devices have many features, and the protocol between device and driver can be complex and poorly documented. In many operating systems, the drivers account for more code than the core kernel.

在典型的计算机上全面支持所有设备是一项艰巨的工作，因为设备数量众多，功能各异，而且设备和驱动程序之间的协议可能非常复杂，文档也很少。在许多操作系统中，驱动程序的代码量甚至超过了内核的核心部分的代码量。

> The UART driver retrieves data a byte at a time by reading the UART control registers; this pattern is called programmed I/O, since software is driving the data movement. Programmed I/O is simple, but too slow to be used at high data rates. Devices that need to move lots of data at high speed typically use direct memory access (DMA). DMA device hardware directly writes incoming data to RAM, and reads outgoing data from RAM. Modern disk and network devices use DMA. A driver for a DMA device would prepare data in RAM, and then use a single write to a control register to tell the device to process the prepared data.

UART 驱动程序通过读取 UART 控制寄存器，一次获取一个字节的数据；由于数据的搬运由软件驱动，这种模式称为 程序控制输入输出（programmed I/O）。程序控制输入输出实现简单简单，但速度太慢，无法支持高速吞吐的需求。需要高速搬运大量数据的设备通常使用 直接内存访问（direct memory access）（简称 DMA)。DMA 设备硬件直接将读取到的数据写入 RAM，并从 RAM 中将数据取出发送出去。现代磁盘和网络设备都使用 DMA。DMA 设备的驱动程序会将（需要发送的）数据在内存中准备好，然后只要通过对（DMA 设备的）控制寄存器发起一次写操作，通知设备自己去处理准备好的数据就好了。

> Interrupts make sense when a device needs attention at unpredictable times, and not too often. But interrupts have high CPU overhead. Thus high speed devices, such as network and disk controllers, use tricks that reduce the need for interrupts. One trick is to raise a single interrupt for a whole batch of incoming or outgoing requests. Another trick is for the driver to disable interrupts entirely, and to check the device periodically to see if it needs attention. This technique is called polling. Polling makes sense if the device performs operations at a high rate, but it wastes CPU time if the device is mostly idle. Some drivers dynamically switch between polling and interrupts depending on the current device load.

对于设备来说，如果其等待的事件的发生是不可预期的，且频率不高时，中断是有意义的。但中断的 CPU 开销很高。因此，高速设备（例如网络和磁盘控制器）会使用一些技巧来减少对中断的需求。一个技巧是为需要批处理的输入或输出只触发一个中断。另一个技巧是驱动程序完全禁用中断，而是定期检查设备的状态。这种技术称为 轮询（polling）。如果设备需要高速执行某些操作，轮询是有意义的，但如果设备大部分时间处于空闲状态，轮询会浪费 CPU 时间。某些驱动程序会根据当前设备负载动态地在轮询和中断两种处理方式之间切换。

> The UART driver copies incoming data first to a buffer in the kernel, and then to user space. This makes sense at low data rates, but such a double copy can significantly reduce performance for devices that generate or consume data very quickly. Some operating systems are able to directly move data between user-space buffers and device hardware, often with DMA.

UART 驱动程序会将传入的数据先复制到内核的缓冲区中，然后再复制到用户空间去。这在低速处理下是合理的，但对于产生数据或消费数据速度都非常快的设备来说，这种两次复制会显著降低性能。某些操作系统能够直接在用户空间缓冲区和设备硬件之间搬运数据，这通常也是通过使用 DMA。

> As mentioned in Chapter 1, the console appears to applications as a regular file, and applications read input and write output using the `read` and `write` system calls. Applications may want to control aspects of a device that cannot be expressed through the standard file system calls (e.g., enabling/disabling line buffering in the console driver). Unix operating systems provide an `ioctl` system call for such cases.

如第 1 章所述，控制台对应用程序而言就像一个普通文件，应用程序同样可以使用 `read` 和 `write` 这一对系统调用实现读取输入和发送输出。此外，应用程序可能需要对设备发起一些操作但这些操作无法简单地通过类似 `read` 和 `write` 这样的标准文件系统调用的方式来表达（例如，在控制台驱动程序中启用或者禁用行缓冲）。Unix 操作系统提供一个像 `ioctl` 这样的系统调用来处理这种情况。

> Some uses of computers require “real-time” responses to external events: responses guaranteed to occur within a bounded time. For example, in safety-critical systems missing a deadline can lead to disasters. Xv6 is not suitable for real-time settings. Among other things, xv6’s scheduler does not take into account real-time deadlines when it decides what process to run next, and xv6 has long kernel code paths with interrupts disabled, so that it may not respond to interrupts quickly. A real-time operating system must not only fix these problems, but also be structured in a way that allows analysis of worst-case response times.

在计算机的某些使用中需要对外部事件 “实时（real-time）” 地响应：即保证在限定的时间内产生响应。例如，在一个 “安全关键型（safety-critical）” 系统中，错过截止时间可能会导致严重的灾难后果。xv6 并不适合实时场景。首先，xv6 的调度程序在决定下一个可运行的进程时并没有在截止时间上考虑要满足实时要求，而且 xv6 在禁止中断的情况下其内核代码的执行路径过长，因此可能无法快速响应中断。实时操作系统不仅必须解决这些问题，还必须在设计上支持对最坏情况下的响应时间进行分析。

6.6 练习（Exercises）

> 1. Modify `uart.c` to not use interrupts at all. You may need to modify `console.c` as well.

1. 修改 `uart.c` 使其完全不使用中断。您可能还需要修改 `console.c` 。

> 1. Add a driver for an Ethernet card.

1. 添加以太网卡的驱动程序。

第 7 章 锁（Chapter 7 Locking）

> Most kernels, including xv6, interleave the execution of multiple activities. One source of interleaving is multiprocessor hardware: computers with multiple CPUs executing independently, such as xv6’ s RISC-V. These multiple CPUs share physical RAM, and xv6 exploits the sharing to maintain kernel data structures that all CPUs read and write. This sharing raises the possibility of one CPU reading a data structure while another CPU is mid-way through updating it, or even multiple CPUs updating the same data simultaneously; without careful design such parallel access is likely to yield incorrect results or a broken data structure. Even on a uniprocessor, the kernel may switch the CPU among a number of threads, causing their execution to be interleaved. Finally, a device interrupt handler that modifies the same data as some interruptible code could damage the data if the interrupt occurs at just the wrong time. The word concurrency refers to situations in which multiple instruction streams are interleaved, due to multiprocessor parallelism, thread switching, or interrupts.

大多数内核（包括 xv6）都会交错执行多个活动。交错的原因之一是多处理器系统（即具有多个独立执行的 CPU 的计算机）的存在，运行 xv6 的 RISC-V（模拟器）就是一个多处理器系统。这些 CPU 共享物理内存，xv6 基于这种共享特性维护着供所有 CPU 都可读写的内核数据结构。这种共享特性增加了一个 CPU 读取数据结构期间另一个 CPU 更新该数据结构的可能性，甚至多个 CPU 同时更新同一份数据；如果没有精心的设计，这种并行访问很可能会产生错误的结果甚至破坏数据结构。即使在单处理器系统上，内核也可能在多个线程之间切换 CPU，导致它们交错执行。最后，如果设备中断处理程序与某些可能被中断打断的代码会访问和修改同一份数据，如果不做额外的保护，数据同样会被损坏。并发（concurrency） 一词指的正是上面所提到的由于多处理器并行、线程切换或中断导致多个指令流交错执行的情况。

> Kernels are full of concurrently-accessed data. For example, two CPUs could simultaneously call `kalloc`, thereby concurrently popping from the head of the free list. Kernel designers like to allow for lots of concurrency, since it can yield increased performance through parallelism, and increased responsiveness. However, as a result kernel designers must convince themselves of correctness despite such concurrency. There are many ways to arrive at correct code, some easier to reason about than others. Strategies aimed at correctness under concurrency, and abstractions that support them, are called concurrency control techniques.

内核中充满了并发访问的数据。例如，两个 CPU 可以同时调用 `kalloc`，导致同时从空闲列表的头部弹出数据。内核设计人员喜欢大量使用并发，因为利用并行性可以提高性能，从而加快响应速度。然而，这么做的代价是，内核设计人员必须确保在并发情况下代码仍然能够正确工作。有很多方法可以实现这个目标，有些方法比较好理解，而有些则比较复杂。旨在确保并发环境下代码正确性的策略，以及支持这些策略的函数设计，被称为 并发控制（concurrency control） 技术。

> Xv6 uses a number of concurrency control techniques, depending on the situation; many more are possible. This chapter focuses on a widely used technique: the lock. A lock provides mutual exclusion, ensuring that only one CPU at a time can hold the lock. If the programmer associates a lock with each shared data item, and the code always holds the

> associated lock when using an item, then the item will be used by only one CPU at a time. In this situation, we say that the lock protects the data item. Although locks are an easy-to-understand concurrency control mechanism, the downside of locks is that they can limit performance, because they serialize concurrent operations.

xv6 视具体情况使用了多种并发控制技术；这类技术还有很多。本章重点介绍一种广泛使用的技术：锁（lock）。锁提供互斥功能，确保同一时间只有一个 CPU 可以持有锁。如果程序员为每个共享数据项关联一个锁，并且代码在访问该数据项时始终持有关联的锁，那么该数据项同一时间将只能被一个 CPU 使用。在这种情况下，我们称锁保护了该数据项。虽然锁是一种易于理解的并发控制机制，但它的缺点是会影响性能，因为它们会导致一些并发操作退化成串行操作。

> The rest of this chapter explains why xv6 needs locks, how xv6 implements them, and how it uses them.

本章的其余部分解释了为什么 xv6 需要锁、xv6 如何实现锁以及又是如何使用它们的。

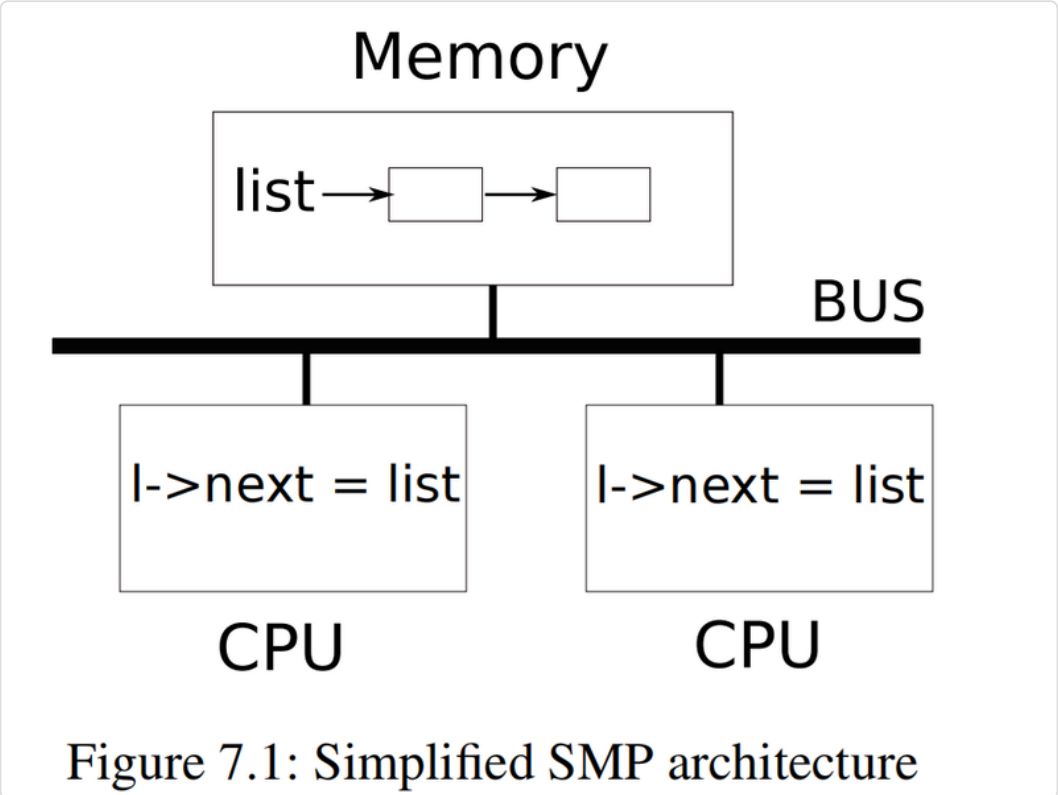

Figure 7.1: Simplified SMP architecture

7.1 竞争（Races）

> As an example of why we need locks, consider two processes with exited children calling the `wait` system call on two different CPUs. `wait` frees the child’s memory. Thus on each CPU, the kernel will call `kfree` to free the children’s memory pages. The kernel allocator maintains a linked list of free pages: `kalloc()` (3027) pops a page of memory from the list, and `kfree()` (3005) pushes a page onto the free list. For best performance, we might hope that the `kfree`s of the two parent processes would execute in parallel without either having to wait for the other, but this would not be correct given xv6’s `kfree` implementation.

举个例子来说明为什么我们需要锁，假设存在两个进程，它们的子进程已退出，这两个进程分别在两个不同的 CPU 上调用系统调用 `wait`。`wait` 会释放子进程的内存。因此，在两个 CPU 上，内核都会调用 `kfree` 来释放子进程的内存页。内核的内存分配器维护了一个空闲物理页的链表：`kalloc()` (3027) 从这个链表中弹出一个内存页，`kfree()` (3005) 将一个内存页插入到这个链表中。为了获得最佳性能，我们当然希望两个父进程的 `kfree` 能够并行执行，而无需等待对方，但考虑到 xv6 的 `kfree` 实现，这显然是行不通的。

> Figure 7.1 illustrates the setting in more detail: the linked list of free pages is in memory that is shared by the two CPUs, which manipulate the list using load and store instructions. (In reality, the processors have caches, but conceptually multiprocessor systems behave as if there were a single, shared memory.) If there were no concurrent requests, you might implement a list `push` operation as follows:

图 7.1 更详细地说明了这种场景：空闲页的链表位于两个 CPU 共享的内存中，这两个 CPU 使用“加载（load）”和“存储（store）”指令来操作该链表（译者注：在谈及 CPU 时，通常会把对内存的读操作称为 load，对内存的写操作称为 store）。（实际上，处理器都有缓存，但从概念上讲，多处理器系统的行为就像只有一个共享的内存一样。）如果没有并发请求，我们可以按如下方式对链表实现 `push` 操作：

```
1 struct element {
2   int data;
3   struct element *next;
4 };
5
6 struct element *list = 0;
7
8 void
9 push(int data)
10 {
11   struct element *l;
12
13   l = malloc(sizeof *l);
14   l->data = data;
15   l->next = list;
16   list = l;
17 }
```

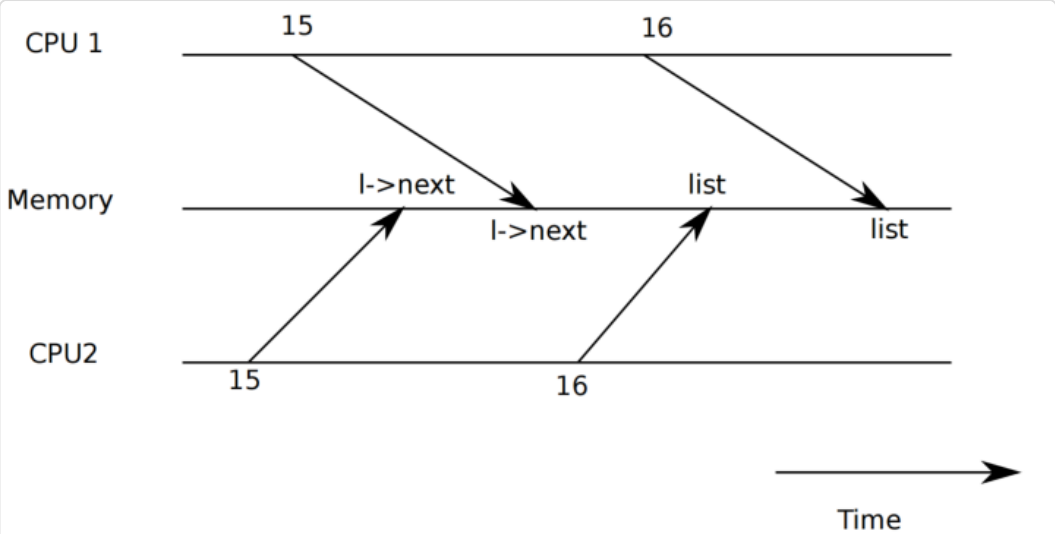

Figure 7.2: Example race

> This implementation is correct if executed in isolation. However, the code is not correct if more than one copy executes concurrently. If two CPUs execute `push` at the same time, both might execute line 15 as shown in Fig 7.1, before either executes line 16, which results in an incorrect outcome as illustrated by Figure 7.2. There would then be two list elements with `next` set to the same former value of `list`. When the two assignments to `list` happen at line 16, the second one will overwrite the first; the element involved in the first assignment will be lost.

如果一次只有一个实例执行，这么实现是没有问题的。但是，如果多个实例同时运行则会出问题。具体来说就是说如果两个 CPU 同时执行 `push`，则两个 CPU 可能都先执行上面代码所示的第 15 行（译者注：即图 7.1 中 CPU 上描述的 `l->next = list`），然后再执行第 16 行，从而导致如图 7.2 所示的错误结果。（CPU1 和 CPU2 分别执行第 15 行后）会有两个（新增）链表节点的 `next` 都指向 `list` 先前所指向的 element 对象。同时当第 16 行对 `list` 进行两次赋值时，第二个赋值将覆盖第一个赋值；也就是说第一个赋值中涉及的 element 对象将会丢失。

> The lost update at line 16 is an example of a race. A race is a situation in which a memory location is accessed concurrently, and at least one access is a write. A race is often a sign of a bug, either a lost update (if the accesses are writes) or a read of an incompletely-updated data structure. The outcome of a race depends on the machine code generated by the compiler, the timing of the two CPUs involved, and how their memory operations are ordered by the memory system, which can make race-induced errors difficult to reproduce and debug. For example, adding print statements while debugging `push` might change the timing of the execution enough to make the race disappear.

第 16 行导致赋值的丢失就是一个 竞争（race） 的例子。竞争是指同时访问某个内存位置，并且至少有一个访问是写入的情况。竞争通常很容易引入 bug，要么是丢失更新（如果访问是写入操作），要么是读取了未更新完全的数据结构。竞争的结果不仅取决于编译器生成的指令顺序，也涉及两个 CPU 上的运行时序以及内存系统对它们操作内存的排序方式，这会使竞争导致的错误难以复现和调试。例如，在调试 `push` 时添加打印语句可能会改变执行时序，从而导致竞争消失。

> The usual way to avoid races is to use a lock. Locks ensure mutual exclusion, so that only one CPU at a time can execute the sensitive lines of `push`; this makes the scenario above impossible. The correctly locked version of the above code adds just a few lines (highlighted in yellow):

避免竞争的常用方法是使用锁。锁确保了 互斥（mutual exclusion），即同时只有一个 CPU 可以执行 `push` 这段敏感代码；从而避免了上述竞争的情况发生。对上述代码正确加锁后的版本仅添加了几行代码（以黄色突出显示, 译者注：对应下面这段代码的第 7，16 和 19 行）：

```
 6 struct element *list = 0;
 7 struct lock listlock;
 8
 9 void
10 push(int data)
11 {
12   struct element *l;
13   l = malloc(sizeof *l);
14   l->data = data;
15
16   acquire(&listlock);
17   l->next = list;
18   list = l;
19   release(&listlock);
20 }
```

> The sequence of instructions between `acquire` and `release` is often called a critical section. The lock is said to be protecting `list`.

`acquire` 和 `release` 之间的指令序列通常被称为 临界区（critical section）。我们称锁保护了 `list`。

> When we say that a lock protects data, we really mean that the lock protects some collection of invariants that apply to the data. Invariants are properties of data structures that are maintained across operations. Typically, an operation's correct behavior depends on the invariants being true when the operation begins. The operation may temporarily violate the invariants but must reestablish them before finishing. For example, in the linked list case, the invariant is that `list` points at the first element in the list and that each element's `next` field points at the next element. The implementation of `push` violates this invariant temporarily: in line 17, `l` points to the next list element, but `list` does not point at `l` yet (reestablished at line 18). The race we examined above happened because a second CPU executed code that depended on the list invariants while they were (temporarily) violated. Proper use of a lock ensures that only one CPU at a time can operate on the data structure in the critical section, so that no CPU will execute a data structure operation when the data structure's invariants do not hold.

当我们说某个锁保护了某某数据时，实际上是指该锁保护了这些数据上的一种 “不变性（invariant）”。所谓 invariant 是指数据结构的一种特性，并且该特性需要在一系列操作前后维持不变。通常，操作开始时 invariant 必须成立，这是操作的行为是否正确的前提。操作过程中可能会暂时破坏 invariant，但必须在完成之前重新恢复它。以前述的链表为例，invariant 指是 `list` 应该指向链表的第一个元素，并且每个元素的 `next` 字段指向下一个元素。`push` 函数执行过程中会

暂时不满足这个 invariant：譬如在第 17 行，`l` 指向下一个元素，但此时 `list` 尚未指向 `l`（在第 18 行会重新建立）。我们上面介绍的竞争情况之所以发生是因为第二个 CPU 在链表的 invariant（暂时）被破坏时执行了依赖于这些 invariant 的代码。正确使用锁确保了每次只有一个 CPU 可以对临界区中的数据结构进行操作，这样当数据结构的 invariant 不成立时，任何其他 CPU 都不会访问该数据结构。

> You can think of a lock as serializing concurrent critical sections so that they run one at a time, and thus preserve invariants (assuming the critical sections are correct in isolation). You can also think of critical sections guarded by the same lock as being atomic with respect to each other, so that each sees only the complete set of changes from earlier critical sections, and never sees partially-completed updates.

你可以认为对可能并发的临界区采用同一把锁进行保护相当于 串行化（serializing） 了它们的运行，即同时只能有一个临界区可以运行，这保证了 invariant（前提是对于单个临界区来说，其执行逻辑本身必须保证正确）。同样，我们也可以认为这些被同一把锁保护的临界区之间保持了 “原子性（atomic）”，也就是说当我们进入一个临界区时可以确定已经执行过的临界区的操作都必然是完整的，不会出现只看到部分被执行而部分未执行的情况。

> Though useful for correctness, locks inherently limit performance. For example, if two processes call `kfree` concurrently, the locks will serialize the two critical sections, so that there is no benefit from running them on different CPUs. We say that multiple processes conflict if they want the same lock at the same time, or that the lock experiences contention. A major challenge in kernel design is avoidance of lock contention in pursuit of parallelism. Xv6 does little of that, but sophisticated kernels organize data structures and algorithms specifically to avoid lock contention. In the list example, a kernel may maintain a separate free list per CPU and only touch another CPU’s free list if the current CPU’s list is empty and it must steal memory from another CPU. Other use cases may require more complicated designs.

虽然锁有助于确保正确性，但它天然地会降低性能。例如，如果两个进程并发调用 `kfree`，锁会将两个临界区串行化，因此即使这个 `kfree` 运行在不同的 CPU 上也不会对性能带来任何提升。如果多个进程同时需要获取同一把锁，我们称进程之间发生了 冲突（conflict），或者说对于锁发生了 争用（contention）。内核设计的一个主要挑战是如何在追求并发的同时避免锁争用。xv6 在这方面做得很少，但成熟的内核会专门组织数据结构和算法来避免锁争用。在链表的示例中，(成熟的) 内核可能会为每个 CPU 维护一个单独的空闲链表，并且只有当当前的 CPU 的链表为空而必须从另一个 CPU “窃取（steal）” 内存时，才会访问另一个 CPU 的空闲链表。对于其他用例可能需要更复杂的设计。

> The placement of locks is also important for performance. For example, it would be correct to move `acquire` earlier in `push`, before line 13. But this would likely reduce performance because then the calls to `malloc` would be serialized. The section “Using locks” below provides some guidelines for where to insert `acquire` and `release` invocations.

锁的放置位置对性能影响也很大。例如，将 `acquire` 移到 `push` 的更靠前位置（第 13 行之前）逻辑上也没有问题。但这可能会降低性能，因为这样对 `malloc` 的调用就会被串行化。下文的 “使用锁” 小节部分提供了一些关于在何处插入 `acquire` 和 `release` 调用的指导。

7.2 代码讲解：锁（Code: Locks）

> Please read `kernel/spinlock.h` and `kernel/spinlock.c`.

请阅读 `kernel/spinlock.h` 和 `kernel/spinlock.c`。

> Xv6 has two types of locks: spinlocks and sleep-locks. We’ ll start with spinlocks. Xv6 represents a spinlock as a `struct spinlock` (1201). The important field in the structure is `locked`, a word that is zero when the lock is available and non-zero when it is held. Logically, xv6 should acquire a lock by executing code like

xv6 中实现的锁有两种类型，它们是：“自旋锁（spinlock）” 和 “睡眠锁（sleep-lock）”。我们先看一下自旋锁。xv6 用结构体类型 `struct spinlock`（1201）来表示自旋锁。该结构体中有个重要的字段叫 `locked`，当锁未被持有时，该项值等于零；当锁被持有时，该项值不为零。从逻辑上讲，xv6 中可以通过执行如下代码来获取锁：

```
21 void
22 acquire(struct spinlock *lk) // does not work!
23 {
24   for(;;) {
25     if(lk->locked == 0) {
26       lk->locked = 1;
27       break;
28     }
29   }
30 }
```

> Unfortunately, this implementation does not guarantee mutual exclusion on a multiprocessor. It could happen that two CPUs simultaneously reach line 25, see that `lk->locked` is zero, and then both grab the lock by executing line 26. At this point, two different CPUs hold the lock, which violates the mutual exclusion property. What we need is a way to make lines 25 and 26 execute as an atomic (i.e., indivisible) step.

可惜这个实现无法保证在多处理器系统上实现互斥。考虑如下可能会发生的场景：两个 CPU 同时执行到第 25 行，发现 `lk->locked` 为零，然后都通过执行第 26 行来获取锁。此时，两个不同的 CPU 都持有了这个锁，这违反了互斥性。我们需要一种方法，使第 25 行和第 26 行以 原子（atomic）（也就是说，不可分割的）的方式执行。

> Because locks are widely used, multi-core processors usually provide an instruction that can be used to make lines 25 and 26 atomic. On the RISC-V this instruction is `amoswap r, a`. `amoswap` reads the value at the memory address `a`, writes the contents of register `r` to that address, and puts the value it read into `r`. That is, it swaps the contents of the register and the addressed memory location. It performs this sequence atomically, using special hardware to prevent any other CPU from using the memory address between the read and the write.

由于锁被广泛使用，多核处理器通常会提供一条以原子方式实现第 25 行和第 26 行的指令。在 RISC-V 架构上，该指令为 `amoswap r, a`。`amoswap` 读取内存地址 `a` 处的值，将寄存器 `r` 的内容写入地址 `a`，同时将读取的值存入 `r`。也就是说，它将寄存器（`r`）中的值和内存地址 `a` 处的值进行

了交换。它以原子方式（译者注：单条机器指令）执行了这个交换的动作，从硬件层面阻止任何其他 CPU 在读取和写入操作步骤之间能够访问该内存地址。

> The portable C library call `__sync_lock_test_and_set(addr, value)` boils down to the `amoswap` instruction; the function returns the old (swapped) contents of `*addr`. Here's a good way to write the loop in `acquire`:

更具备移植性的 C 库函数 `__sync_lock_test_and_set(addr, value)` 最终编译为 `amoswap` 指令；该函数返回 `*addr` 的旧（交换后的）内容。以下是在 `acquire` 中编写循环的好方法：

```
21  while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
22    ;
```

> Xv6's `acquire` (1271) uses the above loops. Each iteration swaps one into `lk->locked` and checks the previous value; if the previous value is zero, then we've acquired the lock, and the swap will have set `lk->locked` to one. If the previous value is one, then some other CPU holds the lock, and the fact that we atomically swapped one into `lk->locked` didn't change its value.

xv6 的 `acquire` (1271) 中使用了上述循环。每次迭代都会将 1（这个值存放在寄存器中）和 `lk->locked` 中的值进行交换，并检查 `lk->locked` 中交换前的值；如果先前值为 0，则表示成功地获取了锁，同时交换操作也会将 `lk->locked` 的值变成了 1（译者注：参考上文的代码，相当于将 25 行检查和 26 行设置以原子方式完成了）。如果 `lk->locked` 交换前的值本身就为 1，则表示其他 CPU 已经持有该锁，而我们将 `lk->locked` 中的值原子性地交换为 1 也不会改变其原先的值。

> Once the lock is acquired, `acquire` records, for debugging, the CPU that acquired the lock. The `lk->cpu` field is protected by the lock and must only be changed while holding the lock.

一旦获取到锁，`acquire` 会记录获取锁的 CPU，这主要是为了调试。`lk->cpu` 字段受锁保护，只能在持有锁的前提下才能对其进行更改。

> The function `release` (1302) is the opposite of `acquire`: it clears the `lk->cpu` field and then releases the lock. Conceptually, the release just requires assigning zero to `lk->locked`. The C standard allows compilers to implement an assignment with multiple store instructions, so a C assignment might be non-atomic with respect to concurrent code. Instead, `release` uses the C library function `__sync_lock_release` that performs an atomic assignment. This function also boils down to a RISC-V `amoswap` instruction.

函数 `release` (1302) 的行为与 `acquire` 相反：它首先清除 `lk->cpu` 的内容，然后释放锁。从概念上讲，释放操作只需要将 `lk->locked` 赋值为零即可。（虽然从 C 语言上看，赋值为零的操作只是一条语句）但编译器仍然可能将其编译为多条存储指令，因此 C 语言形式的赋值操作对于并发来说仍然可能是非原子的。所以，`release` 没有使用简单的 C 赋值语句，而是调用 C 库函数 `__sync_lock_release` 来执行原子化的赋值。该函数最终也编译为 RISC-V 的 `amoswap` 指令。

7.3 代码讲解：使用锁（Code: Using locks）

> Xv6 uses locks in many places to avoid races. As described above, `kalloc` (3027) and `kfree` (3005) form a good example. Try Exercises 1 and 2 to see what happens if those functions omit the locks. You'll likely find that it's difficult to trigger incorrect behavior, suggesting that it's hard to reliably test whether code is free from locking errors and races. Xv6 may well have as-yet-undiscovered races.

xv6 在很多地方使用锁来避免竞争。如上所述，`kalloc` (3027) 和 `kfree` (3005) 就是一个很好的例子。尝试 “练习 1” 和 “练习 2”，看看如果这些函数省略了锁会发生什么。你可能会发现很难触发错误的行为，因为想要可靠地测试代码并确保没有锁相关的错误和竞争问题可不是一件容易的事情。xv6 同样也很可能存在一些尚未发现的竞争问题。

> A hard part about using locks is deciding how many locks to use and which data and invariants each lock should protect. There are a few basic principles. First, any time a variable can be written by one CPU at the same time that another CPU can read or write it, a lock should be used to keep the two operations from overlapping. Second, remember that locks protect invariants: if an invariant involves multiple memory locations, typically all of them need to be protected by a single lock to ensure the invariant is maintained.

使用锁的难点在于决定使用多少个锁，以及每个锁应该保护哪些数据以及 invariant（译者注：回顾一下 6.1 节中有关 invariant 的定义）。有几个基本原则。首先，当一个变量会被一个 CPU 写入，而同时又有另一个 CPU 会读取或写入该变量时，则应该使用锁来防止这两个操作互相影响。其次，请记住锁可以保护 invariant：如果一个 invariant 涉及多处内存，通常情况下所有这些内存都需要用同一个锁来保护，以确保维持其 invariant。

> The rules above say when locks are necessary but say nothing about when locks are unnecessary, and it is important for efficiency not to lock too much, because locks reduce parallelism. If parallelism isn't important, then one could arrange to have only a single thread and not worry about locks. A simple kernel can do this on a multiprocessor by having a single lock; the kernel acquires the lock every time the kernel is entered from user space, for a system call or interrupt; the kernel releases the lock when it returns to user space. Many uniprocessor operating systems have been converted to run on multiprocessors using this approach, sometimes called a "big kernel lock," but the approach sacrifices parallelism: only one CPU can execute in the kernel at a time. If the kernel consumes significant CPU time, more parallelism could be obtained by protecting different objects or modules with different locks, so that different CPUs could be executing in different parts of the kernel at the same time.

上述规则说明了何时需要锁，但并未说明何时不需要锁。为了提高效率，同样需要注意不要使用过多的锁，因为锁会降低并发效率。如果不考虑并发的话，甚至可以安排只使用一个线程，这样就不必担心锁的问题了。为了在多处理器上避免并发，一个内核从简化设计出发，也可以只定义一把锁，但凡通过系统调用或者中断从用户空间进入内核时就先持有这把锁；在退出内核返回用户态时释放它。许多原先针对单处理器设计的操作系统在移植到多处理器系统上时会采用这种方法（有时称为 “大内核锁（big kernel lock）”），但这种方法牺牲了并发性：即内核一次只能在一个 CPU 上执行。如果内核会执行繁重的计算，则使用一组保护不同对象和模块的锁会更好地支持并发性，这样内核代码的不同部分就可以同时在多个 CPU 上执行了。

> As an example of coarse-grained locking, xv6's `kalloc.c` allocator has a single free list protected by a single lock. If multiple processes on different CPUs try to allocate pages at the same time, each will have to wait for its turn by spinning in `acquire`. Spinning wastes CPU time, since it's not useful work. If contention for the lock wasted a significant fraction of CPU time, perhaps performance could be improved by changing the allocator to have a separate free list per CPU, each with its own lock, to allow truly parallel allocation.

举个粗粒度锁的例子，xv6 的内核内存分配器（定义在 `kalloc.c` 中）维护了一个空闲页的链表，这个链表被一把锁保护。如果不同 CPU 上的多个进程同时尝试分配内存页，则每个进程都必须通过调用 `acquire` （该函数会自旋）来等待轮到自己。自旋过程中处理器无法做其他的事情，这明显是对处理器的浪费。考虑到锁的争用浪费了大量 CPU 时间，从提高性能出发，或许可以修改分配器的设计，使其针对每个 CPU 都安排一个空闲链表，每个链表都拥有自己的锁，从而允许实现真正的并行分配。

> As an example of fine-grained locking, xv6 has a separate lock for each file, so that processes that manipulate different files can often proceed without waiting for each other's locks. The file locking scheme could be made even more fine-grained if one wanted to allow processes to simultaneously write different areas of the same file. Ultimately lock granularity decisions need to be driven by performance measurements as well as complexity considerations.

再看一个细粒度锁的例子，xv6 为每个文件设置了单独的锁，因此操作不同文件的进程通常无需等待彼此的锁即可继续操作。如果希望允许多个进程同时写入同一文件的不同区域，则可以将文件的锁方案进一步细分。总而言之，锁的粒度究竟如何决定，这取决于性能指标和实现上的复杂性这两者之间的权衡。

> As subsequent chapters explain each part of xv6, they will mention examples of xv6's use of locks to deal with concurrency. As a preview, Figure 7.3 lists all of the locks in xv6.

后续章节在讲解 xv6 的各个部分时，会涉及到 xv6 使用锁处理并发的示例。作为预览，图 7.3 列出了 xv6 中定义的所有锁。

Lock	Description
bcache.lock	Protects allocation of block buffer cache entries
cons.lock	Serializes read processing of console input
tx_lock	Serializes access to console (uart) output hardware
ftable.lock	Serializes allocation of a struct file in file table
itable.lock	Protects allocation of in-memory inode entries
vdisk_lock	Serializes access to disk hardware and queue of DMA descriptors
kmem.lock	Serializes allocation of memory
log.lock	Serializes operations on the transaction log
pipe's pi->lock	Serializes operations on each pipe
pid_lock	Serializes increments of next_pid
proc's p->lock	Serializes changes to process's state
wait_lock	Helps wait avoid lost wakeups
tickslock	Serializes operations on the ticks counter
inode's ip->lock	Serializes operations on each inode and its content
buf's b->lock	Serializes operations on each block buffer

Figure 7.3: Locks in xv6

7.4 死锁和加锁的顺序（Deadlock and lock ordering）

> Suppose the functions running on CPUs C1 and C2 both have a point at which each needs to hold both lock A and lock B, and they acquire them in different orders:

假设在 CPU C1 和 C2 上运行的函数都需要同时持有锁 A 和锁 B，但它们以不同的顺序获取它们：

```
  CPU C1         CPU C2
acquire(&A);   acquire(&B);
acquire(&B);   acquire(&A);
...            ...
release(&B);   release(&A);
release(&A);   release(&B);
```

> With a bit of bad luck, C1 and C2 might both execute their first acquire at exactly the same moment; both can succeed, since they are asking for different locks. But then both C1 and C2 will have to wait in their second calls to `acquire()`, since both locks are already held by the other CPU. Because both CPUs are waiting for each other, neither will ever release a lock, and both will wait forever. This situation is called deadlock.

如果运气不好，C1 和 C2 可能会同时执行第一次获取锁的操作；由于它们请求的是不同的锁，所以两者都可能成功。但是，由于两个锁都已经被对方 CPU 持有，因此 C1 和 C2 都必须在第二次调用 `acquire()` 时等待。由于两个 CPU 都在等待对方，所以它们都不会释放锁，而是永远等待下去。这种情况称为 死锁（deadlock）。

> The key problem in the C1/C2 example is that the two CPUs acquired the locks in different orders. If they had both tried to acquire A first, one would have acquired A and then B and then released them both, and then the other CPU could have proceeded. More generally, locking code must follow this rule to avoid deadlock: all code paths that hold multiple locks

> must acquire locks in the same order. The need for this global lock acquisition order means that locks are effectively part of each function's specification: callers must invoke functions in a way that causes locks to be acquired in the agreed-on order.

C1/C2 示例中的关键问题在于两个 CPU 获取锁的顺序不同。如果它们都先尝试获取 A，那么其中一个 CPU 就会先获取 A，再获取 B，然后释放它们，这样另一个 CPU 就可以继续执行。更一般地说，锁定代码必须遵循以下规则才能避免死锁：所有持有多个锁的代码路径都必须以相同的顺序获取锁。这种全局锁获取顺序的必要性意味着锁实际上是每个函数编写需要遵循的规范的一部分，也就是说调用者必须在调用函数时以约定的顺序获取锁。

> Xv6 has many lock-order chains of length two involving per-process locks (the lock in each `struct proc`) due to the way that `sleep` works (see Chapter 9). For example, `consoleintr` (7107) is the interrupt routine which handles typed characters. When a newline arrives, any process that is waiting for console input should be woken up. To do this, `consoleintr` holds `cons.lock` while calling `wakeup`, which acquires the waiting process's lock in order to wake it up. In consequence, the global deadlock-avoiding lock order includes the rule that `cons.lock` must be acquired before any process lock. The file-system code contains xv6's longest lock chains. For example, creating a file requires simultaneously holding a lock on the directory, a lock on the new file's inode, a lock on a disk block buffer, the disk driver's `vdisk_lock`, and the calling process's `p->lock`. To avoid deadlock, file-system code always acquires locks in the order mentioned in the previous sentence.

由于 `sleep` 的工作方式（参见第 9 章），xv6 在不少代码执行路径中依次持有锁的个数大于 2 个（译者注：下文将这个依次持有多个锁的代码执行路径形象比喻为 “锁链（lock chain）”），这些锁涉及每个进程的锁（每个 `struct proc` 中的 `lock` 成员）。例如，`consoleintr` (7107) 是处理输入字符的中断例程。当我们按下回车键（对应输入 “换行（newline）” 符）时，所有等待控制台输入的进程都会被唤醒。为此，`consoleintr` 会先持有锁 `cons.lock`，然后调用 `wakeup`，后者会获取等待进程的锁以试图将其唤醒。这导致我们为了避免发生全局死锁，需要在有关加锁顺序规则中新增一个要求，即必须在获取任何进程的锁之前先获取 `cons.lock`。xv6 中最长的锁链出现在文件系统的代码里。例如，创建一个文件需要同时持有目录上的锁、新文件的 inode 上的锁、磁盘块缓冲区的锁、磁盘驱动程序的锁 `vdisk_lock` 以及调用进程的 `p->lock` 锁。为了避免死锁，文件系统中的代码会始终按照上一句中提到的顺序获取锁。

> Honoring a global deadlock-avoiding order can be surprisingly difficult. Sometimes the lock order conflicts with logical program structure, e.g., perhaps code module M1 calls module M2, but the lock order requires that a lock in M2 be acquired before a lock in M1. Sometimes the identities of locks aren't known in advance, perhaps because one lock must be held in order to discover the identity of the lock to be acquired next. This kind of situation arises in the file system as it looks up successive components in a path name, and in the code for the `wait` and `exit` system calls as they search the table of processes looking for child processes. Finally, the danger of deadlock is often a constraint on how fine-grained one can make a locking scheme, since more locks often means more opportunity for deadlock. The need to avoid deadlock is often a major factor in kernel implementation.

为了避免全局死锁而严格遵守加锁顺序可能出奇地困难。有时，锁定顺序与程序的逻辑结构会冲突，例如，代码模块 M1 可能调用模块 M2，但锁定顺序要求先获取 M2 中的锁，然后再获取 M1 中的锁。有时，无法提前知道要获取哪些锁，这或许是因为必须先持有前一个锁才能知道下一个要获

取的锁是谁。这种情况会出现在文件系统中，因为它会查找路径名中（以 “/” 分隔的）组成部分；也会出现在 `wait` 和 `exit` 系统调用函数中，因为它们会在进程表中搜索子进程。总之，由于死锁风险的存在，通常我们要考虑尽量限制加锁方案中涉及的锁的个数，因为锁越多，发生死锁的可能性也就越大。避免死锁通常是一个内核要实现的主要目标之一。

> A question that sometimes arises is what should happen if a CPU tries to acquire a lock that the same CPU already holds. One line of reasoning is that this should be allowed: no other CPU can hold the lock, so there’ s no need to worry about CPUs interfering with each others’ use of the protected data. Locking systems that allow a CPU to re-aquire a lock it alread holds are called a re-entrant or recursive. On the other hand, if a lock is already held, even on the same CPU, that means an operation may have temporarily violated and not yet restored some invariants; to allow a new operation to commence while the invariants don’ t hold seems like an invitation to bugs. Xv6 takes this latter view, and forbids a CPU that currently holds a lock from re-acquiring it. Detecting this situation is the purpose of the call to `holding` in `acquire` .

有人可能会提出一个问题：如果一个 CPU 尝试获取同一个 CPU 已经持有的锁，会发生什么情况？一种回答是应该允许这种情况发生：没有其他 CPU 可以持有该锁，因此无需担心 CPU 之间会干扰彼此对受保护数据的使用。这种允许 CPU 重新获取其已持有锁的锁系统被称为是 可重入的（re-entrant） 或 递归的（recursive）。另一种考虑是，即使同一个 CPU 上已经持有锁，也意味着某个操作可能暂时违反了某些 invariant，并且尚未恢复；在 invariant 不成立的情况下允许新的操作似乎会引发 bug。xv6 采取了后一种观点，禁止当前持有锁的 CPU 重新获取该锁。为此， `acquire` 通过调用 `holding` 检测这种情况。

7.5 锁和中断处理（Locks and interrupt handlers）

> Some xv6 spinlocks protect data that is used by both threads and interrupt handlers. For example, the `clockintr` timer interrupt handler might increment `ticks` (3482) at about the same time that a kernel thread reads `ticks` in `sys_pause` (3836). The lock `tickslock` serializes the two accesses.

xv6 会使用一些自旋锁来保护线程和中断处理程序之间共享的数据。例如， `clockintr` 这个定时器中断处理函数会增加 `ticks` (3482) 的值，而同时内核线程可能在 `sys_pause` (3836) 中读取 `ticks` 的值。xv6 用锁 `tickslock` 串行化两者对 `ticks` 的访问。

> The interaction of spinlocks and interrupts raises a potential danger. Suppose `sys_pause` holds `tickslock` , and its CPU is interrupted by a timer interrupt. `clockintr` would try to acquire `tickslock` , see it was held, and wait for it to be released. In this situation, `tickslock` will never be released: only `sys_pause` can release it, but `sys_pause` will not continue running until `clockintr` returns. So the CPU will deadlock, and any code that needs either lock will also freeze.

自旋锁和中断的共同作用会引发潜在的风险。假设（一个线程通过执行） `sys_pause` 持有了 `tickslock` ，然后一个定时器中断打断了该函数所在线程在当前 CPU 上的执行。 `clockintr` 会尝试获取 `tickslock` ，因为发现它已经被他人持有，于是中断函数开始进入自旋等待锁被释放。在这种情况下， `tickslock` 永远也得不到释放：因为只有 `sys_pause` 会释放它，但是执行

`sys_pause` 的线程却只有等中断处理函数 `clockintr` 退出后才能继续。于是 CPU 进入死锁状态，并且任何需要获取锁的代码也会因为拿不到锁而被阻塞。

> To avoid this situation, if a spinlock is used by an interrupt handler, a CPU must never hold that lock with interrupts enabled. Xv6 is more conservative: when a CPU acquires any lock, xv6 always disables interrupts on that CPU. Interrupts may still occur on other CPUs, so an interrupt's `acquire` can wait for a thread to release a spinlock; just not on the same CPU.

为了避免这种情况，如果中断处理函数会试图获取一个自旋锁的话，那么其他代码（译者注：特指线程代码）必须保证在持有锁的同时关闭中断（译者注：正如上一段所描述的，因为中断的优先级更高，当锁已经被线程持有后，一旦中断抢占了处理器并试图持有同一把锁时就会被阻塞，这会导致中断处理无法退出，线程也就得不到执行，更没有机会释放锁，最终导致两边进入相互等待而死锁的境地）。xv6 的实现更保守：只要一个 CPU 持有一把锁，xv6 总是同时禁用该 CPU 上的中断。中断仍然可能发生在其他 CPU 上，但即使这个（发生在其他 CPU 上的）中断调用了 `acquire` （尝试获取同一把锁）也没关系，因为这不会阻塞太久，已经持有锁的线程一旦释放这个锁则中断就能立即抢到锁并继续，只要它们两者不在同一个 CPU 上，就不会造成死锁。

> Xv6 re-enables interrupts when a CPU holds no spinlocks; it must do a little book-keeping to cope with nested critical sections. `acquire` calls `push_off` (1355) and `release` calls `pop_off` (1369) to track the nesting level of locks on the current CPU. When that count reaches zero, `pop_off` restores the interrupt enable state that existed at the start of the outermost critical section. The `intr_off` and `intr_on` functions execute RISC-V instructions to disable and enable interrupts, respectively.

当 CPU 不再持有自旋锁时，xv6 会重新启用中断；考虑到可能存在嵌套的临界区，所以还需要一些额外的工作来处理这种情况。这包括在 `acquire` 中调用 `push_off` (1355)，以及在 `release` 中调用 `pop_off` (1369)，其目的是跟踪当前 CPU 上持有锁的嵌套层级。当该计数达到零时， `pop_off` 会恢复最外层临界区开始时的中断启用状态。禁用和启用中断在 RISC-V 上分别通过调用 `intr_off` 和 `intr_on` 函数来实现。

> It is important that `acquire` call `push_off` strictly before setting `lk->locked` (1277). If the two were reversed, there would be a brief window when the lock was held with interrupts enabled, and an unfortunately timed interrupt would deadlock the system. Similarly, it is important that `release` call `pop_off` only after releasing the lock (1321).

重要的是， `acquire` 必须在设置 `lk->locked` (1277) 之前调用 `push_off` 。如果两者颠倒，会出现一个短暂的窗口，在此期间既持有锁，而同时中断又没有关闭，如果此时很不幸来了一个定时器中断，则可能会导致系统死锁。同样重要的是， `release` 必须在释放锁 (1321) 之后再调用 `pop_off` 。

7.6 指令和内存排序（Instruction and memory ordering）

> It is natural to think of programs executing in the order in which source code statements appear. That's a reasonable mental model for single-threaded code, but is incorrect when multiple threads interact through shared memory [2, 4]. One reason is that compilers emit load and store instructions in orders different from those implied by the source code, and may entirely omit them (for example by caching data in registers). Another reason is that the

> CPU may execute instructions out of order to increase performance. For example, a CPU may notice that in a serial sequence of instructions A and B are not dependent on each other. The CPU may start instruction B first, either because its inputs are ready before A's inputs, or in order to overlap execution of A and B.

我们很自然地认为程序执行的顺序与源代码语句出现的顺序一致。对于单线程程序来说，这种想法是合理的，但当多个线程通过共享内存交互时，这种想法就不成立了 [2, 4]。原因之一是，经过编译器处理后生成的加载指令和存储指令的顺序可能与源代码表达的顺序不同，甚至这些访存指令可能完全被优化掉（例如，通过将数据缓存在寄存器中）。另一个原因是，CPU 可能会为了提高性能而乱序执行指令。例如，CPU 可能会注意到，在一系列指令中，A 和 B 彼此并不依赖。CPU 可能会首先执行指令 B，这是因为可能 B 的输入在 A 的输入完成之前已经就绪，或者仅仅是为了 “叠加执行（overlap execution）” A 和 B。

> As an example of what could go wrong, in this code for `push`, it would be a disaster if the compiler or CPU moved the store corresponding to line 4 to a point after the `release` on line 6:

举一个可能出错的例子，在 `push` 的这个代码中，如果编译器或 CPU 将第 4 行对应的存储指令移到第 6 行 `release` 之后的某个位置，那将会引起一场灾难：

```
1 l = malloc(sizeof *l);
2 l->data = data;
3 acquire(&listlock);
4 l->next = list;
5 list = l;
6 release(&listlock);
```

> If such a re-ordering occurred, there would be a window during which another CPU could acquire the lock and observe the updated list, but see an uninitialized `list->next`.

如果发生这样的 “指令重排（re-ordering）”，就会出现一个空窗期，在此期间另一个 CPU 可以获取锁并观察到更新后的列表，但会看到 `list->next` 仍未被赋值。

> The good news is that compilers and CPUs help concurrent programmers by following a set of rules called the memory model, and by providing some primitives to help programmers control re-ordering.

好消息是，编译器和 CPU 通过遵循一组称为 “内存模型（memory model）” 的规则并提供一些原语来帮助编写并发程序的程序员们控制指令重排。

> To tell the hardware and compiler not to re-order, xv6 uses `__sync_synchronize()` in both `acquire` (1271) and `release` (1302). `__sync_synchronize()` is a memory barrier: it tells the compiler and CPU to not reorder loads or stores across the barrier. The barriers in xv6's `acquire` and `release` force order in almost all cases where it matters, since xv6 uses locks around accesses to shared data. Chapter 11 discusses a few exceptions.

为了告诉硬件和编译器不要对指令进行重新排序，xv6 在 `acquire` (1271) 和 `release` (1302) 中都使用了 `__sync_synchronize()` 这个函数。`__sync_synchronize()` 是一个 “内存屏障（memory barrier）” 函数：它告诉编译器和 CPU 不要将屏障前的加载或存储操作移到屏障后面

去执行。xv6 在访问共享数据时会使用锁，所以 xv6 的 `acquire` 和 `release` 中的屏障几乎在其涉及的所有情况下都会强制程序按代码顺序执行。第 11 章讨论了一些例外情况。

7.7 睡眠锁（Sleep locks）

> Sometimes xv6 needs to hold a lock for a long time. For example, the file system (Chapter 10) keeps a file locked while reading and writing its content on the disk, and these disk operations can take tens of milliseconds. Holding a spinlock that long would lead to waste if another process wanted to acquire it, since the acquiring process would waste CPU for a long time while spinning. Another drawback of spinlocks is that a process cannot yield the CPU while retaining a spinlock; we'd like to do this so that other processes can use the CPU while the process with the lock waits for the disk. Yielding while holding a spinlock is illegal because it might lead to deadlock if a second thread then tried to acquire the spinlock; since `acquire` doesn't yield the CPU, the second thread's spinning might prevent the first thread from running and releasing the lock. Yielding while holding a lock would also violate the requirement that interrupts must be off while a spinlock is held. Thus we'd like a type of lock that yields the CPU while waiting to acquire, and allows yields (and interrupts) while the lock is held.

有时 xv6 需要长时间持有锁。例如，文件系统（第 10 章）在磁盘上读写文件内容时会保持文件锁定，而这些磁盘操作可能需要数十毫秒。如果采用自旋锁，则当其他进程想要获取锁（而不可得）时，会进入长时间自旋，这是一种十分浪费处理器的行为。自旋锁的另一个缺点是，进程在持有自旋锁时无法让出 CPU；而我们希望的是当一个进程持有了锁并等待磁盘时，其他进程有机会使用 CPU。一个线程在持有自旋锁后主动让出 CPU 是危险的，因为此时如果有第二个线程也尝试获取这把锁，可能会导致死锁；这是因为由于 `acquire` 不会让出 CPU，第二个线程进入自旋后，让出 CPU 的第一个线程有可能再也无法恢复执行并释放锁。在持有锁时让出 CPU 和我们在持有自旋锁期间必须关闭中断的要求也有冲突（译者注：在单处理器上一旦关闭中断相当于停止了调度，让出 CPU 的线程就此永远进入睡眠）。因此，我们需要一种新的锁，这种锁支持在尝试等待锁时能让出 CPU，同时在持有锁时也允许主动让出 CPU（包括不会关闭中断）。

> Xv6 provides such locks in the form of sleep-locks. `acquiresleep` (4471) yields the CPU while waiting, using techniques that will be explained in Chapter 9. At a high level, a sleep-lock has a `locked` field that is protected by a spinlock, and `acquiresleep`'s call to `sleep` atomically yields the CPU and releases the spinlock. The result is that other threads can execute while `acquiresleep` waits.

xv6 通过提供 睡眠锁（sleep-lock） 满足了以上要求。`acquiresleep` (4471) 在等待锁时会让出 CPU，其使用的技术将在第 9 章中讲解。简单来说，睡眠锁有一个 `locked` 成员，该成员受一把自旋锁保护，`acquiresleep` 中会（先尝试获取这把自旋锁，然后）调用 `sleep`，在 `sleep` 中会原子性地让出 CPU 以及释放这把自旋锁。这样，在 `acquiresleep` 等待锁期间，其他线程就有机会可以执行。

> Because sleep-locks leave interrupts enabled, they cannot be used in interrupt handlers. Because `acquiresleep` may yield the CPU, sleep-locks cannot be used inside spinlock critical sections (though spinlocks can be used inside sleep-lock critical sections).

由于睡眠锁不会关闭中断，因此它们不能在中断处理程序中使用。同时由于 `acquiresleep` 可能会让出 CPU，因此睡眠锁也不能在自旋锁保护的临界区内使用（但反过来自旋锁可以在被睡眠锁保护的临界区内使用）。

> Spin-locks are best suited to short critical sections, since waiting for them wastes CPU time; sleep-locks work well for lengthy operations.

自旋锁最适合保护执行时间比较短的临界区，因为自旋等待实在太浪费 CPU 时间；睡眠锁适用于保护执行时间比较长的临界区。

7.8 现实世界（Real world）

> Programming with locks remains challenging despite years of research into concurrency primitives and parallelism. It is often best to conceal locks within higher-level constructs like synchronized queues, although xv6 does not do this. If you program with locks, it is wise to use a tool that attempts to identify races, because it is easy to miss an invariant that requires a lock.

尽管业界多年来对并发原语和并行处理进行了深入研究，但使用锁进行编程对程序员来说仍然充满挑战。通常情况下，最好将锁封装在更高级的结构中（譬如同步队列等），然而 xv6 并没有这样做。编程中使用锁时，最好使用一个能够识别竞争的工具，因为很容易忘记对需要保护的 invariant 上锁。

> Most operating systems support POSIX threads (Pthreads), which allow a user process to have several threads running concurrently on different CPUs. Pthreads has support for user-level locks, barriers, etc. Pthreads also allows a programmer to optionally specify that a lock should be reentrant.

大多数操作系统都支持 POSIX 线程（POSIX threads，简称 Pthreads），它允许用户进程在不同的 CPU 上并发运行多个线程。Pthreads 支持用户模式下的锁和屏障等。Pthreads 还允许程序员选择指定锁是否可重入。

> Supporting Pthreads at user level requires support from the operating system. For example, it should be the case that if one pthread blocks in a system call, another pthread of the same process should be able to run on that CPU. As another example, if a pthread changes its process’ s address space (e.g., maps or unmaps memory), the kernel must arrange that other CPUs that run threads of the same process update their hardware page tables to reflect the change in the address space.

在用户模式下支持 Pthreads 需要操作系统的支持。例如，如果一个 pthread 在系统调用中阻塞，同一进程的另一个 pthread 应该仍然能够在该 CPU 上运行。再譬如，如果一个 pthread 更改了其进程的地址空间（例如，映射或取消映射了某些内存），则对于那些运行同一进程中的线程的 CPU 来说，内核必须同时更新这些 CPU 所使用的页表，从而及时反映进程地址空间的变化。

> It is possible to implement locks without atomic instructions [10], but it is expensive, so most operating systems use atomic instructions.

不使用 “原子指令（atomic instructions）” [10] 来实现锁是可能的，但是成本很高，所以大多数操作系统都使用原子指令。

> Locks can be expensive if many CPUs try to acquire the same lock at the same time. If one CPU has a lock cached in its local cache, and another CPU must acquire the lock, then the atomic instruction to update the cache line that holds the lock must move the line from the one CPU's cache to the other CPU's cache, and perhaps invalidate any other copies of the cache line. Fetching a cache line from another CPU's cache can be orders of magnitude more expensive than fetching a line from a local cache.

多个 CPU 争抢同一把锁可能会引入很大的开销。假设一个 CPU 的本地缓存中缓存了一把锁，当另一个 CPU 尝试获取该锁时，更新持有该锁的缓存行的原子指令必须确保对于该缓存行的更新从当前 CPU 的缓存同步到另一个 CPU 的缓存中，这可能还包括使该缓存行的任何其他副本失效的动作。从另一个 CPU 的缓存中获取缓存行的开销比从本地缓存中获取缓存行的开销要高出几个数量级。

> To avoid the expenses associated with locks, many operating systems use lock-free data structures and algorithms [6, 12]. For example, it is possible to implement a linked list like the one in the beginning of the chapter that requires no locks during list searches, and one atomic instruction to insert an item in a list. Lock-free programming is more complicated, however, than programming locks; for example, one must worry about instruction and memory reordering. Programming with locks is already hard, so xv6 avoids the additional complexity of lock-free programming.

为了避免与锁相关的开销，许多操作系统在数据结构和算法实现中采用了无锁设计 [6, 12]。例如，可以实现类似本章开头的链表，它在对链表执行搜索期间无需上锁，并且只需一条原子指令即可实现对链表的插入。然而，无锁编程比有锁编程更复杂；例如，必须考虑指令和内存的重新排序问题。使用锁进行编程本身就已经很困难，因此 xv6 为了避免引入额外的复杂性没有采用无锁设计。

7.9 练习（Exercises）

> 1. Comment out the calls to `acquire` and `release` in `kalloc` (3027). This seems like it should cause problems for kernel code that calls `kalloc`; what symptoms do you expect to see? When you run xv6, do you see these symptoms? How about when running `usertests`? If you don't see a problem, why not? See if you can provoke a problem by inserting dummy loops into the critical section of `kalloc`.

1. 注释掉 `kalloc` (3027) 中调用 `acquire` 和 `release` 的语句。这似乎会导致调用 `kalloc` 的内核代码出现问题；您预计会看到什么现象？运行 xv6 时，您会看到这些现象吗？运行 `usertests` 时又会看到什么呢？如果您没有看到问题，请说明原因。看看能否通过在 `kalloc` 的临界区中加入循环来引发问题。

> 1. Suppose that you instead commented out the locking in `kfree` (after restoring locking in `kalloc`). What might now go wrong? Is lack of locks in `kfree` less harmful than in `kalloc`?

1. 假设你注释掉了 `kfree` 中的锁定（在恢复 `kalloc` 中的锁定之后）。现在可能出现什么问题？ `kfree` 中缺少锁定的危害比 `kalloc` 中小吗？

> 1. If two CPUs call `kalloc` at the same time, one will have to wait for the other, which is bad for performance. Modify `kalloc.c` to have more parallelism, so that

> simultaneous calls to `kalloc` from different CPUs can proceed without waiting for each other.

1. 如果两个 CPU 同时调用 `kalloc`，其中一个 CPU 必须等待另一个 CPU 的执行，这会降低性能。请修改 `kalloc.c`，使其具有更高的并行性，以便不同 CPU 同时调用 `kalloc` 时无需相互等待。

> 1. Write a parallel program using POSIX threads, which is supported on most operating systems. For example, implement a parallel hash table and measure if the number of puts/gets scales with increasing number of CPUs.

1. 使用大多数操作系统都支持的 POSIX 线程编写一个并行程序。例如，实现一个并行哈希表，并测量 put/get 操作的数量是否随着 CPU 数量的增加而变化。

> 1. Implement a subset of Pthreads in xv6. That is, implement a user-level thread library so that a user process can have more than 1 thread and arrange that these threads can run in parallel on different CPUs. Come up with a design that correctly handles a thread making a blocking system call and changing its shared address space.

1. 在 xv6 中实现 Pthreads 的一个子集。也就是说，实现一个用户级线程库，使一个用户进程可以拥有多个线程，并安排这些线程在不同的 CPU 上并行运行。设计一个方案，能够支持线程运行过程中执行会导致阻塞的系统调用，以及支持更改其共享地址空间。

第 8 章 调度（Chapter 8 Scheduling）

> Any operating system is likely to run with more processes than the computer has CPUs, so a plan is needed to time-share the CPUs among the processes. Ideally the sharing would be transparent to user processes. A common approach is to provide each process with the illusion that it has its own virtual CPU by multiplexing the processes onto the hardware CPUs. This chapter explains how xv6 achieves this multiplexing.

一个操作系统上运行的进程数几乎总会超过计算机上处理器的数量，因此需要一个方案，在进程之间分时共享处理器。理想情况下，这种共享对用户进程透明。一种常见的方法是让进程 复用（multiplex） 硬件（处理器），让每个进程都感觉拥有自己专有的虚拟处理器。本章将解释 xv6 如何实现这种复用。

> Before proceeding with this chapter, please read `kernel/proc.h`, `kernel/swtch.S`, and `yield()`, `sched()`, and `schedule()` in `kernel/proc.c`.

在继续阅读本章之前，请先阅读 `kernel/proc.h`、`kernel/swtch.S` 以及 `kernel/proc.c` 中的 `yield()`、`sched()` 和 `schedule()`。

8.1 复用（Multiplexing）

> Xv6 multiplexes by switching each CPU from one process to another in two situations. First, xv6 switches when a process makes a system call that blocks (has to wait), for example `read` or `wait`. Second, xv6 periodically forces a switch to cope with processes that compute for long periods without blocking. The former are called voluntary switches, the latter involuntary.

xv6 会在两种情况下将一个处理器上的进程从一个切换到另一个来实现复用。一种情况是，一个进程调用了可能导致阻塞（必须等待）的系统调用，譬如 `read`、`wait`。另一种情况是，xv6 会定期进行强制切换，以应对那些计算密集且不发生阻塞的进程。前者属于主动切换；后者是被动切换。（译者注：multiplex 也可以翻译为 “多路复用”，这里的 “多路” 指多个进程复用一个处理器，本文直接翻译为 “复用”，简洁一点）。

> Implementing multiplexing poses a few challenges. First, how to switch from one process to another? The basic idea is to save and restore CPU registers, though the fact that this cannot be expressed in C makes it tricky. Second, how to force switches in a way that is transparent to user processes? Xv6 uses the standard technique in which a hardware timer’ s interrupts drive context switches. Third, all of the CPUs switch among the same set of processes, so a locking plan is necessary to avoid mistakes such as two CPUs deciding to run the same process at the same time. Fourth, a process’ s memory and other resources must be freed when the process exits, but it cannot finish all of this itself. Fifth, each CPU of a multi-core machine must remember which process it is executing so that system calls affect the correct process’ s kernel state.

实现复用面临一些挑战。首先，如何从一个进程切换到另一个进程？基本思想是需要保存和恢复处理器的寄存器，但对于这些操作，由于无法用 C 语言编写，因此比较棘手。其次，如何对用户进程以透明的方式实现强制切换？xv6 采用的是常用的方法，即通过硬件定时器的中断触发上下文切换。第三，因为需要支持同一组进程在多个处理器之间切换，因此需要引入锁机制来避免两个 CPU 同时决定运行同一个进程等错误。第四，进程退出时必须释放其拥有的内存和其他资源，但它无法自行完成所有这些操作。第五，多核机器的每个处理器必须记住它正在执行哪个进程，以便系统调用能够确保只会涉及正确的进程及其内核的状态。

8.2 上下文切换概述（Context switch overview）

> The term “context switch” refers to the steps involved in a CPU leaving off execution of one kernel thread (usually for later resumption), and resuming execution of a different kernel thread; this switching is the heart of multiplexing. Xv6 does not directly context switch from one process’ s kernel thread to another process’ s kernel thread; instead, a kernel thread gives up the CPU by context-switching to that CPU’ s “scheduler thread,” and the scheduler thread picks a different process’ s kernel thread to run, and context-switches to that thread.

“上下文切换（context switch）” 是指 CPU 停止执行一个内核线程（通常是为了稍后恢复），并恢复执行另一个内核线程的操作步骤；这种切换是多路复用的核心。xv6 不会直接从一个进程的内核线程切换到另一个进程的内核线程；相反，内核线程会通过上下文切换到该 CPU 的 “调度线程（scheduler thread）” 来放弃 CPU，然后调度线程会选择另一个进程的内核线程来运行，并将上下文切换到该线程。

> At a broader scope, the steps involved in switching from one user process to another are illustrated in Figure 8.1: a trap (system call or interrupt) from the old process’ s user space to its kernel thread, a context switch to the current CPU’ s scheduler thread, a context switch to a new process’ s kernel thread, and a trap return to the user-level process.

从更广泛的角度来看，图 8.1 说明了从一个用户进程切换到另一个用户进程所涉及的步骤：通过一个系统调用或中断从上一个进程的用户空间 “陷入（trap）” 到其内核线程、然后上下文切换到当前 CPU 的调度线程、再上下文切换到新进程的内核线程，最后通过 trap 返回到新进程的用户态。

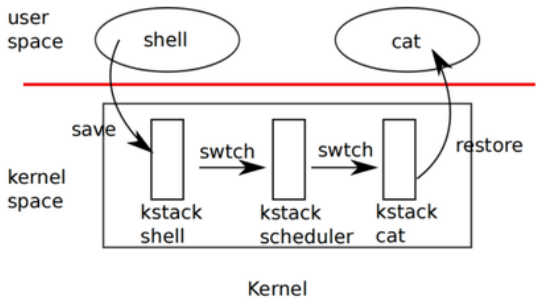

Figure 8.1: Switching from one user process to another. In this example, xv6 runs with one CPU (and thus one scheduler thread).

8.3 代码讲解：上下文切换（Code: Context switching）

> The function `swtch()` in `kernel/swtch.S` contains the heart of thread context switching: it saves the switched-from thread's CPU registers, and restores the previously-saved registers of the switched-to thread. The basic reason this is sufficient is that a thread's state consist of data in memory (e.g. its stack) plus its CPU registers; thread memory need not saved and restored because different threads keep their data in different areas of RAM; but the CPU has only one set of registers so they must be switched (saved and restored) between threads.

`kernel/swtch.S` 中的 `swtch()` 函数是线程上下文切换的核心函数：它保存切换前上一个线程的 CPU 寄存器，并恢复切换后下一个目标线程先前保存的寄存器。这样做的基本原因是，线程的状态由内存中的数据（例如，线程的栈）加上其 CPU 寄存器组成；线程的内存数据无需保存和恢复，因为不同的线程将其数据保存在 RAM 的不同区域中；但 CPU 只有一组寄存器，因此必须在线程之间切换（保存和恢复）。

> Each thread's `struct proc` includes a `struct context` that holds the thread's saved registers when it is not running. A CPU's scheduler thread's `struct context` is in that CPU's `struct cpu`. When thread X wishes to switch to thread Y, thread X calls `swtch(&X's context, &Y's context)`. `swtch()` saves the current CPU registers in X's context, then loads the content of Y's context into the CPU registers, then returns.

每个线程的 `struct proc` 都包含一个 `struct context`，用于存放当一个线程不在 CPU 上运行时需要保存的寄存器的内容。用于表示一个 CPU 的结构体 `struct cpu` 中也有一个 `struct context` 用于存放每个 CPU 对应的调度线程在其非运行时的需要保存的寄存器的内容。当线程 X 希望切换到线程 Y 时，线程 X 会调用 `swtch(&X's context, &Y's context)`。`swtch()` 在返回前将当前 CPU 的寄存器内容保存到 X 的上下文中，然后将 Y 上下文的内容加载到 CPU 寄存器中。

> Here's an abbreviated copy of `swtch`:

下面是 `swtch` 函数的简略版本：

```
swtch:
        sd ra, 0(a0)
        sd sp, 8(a0)
        sd s0, 16(a0)
        ...
        sd s11, 104(a0)

        ld ra, 0(a1)
        ld sp, 8(a1)
        ld s0, 16(a1)
        ...
        ld s11, 104(a1)

        ret
```

> `a0` holds the first function argument, and `a1` the second; in this case, the two `struct context` pointers. `16(a0)` refers to an offset 16 bytes into the memory pointed to by `a0`;

> referring to the definition of `struct context` in `kernel/proc.h` (1951), this is the structure field called `s0`.

`a0` 保存第一个函数参数，`a1` 保存第二个函数参数；在本例中，是两个 `struct context` 类型的指针。`16(a0)` 指向的地址是以 `a0` 中的值为基地址加上 16 个字节的偏移量；参考 `kernel/proc.h` (1951) 中 `struct context` 的定义，也就是名为 `s0` 的结构体字段所在位置。

> Where does `swtch`'s `ret` return to? It returns to the instruction that the `ra` register points to. In the example in which thread X calls `swtch()` to switch to Y, when `ret` executes, `ra` has just been loaded from Y's `struct context`. And the `ra` in Y's `struct context` was originally saved by Y's call to `swtch` when Y gave up the CPU in the past. So the `ret` returns to the instruction after the point at which Y called `swtch()`; that is, X's call to `swtch()` returns as if returning from Y's original call to `swtch()`. And `sp` will be Y's stack pointer, since `swtch` loaded `sp` from Y's `struct context`; thus on return, Y will execute on its own stack. `swtch()` need not directly save or restore the program counter; it's enough to save and restore `ra`.

`swtch` 中的 `ret` 执行后程序会返回到哪里呢？它会返回到 `ra` 寄存器所指向的指令。在上述线程 X 调用 `swtch()` 切换到 Y 的例子中，当 `ret` 执行时，我们已经从 Y 的 `struct context` 中恢复了 `ra` 寄存器的值。而 Y 线程的 `struct context` 中的 `ra` 保存了先前 Y 在放弃 CPU 时调用 `swtch` 保存的值。因此 `ret` 会返回到 Y 调用 `swtch()` 之后的指令；也就是说，X 调用 `swtch()` 返回时，就像从 Y 最初调用 `swtch()` 返回一样。此外，由于 `swtch` 从 Y 的 `struct context` 中加载了 `sp`，因此 `sp` 将指向 Y 的栈；因此，返回时 Y 将在自己的栈上运行。`swtch()` 不需要直接保存或恢复 “程序计数器（program counter）”；只需保存和恢复 `ra` 就足够了。

> `swtch` (2902) saves callee-saved registers (`ra` , `sp` , `s0` .. `s11`) but not caller-saved registers. The RISC-V calling convention requires that if code needs to preserve the value in a caller-saved register across a function call, the compiler must generate instructions that save the register to the stack before the function call, and restore from the stack when the function returns. So `swtch` can rely on the function that called it having already saved the caller-saved registers (either that, or the calling function didn't need the values in the registers).

`swtch` (2902) 仅保存 “被调用方负责保存（callee-saved）” 的寄存器（譬如 `ra`、`sp`、`s0` .. `s11`），但不保存 “调用方负责保存（caller-saved）” 的寄存器。这是因为根据 RISC-V 规范定义的函数调用约定的要求，如果代码需要在调用某个函数前后使用某个 caller-saved 寄存器，则由编译器负责生成指令，在函数调用前将该寄存器的值保存到栈中，并在函数返回后从栈中恢复该寄存器。因此，`swtch` 可以认为调用它的函数已经保存了 caller-saved 寄存器（或者调用者本身不关心这些寄存器中的值，所以更没有保存这些寄存器的必要）。

8.4 代码讲解：调度（Code: Scheduling）

> The last section looked at the internals of `swtch`; now let's take `swtch` as a given and examine switching from one process's kernel thread through the scheduler to another process. The scheduler exists in the form of a special thread per CPU, each running the `scheduler` function. This function is in charge of choosing which process to run next. Each CPU has its own scheduler thread because more than one CPU may be looking for

> something to run at any given time. Process switching always goes through the scheduler thread, rather than direct from one process to another, to avoid some situations in which there would be no stack on which to execute the scheduler (e.g. if the old process has exited, or there is no other process that currently wants to run).

上一节介绍了 `swtch` 的内部实现；现在，我们以 `swtch` 为例，分析如何通过调度器（scheduler）从一个进程的内核线程切换到另一个进程。调度器在每个 CPU 上都安排了一个特殊的线程，这个特殊的线程运行 `scheduler` 函数（译者注，下文将调度器在每个 CPU 上安排的线程也称为 scheduler 线程或调度线程）。该函数负责选择下一个在该 CPU 上运行的进程。每个 CPU 都有自己的 scheduler 线程，因为在任意时刻，可能有多个 CPU 正在等待执行任务。进程切换总是通过调度线程进行，而不是直接从一个进程切换到另一个进程，以避免出现没有栈来执行调度程序的情况（例如，如果旧进程已经退出，而当前也没有其他进程想要运行）。

> A process that wants to give up the CPU must acquire its own process lock `p->lock`, release any other locks it is holding, update its own state (`p->state`), and then call `sched`. You can see this sequence in `yield` (2629), `sleep` and `kexit`. `sched` calls `swtch` to save the current context in `p->context` and switch to the scheduler context in `cpu->context`. `swtch` returns on the scheduler's stack as though `scheduler`'s `swtch` had returned (2582).

想要放弃 CPU 的进程必须先获取到自己的进程锁 `p->lock`，释放其持有的任何其他锁，更新自身状态（`p->state`），然后调用 `sched`。我们可以在 `yield` (2629)、`sleep` 和 `exit` 中看到上述类似的程序执行步骤。`sched` 调用 `swtch` 将当前上下文保存在 `p->context` 中，并根据 `cpu->context` 中的值切换到 scheduler 线程的上下文。`swtch` 在 scheduler 线程的栈上返回，具体返回地址在 `scheduler` 函数中 (2582)，看上去就好像 `scheduler` 中的 `swtch` 函数返回了一样。

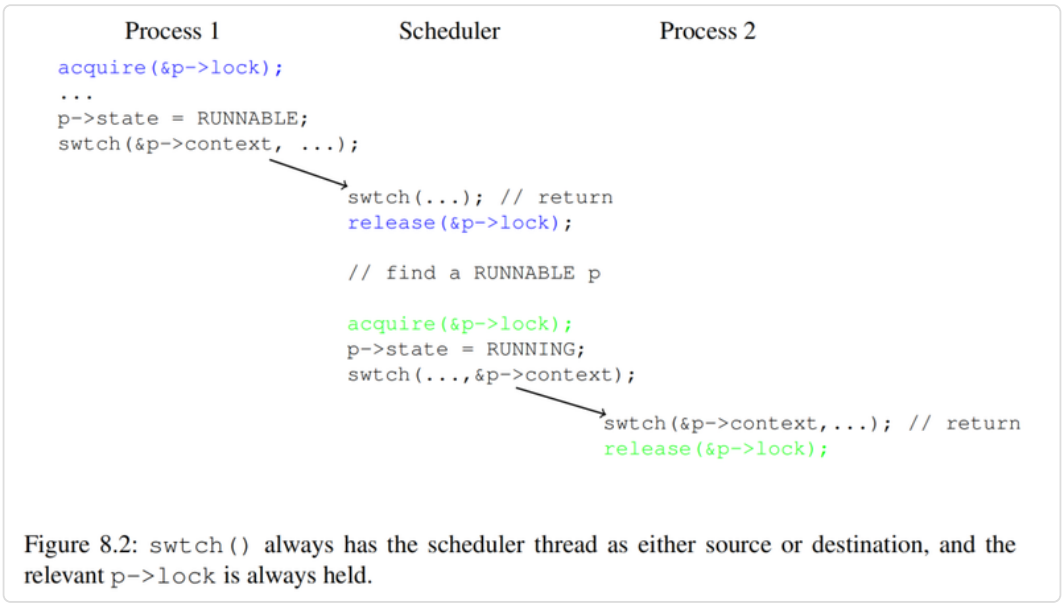

Figure 8.2: `swtch()` always has the scheduler thread as either source or destination, and the relevant `p->lock` is always held.

> `scheduler` (2558) runs a loop: find a process to run, `swtch()` to it, eventually it will `swtch()` back to the scheduler, which continues its loop. The scheduler loops over the process table looking for a runnable process, one that has `p->state == RUNNABLE`. Once it finds a process, it sets the per-CPU current process variable `c->proc`, marks the process as `RUNNING`, and then calls `swtch` to start running it (2577-2582). At some point in the past,

> the target process must have called `swtch()`; the scheduler's call to `swtch()` effectively returns from that earlier call. Figure 8.2 illustrates this pattern.

`scheduler` (2558) 通过一个循环来找到一个要运行的进程，然后通过调用 `swtch()` 切换到该进程，最终该进程也会通过调用 `swtch()` 将 CPU 返回给调度器，而调度器则继续运行这个循环。调度器在循环中遍历进程表，寻找一个可运行的进程，即 `p->state == RUNNABLE` 的进程。一旦找到一个（满足要求的）进程，它会将这个进程的句柄设置到当前 CPU 上用来记录当前运行进程的变量 `c->proc` 上，同时将该进程标记为 `RUNNING`，然后调用 `swtch` 开始运行它 (2577-2582)。在过去的某个时刻，该进程一定调用过 `swtch()`；调度器对 `swtch()` 的调用实际上会导致执行流从之前它调用 `swtch()` 处返回。图 8.2 说明了这种运行模式。

> xv6 holds `p->lock` across calls to `swtch`: the caller of `swtch` acquires the lock, but it's released in the target after `swtch` returns. This arrangement is unusual: it's more common for the thread that acquires a lock to also release it. Xv6's context switching breaks this convention because `p->state` and `p->context` must be updated together atomically. For example, if `p->lock` were released before invoking `swtch`, a different CPU `c` might decide to run the process because its state is `RUNNABLE`. CPU `c` will invoke `swtch` which will restore from `p->context` while the original CPU is still saving into `p->context`. The result would be that the process would be restored with partially-saved registers on CPU `c` and that both CPUs will be using the same stack, which would cause chaos. Once `yield` has started to modify a running process's state to make it `RUNNABLE`, `p->lock` must remain held until the process has saved all its registers and the scheduler is running on its stack. The earliest correct release point is after `scheduler` (running on its own stack) clears `c->proc`. Similarly, once `scheduler` starts to convert a `RUNNABLE` process to `RUNNING`, the lock cannot be released until the process's kernel thread is completely running (after the `swtch`, for example in `yield`).

xv6 在调用 `swtch` 时会持有 `p->lock`：锁的获取由 `swtch` 的调用者负责，而释放则由目标线程在其从 `swtch` 返回后完成。这种安排并不常见：一般情况下获取锁的线程也负责释放。xv6 的上下文切换打破了这种惯例，因为我们必需要确保在更新 `p->state` 和 `p->context` 时不被打扰。例如，如果在调用 `swtch` 之前释放了 `p->lock`，另一个 CPU `c` 可能会决定运行该进程，因为它的状态是 `RUNNABLE`。CPU `c` 将调用 `swtch`，而 `swtch` 会根据 `p->context` 对上下文进行恢复，而此时原先得那个 CPU 仍在将数据写入到 `p->context` 中。结果造成，该进程针对 CPU `c` 所恢复的寄存器上下文信息并不完整，甚至两个 CPU 都使用相同的栈，这将导致混乱。所以对于一个正在运行的进程，一旦 `yield` 开始修改其状态，将其变为 `RUNNABLE`， `p->lock` 必须保持锁定，直到该进程保存了所有寄存器，并且调度器开始在其栈上运行。最早的正确释放时间点是在 `scheduler`（在其自己的栈上运行）清除 `c->proc` 之后。同样，一旦 `scheduler` 开始将 `RUNNABLE` 进程转换为 `RUNNING`，在该进程的内核线程完全运行之前（例如，在 `yield` 中的 `swtch` 之后），锁都不能被释放。

> There is one case when the scheduler's call to `swtch` does not end up in `sched`. `allocproc` sets the context `ra` register of a new process to `forkret` (2653), so that its first `swtch` "returns" to the start of that function. `forkret` exists to release the `p->lock` and set up some control registers and trapframe fields that are required in order to return to user space. At the end, `forkret` simulates the normal return path from a system call back to user space.

存在一种例外，调度器在调用 `swtch` 后不会跳转到 `sched` 中。`allocproc` 将新进程的 `context.ra` 字段设置为 `forkret` (2653) 的地址，这样 `scheduler` 中第一次执行 `swtch` 时“返回”的位置将是 `forkret` 函数的首地址。`forkret` 的作用是释放 `p->lock`，并设置一些返回用户空间所需的控制寄存器和 trapframe 字段。最后，`forkret` 模拟了从系统调用返回用户空间的正常路径。

8.5 代码讲解：`mycpu` 和 `myproc`（Code: `mycpu` and `myproc`）

> Xv6 often needs a pointer to the current process's `proc` structure. On a uniprocessor one could have a global variable pointing to the current `proc`. This doesn't work on a multi-core machine, since each CPU executes a different process. The way to solve this problem is to exploit the fact that each CPU has its own set of registers.

xv6 中经常需要使用一个指向当前进程的 `proc` 结构体的指针。在单处理器上，可以定义一个指向当前 `proc` 的全局变量。但这在多核机器上行不通，因为每个 CPU 上当前执行的进程都不相同。解决这个问题的方法是利用每个 CPU 专属的那组寄存器。

> While a given CPU is executing in the kernel, xv6 ensures that the CPU's `tp` register always holds the CPU's hartid. RISC-V numbers its CPUs, giving each a unique hartid. `mycpu` (2178) uses `tp` to index an array of `cpu` structures and return the one for the current CPU. A `struct cpu` (1971) holds a pointer to the `proc` structure of the process currently running on that CPU (if any), saved registers for the CPU's scheduler thread, and the count of nested spinlocks needed to manage interrupt disabling.

当一个给定的 CPU 在内核态下运行时，xv6 会确保 CPU 的 `tp` 寄存器始终保存该 CPU 的 hartid 的值。RISC-V 规定每个 CPU 都有一个唯一的 hartid 作为其编号。`mycpu` (2178) 使用 `tp` 的值来索引结构体 `cpu` 的数组，并返回当前 CPU 对应的结构体。`struct cpu` (1971) 的成员包括一个指向当前在该 CPU 上运行的进程（如果有）的结构体 `proc` 的指针（`cpu.proc`）；一个用于保存该 CPU 对应的 scheduler 线程的寄存器上下文的成员变量（`cpu.context`）；以及一个用于记录嵌套的自旋锁数量的成员变量（`cpu.noff`），这个成员变量可用于管理是否关闭中断。

> Ensuring that a CPU's `tp` holds the CPU's hartid is a little involved, since user code is free to modify `tp`. `start` sets the `tp` register early in the CPU's boot sequence, while still in machine mode (1094). While preparing to return to user space, `prepare_return` saves `tp` in the trampoline page, in case user code modifies it. Finally, `uservec` restores that saved `tp` when entering the kernel from user space (3127). The compiler guarantees never to modify `tp` in kernel code. It would be more convenient if xv6 could ask the RISC-V hardware for the current hartid whenever needed, but RISC-V allows that only in machine mode, not in supervisor mode.

确保 CPU 的 `tp` 保存 CPU 的 hartid 有点复杂，因为用户态代码有权限自由修改 `tp`。在 CPU 启动的早期，`start` 会设置 `tp` 寄存器，此时 CPU 仍处于机器模式 (1094)。准备返回用户空间时，`prepare_return` 将 `tp` 保存在 trampoline 内存页中，以防用户代码修改它。最后，当进程从用户空间进入内核时 `uservec` 会恢复保存的 `tp` (3127)。编译器会保证永远不会在内核代码中修改 `tp`（译者注：这是 RISC-V ABI 的规定）。如果 xv6 可以随时在需要时向 RISC-V 硬件获取当前的 hartid 就更方便了，但 RISC-V 仅在机器模式下允许这样做，而在管理员模式下则不允许。

> The return values of `cpuid` and `mycpu` are fragile: if the timer were to interrupt and cause the thread to yield and later resume execution on a different CPU, a previously returned value would no longer be correct. To avoid this problem, xv6 requires code to disable interrupts before calling `cpuid()` or `mycpu()`, and only enable interrupts when done using the returned value.

`cpuid` 和 `mycpu` 的返回值需要被保护起来：因为定时器中断有可能会导致当前（调用了 `cpuid` 和 `mycpu` 的）线程放弃 CPU，而该线程稍后又在另一个 CPU 上恢复执行，则先前返回的值将不再正确。为了避免这个问题，xv6 要求代码在调用 `cpuid()` 或 `mycpu()` 之前禁用中断，并且仅在使用返回值完成后启用中断。

> The function `myproc` (2187) returns the `struct proc` pointer for the process that is running on the current CPU. `myproc` disables interrupts, invokes `mycpu`, fetches the current process pointer (`c->proc`) out of the `struct cpu`, and then enables interrupts. The return value of `myproc` is safe to use even if interrupts are enabled: if a timer interrupt moves the calling process to a different CPU, its `struct proc` pointer will stay the same.

函数 `myproc`（2187）返回当前 CPU 上运行的进程的 `struct proc` 指针。`myproc` 中会先禁用中断，然后调用 `mycpu`，从 `struct cpu` 中获取当前进程指针（`c->proc`），然后启用中断。虽然此时启用了中断，`myproc` 的返回值也是可以安全使用的：因为即使定时器中断将调用 `mycpu` 的进程移动到不同的 CPU，但此时获得的 `struct proc` 指针的值并不会被改变（译者注：可以这么理解，`myproc` 本质是一个进程找自己，如果已经找到自己后即使自己被迁移到别的 CPU 对结果也没有影响）。

8.6 现实世界（Real world）

> The xv6 scheduler implements a simple scheduling policy that runs each process in turn. This policy is called round robin. Real operating systems implement more sophisticated policies that, for example, allow processes to have priorities. The idea is that a runnable high-priority process will be preferred by the scheduler over a runnable low-priority process. These policies can become complex because there are often competing goals: for example, the operating system might also want to guarantee fairness and high throughput.

xv6 调度器实现了一个简单的调度策略，即依次运行每个进程。这个策略被称为 轮转（round robin）。真正的操作系统会实现更复杂的策略，例如允许进程拥有优先级。这么做的目的是允许调度器优先选择 “就绪（runnable）” 的高优先级进程，而不是就绪的低优先级进程。这些策略可能会变得复杂，因为通常存在相互冲突的目标：例如，操作系统可能希望在保证公平性的同时又维持高的吞吐量。

8.7 练习（Exercises）

> 1. Modify xv6 to use only one context switch when switching from one process’s kernel thread to another, rather than switching through the scheduler thread. The yielding thread will need to select the next thread itself and call `swtch`. The challenges will be

> to prevent multiple CPUs from executing the same thread accidentally; to get the locking right; and to avoid deadlocks.

1. 修改 xv6，使其在从一个进程的内核线程切换到另一个进程时仅使用一次上下文切换，而不是通过 scheduler 线程进行切换。放弃 CPU 的线程需要自行选择下一个线程并调用 `swtch`。挑战在于如何防止多个 CPU 意外执行同一个线程；如何正确锁定；以及如何避免死锁。

第 9 章 睡眠与唤醒（Chapter 9 Sleep and Wakeup）

> Scheduling and locks help conceal the actions of one thread from another, but we also need abstractions that help threads intentionally interact. For example, the reader of a pipe in xv6 may need to wait for a writing process to produce data; a parent's call to `wait` may need to wait for a child to exit; and a process reading the disk needs to wait for the disk hardware to finish the read. The xv6 kernel uses a mechanism called sleep and wakeup in these situations (and many others). Sleep allows a kernel thread to wait for some condition to be true; another thread or an interrupt handler can cause the condition to be true (typically by modifying some variable(s)) and then call wakeup to indicate that threads waiting for the condition should resume. Sleep and wakeup are often called sequence coordination or conditional synchronization mechanisms.

调度和锁有助于隐藏线程之间的相互作用，但我们也需要一些明确的原语来表达线程之间有意识的交互。例如，xv6 中管道的 “读端（reader）” （译者注：即从管道中读取数据的进程，后面的 “写端” 同理）可能需要等待管道的 “写端（writer）” 写入数据；父进程调用 `wait` 等待子进程退出；读取磁盘的进程需要等待磁盘硬件完成读取。xv6 内核针对这些情况（以及许多其他情况）引入了一种称为 “睡眠（sleep）” 和 “唤醒（wakeup）” 的机制。“睡眠” 允许内核线程等待特定条件变成真；另一个线程或者中断处理函数负责将条件变为真（典型地通过修改一些变量）然后调用 “唤醒” 来通知处于等待该条件的线程恢复运行。“睡眠” 和 “唤醒” 通常被称为 次序协调（sequence coordination） 或 条件同步（conditional synchronization） 机制。

> Before proceeding, please read the functions `sleep()` and `wakeup()` in `kernel/proc.c`, and all of file `kernel/pipe.c`.

在继续之前，请阅读 `kernel/proc.c` 中的函数 `sleep()` 和 `wakeup()`，以及整个 `kernel/pipe.c` 文件。

9.1 概述（Overview）

> The sleep/wakeup interface looks like:

“睡眠” 和 “唤醒” 函数的声明如下：

```
void sleep(void *chan, struct spinlock *lk)
void wakeup(void *chan)
```

> `sleep()` marks the calling process as `SLEEPING` (not `RUNNABLE`) and releases the CPU by context-switching to the scheduler, so that other processes can run. The `chan` argument is called the wait channel. `wakeup(chan)` wakes up all processes (if any) that have called `sleep(chan, ...)` with the same `chan` value. `sleep` and `wakeup` treat `chan` as an opaque 64-bit value; the only thing they do with it is compare for equality. The usual pattern is for callers to pass the address of some convenient object as the `chan` argument.

`sleep()` 将调用进程标记为 `SLEEPING`（而非 `RUNNABLE`），并通过上下文切换到调度程序释放 CPU，以便其他进程可以运行。参数 `chan` 称为 等待通道（wait channel）。`wakeup(chan)` 会唤醒所有使用相同 `chan` 值调用 `sleep(chan, ...)` 的进程（如果没有则什么也不做）。`sleep` 和 `wakeup` 并不关心 `chan` 具体是什么，对于它们来说 `chan` 只是一个64 位的值；对 `chan` 的唯一处理操作是比较其相等性。通常情况下调用者会将某个对象的地址作为实参传入 `chan`。

> Kernel code calls `sleep` to wait for some condition to become true. For example, the kernel code that reads from a pipe calls `sleep` if the pipe buffer is currently empty; the condition in this case is the pipe buffer becoming non-empty (due to another process writing to the pipe). `sleep` and `wakeup` do not know what the condition is: only the calling code knows. The usual pattern is for the caller to first check the condition, and call `sleep` if it is not true; code that later makes the condition true calls `wakeup`.

内核代码调用 `sleep` 来等待某些 条件 变为真。例如，如果当前管道的缓冲区为空，则从管道读取数据的内核代码会调用 `sleep`；在这种情况下，等待的条件是管道缓冲区变为非空（另一个进程会写入管道）。`sleep` 和 `wakeup` 并不关心条件是什么：只有调用代码知道。通常的模式是，调用者首先检查条件，如果条件不成立，则调用 `sleep`；之后使条件成立的代码会调用 `wakeup`。

> Here’s a sketch of how the xv6 kernel pipe code uses `sleep` and `wakeup`:

下面是 xv6 内核中实现管道的代码使用 `sleep` 和 `wakeup` 的简单示例：

```
piperead(pipe) {
  acquire(&pipe->lock);
  while(there's no data in pipe->buffer){
    // ZZZ
    sleep(&pipe, &pipe->lock);
  }
  remove the data from the pipe;
  release(&pipe->lock);
}

pipewrite(pipe) {
  acquire(&pipe->lock);
  append data to pipe->buffer;
  wakeup(&pipe);
  release(&pipe->lock);
}
```

> This code uses the address of the pipe data structure as the wait channel.

此代码使用管道结构体对象的地址作为等待通道。

> What is the `lk` argument to `sleep`? In all uses of `sleep`/`wakeup` the condition involves shared data, used by both the thread that sleeps and the thread that calls `wakeup`, so there always turns out to be a lock that protects the condition. That lock is called the condition lock. In the pipe code above, both functions use the pipe and its buffer while holding the pipe lock, which in this case is also the condition lock. It’s a rule that any code that calls `sleep` or `wakeup` must hold the condition lock, and that the lock must be passed to `sleep` as the second argument.

`sleep` 的参数 `lk` 又是做什么用的呢？在所有 `sleep` / `wakeup` 的使用中，涉及的等待条件都和共享数据有关，这些数据被需要休眠的线程和调用 `wakeup` 的线程（译者注，负责唤醒睡眠线程）使用，因此总会需要有一个锁来保护这个等待条件。因此这个锁被称为 条件锁（condition lock）。在上面的管道实现代码中，两个函数都需要在持有管道锁的前提下使用管道及其缓冲区，这里的锁就是我们说的条件锁。因此处理原则是，任何调用 `sleep` 或 `wakeup` 的代码都必须持有条件锁，并且必须将该锁作为第二个参数传递给 `sleep`。

> The reason that the condition lock must be held when `sleep` is called, and that it must be passed to `sleep`, is to prevent the possibility that another thread might call `wakeup` between the check of the condition and the call to `sleep`. A call to `wakeup` at that point would find no sleeping process to wake up; the `wakeup` would simply return. But then the call to `sleep` might never wake up, since the `wakeup` intended for it has already happened. This undesirable situation is called a lost wake-up.

之所以在调用 `sleep` 时必须持有条件锁，并且必须将锁传递给 `sleep`，这是考虑到我们需要避免一种情况，就是在检查条件和调用 `sleep` 之间，另一个线程可能会调用 `wakeup`（译者注：简单来说，就是在调用 `sleep` 之前提前发生了 `wakeup`）。此时调用 `wakeup` 将找不到任何可唤醒的休眠进程；也就是说在这种情况下 `wakeup` 什么也做不了。这会导致 `sleep` 调用可能永远不会被唤醒，因为预期的 `wakeup` 已经发生（但什么也没有做）。对于这种我们不希望看到的情况我们将其称为 丢失唤醒（lost wake-up）。

> In the pipe example above, the lost wake-up being avoided is the possibility that a thread on another CPU might call `pipewrite` at the point marked `ZZZ`, between `piperead`'s check of the condition and its call to `sleep`. The fact that `piperead` holds the pipe lock during the time between when it checks the condition and calls `sleep` prevents `pipewrite` from executing, and thus prevents a lost wake-up.

在上面的管道示例中，我们需要着力避免的丢失唤醒问题在于另一个 CPU 上的线程可能在标记为 `ZZZ` 的位置调用 `pipewrite`，该位置位于 `piperead` 检查条件和调用 `sleep` 之间。正如代码所示，`piperead` 在检查条件和调用 `sleep` 之间的时间段内确保持有了管道锁，从而阻止了 `pipewrite` 的执行，避免了丢失唤醒。

> `sleep()` releases the condition lock so that the code calling `wakeup()` can proceed. `sleep()` also context-switches to the scheduler in order to let other threads run while it is waiting. The implementation performs these two steps in a way that is atomic (indivisible) with respect to `wakeup()`, to prevent lost wake-ups.

`sleep()` 中会释放条件锁，以便调用 `wakeup()` 的代码可以继续执行。`sleep()` 还会将上下文切换到调度器，以便在其（即调用 `sleep` 的线程）等待期间让其他线程运行。具体实现中以原子的（不可分割）的方式执行这两个步骤，从而避免了丢失唤醒情况的发生。

9.2 代码讲解：睡眠与唤醒（Code: Sleep and wakeup）

> Xv6's `sleep` (2703) and `wakeup` (2734) implement the interface used in the example above. The basic idea is to have `sleep` mark the current process as `SLEEPING` and then call `sched` to release the CPU; `wakeup` looks for a process sleeping on the given wait channel and marks it as `RUNNABLE`. Callers of `sleep` and `wakeup` can use any mutually

> convenient number as the channel. Xv6 often uses the address of a kernel data structure involved in the waiting.

xv6 的 `sleep` (2703) 和 `wakeup` (2734) 实现了上面最后一个示例中使用的接口。其基本思想是让 `sleep` 将当前进程标记为 `SLEEPING`，然后调用 `sched` 释放 CPU；`wakeup` 查找在给定 “等待通道” 上睡眠的进程，并将其状态切换为 `RUNNABLE`。`sleep` 和 `wakeup` 的调用者可以使用任何彼此方便的值作为通道。xv6 通常使用等待过程中涉及的内核数据结构的地址（译者注：譬如上面代码例子中的 `&pipe`）。

> `sleep` acquires `p->lock` (2714) and only then releases the condition lock `lk`. The fact that `sleep` holds one or the other of these locks at all times is what prevents a concurrent `wakeup` (which must acquire and hold both) from acting, and thus prevents a lost wake-up. Now that `sleep` holds just `p->lock`, it can put the process to sleep by recording the wait channel, changing the process state to `SLEEPING`, and calling `sched` (2718-2721). In a moment it will be clear why it’s critical that `p->lock` is not released (by `scheduler`) until after the process is marked `SLEEPING`.

`sleep` 首先尝试获取 `p->lock` (kernel/proc.c:559)，只有拿到这把锁才会释放 `lk`。`sleep` 始终确保只会持有 `p->lock` 或者 `lk` 中的一个，这么做阻止了并发的 `wakeup`，从而避免了一次丢失唤醒（执行 `wakeup` 要求同时获取并持有这两把锁（译者注：参考上面的例子代码，`lk`（即 `&pipe->lock`）是在进入 `wakeup` 之前所获取，而对于 `p->lock`，参考 xv6 中 `wakeup` 函数的实现，是在 `wakeup` 函数中被获取）。（译者注，这里继续 `sleep` 函数的解释）`sleep` 此时只持有 `p->lock`，接下来它记录等待通道、再将进程状态更改为 `SLEEPING`，最后调用 `sched` 将进程进入睡眠状态（2718-2721）。稍后我们将解释为什么只有在进程被标记为 `SLEEPING` 之后，`p->lock` 才能被 `scheduler` 释放，这一点很重要。

> At some point, a process will acquire the condition lock, set the condition that the sleeper is waiting for, and call `wakeup(chan)`. It’s important that `wakeup` is called while holding the condition lock (Strictly speaking it is sufficient if `wakeup` merely follows the `acquire` (that is, one could call `wakeup` after the `release`).). `wakeup` loops over the process table (2734). It acquires the `p->lock` of each process it inspects. When `wakeup` finds a process in state `SLEEPING` with a matching `chan`, it changes that process’s state to `RUNNABLE`. The next time `scheduler` runs, it will see that the process is ready to be run.

总会在其他某个时间点上，另一个进程会获取 “条件锁（condition lock）”，设置睡眠进程正在等待的条件，然后调用 `wakeup(chan)`。重要的是，调用 `wakeup` 的前提是要先持有条件锁（严格来说，如果 `wakeup` 仅仅跟在 `acquire` 之后就足够了（也就是说，可以在 `release` 之后调用 `wakeup`）。）。`wakeup` 会循环遍历进程表 (2734)。它会尝试获取每个进程的 `p->lock`。当 `wakeup` 发现一个进程的状态处于 `SLEEPING` 且 `chan` 匹配时，它会将该进程的状态更改为 `RUNNABLE`。下次 `scheduler` 运行时，它就会发现该进程已准备好可以再次运行。

> Why do the locking rules for `sleep` and `wakeup` ensure that a process that’s going to sleep won’t miss a concurrent wakeup? The going-to-sleep process holds either the condition lock or its own `p->lock` or both from before it checks the condition until after it has marked itself as `SLEEPING`; see Figure 9.1. The process calling `wakeup` needs to acquire both locks. The waker might acquire the locks first, which measn it will make the condition true before the consuming thread checks the condition, and the consuming thread won't need to call `sleep`; or the waker’s `acquire()`s might have to wait until the consuming

> thread has completely finished going to sleep and releases the locks, in which case the waker will then see that the consuming thread is marked `SLEEPING` and will wake it up.

为什么 `sleep` 和 `wakeup` 函数中对锁的使用能够确保即将进入睡眠的进程不会错过一次并发的唤醒呢？这是因为即将进入睡眠态的进程在它检查条件 之前 直到被标记为 `SLEEPING` 之后，会一直持有 “条件锁” 或者自身的进程锁 `p->lock`，或者两者都持有；参考图 9.1。调用 `wakeup` 的进程需要获取 两个 锁。负责唤醒的线程可能先获取到锁，这意味着它会在消费线程检查条件之前使条件成立，对于这种情况消费线程将无需调用 `sleep`；另外一种情况就是负责唤醒的线程调用 `acquire()` 后可能需要一直等待，等到消费线程完全进入睡眠状态并释放锁，在这种情况下，负责唤醒的线程会发现消费线程被标记为 `SLEEPING`，并将其唤醒。

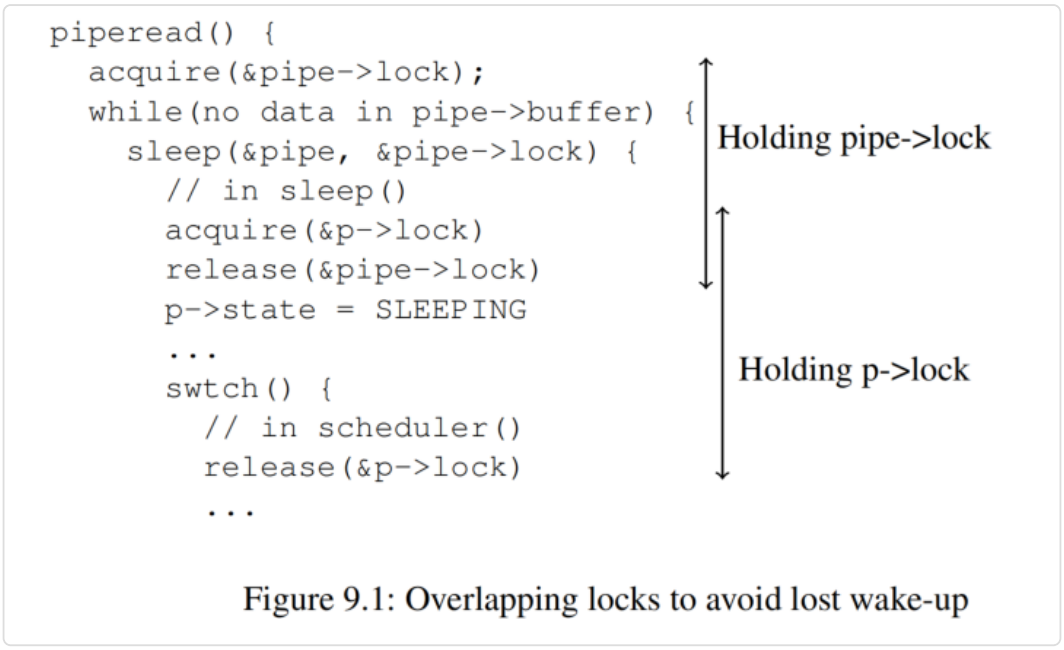

Figure 9.1: Overlapping locks to avoid lost wake-up

> Sometimes multiple processes are sleeping on the same channel; for example, more than one process reading from a pipe. A single call to `wakeup` will wake them all up. One of them will run first and acquire the lock that `sleep` was called with, and (in the case of pipes) read whatever data is waiting. The other processes will find that, despite being woken up, there is no data to be read. From their point of view the wakeup was “spurious,” and they must sleep again. For this reason `sleep` is always called inside a loop that re-checks the condition, as in P above.

有时多个进程在同一个通道上休眠；例如，多个进程正在从管道读取数据。只需调用一次 `wakeup` 即可唤醒所有进程。这期间总有一个进程将首先运行并抢到调用 `sleep` 时传入的锁（译者注：即 `sleep` 函数的第二个参数 `lk`），然后（依然以管道为例）读取所有正在等待的数据。其他进程会发现，尽管自己被唤醒，但没有数据可读取。从它们的角度来看，这次唤醒是 “虚假的（spurious）”，它们必须再次休眠。因此，我们需要确保在一个反复检查条件的循环中调用 `sleep`。

> No harm is done if two uses of sleep/wakeup accidentally choose the same channel: they will see spurious wakeups, but looping as described above will tolerate this problem. Much of the charm of sleep/wakeup is that it is both lightweight (no need to create special data

> structures to act as wait channels) and provides a layer of indirection (callers need not know which specific process they are interacting with).

如果睡眠和唤醒各执行了两次并且意外选择了同一个通道，也不会有什么问题：它们会检测到虚假的唤醒，如上所述的循环可以处理这个问题。睡眠和唤醒机制的最大魅力在于不仅其实现简单（无需创建特殊的数据结构来充当等待通道），又提供了一层封装（调用者无需知道它们正在与哪个特定进程交互）。

9.3 代码讲解：管道（Code: Pipes）

> xv6's pipes are an example of code that uses `sleep` and `wakeup` to synchronize producers and consumers is xv6's implementation of pipes. We saw the interface for pipes in Chapter 1: bytes written to one end of a pipe are copied to an in-kernel buffer and then can be read from the other end of the pipe. Future chapters will examine the file descriptor support surrounding pipes, but let's look now at the implementations of `pipewrite` and `piperead`.

xv6 中实现的 “管道（pipe）” 是一个使用 `sleep` 和 `wakeup` 来同步生产者和消费者例子。我们在第 1 章中了解了管道的接口定义：写入管道一端的字节会被复制到内核中的一块内存缓冲区中，然后可以被管道的另一端读取。后续章节将探讨如何通过文件描述符操作管道，现在我们先来看看 `pipewrite` 和 `piperead` 的实现。

> Each pipe is represented by a `struct pipe`, which contains a `lock` and a `data` buffer. The fields `nread` and `nwrite` count the total number of bytes read from and written to the buffer. The buffer wraps around: the next byte written after `buf[PIPESIZE-1]` is `buf[0]`. The counts do not wrap. This convention lets the implementation distinguish a full buffer (`nwrite == nread+PIPESIZE`) from an empty buffer (`nwrite == nread`), but it means that indexing into the buffer must use `buf[nread % PIPESIZE]` instead of just `buf[nread]` (and similarly for `nwrite`).

每个管道由一个 `struct pipe` 的结构体表示，该结构体中包含一个 `lock` 和一个 `data` 缓冲区。`nread` 和 `nwrite` 字段分别用于计数从缓冲区读取和写入的字节总数。对缓冲区的写入会回绕：超出 `buf[PIPESIZE-1]` 写入的下一个字节会写入 `buf[0]`。计数不会回绕。基于以上约定我们可以区分缓冲区的状态是 “满（`nwrite == nread+PIPESIZE`）” 还是 “空（`nwrite == nread`）”，这也意味着当我们需要索引缓冲区中的某个字节时必须使用 `buf[nread % PIPESIZE]`，而不能使用 `buf[nread]`（对 `nwrite` 也是如此）。

> Let's suppose that calls to `piperead` and `pipewrite` happen simultaneously on two different CPUs. `pipewrite` (6679) begins by acquiring the pipe's lock, which protects the counts, the data, and their associated invariants. `piperead` (6708) then tries to acquire the lock too, but cannot. It spins in `acquire` (1271) waiting for the lock. While `piperead` waits, `pipewrite` loops over the bytes being written (`addr[0..n-1]`), adding each to the pipe in turn (6697). During this loop, it could happen that the buffer fills (6690). In this case, `pipewrite` calls `wakeup` to alert any sleeping readers to the fact that there is data waiting in the buffer and then sleeps on `&pi->nwrite` to wait for a reader to take some bytes out of the buffer. `sleep` releases the pipe's lock as part of putting `pipewrite`'s process to sleep.

假设对 `piperead` 和 `pipewrite` 的调用同时发生在两个不同的 CPU 上。而且假定 `pipewrite` (6679) 首先抢到管道的锁，该锁用于保护计数、数据及其相关的 invariant。`piperead` (6708) 随后尝试获取锁，但此时获取不成功。进入 `acquire` (1271) 后等待锁被释放。在 `piperead` 等待期间，`pipewrite` 循环遍历正在写入的字节 (`addr[0..n-1]`)，并将每个字节依次加入管道的缓冲区 (6697)。在此循环期间，缓冲区可能会被填满 (6690)。在这种情况下，`pipewrite` 调用 `wakeup` 来提醒处于休眠状态的读端此时缓冲区中有数据在等待读取，然后 `pipewrite` 在 `&pi->nwrite` 上休眠，等待读端从缓冲区中取出一些字节。对 `sleep` 的调用在将运行 `pipewrite` 的进程进入休眠的同时会释放管道的锁。

> `piperead` now acquires the pipe's lock and enters its critical section: it finds that `pi->nread != pi->nwrite` (6715) (`pipewrite` went to sleep because `pi->nwrite == pi->nread + PIPESIZE` (6690)), so it falls through to the `for` loop, copies data out of the pipe (6722), and increments `nread` by the number of bytes copied. That much space in the buffer is now available for writing, so `piperead` calls `wakeup` (6729) to wake any sleeping writers before it returns. `wakeup` finds a process sleeping on `&pi->nwrite` , the process that was running `pipewrite` but stopped when the buffer filled. It marks that process as `RUNNABLE` .

`piperead` 现在获取了管道的锁并进入其临界区：它会发现 `pi->nread != pi->nwrite` (6715)（`pipewrite` 之所以进入睡眠状态正是因为 `pi->nwrite == pi->nread + PIPESIZE` (6690)），因此它进入 `for` 循环，将数据从管道的缓冲区往外复制 (6722)，并对 `nread` 增加已写出的字节数。此时说明缓冲区有足够的空间可供写入，因此 `piperead` 在其返回之前会调用 `wakeup` (6729)，唤醒所有睡眠中的写端进程。`wakeup` 会发现一个在 `&pi->nwrite` 上睡眠的进程而该进程正是前面运行 `pipewrite` 并在缓冲区填满后停止的那个。`wakeup` 随即将该进程标记为 `RUNNABLE` 。

> The pipe code uses separate wait channels for reader and writer (`pi->nread` and `pi->nwrite`); this might make the system more efficient in the unlikely event that there are lots of readers and writers waiting for the same pipe. The pipe code sleeps inside a loop checking the sleep condition; if there are multiple readers or writers, all but the first process to wake up will see the condition is still false and sleep again.

管道的代码为读取端和写入端维护单独的等待通道（`pi->nread` 和 `pi->nwrite` ）；这对于存在多个读取端和写入端等待同一个管道的情况来说（虽然这种情况不太可能发生）可能会提高系统效率。`piperead` 和 `pipewrite` 中的代码将执行睡眠的操作包在一个检查睡眠条件的循环中；因此如果有多个读取端或写入端，除了第一个被唤醒的进程之外，其他所有（被唤醒的）进程都会发现条件仍然为假，并再次进入睡眠状态。

9.4 代码讲解：`wait` , `exit` 和 `kill` （Code: Wait, exit, and kill）

> Please read the code for functions `kwait()` , `kexit()` , and `kkill()` in `kernel/proc.c` ; these are the internal implementations of the corresponding system calls.

请阅读 `kernel/proc.c` 中函数 `kwait()` 、`kexit()` 和 `kkill()` 的代码；这些是相应系统调用的内部实现。

> `sleep` and `wakeup` can be used for many kinds of waiting. An interesting example, introduced in Chapter 1, is the interaction between a child's `exit` and its parent's `wait`. At the time of the child's death, the parent may already be sleeping in `wait`, or may be doing something else; in the latter case, a subsequent call to `wait` must observe the child's death, perhaps long after it calls `exit`. The way that xv6 records the child's demise until `wait` observes it is for `exit` to put the caller into the `ZOMBIE` state, where it stays until the parent's `wait` notices it, changes the child's state to `UNUSED`, copies the child's exit status, and returns the child's process ID to the parent. If the parent exits before the child, the parent gives the child to the `init` process, which perpetually calls `wait`; thus every child has a parent to clean up after it. A challenge is to avoid races and deadlock between simultaneous parent `wait` and child `exit`, as well as simultaneous `exit` and `exit`.

`sleep` 和 `wakeup` 可用于多种等待的场景。第 1 章介绍过一个有趣的例子，即子进程的 `exit` 与其父进程的 `wait` 之间的交互。子进程退出时，父进程可能已经在 `wait` 中休眠，或者正在执行其他操作；在后一种情况下，后续对 `wait` 的调用必须观察到子进程已退出，这很可能发生在（子进程）调用 `exit` 很久之后。xv6 记录子进程结束的方式是，在 `wait` 观察到该结果之前，`exit` 会将调用者的状态设置为 `ZOMBIE`，该状态会一直保持到父进程调用 `wait` 发现它已退出，`wait` 会将子进程的状态更改为 `UNUSED`，复制子进程的退出状态，并将子进程的进程 ID 返回给父进程。如果父进程先于子进程退出，则父进程会将子进程托付给 `init` 进程，而后者会循环调用 `wait`；所以，每个子进程都会有一个父进程来负责回收它。存在的挑战是如何避免当父子进程同时调用 `wait` 和 `exit` 或者同时调用 `exit` 和 `exit` 时可能引起的竞争和死锁。

> `kwait`, the kernel implementation for `wait`, starts by acquiring `wait_lock` (2503), which acts as the condition lock that helps ensure that `kwait` doesn't miss a `wakeup` from an exiting child. Then `kwait` scans the process table. If it finds a child in `ZOMBIE` state, it frees that child's resources and its `proc` structure, copies the child's exit status to the address supplied to `wait` (if it is not 0), and returns the child's process ID. If `kwait` finds children but none have exited, it calls `sleep` to wait for any of them to exit (2545), then scans again. `kwait` often holds two locks, `wait_lock` and some process's `pp->lock`; the deadlock-avoiding order is first `wait_lock` and then `pp->lock`.

系统调用 `wait` 在内核中的实现函数 `kwait` 会首先尝试获取 `wait_lock` (2503) 这把锁，该锁充当条件锁，有助于确保 `kwait` 不会错过正在退出的子进程（译者注：在 `kexit` 中）对 `wakeup` 的调用。获取 `wait_lock` 后 `kwait` 会扫描进程表。如果它发现一个处于 `ZOMBIE` 状态的子进程，它会释放该子进程的资源及其 `proc` 结构，将子进程的退出状态复制到 `wait` 函数的参数传入的地址处（前提是该地址不为 0），并返回该子进程的进程 ID。如果 `kwait` 发现存在子进程但目前都还没有退出，那么它会调用 `sleep` 进入等待 (2545)，直到任何一个子进程退出而被唤醒，然后 `kwait` 会进入下一个循环再次扫描进程表 (译者注：此时再次扫描必然能够发现有子进程状态变为 `ZOMBIE`)。`kwait` 通常持有两把锁，`wait_lock` 和某个进程的 `pp->lock`；避免死锁的顺序是确保先获取 `wait_lock`，后获取 `pp->lock`。

> `kexit` (2454) records the exit status, frees some resources, calls `reparent` to give any children to the `init` process, wakes up the parent in case it is in `wait`, marks the caller as a zombie, and permanently yields the CPU. `kexit` holds both `wait_lock` and `p->lock` during this sequence. It holds `wait_lock` because it's the condition lock for the `wakeup(p->parent)`, preventing a parent in `wait` from losing the wakeup. `kexit` must

> hold `p->lock` for this sequence also, to prevent a parent in `wait` from seeing that the child is in state `ZOMBIE` before the child has finally called `swtch`. `kexit` acquires these locks in the same order as `kwait` to avoid deadlock.

`kexit` (2454) 会记录退出状态，释放一些资源，调用 `reparent` 将其所有的子进程委托给 `init` 进程，唤醒有可能在 `wait` 中等待的父进程，将调用者本身标记为 “僵尸（zombie）” 进程，并永久交出 CPU。`kexit` 在上述执行过程中同时持有 `wait_lock` 和 `p->lock`。它持有 `wait_lock` 是因为它是 `wakeup(p->parent)` 的条件锁，其目的是为了防止处于 `wait` 状态的父进程错过唤醒通知。`kexit` 也必须在上述执行过程中持有 `p->lock`，以防止在 `wait` 中等待的父进程在子进程最终调用 `swtch` 之前就看到子进程已处于 `ZOMBIE` 状态。另外，`kexit` 也需要按照与 `kwait` 相同的顺序获取这些锁以避免死锁。

> It may look incorrect for `kexit` to wake up the parent before setting its state to `ZOMBIE`, but that is safe: although `wakeup` may cause the parent to run, the loop in the parent's `kwait` cannot examine the child until the child’s `p->lock` is released by `scheduler`, so `kwait` can’t look at the exiting process until well after `kexit` has set its state to `ZOMBIE` (2486).

在将子进程的状态设置为 `ZOMBIE` 之前，`kexit` 会唤醒其父进程，这看上去似乎不太正确，但这是安全的：尽管 `wakeup` 可能唤醒（译者注：在 `wait` 中睡眠的）父进程，但在 `kwait` 的循环中，在子进程的 `p->lock` 被 `scheduler` 释放之前是无法查看子进程的状态的，相反，`kwait` 会一直阻塞，直到 `kexit` 将子进程的状态设置为 `ZOMBIE`（2486）之后才能继续并看到已退出的进程（译者注：严格地说这个时间点是在 `kexit` 中调用 `sched` 后，上下文切换到调度线程，并在 `scheduler` 中释放 `p->lock` 之后）。

> While `exit` allows a process to terminate itself, the `kill` system call (2754) lets one process request that another terminate. It would be too complex for `kill` to directly destroy the victim process, since the victim might be executing on another CPU, perhaps in the middle of a sensitive sequence of updates to kernel data structures. Thus `kkill` does very little: it just sets the victim’s `p->killed` and, if it is sleeping, wakes it up. Eventually the victim will enter or leave the kernel, at which point code in `usertrap` will call `kexit` if `p->killed` is set (it checks by calling `killed` (2783)). If the victim is running in user space, it will see that it has been killed the next time it enters the kernel by making a system call or because the timer (or some other device) interrupts.

`exit` 用于一个进程终止自己，系统调用 `kill` (2754) 则用于一个进程请求终止另一个进程。希望通过 `kill` 直接销毁某个进程（该进程我们形象地称之为 受害者（victim））可没有那么简单，因为 victim 进程可能正在另一个 CPU 上执行，甚至可能正在对内核数据结构进行一系列敏感的更新。因此，`kkill` 函数中的代码并不会做实际的销毁工作：它只是简单地设置 victim 进程的 `p->killed` 标志位，如果 victim 进程处于休眠状态，则将其唤醒。最终，victim 进程会进入或离开内核态，此时 `usertrap` 中的代码会检查 `p->killed` 是否已设置（通过调用 `killed` (2783)），如果被设置了则调用 `kexit`（完成实际的销毁）。如果 victim 进程此时正在用户空间运行，它很快就会通过系统调用或因为定时器（或其他外设）中断再一次进入内核态并看到它已经被杀死了。

> If the victim process is in `sleep`, `kkill`’s call to `wakeup` will cause the victim to return from `sleep`. This is potentially dangerous because the condition being waited for for may not be true. However, xv6 calls to `sleep` are always wrapped in a `while` loop that re-tests the condition after `sleep` returns. Some calls to `sleep` also test `p->killed` in the loop,

> and abandon the current activity if it is set. This is only done when such abandonment would be correct. For example, the pipe read and write code (6686) returns if the killed flag is set; eventually the code will return back to trap, which will again check `p->killed` and exit.

当 victim 进程阻塞在 `sleep` 中时，`kkill` 通过调用 `wakeup` 将导致 victim 进程从 `sleep` 函数返回。但这么做存在潜在的危险，因为等待的条件可能并不成立。但所幸的是，由于 xv6 总是用一个 `while` 循环包住了对 `sleep` 的调用，该循环会在 `sleep` 返回后重新测试条件。一些对 `sleep` 的调用还会在循环中测试 `p->killed` 这个条件，如果设置了该标志，则放弃当前活动。只有确保这种放弃是正确的才会执行此操作。例如，如果设置了 killed 标志，管道的读写代码 (6686) 就会返回；最终代码将返回到 trap，并再次检查 `p->killed`，如果条件满足则调用 `exit` 结束该 victim 进程。

> Some xv6 `sleep` loops do not check `p->killed` because the code is in the middle of a multi-step system call that should be atomicc (i.e., would be incorrect if abandoned midway through). The virtio driver (7688) is an example: it does not check `p->killed` because a disk operation may be one of a set of writes that are all needed in order for the file system to be left in a correct state. A process that is killed while waiting for disk I/O won’t exit until it completes the current system call and `usertrap` sees the killed flag.

xv6 中有些调用 `sleep` 循环并没有检查 `p->killed`，因为这些代码正处于一个多步骤系统调用过程的中间阶段，而这些调用是不能被打断的（即如果中途放弃，那就会产生问题）。virtio 驱动程序 (7688) 就是一个例子：它不会检查 `p->killed`，因为磁盘操作作为一系列写操作的组成部分，如果中间（被 `kill`）打断将有可能使得文件系统处于一种不正确的状态。所以如果当一个进程在等待磁盘 I/O 时被杀死，则我们会一直等到该进程完成当前系统调用后，`usertrap` 会再次检查 killed 标志，如果被设置则调用 `exit` 结束该 victim 进程。

9.5 进程锁（Process Locking）

> The lock associated with each process (`p->lock`) is the most complex lock in xv6. A simple way to think about `p->lock` is that it must be held while reading or writing any of the following `struct proc` fields: `p->state`, `p->chan`, `p->killed`, `p->xstate`, and `p->pid`. These fields can be used by other processes, or by scheduler threads on other CPUs, so it’s natural that they must be protected by a lock.

与每个进程关联的锁（`p->lock`）是 xv6 中最复杂的锁。一种对 `p->lock` 的简单理解是，在对 `struct proc` 读取或写入以下任何字段时必须持有该锁，它们包括：`p->state`、`p->chan`、`p->killed`、`p->xstate` 和 `p->pid`。这些字段可能被其他进程或其他 CPU 上的 scheduler 线程访问，因此它们必须受到锁的保护也就不足为奇了。

> However, most uses of `p->lock` are protecting higher-level invariants of xv6’s process data structures and algorithms. Here’s the full set of things that `p->lock` does:

然而，`p->lock` 的大多数用途是用于保护 xv6 中处于更高层次的进程数据结构和处理逻辑的 “不变性（invariant）”。以下是对 `p->lock` 功能的总结：

> - Along with `p->state`, it prevents races in allocating `proc[]` slots for new processes.
> - It conceals a process from view while it is being created or destroyed.

> - It prevents a parent's `wait` from collecting a process that has set its state to `ZOMBIE` but has not yet yielded the CPU.
> - It prevents another CPU's scheduler from deciding to run a yielding process after it sets its state to `RUNNABLE` but before it finishes `swtch`.
> - It ensures that only one CPU's scheduler decides to run a `RUNNABLE` processes.
> - It prevents a timer interrupt from causing a process to yield while it is in `swtch`.
> - Along with the condition lock, it helps prevent `wakeup` from overlooking a process that is calling `sleep` but has not finished yielding the CPU.
> - It prevents the victim process of `kill` from exiting and perhaps being re-allocated between `kkill`'s check of `p->pid` and setting `p->killed`.
> - It makes `kkill`'s check and write of `p->state` atomic.

- 和 `p->state` 一起，避免从 `proc[]` 数组中为新进程分配对象时发生竞争。
- 它在进程创建或销毁时将其 “隐藏（conceal）” 起来（译者注：这里的隐藏指的是在创建和销毁进程的过程中涉及修改进程的 `state` 等字段时通过加锁避免其他进程的并发访问）。
- 子进程退出时，在将其状态设置为 `ZOMBIE` 和完全让出 CPU 之间有一段窗口期，进程锁用于避免父进程在调用 `wait` 过程中在这段窗口期中操作子进程。
- 一个进程在让出（yield）CPU 时，在它将自己状态设置为 `RUNNABLE` 和调用 `swtch` 完成切换之间有一段窗口期，进程锁用于防止另一个 CPU 上的 scheduler 在这段窗口期之间操作（恢复运行）该进程。
- 它确保只有一个 CPU 上的 scheduler 可以决定运行一个 `RUNNABLE` 的进程。
- 它可以避免因为定时器中断导致一个正在执行 `swtch` 进程让出 CPU。
- 与条件锁一起使用，它有助于避免一个消费者进程执行 `sleep` 但尚未完成交出 CPU 时，因为并发 `wakeup` 而导致的 “丢失唤醒”。
- 在 `kkill` 中，检查 `p->pid == pid` 和设置 `p->killed` 之间存在一段窗口期，进程锁可以防止在这两步操作之间因为 victim 主动退出导致进程结构体被重新分配给一个新的进程，而这个新进程因为 `kill` 继续执行设置 `p->killed` 而被误杀。
- 它确保 `kkill` 中对 `p->state` 的检查和改写操作的原子性。

> The `p->parent` field is protected by the global lock `wait_lock rather than by` p->lock`. Only a process's parent modifies` p->parent`, though the field is read both by the process itself and by other processes searching for their children. The purpose of` wait_lock `is to act as the condition lock when` wait `sleeps waiting for any child to exit. An exiting child holds either` wait_lock `or` p->lock `until after it has set its state to` ZOMBIE`, woken up its parent, and yielded the CPU.` wait_lock `also serializes concurrent` exit`s by a parent and child, so that the` init `process (which inherits the child) is guaranteed to be woken up from its` wait`. wait_lock` is a global lock rather than a per-process lock in each parent, because, until a process acquires it, it cannot know who its parent is.

`p->parent` 字段受全局锁 `wait_lock` 保护，而不是 `p->lock`。只有进程的父进程才会修改 `p->parent`，但该字段既可以被进程本身读取，也可以被其他搜索其子进程的进程读取。`wait_lock` 的作用是在一个进程调用 `wait` 等待子进程退出时充当条件锁。正在退出的子进程会持有 `wait_lock` 或 `p->lock`，直到它将其自身状态设置为 `ZOMBIE`、唤醒其父进程并交出 CPU。`wait_lock` 也串行化了父进程和子进程对 `exit` 的并发调用，以便保证在 `wait` 中等待的 `init` 能被正确地唤醒（当子进程原来的父进程提前退出后会将子进程托付给 `init`，于是 `init` 成为子

进程的 “继父”）。`wait_lock` 被定义成一把全局锁，而不是作为每个父进程所自己拥有的锁，因为在进程获取它之前，它无法知道它的父进程是谁。

9.6 现实世界（Real world）

> `sleep` and `wakeup` are a simple and effective synchronization method, but there are many others; semaphores [5] are an example. The first challenge in all of them is to avoid the “lost wakeups” problem we saw at the beginning of the chapter. The original Unix kernel’s `sleep` simply disabled interrupts, which sufficed because Unix ran on a single-CPU system. Because xv6 runs on multiprocessors, it adds an explicit lock to `sleep`. FreeBSD’s `msleep` takes the same approach. Plan 9’s `sleep` uses a callback function that runs with the scheduling lock held just before going to sleep; the function serves as a last-minute check of the sleep condition, to avoid lost wakeups. The Linux kernel’s `sleep` uses an explicit process queue, called a wait queue, instead of a wait channel; the queue has its own internal lock.

`sleep` 和 `wakeup` 是一种简单有效的同步方法，当然还有许多其他方法；“信号量（semaphore）” [5] 是一个例子。所有这些方法中的第一个挑战是避免我们在本章开头看到的“丢失唤醒（lost wakeups）” 问题。早期 Unix 内核的 `sleep` 实现只是禁用中断就足够了，因为当时 Unix 只在单处理器系统上运行。但 xv6 需要在多处理器系统上运行，所以它在 `sleep` 的实现中添加了显式的上锁操作（译者注：即 `acquire(&p->lock)`）。FreeBSD 的 `msleep` 采用相同的方法。Plan 9 的 `sleep` 使用一个回调函数，该函数在进入睡眠状态之前持有调度锁并运行；该函数用作睡眠条件的最后一刻检查，以避免丢失唤醒。Linux 内核的 `sleep` 使用一个显式进程队列，称为 “等待队列（wait queue）”，而不是等待通道；该队列有自己的内部锁。

> Scanning the entire set of processes in `wakeup` is inefficient. A better solution is to replace the `chan` in both `sleep` and `wakeup` with a data structure that holds a list of processes sleeping on that structure, such as Linux’s wait queue. Plan 9’s `sleep` and `wakeup` call that structure a rendezvous point. Many thread libraries refer to the same structure as a condition variable; in that context, the operations `sleep` and `wakeup` are called `wait` and `signal`. All of these mechanisms share the same flavor: the sleep condition is protected by some kind of lock dropped atomically during sleep.

在 `wakeup` 中扫描所有进程是效率低下的。更好的解决方案是将 `sleep` 和 `wakeup` 中的 `chan` 替换为一个数据结构并将在该结构上休眠的进程列表维护在这个结构中，譬如 Linux 中的 “等待队列（wait queue）”。Plan 9 的 `sleep` 和 `wakeup` 将该结构称为 “汇合点（rendezvous point）”。许多线程库将类似的结构称为 “条件变量（condition variable）”；此时，`sleep` 和 `wakeup` 操作分别称为 `wait` 和 `signal`。所有这些方案的思路都类似：在睡眠期间维持睡眠的条件都要受到某种原子释放的锁的保护。

> xv6’s `wakeup` wakes up all processes that are waiting on a particular wait channel. If there are more than one of them, they will all try to acquire the condition lock and re-check the condition; in many cases only one will be able to do anything useful (e.g., read all the data waiting in a pipe). The rest will find the condition is no longer true and go back to sleep; it was a waste of CPU time to wake them up. As a result, most condition variable designs provide two primitives: `signal`, which wakes up one of the processes waiting for the condition variable, and `broadcast`, which wakes up all of them.

xv6 的 `wakeup` 函数会唤醒所有在特定等待通道上等待的进程。如果存在多个进程，它们都会尝试获取条件锁并重新检查条件；很多情况下，只有一个进程能够执行有用的操作（例如，读取管道中所有等待的数据）。其余进程会发现条件不再成立，然后重新进入睡眠状态；唤醒它们纯粹是浪费 CPU 时间。因此，大多数条件变量设计都提供了两种原语：`signal`，用于唤醒等待条件变量的某个进程；`broadcast`，用于唤醒所有进程。

> Forcibly killing processes poses some problems. For example, a killed process may be deep inside the kernel sleeping, and unwinding its stack requires care, since each function on the call stack may need to do some clean-up. Some languages help out by providing an exception mechanism, but not C. Furthermore, there are other events that can cause a sleeping process to be woken up, even though the event it is waiting for has not happened yet. For example, when a Unix process is sleeping, another process may send a `signal` to it. In this case, the process will return from the interrupted system call with the value -1 and with the error code set to EINTR. The application can check for these values and decide what to do. Xv6 doesn’t support signals and this complexity doesn’t arise.

强制终止进程会带来一些问题。例如，一个被杀死的进程的休眠点可能发生在内核的多次函数嵌套调用中，而从睡眠的地方逐层退出返回需要非常小心，因为调用栈上的每个函数都可能需要进行一些清理工作。有些语言提供了异常机制来解决这个问题，但 C 语言没有。此外，唤醒休眠进程的可能不是它正在等待的事件，而是其他事件。例如，当一个 Unix 进程正在休眠时，另一个进程可能会向它发送一个 “信号（signal）”。在这种情况下，该进程将从中断的系统调用返回，返回值为 `-1`，错误代码设置为 `EINTR`。应用程序可以检查这些值并决定如何处理。xv6 不支持信号，因此不考虑这种复杂性。

> Xv6’s support for `kill` is not entirely satisfactory: there are sleep loops which probably should check for `p->killed`. A related problem is that, even for `sleep` loops that check `p->killed`, there is a race between `sleep` and `kill`; the latter may set `p->killed` and try to wake up the victim just after the victim’s loop checks `p->killed` but before it calls `sleep`. If this problem occurs, the victim won’t notice the `p->killed` until the condition it is waiting for occurs. This may be quite a bit later or even never (e.g., if the victim is waiting for input from the console, but the user doesn’t type any input).

xv6 对 `kill` 的支持并不完全令人满意：代码中有些包裹 `sleep` 的循环会检查 `p->killed`。与之相关的是，即使存在这些检查 `p->killed` 的循环，`sleep` 和 `kill` 之间也依然存在竞争；在 victim 的循环内部，在检查 `p->killed` 之后和调用 `sleep` 之前存在一个窗口期，假设 `kill` 恰好在这个窗口期中设置了 `p->killed` 并尝试唤醒 victim 进程（译者注：`kill` 并不会实际唤醒 victim，它只是设置 `p->state = RUNNABLE`）。这种情况一旦出现，victim 将无法立即观察到 `p->killed` 被设置这个条件成立（因为它已经检查过一次），victim 只会进入睡眠等待被唤醒。这可能要等相当长一段时间，甚至永远不会发生（例如，如果 victim 正在等待的是来自控制台的输入，但用户一直没有输入任何内容）。

9.7 练习（Exercises）

> 1. Implement counting semaphores in xv6. Choose a few of xv6’s uses of sleep and wakeup and replace them with semaphores. Judge the result.

1. 在 xv6 中实现计数信号量。选择 xv6 中一些使用 `sleep` 和 `wakeup` 的地方，用信号量替换它们。看看结果。

> 1. Can you implement a variant of `sleep()` that takes just one argument, the channel, and doesn’t need a lock argument?

1. 你能修改一下 `sleep()`？让它只接受一个参数，即通道，而不需要那个锁参数。

> 1. Fix the race mentioned above between `kill` and `sleep`, so that a `kill` that occurs after the victim’s sleep loop checks `p->killed` but before it calls `sleep` results in the victim abandoning the current system call.

1. 修复上面提到的 `kill` 和 `sleep` 之间的竞争，以便在 victim 的睡眠循环中检查 `p->killed` 之后但在调用 `sleep` 之前发生的 `kill` 会导致 victim 放弃当前的系统调用。

> 1. Design a plan so that every sleep loop checks `p->killed` so that, for example, a process that is in the virtio driver can return quickly from the while loop if it is killed by another process.

1. 设计一个计划，以便每个睡眠循环检查 `p->killed`，这样，如果 virtio 驱动程序中的进程被另一个进程杀死，它可以从 while 循环快速返回。

第 10 章 文件系统（Chapter 10 File system）

> The purpose of a file system is to organize and store data. File systems typically support sharing of data among users and applications, as well as persistence so that data is still available after a reboot.

文件系统（file system）的目的是组织和存储数据。文件系统通常支持在多个用户或者多个应用程序之间共享数据，以及支持 持久化（persistence），即在系统重启后数据仍然存在。

> The xv6 file system provides Unix-like files, directories, and pathnames (see Chapter 1), and stores its data on a virtio disk for persistence. The file system addresses several challenges:
>
> - The file system needs on-disk data structures to represent the tree of named directories and files, to record the identities of the blocks that hold each file’ s content, and to record which areas of the disk are free.
> - The file system must support crash recovery. That is, if a crash (e.g., power failure) occurs, the file system must still work correctly after a restart. The risk is that a crash might interrupt a sequence of updates and leave inconsistent on-disk data structures (e.g., a block that is both used in a file and marked free).
> - Different processes may operate on the file system at the same time, so the file-system code must coordinate to maintain invariants.
> - Accessing a disk is orders of magnitude slower than accessing memory, so the file system must maintain an in-memory cache of popular blocks.

xv6 的文件系统支持 “类似 Unix（Unix-like）” 的 “文件（file）”、“目录（directory）” 和 “路径名（pathname）” 等概念（参考第 1 章），并将其数据存储在 virtio 磁盘（译者注：virtio 磁盘是 QEMU 模拟的一种虚拟化磁盘）上实现 “持久化（persistence）”。文件系统解决了以下几个难题：

- 文件系统需要将存储在磁盘上的数据以目录和文件的形式组织成树状结构，每个目录和文件都有自己的名字标识，同时这个树状的数据结构本身也需要存储在磁盘上。文件系统还需要知道磁盘上哪些 “块（block）” 保存有文件的内容（通过记录这些块的 “标识符（identity）”），以及记录磁盘上哪些区域是空闲的（没有存放数据）。
- 文件系统必须支持 崩溃恢复（crash recovery）。也就是说，如果发生崩溃（例如，电源故障），文件系统必须在重新启动后仍能正常工作。而实现这个功能的挑战在于突然发生的崩溃可能会打断当前对磁盘的更新操作序列，这会导致磁盘上的实际数据状态和记录数据状态的数据结构内容之间不一致（例如，一个块被某个文件使用但同时却被标记为空闲）。
- 多个进程可能同时访问文件系统，因此文件系统代码必须协调它们的操作以维持一些 “不变性（invariants）”。
- 访问磁盘的速度比访问内存慢几个数量级，因此文件系统必须利用内存缓存一些经常访问的块的内容（以提高访问速度）。

> The rest of this chapter explains how xv6 addresses these challenges.

本章的其余部分将解释 xv6 如何应对这些挑战。

10.1 概述（Overview）

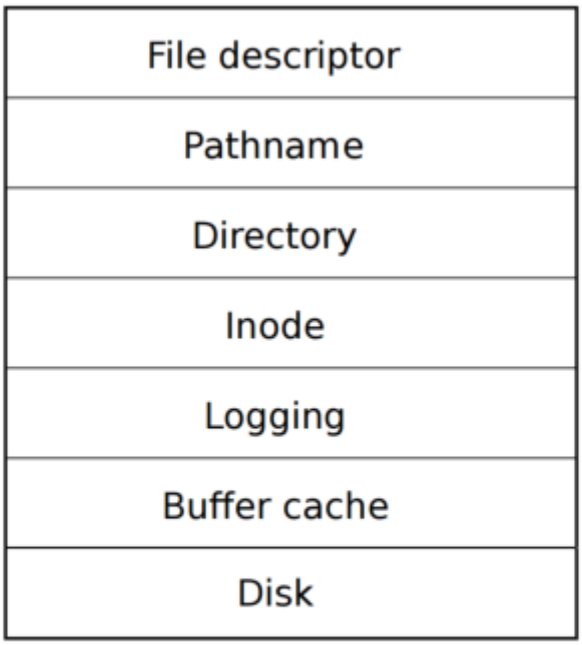

Figure 10.1: Layers of the xv6 file system.

> The xv6 file system implementation is organized in seven layers, shown in Figure 10.1. The disk layer reads and writes blocks on an virtio hard drive. The buffer cache layer caches disk blocks and synchronizes access to them, making sure that only one kernel process at a time can modify the data stored in any particular block. The logging layer allows higher layers to wrap updates to several blocks in a transaction, and ensures that the blocks are updated atomically in the face of crashes (i.e., all of them are updated or none). The inode layer provides individual files, each represented as an inode with a unique i-number and some blocks holding the file’ s data. The directory layer implements each directory as a special kind of inode whose content is a sequence of directory entries, each of which contains a file’ s name and i-number. The pathname layer provides hierarchical path names like `/usr/rtm/xv6/fs.c`, and resolves them with recursive lookup. The file descriptor layer abstracts many Unix resources (e.g., pipes, devices, files, etc.) using the file system interface, simplifying the lives of application programmers.

（译者注：原文称呼磁盘上的块为 block，称呼对应的内存中的副本为 buffer。为方便区分，本译文中将对这两个概念直接使用英文原文，不再翻译为中文。）

xv6 的文件系统实现分为七层，如图 10.1 所示。“磁盘（Disk）” 层读取和写入 virtio 硬盘上的块。“缓存（Buffer cache）” 层缓存 block 并同步对它们的访问，确保每次只有一个内核进程可以修改存储在某个特定 block 中的数据。“日志（Logging）” 层允许上层将对涉及多个 block 的更新操作封装为 事务（transaction），并在遇到崩溃时能够确保更新操作的 “原子性（atomic）”（即，对所有的这些块，要么都更新，要么都不更新）。“索引结点（Inode）” 层向上层提供单个文件的概念，每个文件表示为一个 “索引节点（inode）”，每个 inode 具有唯一

的 “索引号（i-number）” 并记录了哪些 block 保存了该文件的内容。“目录（Directory）” 层将每个目录实现为一种特殊的 inode，其内容是一组 “目录项（directory entry）”，每个目录项包含一个文件名和它的索引号。“路径名（Pathname）” 层提供了分层的路径名，例如 `/usr/rtm/xv6/fs.c`，并通过递归查找来解析它们。“文件描述符 (File Descriptor)” 层提供文件系统接口抽象了众多 Unix 资源（例如，管道、设备、文件等），简化了面向应用的编程人员的工作。

> Disk hardware traditionally presents the data on the disk as a numbered sequence of 512-byte blocks (also called sectors): sector 0 is the first 512 bytes, sector 1 is the next, and so on. The block size that an operating system uses for its file system maybe different than the sector size that a disk uses, but typically the block size is a multiple of the sector size. Xv6 holds copies of blocks that it has read into memory in objects of type `struct buf` (3900). The data stored in this structure is sometimes out of sync with the disk: it might have not yet been read in from disk (the disk is working on it but hasn’t returned the sector’s content yet), or it might have been updated by software but not yet written to the disk.

传统上磁盘硬件将磁盘空间划分为 512 字节大小的块（也称为 扇区（sector）），并编号排序：扇区 0 是第一组 512 个字节，扇区 1 是下一组，依此类推。操作系统里的文件系统所定义的的 “块（block）” 的大小可能与磁盘定义的扇区大小不同，但通常块的大小是扇区大小的整数倍。xv6 定义了结构体类型 `struct buf`（3900）来表示读入内存的 block 的副本。内存中此结构体中的数据可能会与磁盘上的内容不同步：譬如数据尚未从磁盘读入（磁盘正在处理但尚未返回扇区的内容），或者软件更新了内存中的数据但还没来得及写回磁盘。

> The file system must have a plan for where it stores inodes and content blocks on the disk. To do so, xv6 divides the disk into several sections, as Figure 10.2 shows. The file system does not use block 0 (it holds the boot sector). Block 1 is called the superblock; it contains metadata about the file system (the file system size in blocks, the number of data blocks, the number of inodes, and the number of blocks in the log). Blocks starting at 2 hold the log. After the log are the inodes, with multiple inodes per block. After those come bitmap blocks tracking which data blocks are in use. The remaining blocks are data blocks; each is either marked free in the bitmap block, or holds content for a file or directory. The superblock is filled in by a separate program, called `mkfs`, which builds an initial file system.

文件系统必须对在磁盘上如何存储 inode 和数据内容有所规划。为此，xv6 将磁盘划分为几个部分，如图 10.2 所示。文件系统不使用编号为 0 的 block（它用于保存 “引导扇区（boot sector）”）。编号为 1 的 block 称为 超级块（superblock）：它包含有关文件系统的 “元数据（metadata）”（譬如文件系统的大小（以 block 为单位）、数据 block 的个数、inode 的个数以及 “日志（log）” 所占用的 block 的个数）。从编号为 2 的 block 开始保存 log。log 存放区的后面是存放 inode 的区域，由多个 block 组成，每个 block 中记录有多个 inode。再往后的 block 用于存放 “位图（bitmap）”，bitmap 用于记录哪些存放数据的 block 正在被使用。其余的 block 存放数据，每个数据 block 要么在 bitmap block 中标记为空闲，要么保存着文件或目录的内容。superblock 中的内容由一个名为 `mkfs` 的程序进行填充，该程序构建了（磁盘上的）初始的文件系统（译者注：`mkfs` 所作的工作类似于我们常说的磁盘格式化）。

> The rest of this chapter discusses each layer, starting with the buffer cache. Look out for situations where well-chosen abstractions at lower layers ease the design of higher ones.

本章的其余部分将从 “缓存（buffer cache）” 层开始逐个讨论每一层。注意那些在较低层上精心实现的抽象接口是如何简化了较高层的设计。

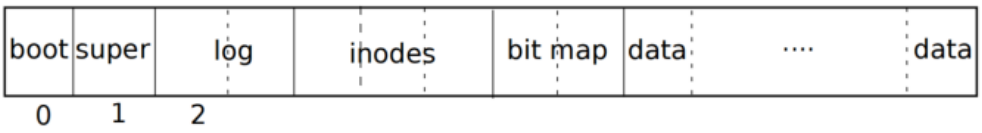

Figure 10.2: Structure of the xv6 file system.

10.2 缓存层（Buffer cache layer）

> The buffer cache has two jobs: (1) synchronize access to disk blocks to ensure that only one copy of a block is in memory and that only one kernel thread at a time uses that copy; (2) cache popular blocks so that they don’ t need to be re-read from the slow disk. The code is in `bio.c`.

Buffer cache 有两个作用：（1）同步对磁盘上 block 的访问，以确保内存中只有一个 block 的副本，并且当前只有一个内核线程访问该副本；（2）缓存常用的 block，这样就不需要每次都从慢速的磁盘重新读取数据。相关代码见 `bio.c`。

> The main interface exported by the buffer cache consists of `bread` and `bwrite`; the former obtains a `buf` containing a copy of a block which can be read or modified in memory, and the latter writes a modified buffer to the appropriate block on the disk. A kernel thread must release a buffer by calling `brelse` when it is done with it. The buffer cache uses a per-buffer sleep-lock to ensure that only one thread at a time uses each buffer (and thus each disk block); `bread` returns a locked buffer, and `brelse` releases the lock.

Buffer cache 的主要接口包括 `bread` 和 `bwrite`；`bread` 会返回一个 `struct buf`（的指针），该结构体在内存中缓存了一个来自 block 的副本供进一步的读写，`bwrite` 将修改后的 buffer 写入磁盘上对应的 block。内核线程在使用完 buffer 后必须通过调用 `brelse` 来释放它。Buffer cache 对每个 buffer 维护一个睡眠锁（per-buffer sleep-lock）来确保每次只有一个线程使用这个 buffer（这也意味着这个 buffer 对应的 block 也受到该睡眠锁的保护）；`bread` 返回一个锁定的 buffer，而 `brelse` 负责释放该锁。

> Let’ s return to the buffer cache. The buffer cache has a fixed number of buffers to hold disk blocks, which means that if the file system asks for a block that is not already in the cache, the buffer cache must recycle a buffer currently holding some other block. The buffer cache recycles the least recently used buffer for the new block. The assumption is that the least recently used buffer is the one least likely to be used again soon.

让我们回到 Buffer cache。Buffer cache 管理的 buffer（译者注：即 `struct buf`）数量是有限的，这意味着对于文件系统来说，如果当前访问的 block 尚未缓存（而管理的 buffer 恰好用完了），则 Buffer cache 将回收一些其他使用中的 buffer。Buffer cache 将回收 “最近最少使用（Least Recently Used，缩写即 LRU）” 的 buffer 来保存新的 block。这么做的前提是假设这个所谓的 “最近最少使用” 的 buffer 在概率上也是最不可能马上会被再次访问到的 buffer。

10.3 代码讲解：缓存（Code: Buffer cache）

> The buffer cache is a doubly-linked list of buffers. The function `binit`, called by `main` (1176), initializes the list with the `NBUF` buffers in the static array `buf` (4792-2301). All other access to the buffer cache refer to the linked list via `bcache.head`, not the `buf` array.

buffer cache 被实现为一个大小为 `NBUF` 的静态数组 `buf`（译者注：即 `bcache.buf`），每个成员是一个 `struct buf` 的结构体，同时这些结构体成员又通过一个双向链表串联起来。`main` 通过调用函数 `binit` (1176) 初始化该链表 (4792-2301)。所有其他对 buffer cache 的访问都通过 `bcache.head` 引用该链表，而不是直接访问 `buf` 数组。

> A buffer has two state fields associated with it. The field `valid` indicates that the buffer contains a copy of the block. The field `disk` indicates that the buffer content has been handed to the disk, which may change the buffer (e.g., write data from the disk into `data`).

每个 buffer 有两个存放状态的成员字段。字段 `valid` 用于表示该 buffer 中是否包含了 block 的副本。字段 `disk` 用于表示是否已经将 buffer 提交给磁盘，注意这里的提交可能会导致 buffer 中的内容发生改变（即，将磁盘上的数据写入 `data`）（译者注：相关内容需要看 `kernel/virtio_disk.c` 中驱动和 virtio 磁盘之间的读写实现逻辑，驱动将读写请求提交给硬件后设置 `disk` 为 1 并等待磁盘处理结束，磁盘完成处理后通过中断通知驱动，驱动再将 `disk` 恢复为 0）。

> `bread` (4352) calls `bget` to get a buffer for the given sector (4356). If the buffer needs to be read from disk, `bread` calls `virtio_disk_rw` to do that before returning the buffer.

`bread` (4352) 调用 `bget` 获取指定扇区对应的 buffer (4356)。如果需要从磁盘读取数据到 buffer，`bread` 会调用 `virtio_disk_rw` 执行此操作，然后再返回 buffer。

> `bget` (4308) scans the buffer list for a buffer with the given device and sector numbers (4314-4322). If there is such a buffer, `bget` acquires the sleep-lock for the buffer. `bget` then returns the locked buffer.

`bget` (4308) 扫描 buffer 链表，根据函数参数给定的设备号（`dev`）和 block 号（`blockno`）查找对应的 buffer (4314-4322)。如果 buffer 已经存在，`bget` 会获取该 buffer 的睡眠锁。然后 `bget` 返回锁定的 buffer。

> If there is no cached buffer for the given sector, `bget` must make one, possibly reusing a buffer that held a different sector. It scans the buffer list a second time, looking for a buffer that is not in use (`b->refcnt = 0`); any such buffer can be used. `bget` edits the buffer metadata to record the new device and sector number and acquires its sleep-lock. Note that the assignment `b->valid = 0` ensures that `bread` will read the block data from disk rather than incorrectly using the buffer’s previous contents.

如果对于给定的扇区（即 block）还没有建立对应的 buffer，则 `bget` 必须创建一个，并可能重用那些已经保存了其他扇区内容的 buffer。它会再次扫描 buffer 链表，查找未使用的 buffer（`b->refcnt = 0`）；只要满足这个条件的 buffer 都可以直接拿来使用。`bget` 会编辑 buffer 的 “元数据（metadata）” 以记录新的设备号和扇区号，并获取其睡眠锁。请注意，`bget` 中的 `b->valid = 0` 操作确保该函数返回后 `bread` 将从磁盘重新读取 block 数据，而不是错误地使用 buffer 中先前的内容。

> It is important that there is at most one cached buffer per disk sector, to ensure that readers see writes, and because the file system uses locks on buffers for synchronization. `bget` ensures this invariant by holding the `bache.lock` `bcache.lock` continuously from the first loop’s check of whether the block is cached through the second loop’s declaration that the block is now cached (by setting `dev`, `blockno`, and `refcnt`). This causes the check for a block’s presence and (if not present) the designation of a buffer to hold the block to be atomic.

每个磁盘扇区（译者注：即 block）最多只能对应有一个 buffer，这很重要，只有这样才能确保读取和写入内容的一致性，当然这也是因为文件系统使用了 buffer 上的锁实现了同步。`bget` 中有两个 `for` 循环，第一个循环检查 block 是否已缓存，（如果检查失败）第二个循环选择一个新的 buffer 并标记 block 已缓存（通过设置 `dev`、`blockno` 和 `refcnt`），为了保证这两个循环操作的原子性（即 invariant），进入 `bget` 后首先要做的就是尝试持有 `bcache.lock`。

> It is safe for `bget` to acquire the buffer’s sleep-lock outside of the `bcache.lock` critical section, since the non-zero `b->refcnt` prevents the buffer from being re-used for a different disk block. The sleep-lock protects reads and writes of the block’s buffered content, while the `bcache.lock` protects information about which blocks are cached.

`bget` 中会在 `bcache.lock` 保护的临界区之外尝试获取 buffer 的睡眠锁（译者注：即 `bget` 代码中的两处 `acquiresleep(&b->lock);`），这么做并没有问题，因为非零的 `b->refcnt` 会阻止该 buffer 被其他磁盘 block 重复使用。睡眠锁用于保护对 buffer 中缓存的 block 内容的读写操作，而 `bcache.lock` 用于保护将 block 和 buffer 关联起来的操作。

> If all the buffers are busy, then too many processes are simultaneously executing file system calls; `bget` panics. A more graceful response might be to sleep until a buffer became free, though there would then be a possibility of deadlock.

如果所有 buffer 都被使用了，则说明此时有太多进程在执行文件系统相关的调用；目前 `bget` 对这种情况的处理是直接让系统 “崩溃（panic）”。当然更优雅的处理方式是将进程休眠直到出现空闲的 buffer，但这么做可能会出现死锁。

> Once `bread` has read the disk (if needed) and returned the buffer to its caller, the caller has exclusive use of the buffer and can read or write the data bytes. If the caller does modify the buffer, it must call `bwrite` to write the changed data to disk before releasing the buffer. `bwrite` (4366) calls `virtio_disk_rw` to talk to the disk hardware.

一旦 `bread` 读取了磁盘（如果需要）并将 buffer 返回给其调用者，调用者就拥有了 buffer 的独占使用权，可以读取或写入数据字节。如果调用者确实修改了 buffer，则必须调用 `bwrite` 将更改后的数据写入磁盘，然后再释放 buffer。`bwrite` (4366) 调用 `virtio_disk_rw` 完成对实际磁盘硬件的读写。

> When the caller is done with a buffer, it must call `brelse` to release it. (The name `brelse`, a shortening of b-release, is cryptic but worth learning: it originated in Unix and is used in BSD, Linux, and Solaris too.) `brelse` (4376) releases the sleep-lock and moves the buffer to the front of the linked list (4387-4392). Moving the buffer causes the list to be ordered by how recently the buffers were used (meaning released): the first buffer in the list is the most recently used, and the last is the least recently used. The two loops in `bget` take advantage of this: the scan for an existing buffer must process the entire list in the worst case, but

> checking the most recently used buffers first (starting at `bcache.head and following next pointers) will reduce scan time when there is good locality of reference. The scan to pick a buffer to reuse picks the least recently used buffer by scanning backward (following prev` pointers).

当调用者使用完 buffer后，必须调用 `brelse` 来释放它。（`brelse` 这个名字是 b-release 的缩写，这个函数的名字理解起来有点晦涩，但仍然值得介绍一下：它起源于 Unix，同样也用于 BSD、Linux 和 Solaris 中。） `brelse` (4376) 释放睡眠锁并将 buffer 移动到链表的开头 (4387-4392) 。移动 buffer 会导致链表按 buffer 的使用频率（释放的最近程度）排序：列表中第一个 buffer 是最近使用的，最后一个是最近最少使用的。`bget` 中的两个循环利用了这一点：在最坏的情况下，对现有缓冲区的扫描必须处理整个列表，但首先检查最近使用的 buffer（从 `bcache.head` 开始，沿着 `next` 指针），在引用局部性良好的情况下将减少扫描时间。选择要重用的缓冲区时，通过自后向前扫描（跟随 `prev` 指针）来选择最近最少使用的 buffer。

10.4 日志层（Logging layer）

> One of the most interesting problems in file system design is crash recovery. The problem arises because many file-system operations involve multiple writes to the disk, and a crash after a subset of the writes may leave the on-disk file system in an inconsistent state. For example, suppose a crash occurs during file truncation (setting the length of a file to zero and freeing its content blocks). Depending on the order of the disk writes, the crash may either leave an inode with a reference to a content block that is marked free, or it may leave an allocated but unreferenced content block.

文件系统设计中最有趣的问题之一是有关 “崩溃恢复（crash recovery）” 。这个问题的出现是因为许多文件系统操作涉及对磁盘的多次写入，而一旦写入操作中途发生了崩溃，则可能导致磁盘上的文件系统处于不一致的状态。例如，假设在文件 “截断（truncation）” （将文件长度设置为零并释放保存文件内容的 block）期间发生崩溃。根据磁盘写入的顺序不同，崩溃发生后，磁盘中的 inode 可能引用了一个 block，但这个 block 在 bitmap 中却是标记为 “空闲（free）” 的，或者反过来，一个 block 在 bitmap 中标记为 “已分配（allocated）” ，但是并没有 inode 引用这个 block。

> The latter is relatively benign, but an inode that refers to a freed block is likely to cause serious problems after a reboot. After reboot, the kernel might allocate that block to another file, and now we have two different files pointing unintentionally to the same block. If xv6 supported multiple users, this situation could be a security problem, since the old file’ s owner would be able to read and write blocks in the new file, owned by a different user.

后者相对来说还不会导致严重的问题，但对于前者，即一个 inode 引用了已被释放的 block 这种情况，在重新启动后可能会导致严重问题。重新启动后，内核可能会将该 block 分配给另一个文件，现在我们有两个不同的文件无意中指向同一个 block。如果 xv6 支持多个用户，这种情况可能会导致一个安全问题，因为旧文件的所有者将能够访问新文件中的 block，而新文件的所有者有可能是另一个用户。

> Xv6 solves the problem of crashes during file-system operations with a simple form of logging. An xv6 system call does not directly write the on-disk file system data structures.

> Instead, it places a description of all the disk writes it wishes to make in a log on the disk. Once the system call has logged all of its writes, it writes a special commit record to the disk indicating that the log contains a complete operation. At that point the system call copies the writes to the on-disk file system data structures. After those writes have completed, the system call erases the log on disk.

xv6 通过实现一套简化版本的日志机制解决了文件系统操作期间的崩溃问题。（引入日志后）每一个 xv6 的系统调用不会直接改写磁盘上的文件系统数据结构。相反，它会将其希望写入磁盘的内容以日志的形式先记录在磁盘上的 log 区域。完成日志记录后系统调用会向磁盘写入一条特殊的 提交（commit） 记录，标识该日志包含了一组完整的操作。直到这个时候系统调才会将写入日志中的内容复制到磁盘上的文件系统数据结构中。完成复制后，系统调用将擦除磁盘上的 log 区域。

> If the system should crash and reboot, the file-system code recovers from the crash as follows, before running any processes. If the log is marked as containing a complete operation, then the recovery code copies the writes to where they belong in the on-disk file system. If the log is not marked as containing a complete operation, the recovery code ignores the log. The recovery code finishes by erasing the log.

如果系统崩溃并重新启动，则在运行任何进程之前，文件系统代码将按如下方式从崩溃中恢复。如果日志标记为包含完整操作，则负责恢复的代码会将日志中记录的内容复制到磁盘文件系统中它们应该所在的位置。如果日志没有标记为包含完整操作，则恢复代码将忽略该日志。恢复代码最后会擦除日志。

> Why does xv6’s log solve the problem of crashes during file system operations? If the crash occurs before the operation commits, then the log on disk will not be marked as complete, the recovery code will ignore it, and the state of the disk will be as if the operation had not even started. If the crash occurs after the operation commits, then recovery will replay all of the operation’s writes, perhaps repeating them if the operation had started to write them to the on-disk data structure. In either case, the log makes operations atomic with respect to crashes: after recovery, either all of the operation’s writes appear on the disk, or none of them appear.

为什么 xv6 的日志机制能够解决文件系统操作期间的崩溃问题呢？如果崩溃发生在操作提交之前，那么磁盘上的日志不会被标记为已完成，负责恢复的代码将忽略它，磁盘的状态将如同操作尚未启动一样。如果崩溃发生在操作提交之后，则负责恢复的代码将把日志中的所有写操作再重新执行一遍，如果崩溃之前操作已开始将它们写入磁盘数据结构，则这些操作相当于被执行了两遍。在任何一种情况下，日志都会确保操作在崩溃发生情况下的原子性：也就是说，一旦崩溃恢复后，操作中的写入结果要么都保存在磁盘上，要么都不保存。

10.5 日志设计（Log design）

> The log resides at a known fixed location, specified in the superblock. It consists of a header block followed by a sequence of updated block copies (“logged blocks”). The header block contains an array of sector numbers, one for each of the logged blocks, and the count of log blocks. The count in the header block on disk is either zero, indicating that there is no transaction in the log, or nonzero, indicating that the log contains a complete committed transaction with the indicated number of logged blocks. Xv6 writes the header block when a

> transaction commits, but not before, and sets the count to zero after copying the logged blocks to the file system. Thus a crash midway through a transaction will result in a count of zero in the log’ s header block; a crash after a commit will result in a non-zero count.

（磁盘上的）log 位于 superblock 中指定的一个固定位置。它由一个 “头块（header block）” 和紧跟着的一组用于更新的 block 的副本（称之为 “logged block”）组成。header block 包含一个记录扇区号（译者注：这里扇区即 block，下不赘述）的数组（每个扇区号对应一个 logged block 需要更新的 block，译者注：即 `struct logheader` 的 `block` 成员）以及一个记录了需要参与更新的 logged block 的个数的整数（译者注：即 `struct logheader` 的 `n` 成员）。磁盘上的 header block 中的计数 `n` 如果值为零，表示 log 中没有事务需要更新；如果其值非零，则表示 log 包含一个完整的已提交的事务，其具体值为事务中需要更新的 logged block 的数量。只有在一个事务被实际 “提交（commit）” 时 xv6 才会将内存中的 header block 副本（译者注：即全局变量 `log.lh`）写入磁盘中的 header block，在此之前并不会写入，此外，提交过程会在将 logged blocks 全部复制到文件系统实际对应的 block 后再将磁盘中的 header block 中的计数值恢复为零。因此，如果是事务中途（即提交前）发生崩溃，则磁盘中 log 的 header block 中的计数值为仍然为零（译者注：这表示对应一个无效的事务，无需恢复）；如果是提交后发生崩溃将导致计数是一个非零值（译者注：对这种情况，记录的 logged block 内容将在系统重启后被恢复）。

> Each system call’ s code indicates the start and end of the sequence of writes that must be atomic with respect to crashes. To allow concurrent execution of file-system operations by different processes, the logging system can accumulate the writes of multiple system calls into one transaction. Thus a single commit may involve the writes of multiple complete system calls. To avoid splitting a system call across transactions, the logging system only commits when no file-system system calls are underway.

考虑到崩溃随时可能发生，为了保证某些对磁盘执行写操作的原子性，系统调用代码中需要明确标注出这些写操作的开始和结束。为了允许不同进程并发执行文件系统操作，日志系统可能会将多个系统调用的写入累积到一个事务中。因此，单个提交可能涉及多个完整的系统调用的写入操作。为了避免一次系统调用中的写操作被拆散在多个事务中，日志系统会确保只有在这些被累积的文件系统调用的写操作全部执行完后才进行提交。

> The idea of committing several transactions together is known as group commit. Group commit reduces the number of disk operations because it amortizes the fixed cost of a commit over multiple operations. Group commit also hands the disk system more concurrent writes at the same time, perhaps allowing the disk to write them all during a single disk rotation. Xv6’ s virtio driver doesn’ t support this kind of batching, but xv6’ s file system design allows for it.

将多个事务放在一起提交的做法称为 组提交（group commit）。group commit 可以减少操作磁盘的次数，因为每次提交的成本相对固定，有利于分摊多次操作的开销。group commit 也可以为磁盘系统支持更多并发写操作，譬如允许磁盘在一次磁盘旋转过程中写入所有的这些操作。虽然 xv6 的 virtio 驱动程序不支持此类批处理，但从 xv6 的文件系统设计角度来看也是可以支持的。

> Xv6 dedicates a fixed amount of space on the disk to hold the log. The total number of blocks written by the system calls in a transaction must fit in that space. This has two consequences. No single system call can be allowed to write more distinct blocks than there is space in the log. This is not a problem for most system calls, but two of them can potentially write many blocks: `write` and `unlink`. A large file write may write many data

> blocks and many bitmap blocks as well as an inode block; unlinking a large file might write many bitmap blocks and an inode. Xv6's write system call breaks up large writes into multiple smaller writes that fit in the log, and `unlink` doesn't cause problems because in practice the xv6 file system uses only one bitmap block. The other consequence of limited log space is that the logging system cannot allow a system call to start unless it is certain that the system call's writes will fit in the space remaining in the log.

xv6 在磁盘上预留了固定的空间来保存日志。一次事务中可能包含了多次系统调用发起的写操作，其涉及的 block 总数必须不能超过这个固定空间的大小。这导致两个问题需要考虑：首先任何单个系统调用写入的 block 的个数都不能超过 log 空间的大小。这对于大多数系统调用来说都不是问题，但有两个例外：`write` 和 `unlink`。一个针对大文件的 `write` 可能涉及多个数据 block 和多个 bitmap block 以及一个 inode block；`unlink` 大文件可能会写入许多 bitmap block 和一个 inode block。为此，xv6 的 `write` 系统调用实现中将大的写入分解为适合 log 的多个较小的写入，而 xv6 中的 `unlink` 实际上并不会导致此类问题，因为 xv6 文件系统只使用了一个 bitmap block。由于 log 空间有限，需要考虑的另一个问题是，除非确定 log 中剩余的空间足够系统调用的写入，否则日志系统不会允许发起新的系统调用的写操作。

10.6 代码讲解：日志（Code: logging）

> A typical use of the log in a system call looks like this:

在一个系统调用中典型的日志使用例子如下所示：

```
begin_op();
...
bp = bread(...);
bp->data[...] = ...;
log_write(bp);
...
end_op();
```

> `begin_op` (4702) waits until the logging system is not currently committing, and until there is enough unreserved log space to hold the writes from this call. `log.outstanding` counts the number of system calls that have reserved log space; the total reserved space is `log.outstanding` times `MAXOPBLOCKS`. Incrementing `log.outstanding` both reserves space and prevents a commit from occurring during this system call. The code conservatively assumes that each system call might write up to `MAXOPBLOCKS` distinct blocks.

`begin_op` (4702) 只有当如下条件都满足时才会继续，否则会将调用它的任务进入睡眠并等待：条件之一是日志系统当前不处于提交过程中，还有一个前提条件就是有足够的未被占用的 log 空间来保存此调用的写入。`log.outstanding` 记录了当前有多少个系统调用申请使用 log 空间；预留的 block 的个数为 `log.outstanding` 乘以 `MAXOPBLOCKS`。对 `log.outstanding` 加 1 的操作不仅起到预留空间的作用，也防止在此系统调用期间发生提交（译者注：参考 `end_op` 的代码，只有当 `log.outstanding` 为 0 时才会允许提交）。代码保守地假设每次系统调用最多可以写入 `MAXOPBLOCKS` 个不同的 block。

> `log_write` (4790) acts as a proxy for `bwrite`. It records the block's sector number in memory, reserving it a slot in the log on disk, and pins the buffer in the block cache to prevent the block cache from evicting it. The block must stay in the cache until committed: until then, the cached copy is the only record of the modification; it cannot be written to its place on disk until after commit; and other reads in the same transaction must see the modifications. `log_write` notices when a block is written multiple times during a single transaction, and allocates that block the same slot in the log. This optimization is often called absorption. It is common that, for example, the disk block containing inodes of several files is written several times within a transaction. By absorbing several disk writes into one, the file system can save log space and can achieve better performance because only one copy of the disk block must be written to disk.

`log_write` (4790) 是 `bwrite` 的支持日志的平替版本。它只会将需要写入的 block 的扇区号记录在内存中，为这个 block 在磁盘上的 log 区中预定一个位置，并调用 `bpin` 将（block 对应的）缓存锁定在 buffer cache 中，以阻止其被回收掉。在最终 commit 之前，block 必须一直留在 cache 中：在最终提交之前，cache 中的这个 block 的唯一副本保留了针对该 block 的所有修改；该 buffer 只有在提交后才会被写入磁盘上对应的 block 的位置；（基于该 cache 的 buffer）同一个事务中的其他读取操作都能看到最新的修改内容。考虑到在单个事务中会针对同一个 block 实现多次写入，`log_write` 会在日志中为该 block 分配相同的槽位。这种优化通常称为 合并（absorption）。例如，储存了多个文件 inode 的 block 在一次事务中被多次写入是很常见的情况。通过将多个针对磁盘的写入操作合并到同一个 log 块中，文件系统可以节省日志空间并优化性能，因为最终只有一个 block 的副本被写入磁盘。

> `end_op` (4722) first decrements the count of outstanding system calls. If the count is now zero, it commits the current transaction by calling `commit()`. There are four stages in this process. `write_log()` (4754) copies each block modified in the transaction from the buffer cache to its slot in the log on disk. `write_head()` (4668) writes the header block to disk: this is the commit point, and a crash after the write will result in recovery replaying the transaction's writes from the log. `install_trans` (4619) reads each block from the log and writes it to the proper place in the file system. Finally `end_op` writes the log header with a count of zero; this has to happen before the next transaction starts writing logged blocks, so that a crash doesn't result in recovery using one transaction's header with the subsequent transaction's logged blocks.

`end_op` (4722) 首先减少当前正在写日志的系统调用的计数。如果计数现在为零，则通过调用 `commit()` 提交当前事务。`commit()` 这个过程分为四个阶段。`write_log()`（4754）将事务中修改的每个 block 从 buffer cache 复制到磁盘上预留的日志槽位中。`write_head()`（4668）将 header block 写入磁盘：这是 “提交的关键节点（commit point）”，此后系统发生崩溃将导致在日志恢复处理中对事务进行重写。`install_trans`（4619）从 log 区中读取每个 block 并将其写入文件系统中它应该所在的位置。最后，`end_op` 将 header block 中的计数更新为零；我们需要确保在下一个事务开始写入 logged block 之前完成这项工作，从而避免在崩溃恢复中出现 header block 是当前事务的，而 logged block 却是后续事务的这种错误情况发生。

> `recover_from_log` (4682) is called from `initlog` (4606), which is called from `fsinit` (4891) during boot before the first user process runs (2666). It reads the log header, and mimics the actions of `end_op` if the header indicates that the log contains a committed transaction.

`initlog` (4606) 会调用 `recover_from_log` (4682)，而 `initlog` 是在系统启动过程中，在第一个用户进程 (4891) 运行之前，由 `fsinit` (2666) 调用的。它会读取日志头，如果日志头指示日志包含已提交的事务，则模拟 `end_op` 的操作。

> An example use of the log occurs in `filewrite` (5803). The transaction looks like this:

有关日志的一个使用示例可以参考 `filewrite` 的实现 (5803)。其中实现的一个事务如下所示：

```
begin_op();
ilock(f->ip);
r = writei(f->ip, ...);
iunlock(f->ip);
end_op();
```

> This code is wrapped in a loop that breaks up large writes into individual transactions of just a few sectors at a time, to avoid overflowing the log. The call to `writei` writes many blocks as part of this transaction: the file’ s inode, one or more bitmap blocks, and some data blocks.

这段代码被封装在一个循环中，该循环将涉及大量数据的写入操作分解成每次只写入几个扇区的独立事务，以避免日志溢出。对 `writei` 的调用会在一个事务中对多个 block 进行写入，这些 block 涉及：文件的 inode、一个或多个 bitmap block 以及一些数据 block。

10.7 代码讲解：块分配器（Code: Block allocator）

> File and directory content is stored in disk blocks, which must be allocated from a free pool. Xv6’ s block allocator maintains a free bitmap on disk, with one bit per block. A zero bit indicates that the corresponding block is free; a one bit indicates that it is in use. The program `mkfs` sets the bits corresponding to the boot sector, superblock, log blocks, inode blocks, and bitmap blocks.

文件和目录的内容存储在磁盘 block 中，这些 block 通过统一的资源池机制进行分配和管理。xv6 的“块分配器（block allocator）” 在磁盘上维护一个记录 block 使用状态的 bitmap，每个 block 对应 bitmap 中的一个 bit 位。如果这个 bit 位的值是 0 则表示相应的 block 可用；如果这个 bit 位的值是 1 则表示对应的 block 已被使用。程序 `mkfs` 负责设置与引导扇区、superblock、log、inode 和 bitmap 所占用的 block 对应的比特位。

> The block allocator provides two functions: `balloc` allocates a new disk block, and `bfree` frees a block. `balloc` The loop in `balloc` at (4923) considers every block, starting at block 0 up to `sb.size`, the number of blocks in the file system. It looks for a block whose bitmap bit is zero, indicating that it is free. If `balloc` finds such a block, it updates the bitmap and returns the block. For efficiency, the loop is split into two pieces. The outer loop reads each block of bitmap bits. The inner loop checks all Bits-Per-Block (`BPB`) bits in a single bitmap block. The race that might occur if two processes try to allocate a block at the same time is prevented by the fact that the buffer cache only lets one process use any one bitmap block at a time.

块分配器提供两个函数接口：`balloc` 用于分配一个新的磁盘 block，`bfree` 用于释放一个 block。`balloc` (4923) 中有一个循环遍历每个 block，从 0 开始直到 `sb.size`（文件系统中支持的 block 的最大个数），在 bitmap 中寻找对应的 bit 位的值为零的 block（即空闲的 block）。如果 `balloc` 找到这样的 block，则更新 bitmap 并返回该 block 的下标。为了提高效率，循环被分成两部分。外层循环遍历每个存放 bitmap bit 的 block（译者注：由于 xv6 的文件系统支持的 block 数量有限，实际上访问的 bp 都是同一份 buffer cache。而且外层循环实际上也只会运行一次，因为除非 block 耗尽，总会在内层循环中找到空闲的 block）。内层循环检查单个位图 block 中的所有 bit 位 (Bits-Per-Block，简称 `BPB`)。由于 buffer cache 一次只允许一个进程访问一个 bitmap block，这也避免了两个进程同时尝试分配一个 block 时可能发生的竞争。

> `bfree` (4952) finds the right bitmap block and clears the right bit. Again the exclusive use implied by `bread` and `brelse` avoids the need for explicit locking.

`bfree` (4952) 找到正确的 bitmap 所在 block 并清除对应的位。同样，由于 `bread` 和 `brelse` 中隐含的独占使用，所以在 `bfree` 中我们同样没有为操作显示上锁。

> As with much of the code described in the remainder of this chapter, `balloc` and `bfree` must be called inside a transaction.

与本章其余部分描述的大部分代码一样，对 `balloc` 和 `bfree` 的调用必须包括在一个事务中。

10.8 索引节点层（Inode layer）

> The term inode can have one of two related meanings. It might refer to the on-disk data structure containing a file’ s size and list of data block numbers. Or “inode” might refer to an in-memory inode, which contains a copy of the on-disk inode as well as extra information needed within the kernel.

术语 索引节点（inode） 有两种含义。一种是指磁盘上记录文件大小和一组数据 block 编号的数据结构（译者注：即下文将介绍的 `struct dinode`）。另一种是指内存中的一个数据结构（译者注：即下文将介绍的 `struct inode`），它包含了磁盘上的 inode 的副本以及一些内核所需的额外信息。

> The on-disk inodes are packed into a contiguous area of disk called the inode blocks. Every inode is the same size, so it is easy, given a number n, to find the nth inode on the disk. In fact, this number n, called the inode number or i-number, is how inodes are identified in the implementation.

磁盘上的 inode 占据了磁盘上的一个连续的区域，由多个含有 inode 信息的 block 组成。每个 inode 的大小相同，因此，给定一个数字 n，很容易找到磁盘上的第 n 个 inode。实际上，在实现中我们正是利用这个数字 n（称为 inode number 或 i-number）来标识 inode。

> The on-disk inode is defined by a `struct dinode` (4131). The `type` field distinguishes between files, directories, and special files (devices). A type of zero indicates that an on-disk inode is free. The `nlink` field counts the number of directory entries that refer to this inode, in order to recognize when the on-disk inode and its data blocks should be freed. The `size` field records the number of bytes of content in the file. The `addrs` array records the block numbers of the disk blocks holding the file’ s content.

磁盘上的 inode 对应结构体 `struct dinode` (4131)。其 `type` 字段用于区分这个 inode 是文件、目录还是特殊文件（设备）。如果 `type` 值为零则表示磁盘上的这个 inode 未被使用。`nlink` 字段用于统计有多少个 “目录项（directory entry）” 会引用此 inode，以便识别何时应释放磁盘上的 inode 及其数据块。`size` 字段记录文件实际的大小（单位是字节）。`addrs` 数组记录所有保存该文件内容的 block 的编号。

> The kernel keeps the set of active inodes in memory in a table called `itable`; `struct inode` (4216) is the in-memory copy of a `struct dinode` on disk. The kernel stores an inode in memory only if there are C pointers referring to that inode. The `ref` field counts the number of C pointers referring to the in-memory inode, and the kernel discards the inode from memory if the reference count drops to zero. The `iget` and `iput` functions acquire and release pointers to an inode, modifying the reference count. Pointers to an inode can come from file descriptors, current working directories, and transient kernel code such as `kexec`.

内核将当前所有活跃的 inode 保存在内存中一个名为 `itable` 的表中；`struct inode`（4216）是磁盘上 `struct dinode` 在内存中的副本。只有当有 C 语言指针引用该 inode 时，内核才会将该 inode 存储在内存中。`ref` 字段统计内存中 inode 的引用数量，如果引用计数降至零，内核会从内存中丢弃该 inode。`iget` 和 `iput` 函数用于获取和释放指向 inode 的指针，从而修改引用计数。对 inode 的访问（一般通过指针）可能来自文件描述符、当前工作目录以及诸如 `kexec` 之类的内核代码。

译者注：注意原文提到 inode 时可能根据上下文的不同会有不同的含义。譬如，当上下文指的是磁盘中的 inode 时，对应的是 `struct dinode`；当上下文指的是内存中的 inode 时，对应的是 `itable` 的表中 `struct inode` 项。涉及指针访问的 inode 时不加特殊说明指的当然是内存中的 `struct inode`。如果没有特别的上下文限定，inode 更多表达的是文件系统中索引节点的概念，此时会更倾向于指的是磁盘中的 inode。

> There are four lock or lock-like mechanisms in xv6’s inode code. `itable.lock` protects the invariant that an inode is present in the inode table at most once, and the invariant that an in-memory inode’s `ref` field counts the number of in-memory pointers to the inode. Each in-memory inode has a `lock` field containing a sleep-lock, which ensures exclusive access to the inode’s fields (such as file length) as well as to the inode’s file or directory content blocks. An inode’s `ref`, if it is greater than zero, causes the system to maintain the inode in the table, and not re-use the table entry for a different inode. Finally, each inode contains a `nlink` field (on disk and copied in memory if in memory) that counts the number of directory entries that refer to a file; xv6 won’t free an inode if its link count is greater than zero.

xv6 的 inode 相关处理代码中使用了四种锁或类似锁的机制。`itable.lock` 确保每个磁盘上的 inode 在 inode 表中只会有一个副本，此外该锁也保护了对内存中 inode 的 `ref` 字段的更新动作（该字段用于计数指向该 inode 的指针个数）。每个 `struct inode` 都有一个 `lock` 字段，这是一把睡眠锁，用于确保对 `struct inode` 中成员字段（例如 `size`）以及存放 inode 的文件或目录内容的 block 的独占访问。当 `struct inode` 的 `ref` 大于零时，则系统确保不会将 inode 表的该项回收并重复用于其他的 inode。最后，每个 inode 都包含一个 `nlink` 字段（磁盘上的 `struct dinode` 和内存中的 `struct inode` 都有这个字段，`struct inode` 的值复制自 `struct`

`dinode`），该字段用于计数引用某个文件的 directory entry 的数量；只要该值大于零，则 xv6 不会释放该 inode。

> A `struct inode` pointer returned by `iget()` is guaranteed to be valid until the corresponding call to `iput()`; the inode won't be deleted, and the memory referred to by the pointer won't be re-used for a different inode. `iget()` provides non-exclusive access to an inode, so that there can be many pointers to the same inode. Many parts of the file-system code depend on this behavior of `iget()`, both to hold long-term references to inodes (as open files and current directories) and to prevent races while avoiding deadlock in code that manipulates multiple inodes (such as pathname lookup).

内核保证 `iget()` 返回的 `struct inode` 指针在调用对应 `iput()` 之前一直有效；即该 inode 不会被删除，并且该指针指向的内存不会被其他 inode 重复使用。`iget()` 提供对 inode 的非独占访问，因此可以有多个指针指向同一个 inode。文件系统代码的许多部分都依赖于 `iget()` 的这种行为，既可以保存对 inode 的长期引用（例如打开的文件和当前目录），也可以防止争用，同时避免在操作多个 inode 的代码（例如路径名查找）中出现死锁。

> The `struct inode` that `iget` returns may not have any useful content. In order to ensure it holds a copy of the on-disk inode, code must call `ilock`. This locks the inode (so that no other process can `ilock` it) and reads the inode from the disk, if it has not already been read. `iunlock` releases the lock on the inode. Separating acquisition of inode pointers from locking helps avoid deadlock in some situations, for example during directory lookup. Multiple processes can hold a C pointer to an inode returned by `iget`, but only one process can lock the inode at a time.

`iget` 返回 `struct inode` 时可能还没有将磁盘上对应的 inode 的内容读入缓存。为了确保它已经加载了这部分内容，代码必须继续调用 `ilock`。这会锁定 inode（以便其他进程无法 `ilock` 它），并从磁盘读取该 inode（如果尚未读取）。`iunlock` 会释放对 inode 的锁定。将 inode 指针的获取与锁定分离有助于避免在某些情况下（例如在目录查找期间）发生死锁。多个进程可以持有指向 `iget` 返回的 inode 的 C 指针，但一次只能有一个进程可以锁定该 inode。

> The inode table only stores inodes to which kernel code or data structures hold C pointers. Its main job is synchronizing access by multiple processes. The inode table also happens to cache frequently-used inodes, but caching is secondary; if an inode is used frequently, the buffer cache will probably keep it in memory. Code that modifies an in-memory inode writes it to disk with `iupdate`.

inode 表（即 `itable`）仅存储内核代码或数据结构中 C 指针指向的 inode。它的主要作用是同步多个进程的访问。inode 表也会缓存常用的 inode，但这里缓存的作用是次要的；如果某个 inode 被频繁使用，buffer cache 也可能会将其保留在内存中。如果代码修改了内存中的 inode 还需要使用 `iupdate` 将其同步回磁盘中。

10.9 代码讲解：索引节点（Code: Inodes）

> To allocate a new inode (for example, when creating a file), xv6 calls `ialloc` (5059). `ialloc` is similar to `balloc`: it loops over the inode structures on the disk, one block at a time, looking for one that is marked free. When it finds one, it claims it by writing the new

> `type` to the disk and then returns an entry from the inode table with the tail call to `iget` (5073). The correct operation of `ialloc` depends on the fact that only one process at a time can be holding a reference to `bp`: `ialloc` can be sure that some other process does not simultaneously see that the inode is available and try to claim it.

为了分配新的 “索引节点（inode）”（例如，创建文件时），xv6 会调用 `ialloc` (5059)。`ialloc` 类似于 `balloc`，该函数会在磁盘上存放 inode 的 block 中进行遍历，寻找标记为空闲的项。找到后，它会更新其 `type` 字段为非零值（表示占用），最后以尾调用的方式通过 `iget` (5073) 返回 inode 表中的项。`ialloc` 正确工作的前提取决于同一时间只有一个进程可以持有对 `bp` 的引用：`ialloc` 可以确保其他进程不会同时发现该 inode 可用并尝试占用它。

> `iget` (5107) looks through the inode table for an active entry (`ip->ref > 0`) with the desired device and inode number. If it finds one, it returns a new reference to that inode (5116-5120). As `iget` scans, it records the position of the first empty slot (5121-5122), which it uses if it needs to allocate a table entry.

`iget` (5107) 在 “索引节点表（inode table）” 中查找有效的 (`ip->ref > 0`) 的项，同时将该项的设备号与 inode 编号和给定的入参值进行匹配。如果找到，它会递增该 inode 的 `ref` 值并返回其指针（5116-5120）。在扫描过程中 `iget` 会记录第一个可用项的位置 (5121-5122)，以便需要在索引节点表中分配新的项时直接定位到第一个可用项的位置（避免再扫描一遍）。

> Code must lock the inode using `ilock` before reading or writing its metadata or content. `ilock` (5153) uses a sleep-lock for this purpose. Once `ilock` has exclusive access to the inode, it reads the inode from disk (more likely, the buffer cache) if needed. The function `iunlock` (5181) releases the sleep-lock, which may cause any processes sleeping to be woken up.

在读取或写入 inode 的 “元数据（metadata）” 或文件内容之前，代码必须使用 `ilock` 锁定 inode。`ilock` (5153) 使用睡眠锁来实现此目的。一旦 `ilock` 获得 inode 的独占访问权限，它会根据需要从磁盘（更可能是 buffer cache）读取 inode。函数 `iunlock` (5181) 会释放睡眠锁，这可能会导致其他处于休眠状态的进程被唤醒。

> `iput` (5208) releases a C pointer to an inode by decrementing the reference count (5231). If this is the last reference, the inode’s slot in the inode table is now free and can be re-used for a different inode.

`iput` (5208) 通过减少引用计数 (5231) 释放指向某个 inode 的 C 指针。如果这是最后一处引用，则此时我们可以释放该 inode 在索引节点表中的槽位，并将其重新用于其他 inode。

> If `iput` sees that there are no C pointer references to an inode and that the inode has no links to it (occurs in no directory), then the inode and its data blocks must be freed. `iput` calls `itrunc` to truncate the file to zero bytes, freeing the data blocks; sets the inode type to 0 (unallocated); and writes the inode to disk (5213).

如果 `iput` 发现不再有 C 指针引用某个 inode，并且该 inode 没有被任何链接所引用（换句话说，在任何目录下我们都找不到该文件），此时我们必须释放该 inode 及其相关联的数据 block。`iput` 调用 `itrunc` 将文件长度 “截断（truncate）” 为零，这会释放占用的数据 block（译者注：仅仅修改 bitmap，并不会清理实际 block 上的内容）；然后将 inode 类型设置为 0（表示未分配）；最后将内存中 inode 的修改回写入磁盘（5213）。

> The locking protocol in `iput` in the case in which it frees the inode deserves a closer look. One danger is that a concurrent thread might be waiting in `ilock` to use this inode (e.g., to read a file or list a directory), and won't be prepared to find that the inode is no longer allocated. This can't happen because there is no way for a system call to get a pointer to an in-memory inode if it has no links to it and `ip->ref` is one. That one reference is the reference owned by the thread calling `iput`. The other main danger is that a concurrent call to `ialloc` might choose the same inode that `iput` is freeing. This can happen only after the `iupdate` writes the disk so that the inode has type zero. This race is benign; the allocating thread will politely wait to acquire the inode's sleep-lock before reading or writing the inode, at which point `iput` is done with it.

在 `iput` 函数中涉及释放 inode 的处理过程中的锁机制设计值得仔细研究一下。一个需要考虑的风险是，另一个并发线程可能正在 `ilock` 中等待使用此 inode（例如，读取文件或列出目录），如果当它醒来后发现该 inode 已经被释放掉肯定是不行的。这种情况实际上并不会发生，因为当一个系统调用得到指向该 inode 的指针时，该 inode 的 `nlink` 的值不可能为 0 而且其 `ref` 也不可能为 1。该引用是调用 `iput` 的线程所拥有的引用。另一个主要风险是，对 `ialloc` 的并发调用可能会选择 `iput` 正在释放的同一个 inode。这种情况只有在 `iupdate` 写入磁盘后，即 inode 的 `type` 为零时才会发生。这种竞争不会导致什么问题；调用 `ialloc` 的线程会礼貌地等待获取 inode 的睡眠锁，然后再读取或写入 inode，此时 `iput` 已完成对它的处理。

> `iput()` can write to the disk. This means that any system call that uses the file system may write to the disk, because the system call may be the last one having a reference to the file. Even calls like `read()` that appear to be read-only, may end up calling `iput()`. This, in turn, means that even read-only system calls must be wrapped in transactions if they use the file system.

`iput()` 涉及对磁盘的写入操作。这意味着任何使用文件系统的系统调用都会涉及对磁盘的写入操作，因为该系统调用可能是最后一个引用该文件的调用。即使像 `read()` 这样看似只读的调用，最终也可能调用 `iput()`。反过来，这意味着即使是只读的系统调用，如果使用文件系统，也必须封装在事务中。

> There is a challenging interaction between `iput()` and crashes. `iput()` doesn't truncate a file immediately when the link count for the file drops to zero, because some process might still hold a reference to the inode in memory: a process might still be reading and writing to the file, because it successfully opened it. But, if a crash happens before the last process closes the file descriptor for the file, then the file will be marked allocated on disk but no directory entry will point to it.

针对系统崩溃场景，`iput()` 的处理逻辑中存在一个棘手的问题。当文件的链接计数降至零时，`iput()` 不会立即清除文件的内容（truncate），因为某些进程可能仍在内存中持有对 inode 的引用：进程可能仍在准备读取和写入该文件，因为它已成功打开该文件。但是，如果崩溃发生在最后一个进程关闭文件的文件描述符之前，那么该文件将在磁盘上被标记为已分配，但没有 “目录项（directory entry）” 指向它(译者注：有关目录项的概念见下文第 10.11 节介绍)。

> File systems handle this case in one of two ways. The simple solution is that on recovery, after reboot, the file system scans the whole file system for files that are marked allocated, but have no directory entry pointing to them. If any such file exists, then it can free those files.

文件系统有两种方法来处理这种情况。一种简单的解决方案是，在系统恢复时，也就是重启后，文件系统会扫描整个文件系统，查找那些标记为 “已分配” 但没有 “目录项（directory entry）” 指向的文件。如果存在这样的文件，就可以释放它们。

> The second solution doesn’ t require scanning the file system. In this solution, the file system records on disk (e.g., in the super block) the inode inumber of a file whose link count drops to zero but whose reference count isn’ t zero. If the file system removes the file when its reference count reaches 0, then it updates the on-disk list by removing that inode from the list. On recovery, the file system frees any file in the list.

第二种解决方案不需要扫描文件系统。在该解决方案中，文件系统会在磁盘上（例如，在 superblock 中）记录链接计数降至零但引用计数不为零的文件的 inode 编号。如果文件系统在引用计数达到 0 时删除该文件，则会通过从列表中删除该 inode 来更新磁盘上的列表。恢复后，文件系统将释放列表中所有的文件。

> Xv6 implements neither solution, which means that inodes may be marked allocated on disk, even though they are not in use anymore. This means that over time xv6 runs the risk that it may run out of disk space.

上述的两种方案 xv6 都没有实现，这意味着即使 inode 不再使用，它们也可能被标记为已分配。所以随着时间的推移，xv6 可能会面临耗尽磁盘空间的风险。

10.10 代码讲解：索引节点的内容（Code: Inode content）

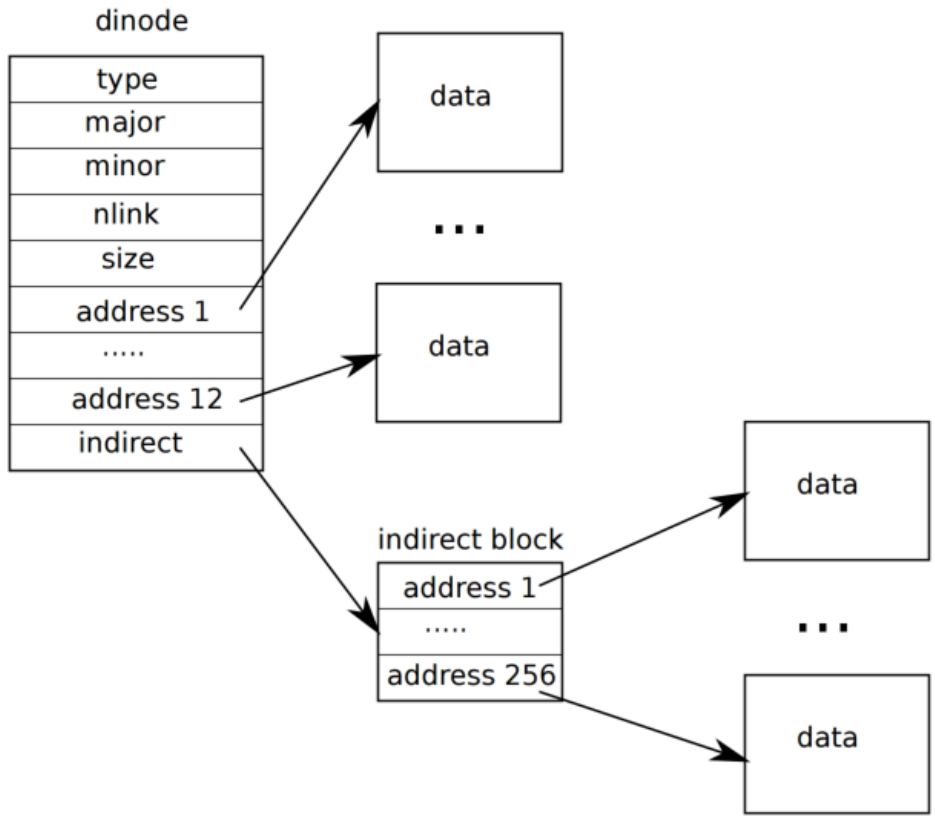

Figure 10.3: The representation of a file on disk.

> The on-disk inode structure, `struct dinode`, contains a size and an array of block numbers (see Figure 10.3). The inode data is found in the blocks listed in the `dinode`'s `addrs` array. The first `NDIRECT` blocks of data are listed in the first `NDIRECT` entries in the array; these blocks are called direct blocks. The next `NINDIRECT` blocks of data are listed not in the inode but in a data block called the indirect block. The last entry in the `addrs` array gives the address of the indirect block. Thus the first 12 kB (`NDIRECT x BSIZE`) bytes of a file can be loaded from blocks listed in the inode, while the next `256` kB (`NINDIRECT x BSIZE`) bytes can only be loaded after consulting the indirect block. This is a good on-disk representation but a complex one for clients. The function `bmap` manages the representation so that higher-level routines, such as `readi` and `writei`, which we will see shortly, do not need to manage this complexity. `bmap` returns the disk block number of the `bn`'th data block for the inode `ip`. If `ip` does not have such a block yet, `bmap` allocates one.

磁盘上的 inode 结构体 `struct dinode` 包含一个 `size` 字段和一个存放 block 编号的数组（参见图 10.3）。inode 的数据存放于 `dinode` 的 `addrs` 数组中列出的 block 中。数组的前 `NDIRECT` 项对应 `NDIRECT`（译者注：常量值 12）个数据 block，这些 block 称为 直接块（direct block）。还有 `NINDIRECT`（译者注：常量值 256）个数据 block 的编号没有直接存放在 `addrs` 数组中，而

是存放在一个称为 间接块（indirect block） 的数据 block 中。`addrs` 数组中的最后一项给出了 indirect block 的编号。因此，文件的前 12 kB（`NDIRECT x BSIZE`）字节可以从 inode 中列出的 block 加载，而接下来的 `256` kB（`NINDIRECT x BSIZE`）字节只能在查阅 indirect block 后才能加载。这是一种优化设计，但对于使用者来说比较复杂。函数 `bmap` 封装了对 inode 中数据块的管理（译者注：即对 `inode` 的 `addrs` 数组的访问），这样更高级别的函数（例如我们稍后会看到的 `readi` 和 `writei`）就无需考虑这种复杂性了。`bmap` 返回 `ip` 的第 `bn` 个数据块的 block 编号。如果 `ip` 还没有这样的 block，`bmap` 就会分配一个。

> The function `bmap` (5283) begins by picking off the easy case: the first `NDIRECT` blocks are listed in the inode itself (5288-5296). The next `NINDIRECT` blocks are listed in the indirect block at `ip->addrs[NDIRECT]`. `bmap` reads the indirect block (5308) and then reads a block number from the right position within the block (5309). If the block number exceeds `NDIRECT+NINDIRECT`, `bmap` panics; `writei` contains the check that prevents this from happening (5415).

函数 `bmap` (5283) 首先处理相对简单的情况：即前 `NDIRECT` 个列在 inode 中的 block (5288-5296)。后面的 `NINDIRECT` 个 block 列在 indirect block，即 `ip->addrs[NDIRECT]` 中。（为了获取这后面的 `NINDIRECT` 个 block） `bmap` 先读取 indirect block (5308)，然后从这个 block 中的正确位置再读取 block 编号 (5309)。如果 block 编号超过 `NDIRECT+NINDIRECT`， `bmap` 就直接 panic； `writei` 会检查这种异常，防止其发生 (5415)。

> `bmap` allocates blocks as needed. An `ip->addrs[]` or indirect entry of zero indicates that no block is allocated. As `bmap` encounters zeros, it replaces them with the numbers of fresh blocks, allocated on demand (5289-5290) (5302-5303).

`bmap` 会根据需要分配 block。如果 `ip->addrs[]` 或 indirect block 中的项的值为零表示未分配任何块。当 `bmap` 遇到零值时，它会将其替换为按需分配的新的 block 的编号 (5289-5290) (5302-5303)。

> `itrunc` frees a file’s blocks, resetting the inode’s size to zero. `itrunc` (5327) starts by freeing the direct blocks(5333-5338), then the ones listed in the indirect block (5343-5346), and finally the indirect block itself (5348-5349).

`itrunc` 释放文件的 block，将 inode 的 `size` 重置为零。`itrunc` (5327) 首先释放 direct block (5333-5338)，然后释放 indirect block (5343-5346) 中列出的 block，最后释放 indirect block 本身 (5348-5349)。

> `bmap` makes it easy for `readi` and `writei` to get at an inode’s data. `readi` (5373) starts by making sure that the offset and count are not beyond the end of the file. Reads that start beyond the end of the file return an error (5378-5379) while reads that start at or cross the end of the file return fewer bytes than requested (5380-5381). The main loop processes each block of the file, copying data from the buffer into `dst` (5383-5395). `writei` (5408) is identical to `readi`, with three exceptions: writes that start at or cross the end of the file grow the file, up to the maximum file size (5415-5416); the loop copies data into the buffers instead of out (5424); and if the write has extended the file, `writei` must update its size (5432-5433).

通过调用 `bmap`，`readi` 和 `writei` 可以轻松获取 inode 的数据。`readi` (5373) 首先确保起始位置（`off`）和结束位置（`off + n`）不超过文件末尾。读取时起始位置超出了文件末尾将返回错误 (5378-5379)，而结束位置超出文件末尾将按照文件实际长度进行读取（这将比实际请求的字节数更少）(5380-5381)。主循环处理文件的每个 block，将数据从缓冲区复制到 `dst` (5383-5395)。`writei` (5408) 与 `readi` 相同，但有三个例外：首先，写入的结束位置超出文件末尾会使文件增大，直到达到最大文件大小 `MAXFILE*BSIZE` (5415-5416)；其次，`writei` 中的循环是将数据复制到缓冲区中去而不是把缓冲区中的数据往外复制（5424）；最后，如果写入操作扩展了文件大小，则 `writei` 必须更新其 `size`（5432-5433）。

> The function `stati` (5359) copies inode metadata into the `stat` structure, which is exposed to user programs via the `stat` system call.

函数 `stati`（5359）将 inode 的“元数据（metadata）”复制到 `struct stat` 结构体中，用户通过 `stat` 系统调用可以访问该结构体的内容。

10.11 代码讲解：目录层（Code: directory layer）

> A directory is implemented internally much like a file. Its inode has type `T_DIR` and its data is a sequence of directory entries. Each entry is a `struct dirent` (4165), which contains a name and an inode number. The name is at most `DIRSIZ` (14) characters; if shorter, it is terminated by a NULL (0) byte. Directory entries with inode number zero are free.

“目录（directory）”的内部实现与文件非常相似（译者注：即每一个目录也对应一个 inode）。其 inode 的 `type` 为 `T_DIR`，其数据是一组“目录项（directory entry）”。每一个目录项对应一个结构体 `struct dirent`（4165），该结构体包含一个 `name` 和一个 inode 编号（`inum`）。`name` 最多包含 `DIRSIZ`（14）个字符；如果少于 14 个字符，则以 NULL（0）字节结尾。如果 inode 编号为 0 说明该目录项未被使用。

> The function `dirlookup` (5453) searches a directory for an entry with the given name. If it finds one, it returns a pointer to the corresponding inode, unlocked, and sets `*poff` to the byte offset of the entry within the directory, in case the caller wishes to edit it. If `dirlookup` finds an entry with the right name, it updates `*poff` and returns an unlocked inode obtained via `iget`. `dirlookup` is the reason that `iget` returns unlocked inodes. The caller has locked `dp`, so if the lookup was for ., an alias for the current directory, attempting to lock the inode before returning would try to re-lock dp and deadlock. (There are more complicated deadlock scenarios involving multiple processes and .., an alias for the parent directory; . is not the only problem.) The caller can unlock `dp` and then lock `ip`, ensuring that it only holds one lock at a time.

函数 `dirlookup`（5453）在目录中搜索具有给定名称的目录项。如果找到，它将返回该目录项对应的 inode 的指针（此时还未锁定），并将出参 `*poff` 设置为该目录项在目录中的字节偏移量，以便调用者对其进行编辑。如果 `dirlookup` 找到具有正确名称的目录项，它将给 `*poff` 赋值，并通过 `iget` 获取未锁定的 inode 的指针并返回该指针。之所以这里 `iget` 返回未锁定的 inode 的原因在于 `dirlookup` 的调用者已锁定 `dp`（即当前目录），因此如果查找的是“.”（当前目录的别名），则在返回之前尝试锁定 inode 会导致重复锁定 dp 并导致死锁（“.” 不是唯一的问题，多进程并发情况下访问“..”（父目录的别名）同样存在复杂的死锁风险。）。所以我们让 `dirlookup` 的调用者来负责解锁 `dp`，然后再锁定 `ip`，确保它一次只持有一个锁。

> The function `dirlink` (5481) writes a new directory entry with the given name and inode number into the directory `dp`. If the name already exists, `dirlink` returns an error (5487-5491). The main loop reads directory entries looking for an unallocated entry. When it finds one, it stops the loop early (5493-5498), with `off` set to the offset of the available entry. Otherwise, the loop ends with `off` set to `dp->size`. Either way, `dirlink` then adds a new entry to the directory by writing at offset `off` (5502-5503).

函数 `dirlink`（5481）通过给定名字和 inode 编号将一个新的目录项加入 `dp` 指向的 directory。如果该名字已存在，`dirlink` 将返回错误（5487-5491）。函数中的循环会检查目录中的目录项数组，查找尚未分配的目录项。如果找到，则会提前停止循环（5493-5498），并将 `off` 设置为可用目录项的偏移量。否则，循环结束后 `off` 会被设置为 `dp->size`。无论哪种情况，`dirlink` 都会在偏移量 `off` 处为目录添加一个新的目录项（5502-5503）。

10.12 代码讲解：路径名称（Code: Path names）

> Path name lookup involves a succession of calls to `dirlookup`, one for each path component. `namei` (5590) evaluates `path` and returns the corresponding `inode`. The function `nameiparent` is a variant: it stops before the last element, returning the inode of the parent directory and copying the final element into `name`. Both call the generalized function `namex` to do the real work.

路径名查找涉及对 `dirlookup` 的多次连续调用，路径中以 “/” 分隔的每个部分（译者注：原文中称之为 component 或者 element）都对应一次。`namei` (5590) 会解析 `path` 并返回该路径对应的 `inode`。函数 `nameiparent` 是 `namei` 的一个 “变体（variant）”：它会取出 `path` 的最后一个 element 之前的部分（即该 element 的父目录的路径）并返回父目录路径的 inode，最后一个 element 则被复制到 `name` 中返回。`namei` 和 `nameiparent` 内部都是通过调用公共函数 `namex` 来完成实际的工作。

> `namex` (5555) starts by deciding where the path evaluation begins. If the path begins with a slash, evaluation begins at the root; otherwise, the current directory (5559-5562). Then it uses `skipelem` to consider each element of the path in turn (5564). Each iteration of the loop must look up `name` in the current inode `ip`. The iteration begins by locking `ip` and checking that it is a directory. If not, the lookup fails (5565-5569). (Locking `ip` is necessary not because `ip->type` can change underfoot—it can’ t—but because until `ilock` runs, `ip->type` is not guaranteed to have been loaded from disk.) If the call is `nameiparent` and this is the last path element, the loop stops early, as per the definition of `nameiparent`; the final path element has already been copied into `name`, so `namex` need only return the unlocked `ip` (5570-5574). Finally, the loop looks for the path element using `dirlookup` and prepares for the next iteration by setting `ip = next` (5575-5580). When the loop runs out of path elements, it returns `ip`.

`namex` (5555) 首先确定路径解析的开始位置。如果路径以 “/” 开头，则从根目录开始；否则，从当前目录开始 (5559-5562)。然后，它使用 `skipelem` 依次考量路径的每个 element (5564)。每次循环迭代都必须在当前 inode `ip` 中查找 `name`。迭代首先锁定 `ip` 并检查它是否为目录。如果不是，则终止查找 (5565-5569)。（锁定 `ip` 是必要的，不是因为 `ip->type` 可能会被更改，而是因为只有调用了 `ilock` 才能保证 `ip->type` 已被从磁盘加载。）如果本次调用 `namex` 的是 `nameiparent` 并且这是最后一个路径元素，则循环将提前停止；最后一个路径元素此时已复制到

`name` 中，因此 `namex` 只需返回未锁定的 `ip` (5570-5574)。最后，循环中使用 `dirlookup` 查找路径 element，并通过设置 `ip = next` (5575-5580) 为下一次迭代做准备。当循环处理完路径中所有的 element 时，它将返回 `ip`。

> The procedure `namex` may take a long time to complete: it could involve several disk operations to read inodes and directory blocks for the directories traversed in the pathname (if they are not in the buffer cache). Xv6 is carefully designed so that if an invocation of `namex` by one kernel thread is blocked on a disk I/O, another kernel thread looking up a different pathname can proceed concurrently. `namex` locks each directory in the path separately so that lookups in different directories can proceed in parallel.

函数 `namex` 可能需要很长时间才能完成：它可能涉及多个磁盘操作，在扫描路径名的过程中对于每个目录都会读取目录对应的 inode 以及相关的 block（如果它们不在 buffer cache 中）。xv6 的设计非常精妙，即使某个内核线程对 `namex` 的调用因磁盘 I/O 而受阻，另一个查找不同路径名的内核线程仍可并发执行。`namex` 会分别锁定路径中的每个目录，以便不同目录中的查找可以并行进行。

> This concurrency introduces some challenges. For example, while one kernel thread is looking up a pathname another kernel thread may be changing the directory tree by unlinking a directory. A potential risk is that a lookup may be searching a directory that has been deleted by another kernel thread and its blocks have been re-used for another directory or file.

这种并发性带来了一些挑战。例如，当一个内核线程正在查找路径名时，另一个内核线程可能正在通过取消对目录的链接来更改目录树。一个潜在的风险是，查找的目录可能已被另一个内核线程删除，并且其磁盘 block 已被其他目录或文件重用。

> Xv6 avoids such races. For example, when executing `dirlookup` in `namex`, the lookup thread holds the lock on the directory and `dirlookup` returns an inode that was obtained using `iget`. `iget` increases the reference count of the inode. Only after receiving the inode from `dirlookup` does `namex` release the lock on the directory. Now another thread may unlink the inode from the directory but xv6 will not delete the inode yet, because the reference count of the inode is still larger than zero.

xv6 避免了此类竞争。例如，在 `namex` 中执行 `dirlookup` 时，查找线程会持有目录的锁，而 `dirlookup` 会返回一个使用 `iget` 获取的 inode。`iget` 会增加该 inode 的引用计数。只有在 `dirlookup` 接收到该 inode 后，`namex` 才会释放目录的锁。此时，另一个线程可能会取消该 inode 与目录的链接，但 xv6 不会删除该 inode，因为该 inode 的引用计数仍然大于零。

> Another risk is deadlock. For example, `next` points to the same inode as `ip` when looking up ".". Locking `next` before releasing the lock on `ip` would result in a deadlock. To avoid this deadlock, `namex` unlocks the directory before obtaining a lock on `next`. Here again we see why the separation between `iget` and `ilock` is important.

另一个风险是死锁。例如，在查找 “.” 时，`next` 指向与 `ip` 相同的 inode。在释放 `ip` 的锁之前锁定 `next` 会导致死锁。为了避免这种死锁，`namex` 会在获取 `next` 的锁之前解锁目录。这里我们再次看到了 `iget` 和 `ilock` 分离的重要性。

10.13 文件描述符层（File descriptor layer）

> A cool aspect of the Unix interface is that most resources in Unix are represented as files, including devices such as the console, pipes, and of course, real files. The file descriptor layer is the layer that achieves this uniformity.

Unix 接口的一个很酷的设计是，Unix 中的大多数资源都以文件的形式表示，包括控制台、管道等设备，当然还有实际的文件。文件描述符层实现了这种统一性。

> Xv6 gives each process its own table of open files, or file descriptors, as we saw in Chapter 1. Each open file is represented by a `struct file` (4200), which is a wrapper around either an inode or a pipe, plus an I/O offset. Each call to `open` creates a new open file (a new `struct file`): if multiple processes open the same file independently, the different instances will have different I/O offsets. On the other hand, a single open file (the same `struct file`) can appear multiple times in one process' s file table and also in the file tables of multiple processes. This would happen if one process used `open` to open the file and then created aliases using `dup` or shared it with a child using `fork`. A reference count tracks the number of references to a particular open file. A file can be open for reading or writing or both. The `readable` and `writable` fields track this.

正如我们在第 1 章中看到的，xv6 为每个进程维护了自身打开的文件的列表，或者叫文件描述符表。每个打开的文件都由一个 `struct file` (4200) 表示，它封装了 inode、管道以及 I/O 偏移量等概念。每次调用 `open` 都会创建一个新的打开文件（一个新的 `struct file`）：如果多个进程都打开了同一个文件，则每个进程中针对该文件的的不同的文件描述符实例将具有不同的 I/O 偏移量。另一方面，单个打开的文件（对应一个 `struct file`）可以在同一个进程的文件表中出现多次，也可以在多个不同的进程的文件表中出现多次。如果一个进程使用 `open` 打开文件，然后使用 `dup` 创建别名或使用 `fork` 与子进程共享该文件，就会发生这种情况。引用计数跟踪对特定打开文件的引用数量。文件可以处于读、写或两者兼有的状态。`readable` 和 `writable` 字段用于标记这些状态。

> All the open files in the system are kept in a global file table, the `ftable`. The file table has functions to allocate a file (`filealloc`), create a duplicate reference (`filedup`), release a reference (`fileclose`), and read and write data (`fileread` and `filewrite`).

系统中所有打开的文件都保存在一个全局文件表 `ftable` 中。我们提供以下函数操作该文件表：分配文件（`filealloc`）；创建重复引用（`filedup`）；释放引用（`fileclose`）以及读数据（`fileread`）和写数据（`filewrite`）。

> The first three follow the now-familiar form. `filealloc` (5679) scans the file table for an unreferenced file (`f->ref == 0`) and returns a new reference; `filedup` (5702) increments the reference count; and `fileclose` (5714) decrements it. When a file' s reference count reaches zero, `fileclose` releases the underlying pipe or inode, according to the type.

前三个函数遵循我们熟悉的形式。`filealloc` (5679) 扫描文件表，查找未引用的文件 (`f->ref == 0`) 并返回一个新的引用；`filedup` (5702) 增加引用计数；`fileclose` (5714) 减少引用计数。当文件的引用计数达到零时，`fileclose` 会根据文件类型释放底层的管道或 inode。

> The functions `filestat`, `fileread`, and `filewrite` implement the `stat`, `read`, and `write` operations on files. `filestat` (5753) is only allowed on inodes and calls `stati`.

> `fileread` and `filewrite` check that the operation is allowed by the open mode and then pass the call through to either the pipe or inode implementation. If the file represents an inode, `fileread` and `filewrite` use the I/O offset as the offset for the operation and then advance it (5787-5788) (5830-5831). Pipes have no concept of offset. Recall that the inode functions require the caller to handle locking (5759-5761) (5786-5789) (5829-5832). The inode locking has the convenient side effect that the read and write offsets are updated atomically, so that multiple writing to the same file simultaneously cannot overwrite each other's data, though their writes may end up interlaced.

函数 `filestat`、`fileread` 和 `filewrite` 分别实现针对文件的系统调用 `fstat`、`read` 和 `write` 操作（译者注：实际并不存在 `stat` 系统调用，而应该是 `fstat`，原文这里可能是笔误）。`filestat` (5753) 仅允许在 inode 上使用并调用 `stati`。`fileread` 和 `filewrite` 检查打开模式是否允许该操作，然后将调用传递给管道或 inode 的具体实现。如果文件对应的是 inode，则 `fileread` 和 `filewrite` 会根据读写的偏移量修改结构体 file 的 off 成员字段的值 (5787-5788) (5830-5831)。管道没有偏移量的概念。回忆一下，针对 inode 的操作要求调用者使用锁 (5759-5761) (5786-5789) (5829-5832)。针对 inode 的加锁操作带来了一个额外的好处就是确保了读写过程中对偏移量修改的原子性，因此同时对同一文件的多次写入并不会覆盖彼此的数据，尽管最终它们写入的内容可能会交错出现。

10.14 代码讲解：系统调用（Code: System calls）

> With the functions that the lower layers provide, the implementation of most system calls is trivial (see (5850)). There are a few calls that deserve a closer look.

借助底层提供的功能，大多数系统调用的实现都很简单（参见文件（5850））。但有一些调用值得仔细研究。

> The functions `sys_link` and `sys_unlink` edit directories, creating or removing references to inodes. They are another good example of the power of using transactions. `sys_link` (6002) begins by fetching its arguments, two strings `old` and `new` (6007). Assuming `old` exists and is not a directory (6011-6014), `sys_link` increments its `ip->nlink` count. Then `sys_link` calls `nameiparent` to find the parent directory and final path element of `new` (6027) and creates a new directory entry pointing at `old`'s inode (6030). The new parent directory must exist and be on the same device as the existing inode: inode numbers only have a unique meaning on a single disk. If an error like this occurs, `sys_link` must go back and decrement `ip->nlink`.

函数 `sys_link` 和 `sys_unlink` 用于编辑目录，创建或删除对 inode 的引用。它们是另一个很好的展现使用事务强大功能的例子。`sys_link`（6002）首先获取其两个字符串参数 `old` 和 `new`（6007）。在确保 `old` 存在且不是目录后（6011-6014）（译者注：这里的代码行应该是 6011-6021，原文应该是写错了），`sys_link` 会增加其 `ip->nlink` 计数。然后，`sys_link` 调用 `nameiparent` 来查找 `new`（6027）的父目录和最终路径元素，并创建一个指向 `old` inode（6030）的新目录条目。新的父目录必须存在，并且与现有 inode 位于同一设备上：inode 编号只在同一个磁盘设备上具备唯一性。如果以上条件不满足，`sys_link` 必须返回并减少 `ip->nlink` 的值。

> Transactions simplify the implementation because it requires updating multiple disk blocks, but we don’t have to worry about the order in which we do them. They either will all succeed or none. For example, without transactions, updating `ip->nlink` before creating a link, would put the file system temporarily in an unsafe state, and a crash in between could result in havoc. With transactions we don’t have to worry about this.

事务简化了实现，因为它需要更新多个 block，但我们不必担心执行顺序。它们要么全部成功，要么全部失败。假如没有事务机制，在创建链接之前更新了 `ip->nlink` 会使文件系统暂时处于不安全状态，在此期间崩溃可能会导致严重破坏。有了事务，我们就不必担心这个问题了。

> `sys_link` creates a new name for an existing inode. The function `create` (6124) creates a new name for a new inode. It is a generalization of the three file creation system calls: `open` with the `O_CREATE` flag makes a new ordinary file, `mkdir` makes a new directory, and `mkdev` makes a new device file. Like `sys_link`, `create` starts by calling `nameiparent` to get the inode of the parent directory. It then calls `dirlookup` to check whether the name already exists (6089). If the name does exist, `create`’s behavior depends on which system call it is being used for: `open` has different semantics from `mkdir` and `mkdev`. If `create` is being used on behalf of `open` (`type == T_FILE`) and the name that exists is itself a regular file, then open treats that as a success, so `create` does too (6138). Otherwise, it is an error (6139-6140). If the name does not already exist, `create` now allocates a new inode with `ialloc` (6143). If the new inode is a directory, `create` initializes it with . and .. entries. Finally, now that the data is initialized properly, `create` can link it into the parent directory (6158). `create`, like `sys_link`, holds two inode locks simultaneously: `ip` and `dp`. There is no possibility of deadlock because the inode `ip` is freshly allocated: no other process in the system will hold `ip`’s lock and then try to lock `dp`.

`sys_link` 为一个已存在的 inode 创建新的名称。函数 `create` (6124) 为一个新的 inode 创建新名称。它被三个和创建文件相关的系统调用所调用：带有 `O_CREATE` 参数的 `open` 用它创建新的普通文件，`mkdir` 用它创建新的目录，`mknode` 用它创建新的设备文件（译者注：原文将 `mknod` 写成了 `mkdev`,应该是笔误，下文还有类似情况，翻译时一并改之不再说明）。与 `sys_link` 类似，`create` 首先调用 `nameiparent` 获取父目录的 inode。然后，它调用 `dirlookup` 检查该名称是否已存在 (6089)。如果名称已存在，`create` 的行为取决于它正在用于哪个系统调用：特别地，`open` 的语义与 `mkdir` 和 `mknod` 的语义是不同的。如果 `create` 代表 `open`（`type == T_FILE`）使用，并且存在的名称本身是一个常规文件（译者注：根据源码，设备文件也归于符合此条件），则 open 将其视为成功，`create` 将直接返回找到的 inode（6138）。否则，将返回 0 表示失败（6139-6140）。如果名称尚不存在，`create` 将使用 `ialloc` 分配一个新的 inode（6143）。如果新的 inode 是一个目录，`create` 会为它创建 . 和 .. 条目。最后，既然已经正确地完成了初始化，`create` 可以将其加入到父目录中（6158）（译者注：应该是 6162）。`create` 与 `sys_link` 类似，会同时持有两个 inode 锁：`ip` 和 `dp`。由于 inode `ip` 是新分配的，因此不存在死锁的可能性：系统中没有其他进程会在持有 `ip` 的锁的前提下再尝试锁定 `dp`。

> Using `create`, it is easy to implement `sys_open`, `sys_mkdir`, and `sys_mknod`. `sys_open` (6185) is the most complex, because creating a new file is only a small part of what it can do. If `open` is passed the `O_CREATE` flag, it calls `create` (6201). Otherwise, it calls `namei` (6207). `create` returns a locked inode, but `namei` does not, so `sys_open` must lock the inode itself. This provides a convenient place to check that directories are only opened for reading, not writing. Assuming the inode was obtained one way or the other, `sys_open`

> allocates a file and a file descriptor (6225) and then fills in the file (6237-6242). Note that no other process can access the partially initialized file since it is only in the current process’ s table.

基于 `create` 可以轻松实现 `sys_open`、 `sys_mkdir` 和 `sys_mknod`。 `sys_open` (6185) 是最复杂的，因为创建新文件只是它所能做工作中的一小部分。如果 `open` 传入了 `O_CREATE` 标志，它会调用 `create` (6201)。否则，它会调用 `namei` (6207)。 `create` 返回一个锁定的 inode，但 `namei` 不会，因此（针对执行 `namei` 的代码路径） `sys_open` 必须自己负责锁定 inode。（在执行 `namei` 获得文件路径对应的 inode 后）如果该 inode 的类型是目录，我们还需要检查并确保其仅用于读取而不是写入。无论 inode 是通过 `create` 还是 `namei` 获得的， `sys_open` 都会为它分配一个文件和一个文件描述符 (6225)，然后填充文件的结构体内容 (6237-6242)。值得注意的是，由于这个部分初始化的文件仅位于当前进程的表中，所以我们无需担心其他进程会访问到它。

> Chapter 9 examined the implementation of pipes before we even had a file system. The function `sys_pipe` connects that implementation to the file system by providing a way to create a pipe pair. Its argument is a pointer to space for two integers, where it will record the two new file descriptors. Then it allocates the pipe and installs the file descriptors.

第 9 章在介绍文件系统之前讲解了管道的实现。函数 `sys_pipe` 通过创建一对管道文件的方法将其实现与文件系统建立关联。该系统调用的参数是一个指针，该指针指向的内存空间可以放置两个整数，它将在其中记录两个新的文件描述符。函数实现中会分配管道以及分配新的文件描述符。

10.15 现实世界（Real world）

> The buffer cache in a real-world operating system is significantly more complex than xv6’ s, but it serves the same two purposes: caching and synchronizing access to the disk. Xv6’ s buffer cache, like V6’ s, uses a simple least recently used (LRU) eviction policy; there are many more complex policies that can be implemented, each good for some workloads and not as good for others. A more efficient LRU cache would eliminate the linked list, instead using a hash table for lookups and a heap for LRU evictions. Modern buffer caches are typically integrated with the virtual memory system to support memory-mapped files.

实际操作系统中的 buffer cache 比 xv6 的复杂得多，但它有两个相同的用途：缓存和同步磁盘访问。与 V6 类似，xv6 的 buffer cache 使用简单的 “最近最少使用 (LRU)” 驱逐策略；此外，还可以实现许多更复杂的策略，每种策略都适用于某些特定的工作场景，而对其他工作场景则不那么适用。更高效的 LRU 缓存在查找时会使用哈希表而不是链表，并使用堆实现 “LRU 驱逐（LRU evictions）” （译者注：操作系统中的 "LRU evictions" 指的是在使用 LRU (Least Recently Used) 页面置换算法时，当物理内存（页框）已满且有新页面需要载入时，操作系统选择并 “驱逐” （即移出内存）那个“最近最少被使用”的页面的过程。）。现代 buffer cache 通常与虚拟内存系统集成，以支持内存映射文件。

> Xv6’ s logging system is inefficient. A commit cannot occur concurrently with file-system system calls. The system logs entire blocks, even if only a few bytes in a block are changed. It performs synchronous log writes, a block at a time, each of which is likely to require an entire disk rotation time. Real logging systems address all of these problems.

xv6 的日志系统效率低下。提交操作无法与文件系统的系统调用并发执行。即使一个 block 中只有几个字节被更改，系统也会记录整个 block。日志写入实现为同步方式，每次写入一个 block，每次写入都可能需要花费完整的磁盘旋转时间。真正的日志系统解决了所有这些问题。

> Logging is not the only way to provide crash recovery. Early file systems used a scavenger during reboot (for example, the UNIX `fsck` program) to examine every file and directory and the block and inode free lists, looking for and resolving inconsistencies. Scavenging can take hours for large file systems, and there are situations where it is not possible to resolve inconsistencies in a way that causes the original system calls to be atomic. Recovery from a log is much faster and causes system calls to be atomic in the face of crashes.

日志并非解决崩溃恢复的唯一方法。早期的文件系统在重启时使用清理程序（例如，UNIX 的 `fsck` 程序）来检查每个文件、目录以及 block 和 inode 空闲列表，查找并解决不一致问题。对于大型文件系统来说，清理过程可能需要数小时，并且在某些情况下，在解决一致性问题时无法确保原先系统调用的原子性。基于日志方式实现恢复速度更快，并且在遇到崩溃时也能确保系统调用的原子性。

> Xv6 uses the same basic on-disk layout of inodes and directories as early UNIX; this scheme has been remarkably persistent over the years. BSD’ s UFS/FFS and Linux’ s ext2/ext3 use essentially the same data structures. The most inefficient part of the file system layout is the directory, which requires a linear scan over all the disk blocks during each lookup. This is reasonable when directories are only a few disk blocks, but is expensive for directories holding many files. Microsoft Windows’ s NTFS, macOS’ s HFS, and Solaris’ s ZFS, just to name a few, implement a directory as an on-disk balanced tree of blocks. This is complicated but guarantees logarithmic-time directory lookups.

对于 inode 和目录，xv6 使用了与早期 UNIX 相同的基本磁盘布局；这种方案多年来一直非常稳定。BSD 的 UFS/FFS 和 Linux 的 ext2/ext3 也使用基本相同的数据结构。文件系统布局中最低效的部分是目录，每次查找时都需要对所有 block 进行线性扫描。当目录只占用几个 block 时，这是合理的，但对于包含许多文件的目录来说，成本很高。Microsoft Windows 的 NTFS、macOS 的 HFS 和 Solaris 的 ZFS 等针对目录的磁盘 block 采用平衡树的方式实现。虽然比较复杂，但可以保证以对数的时间复杂度实现目录查找。

> Xv6 is naive about disk failures: if a disk operation fails, xv6 panics. Whether this is reasonable depends on the hardware: if an operating systems sits atop special hardware that uses redundancy to mask disk failures, perhaps the operating system sees failures so infrequently that panicking is okay. On the other hand, operating systems using plain disks should expect failures and handle them more gracefully, so that the loss of a block in one file doesn’ t affect the use of the rest of the file system.

xv6 对磁盘故障的处理比较简单：如果磁盘操作失败，xv6 会直接崩溃。这种情况是否合理取决于硬件条件：如果操作系统使用特殊的硬件，并利用冗余机制来屏蔽磁盘故障，那么操作系统遇到故障的频率可能很低，因此进入崩溃是可以接受的。另一方面，使用普通磁盘的操作系统应该能够预料到故障，并更优雅地处理它们，这样即使一个文件中的某个块丢失，也不会影响文件系统其他部分的使用。

> Xv6 requires that the file system fit on one disk device and not change in size. As large databases and multimedia files drive storage requirements ever higher, operating systems

> are developing ways to eliminate the “one disk per file system” bottleneck. The basic approach is to combine many disks into a single logical disk. Hardware solutions such as RAID are still the most popular, but the current trend is moving toward implementing as much of this logic in software as possible. These software implementations typically allow rich functionality like growing or shrinking the logical device by adding or removing disks on the fly. Of course, a storage layer that can grow or shrink on the fly requires a file system that can do the same: the fixed-size array of inode blocks used by xv6 would not work well in such environments. Separating disk management from the file system may be the cleanest design, but the complex interface between the two has led some systems, like Sun’s ZFS, to combine them.

xv6 要求整个文件系统容纳在同一个磁盘设备上，并且大小不能改变。随着大型数据库和多媒体文件对存储需求的不断增长，操作系统正在开发各种方法来消除“每个文件系统一个磁盘（one disk per file system）”的限制。基本方法是将多个磁盘组合成一个逻辑磁盘。RAID 等硬件解决方案仍然是最流行的，但目前的趋势是尽可能多地在软件中实现这种逻辑。这些软件实现通常允许丰富的功能，例如通过动态添加或删除磁盘来增大或缩小逻辑设备的大小。当然，一个可以动态增大或缩小的存储层需要一个文件系统的支持：由于 xv6 使用固定大小的 block 数组存放 inode，所以是无法支持上述环境要求的。将磁盘管理与文件系统分离可能是最简洁的设计，但两者之间复杂的接口导致一些系统（例如 Sun 的 ZFS）将它们合并在一起。

> Xv6’s file system lacks many other features of modern file systems; for example, it lacks support for snapshots and incremental backup.

xv6 的文件系统缺少现代文件系统的许多其他功能；例如，它缺少对“快照（snapshot）”和“增量备份（incremental backup）”的支持。

> Modern Unix systems allow many kinds of resources to be accessed with the same system calls as on-disk storage: named pipes, network connections, remotely-accessed network file systems, and monitoring and control interfaces such as `/proc`. Instead of xv6’s `if` statements in `fileread` and `filewrite`, these systems typically give each open file a table of function pointers, one per operation, and call the function pointer to invoke that inode’s implementation of the call. Network file systems and user-level file systems provide functions that turn those calls into network RPCs and wait for the response before returning.

现代 Unix 系统支持使用和我们访问磁盘存储时所使用的系统调用相同的接口来访问多种资源：这包括了命名管道、网络连接、可远程访问的网络文件系统，以及诸如 `/proc` 之类的用于监控和控制的接口。这些系统通常为每个打开的文件提供一个函数指针表（每个操作一个），并通过调用这些函数指针来调用针对该 inode 的具体函数实现，而不是像 xv6 那样在 `fileread` 和 `filewrite` 中使用 `if` 语句。网络文件系统和用户层文件系统则将这些调用转换为网络 RPC（译者注：RPC 即 Remote Procedure Call 的缩写），这意味着针对这些文件系统，调用这些系统调用后在返回之前将不得不等待较长时间的响应。

10.16 练习（Exercises）

> 1. Why panic in `balloc`? Can xv6 recover?

1. 为什么 `balloc` 会 panic？xv6 能恢复吗？

> 1. Why panic in `ialloc`? Can xv6 recover?

1. 为什么 `ialloc` 会 panic？xv6 能恢复吗？

> 1. Why doesn’t `filealloc` panic when it runs out of files? Why is this more common and therefore worth handling?

1. 为什么 `filealloc` 中遇到文件用完时不 panic？为什么这种情况更常见，因此值得处理？

> 1. Suppose the file corresponding to `ip` gets unlinked by another process between `sys_link`’s calls to `iunlock(ip)` and `dirlink`. Will the link be created correctly? Why or why not?

1. 假设在 `sys_link` 调用 `iunlock(ip)` 和 `dirlink` 之间，另一个进程 unlink 了与 ip 对应的文件。链接会正确创建吗？为什么会这样？

> 1. `create` makes four function calls (one to `ialloc` and three to `dirlink`) that it requires to succeed. If any doesn’t, `create` calls `panic`. Why is this acceptable? Why can’t any of those four calls fail?

1. `create` 需要调用四个函数才能成功（一次调用 `ialloc`，三次调用 `dirlink`）。如果任何一个函数调用失败，`create` 就会引发 `panic`。为什么这是可以接受的？为什么这四个函数调用都不能失败？

> 1. `sys_chdir` calls `iunlock(ip)` before `iput(cp->cwd)`, which might try to lock `cp->cwd`, yet postponing `iunlock(ip)` until after the `iput` would not cause deadlocks. Why not?

1. `sys_chdir` 在 `iput(cp->cwd)` 之前调用 `iunlock(ip)`，这可能会尝试锁定 `cp->cwd`，但将 `iunlock(ip)` 推迟到 `iput` 之后执行不会造成死锁。为什么呢？

> 1. Implement the `lseek` system call. Supporting `lseek` will also require that you modify `filewrite` to fill holes in the file with zero if `lseek` sets off beyond `f->ip->size`.

1. 实现 `lseek` 系统调用。支持 `lseek` 还需要修改 `filewrite`，以便在 `lseek` 超出 `f->ip->size` 时用零填充文件中的空洞。

> 1. Add `O_TRUNC` and `O_APPEND` to `open`, so that `>` and `>>` operators work in the shell.

1. 为 `open` 中添加 `O_TRUNC` 和 `O_APPEND`，以便 `>` 和 `>>` 操作符在 shell 中起作用。

> 1. Modify the file system to support symbolic links.

1. 修改文件系统以支持 “符号链接（symbolic link）”。

> 1. Modify the file system to support named pipes.

1. 修改文件系统以支持 “有名管道（named pipe）”。

> 1. Modify the file and VM system to support memory-mapped files.

1. 修改文件和 VM 系统以支持 “内存映射文件（memory-mapped file）”。

第 11 章 再次讨论并发问题（Chapter 11 Concurrency revisited）

> Simultaneously obtaining good parallel performance, correctness despite concurrency, and understandable code is a big challenge in kernel design. Straightforward use of locks is the best path to correctness, but is not always possible. This chapter highlights examples in which xv6 is forced to use locks in an involved way, and examples where xv6 uses lock-like techniques but not locks.

对于内核并发设计，要同时保证优秀的性能、正确的逻辑以及良好的代码可读性是一个巨大的挑战。直接使用锁是实现正确性的最佳途径，但并非总是可行的。本章重点介绍了 xv6 中不得不以复杂的方式使用锁的案例，此外，也介绍了另外一些案例，在这些案例中 xv6 没有直接使用锁而是使用了其他类似锁的技术。

11.1 加锁的方式（Locking patterns）

> Cached items are often a challenge to lock. For example, the file system’s block cache (4275) stores copies of up to `NBUF` disk blocks. It’s vital that a given disk block have at most one copy in the cache; otherwise, different processes might make conflicting changes to different copies of what ought to be the same block. Each cached block is stored in a `struct buf` (3900). A `struct buf` has a lock field which helps ensure that only one process uses a given disk block at a time. However, that lock is not enough: what if a block is not present in the cache at all, and two processes want to use it at the same time? There is no `struct buf` (since the block isn’t yet cached), and thus there is nothing to lock. Xv6 deals with this situation by associating an additional lock (`bcache.lock`) with the set of identities of cached blocks. Code that needs to check if a block is cached (e.g., `bget` (4308)), or change the set of cached blocks, must hold `bcache.lock` ; after that code has found the block and `struct buf` it needs, it can release `bcache.lock` and lock just the specific block. This is a common pattern: one lock for the set of items, plus one lock per item.

对缓存项上锁通常是一件非常具有挑战性的事。例如，文件系统的 “块缓存（block cache）” (4275) 记录了最多 `NBUF` 个磁盘 block 的副本。至关重要的是，给定的磁盘 block 在缓存中只能有一个副本；否则如果不同的进程对同一个 block 的多个副本进行修改的话，必然会导致冲突。每个缓存都存储在 `struct buf` (3900) 中。 `struct buf` 有一个 `lock` 字段，用于确保同一时间只有一个进程能够访问给定的磁盘 block。然而，仅靠这把锁还不够：如果某个 block 根本不存在于缓存中，而两个进程想要同时使用它，该怎么办？由于没有 `struct buf` （因为该 block 尚未被缓存），因此也就没有可锁定的内容。为了处理这种情况，xv6 为整个缓存 block 集合引入一把额外的锁 (`bcache.lock`) 。在需要检查某个 block 是否已缓存（例如 `bget` (4308)）或更改缓存 block 集合时代码必须先持有 `bcache.lock` ；在找到所需的 block 和 `struct buf` 后，代码就可以释放 `bcache.lock` 而仅仅锁定特定的 block。这是一种常见的模式：对一组对象加一把全局的锁，同时每个对象还有一把私有的锁。

> Ordinarily the same function that acquires a lock will release it. But a more precise way to view things is that a lock is acquired at the start of a sequence that must appear atomic, and released when that sequence ends. If the sequence starts and ends in different functions, or different threads, or on different CPUs, then the lock acquire and release must do the same. The function of the lock is to force other uses to wait, not to pin a piece of data to a particular agent. One example is the `acquire` in `yield` (2629), which is released in the scheduler thread rather than in the acquiring process. Another example is the `acquiresleep` in `ilock` (5153); this code often sleeps while reading the disk; it may wake up on a different CPU, which means the lock may be acquired and released on different CPUs.

通常，获取锁的函数也会负责释放锁。但更准确的理解是，锁在一个原子性的操作序列开始时被获取，并在该序列结束时被释放。如果这个操作序列分布在不同的函数、不同的线程或者在不同的 CPU 上开始和结束，则锁的获取和释放也必须遵循相同的逻辑。锁的作用是强制其他用户等待，而不是将数据绑定到特定的对象上（译者注：指前面提到的函数、线程或者 CPU）。一个例子是 `yield`（2629）中的 `acquire`，该锁在 scheduler 线程中才被释放，而不是在获取锁的进程中。另一个例子是 `ilock`（5153）中的 `acquiresleep`；这段代码在读取磁盘时经常处于休眠状态；执行这段代码的线程可能在不同的 CPU 上被唤醒，这意味着上锁和解锁可能发生在不同的 CPU 上。

> Freeing an object that is protected by a lock embedded in the object is a delicate business, since owning the lock is not enough to guarantee that freeing would be correct. The problem case arises when some other thread is waiting in `acquire` to use the object; freeing the object implicitly frees the embedded lock, which will cause the waiting thread to malfunction. One solution is to track how many references to the object exist, so that it is only freed when the last reference disappears. See `pipeclose` (6661) for an example; `pi->readopen` and `pi->writeopen` track whether the pipe has file descriptors referring to it.

如果一个对象中含有一个用于保护该对象的锁成员（译者注：下文我们把该锁成员称之为 “内嵌锁（embedded lock）”），则在释放该对象时需要额外谨慎，因为拥有锁并不足以保证释放操作的正确性。如果在释放这样的一个对象时存在其他线程正在 `acquire` 中等待使用该对象，则会出现问题；因为释放该对象会隐式地释放内嵌锁，这将导致等待的线程出现问题（译者注：这里的释放对象可以理解为调用 `kfree`，这会释放锁所在的内存并导致内存中的值被改写，从而造成解锁的发生，但这种解锁动作并不是我们显式执行的结果）。一种解决方案是跟踪该对象的引用数量，以便仅在最后一个引用消失时才释放它。例如，请参阅 `pipeclose` (6661)； `pi->readopen` 和 `pi->writeopen` 会跟踪管道是否有文件描述符引用它。

> Usually one sees locks around sequences of reads and writes to sets of related items; the locks ensure that other threads see only completed sequences of updates (as long as they, too, lock). What about situations where the update is a simple write to a single shared variable? For example, `setkilled` and `killed` (2775) lock around their simple uses of `p->killed`. If there were no lock, one thread could write `p->killed` at the same time that another thread reads it. This is a race, and the C language specification says that a race yields undefined behavior, which means the program may crash or yield incorrect results (Note: "Threads and data races" in https://en.cppreference.com/w/c/language/memory_model). The locks prevent the race and avoid the undefined behavior.

通常，我们会看到锁用于保护一个由多个读写操作组成的对相关数据项的访问序列；锁确保其他（使用同一把锁的）其他线程在该执行序列没有完成之前必须等待。但如果操作只是对单个共享变

量的单个写入，情况又会如何呢？例如，`setkilled` 和 `killed` (2775) 会在对 `p->killed` 的简单读写前后上锁和解锁。如果没有锁，可能会出现当一个线程对 `p->killed` 写入的同时另一个线程读取它。这是一种竞争，C 语言规范指出竞争会导致 未定义行为（undefined behavior），这意味着程序可能崩溃或产生不正确的结果（注："Threads and data races" in https://en.cppreference.com/w/c/language/memory_model）。锁可以阻止竞争，从而避免未定义行为。

> One reason races can break programs is that, if there are no locks or equivalent constructs, the compiler may generate machine code that reads and writes memory in ways quite different than the original C code. For example, the machine code of a thread calling `killed` could copy `p->killed` to a register and read only that cached value; this would mean that the thread might never see any writes to `p->killed`. The locks prevent such caching.

竞争会导致程序崩溃的一个原因是，如果没有锁或等效的机制，编译器可能会生成与原始 C 代码截然不同的读写内存的机器码。例如，调用 `killed` 的线程的机器指令可能会将 `p->killed` 复制到寄存器，并只读取该寄存器中缓存的值；这意味着该线程可能永远不会看到对 `p->killed` 更新。锁可以防止这种缓存。

11.2 类似锁的并发保护方式（Lock-like patterns）

> In many places xv6 uses a reference count or a flag in a lock-like way to indicate that an object is allocated and should not be freed or re-used. A process's `p->state` acts in this way, as do the reference counts in `file`, `inode`, and `buf` structures. While in each case a lock protects the flag or reference count, it is the latter that prevents the object from being prematurely freed.

xv6 在很多地方以 “类似锁（lock-like）” 的方式使用引用计数或标志来标识对象已分配且不应被释放或重用。进程的 `p->state` 以这种方式运行，`file`、`inode` 和 `buf` 结构中的引用计数也是如此。虽然在每种情况下，锁都会保护标志或引用计数，但后者会阻止对象被过早释放。

> The file system uses `struct inode` reference counts as a kind of shared lock that can be held by multiple processes, in order to avoid deadlocks that would occur if the code used ordinary locks. For example, the loop in `namex` (5555) locks the directory named by each pathname component in turn. However, `namex` must release each lock at the end of the loop, since if it held multiple locks it could deadlock with itself if the pathname included a dot (e.g., `a/./b`). It might also deadlock with a concurrent lookup involving the directory and ... As Chapter 10 explains, the solution is for the loop to carry the directory inode over to the next iteration with its reference count incremented, but not locked.

文件系统使用 `struct inode` 结构体中的引用计数（译者注：即 `ref` 成员字段）作为一种可由多个进程持有的 “共享锁”，以避免代码使用普通锁时可能发生的死锁。例如，`namex`（5555）中的循环依次锁定每个路径名中的 component 所对应的目录。但是，`namex` 必须在循环结束时释放每个锁，因为如果它持有多个锁，当路径名包含一个 “.”（例如，`a/./b`）时，它可能会与自身发生死锁。它还可能与涉及目录的并发查找发生死锁 ..... 正如第 10 章所解释的，解决方案是让循环将目录的 inode 传递到下一次迭代，其引用计数递增，但不锁定。

> Some data items are protected by different mechanisms at different times, and may at times be protected from concurrent access implicitly by the structure of the xv6 code rather than by explicit locks. For example, when a physical page is free, it is protected by `kmem.lock` (2973). If the page is then allocated as a pipe (6622), it is protected by a different lock (the embedded `pi->lock`). If the page is re-allocated for a new process's user memory, it is not protected by a lock at all. Instead, the fact that the allocator won't give that page to any other process (until it is freed) protects it from concurrent access. The ownership of a new process's memory is complex: first the parent allocates and manipulates it in `fork`, then the child uses it, and (after the child exits) the parent again owns the memory and passes it to `kfree`. There are two lessons here: a data object may be protected from concurrency in different ways at different points in its lifetime, and the protection may take the form of implicit structure rather than explicit locks.

某些数据项在不同时间受不同机制保护，有时可能通过 xv6 代码结构（而非显式锁）隐式地防止并发访问。例如，当一个物理页空闲时，它受 `kmem.lock` (2973) 保护。如果该页随后被分配给管道 (6622)，则受另一个锁（嵌入的 `pi->lock`）保护。如果该页被重新分配给新进程的用户内存，则它完全不受锁保护。相反，分配器不会将该页分配给任何其他进程（直到它被释放），这一事实保护了它免受并发访问。新进程内存的所有权非常复杂：首先，父进程在 `fork` 中分配并操作它，然后子进程使用它，并且（子进程退出后）父进程再次拥有该内存并将其传递给 `kfree`。这里有两条教训：在数据对象的生命周期的不同阶段，可以采用不同的方式保护其免于并发，并且这种保护可以采用隐式结构而不是显式锁的形式。

> A final lock-like example is the need to disable interrupts around calls to `mycpu()` (2187). Disabling interrupts causes the calling code to be atomic with respect to timer interrupts that could force a context switch, and thus move the process to a different CPU.

最后一个类似锁的例子是，需要在 `mycpu()` 调用前后禁用中断 (2187)。禁用中断会导致这段代码不会受定时器中断干扰而具有原子性，因为定时器中断可能强制触发上下文切换，这会导致进程被移动到不同的 CPU 上。

11.3 完全不上锁（No locks at all）

> There are a few places where xv6 shares mutable data with no locks at all. One is in the implementation of spinlocks, although one could view the RISC-V atomic instructions as relying on locks implemented in hardware. Another is the `started` variable in `main.c` (1156), used to prevent other CPUs from running until CPU zero has finished initializing xv6; the `volatile` ensures that the compiler actually generates load and store instructions.

xv6 中有些地方在共享可变数据时并不采用锁进行保护。一个是自旋锁的实现，尽管 RISC-V 的原子指令也可以看作依赖于硬件实现的锁。另一个是 `main.c` (1156) 中的 `started` 变量，用于阻止其他 CPU 在第一个 CPU（编号为 0）完成 xv6 初始化之前运行；`volatile` 这个 C 语言的类型限定符确保编译器确实生成了加载和存储指令。

> Xv6 contains cases in which one CPU or thread writes some data, and another CPU or thread reads the data, but there is no specific lock dedicated to protecting that data. For example, in `fork`, the parent writes the child's user memory pages, and the child (a different thread, perhaps on a different CPU) reads those pages; no lock explicitly protects those

> pages. This is not strictly a locking problem, since the child doesn’t start executing until after the parent has finished writing. It is a potential memory ordering problem (see Chapter 7), since without a memory barrier there’s no reason to expect one CPU to see another CPU’s writes. However, since the parent releases locks, and the child acquires locks as it starts up, the memory barriers in `acquire` and `release` ensure that the child’s CPU sees the parent’s writes.

xv6 中还存在这样的情况：一个 CPU 或线程写入数据，另一个 CPU 或线程读取数据，但没有专门的锁来保护这些数据。例如，在 `fork` 中，父进程写入子进程的用户内存页，而子进程（另一个线程，可能位于不同的 CPU 上）读取这些页；没有锁明确地保护这些页。严格来说，这并非是一个需要引入锁保护的问题，因为子进程直到父进程完成写入后才开始执行。这是一个潜在的 “内存排序（memory ordering）” 问题（参见第 7 章），因为如果没有内存屏障，就无法保证一个 CPU 正确地看到另一个 CPU 的写入结果。但是，由于父进程会释放锁，而子进程会在启动时获取锁，因此 `acquire` 和 `release` 中执行的内存屏障可以确保子进程的 CPU 能够看到父进程的写入。

11.4 并行性（Parallelism）

> Locking is primarily about suppressing parallelism in the interests of correctness. Because performance is also important, kernel designers often have to think about how to use locks in a way that both achieves correctness and allows parallelism. While xv6 is not systematically designed for high performance, it’s still worth considering which xv6 operations can execute in parallel, and which might conflict on locks.

锁定主要是为了正确性而限制了并行性。由于性能也很重要，内核设计人员经常需要考虑如何使用锁，既能实现正确性，又能实现并行性。虽然 xv6 并非一个为追求高性能而设计的系统，但（具体实现 xv6 时）仍然值得花些时间考虑一下哪些操作可以并行执行，哪些操作可能会在上锁时发生冲突。

> Pipes in xv6 are an example of fairly good parallelism. Each pipe has its own lock, so that different processes can read and write different pipes in parallel on different CPUs. For a given pipe, however, the writer and reader must wait for each other to release the lock; they can’t read/write the same pipe at the same time. It is also the case that a read from an empty pipe (or a write to a full pipe) must block, but this is not due to the locking scheme.

xv6 中的管道是展示并行性的相当好的一个示例。每个管道都有自己的锁，因此不同的进程可以在不同的 CPU 上并行地读写不同的管道。然而，对于给定的管道，写入者和读取者必须等待对方释放锁；他们不能同时读写同一个管道。从空管道读取（或向满管道写入）也必须阻塞，但这并不是由于锁方案造成的。

> Context switching is a more complex example. Two kernel threads, each executing on its own CPU, can call `yield`, `sched`, and `swtch` at the same time, and the calls will execute in parallel. The threads each hold a lock, but they are different locks, so they don’t have to wait for each other. Once in `scheduler`, however, the two CPUs may conflict on locks while searching the table of processes for one that is `RUNNABLE`. That is, xv6 is likely to get a performance benefit from multiple CPUs during context switch, but perhaps not as much as it could.

上下文切换是一个更复杂的例子。两个内核线程，每个线程都在各自的 CPU 上执行，可以同时调用 `yield`、`sched` 和 `swtch`，并且这些调用将并行执行。每个线程都持有一个锁，但它们是不同的锁，因此它们不必互相等待。然而，一旦进入 `scheduler`，两个 CPU 在搜索进程表以查找 `RUNNABLE` 的进程时可能会发生锁冲突。也就是说，xv6 在上下文切换期间可能会从多 CPU 中获得性能优势，但可能不如预期的那样显著。

> Another example is concurrent calls to `fork` from different processes on different CPUs. The calls may have to wait for each other for `pid_lock` and `kmem.lock`, and for per-process locks needed to search the process table for an `UNUSED` process. On the other hand, the two forking processes can copy user memory pages and format page-table pages fully in parallel.

另一个例子是来自不同 CPU 上的不同进程的并发 `fork` 调用。这些调用可能需要互相等待 `pid_lock` 和 `kmem.lock`，以及在进程表中搜索 `UNUSED` 进程时需要等待每个进程的锁。另一方面，两个 fork 进程可以完全并行地复制用户内存页并格式化页表页。

> The locking scheme in each of the above examples sacrifices parallel performance in certain cases. In each case it’s possible to obtain more parallelism using a more elaborate design. Whether it’s worthwhile depends on details: how often the relevant operations are invoked, how long the code spends with a contended lock held, how many CPUs might be running conflicting operations at the same time, whether other parts of the code are more restrictive bottlenecks. It can be difficult to guess whether a given locking scheme might cause performance problems, or whether a new design is significantly better, so measurement on realistic workloads is often required.

上述每个示例中的锁定方案在某些情况下都会牺牲并行性能。在每种情况下，使用更精细的设计都可能获得更高的并行性。这是否值得取决于一些细节，譬如：相关操作的调用频率；代码在争用锁的情况下花费的时间；可能会有多少个 CPU 同时运行冲突的操作；代码的其他部分是否是更具限制性的瓶颈等。很难猜测给定的锁定方案是否会导致性能问题，或者新的设计是否显著优于现有方案，因此通常需要基于实际工作负载进行测量。

11.5 练习（Exercises）

> 1. Modify xv6’s pipe implementation to allow a read and a write to the same pipe to proceed in parallel on different CPUs.

1. 修改 xv6 的管道实现，以允许对同一管道的读取和写入在不同的 CPU 上并行进行。

> 1. Modify xv6’s `scheduler()` to reduce lock contention when different CPUs are looking for runnable processes at the same time.

1. 修改 xv6 的 `scheduler()`，以减少不同 CPU 同时寻找可运行进程时的锁争用。

> 1. Eliminate some of the serialization in xv6’s `fork()`.

1. 消除 xv6 的 `fork()` 中的部分串行化代码。

第 12 章 总结（Chapter 12 Summary）

> This text introduced the main ideas in operating systems by studying one operating system, xv6, line by line. Some code lines embody the essence of the main ideas (e.g., context switching, user/kernel boundary, locks, etc.) and each line is important; other code lines provide an illustration of how to implement a particular operating system idea and could easily be done in different ways (e.g., a better algorithm for scheduling, better on-disk data structures to represent files, better logging to allow for concurrent transactions, etc.). All the ideas were illustrated in the context of one particular, very successful system call interface, the Unix interface, but those ideas carry over to the design of other operating systems.

本文通过逐行研究 xv6 操作系统，介绍了操作系统的核心思想。部分代码体现了核心思想的精髓（例如，上下文切换、用户/内核边界、锁等等），每一行都至关重要；其他代码则展示了如何实现特定的操作系统理念，并且可以轻松地以不同的方式实现（例如，更好的调度算法、更好的磁盘数据结构来表示文件、更好的日志记录以支持并发事务等等）。所有这些思想都基于一个非常成功的系统调用接口（Unix 接口）进行了阐述，但这些思想也适用于其他操作系统的设计。

参考书目（Bibliography）

[1] Linux common vulnerabilities and exposures (CVEs). https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=linux.

[2] The RISC-V instruction set manual Volume I: unprivileged specification ISA. https://drive.google.com/file/d/17GeetSnT5wW3xNuAHI95-SI1gPGd5sJ_/view?usp=drive_link, 2024.

[3] The RISC-V instruction set manual Volume II: privileged specification. https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?usp=drive_link, 2024.

[4] Hans-J Boehm. Threads cannot be implemented as a library. ACM PLDI Conference, 2005.

[5] Edsger Dijkstra. Cooperating sequential processes. https://www.cs.utexas.edu/users/EWD/transcriptions/EWD01xx/EWD123.html, 1965.

[6] Maurice Herlihy and Nir Shavit. The Art of Multiprocessor Programming, Revised Reprint. 2012.

[7] Brian W. Kernighan. The C Programming Language. Prentice Hall Professional Technical Reference, 2nd edition, 1988.

[8] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. Sel4: Formal verification of an OS kernel. In Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, page 207–220, 2009.

[9] Donald Knuth. Fundamental Algorithms. The Art of Computer Programming. (Second ed.), volume 1. 1997.

[10] L Lamport. A new solution of dijkstra’ s concurrent programming problem. Communications of the ACM, 1974.

[11] John Lions. Commentary on UNIX 6th Edition. Peer to Peer Communications, 2000.

[12] Paul E. Mckenney, Silas Boyd-wickizer, and Jonathan Walpole. RCU usage in the linux kernel: One decade later, 2013.

[13] Martin Michael and Daniel Durich. The NS16550A: UART design and application considerations. http://bitsavers.trailing-edge.com/components/national/_appNotes/AN-0491.pdf, 1987.

[14] Aleph One. Smashing the stack for fun and profit. http://phrack.org/issues/49/14.html#article.

[15] David Patterson and Andrew Waterman. The RISC-V Reader: an open architecture Atlas. Strawberry Canyon, 2017.

[16] Dave Presotto, Rob Pike, Ken Thompson, and Howard Trickey. Plan 9, a distributed system. In In Proceedings of the Spring 1991 EurOpen Conference, pages 43–50, 1991.

[17] Dennis M. Ritchie and Ken Thompson. The UNIX time-sharing system. Commun. ACM, 17(7):365–375, July 1974.